

Семантические пространства литературных стилей (NLP)

🚀 Pet проект: Обучение Word2Vec на литературных корпусах

Описание проекта

Цель проекта — реализовать с нуля модель Word2Vec (CBOW и Skip-gram) и обучить её на корпусах произведений классических авторов из разных литературных жанров (Шекспир, Достоевский, Азимов, Кафка). Такой подход позволяет исследовать, как стилистика и жанровая специфика отражаются в векторных представлениях слов.

Проект выполнен в Google Colab с использованием GPU, что обеспечивает воспроизводимость, удобство экспериментов и возможность делиться результатами через Jupyter Notebook.

Навыки и технологии в pet-проекте:

- Python, NumPy, Pandas, Keras
- NLP: токенизация, предобработка текста
- Реализация и обучение Word2Vec (CBOW, Skip-gram)
- Визуализация (matplotlib, seaborn, t-SNE, PCA)
- Работа в Google Colab с использованием GPU

Описание данных

- Источники:** открытые литературные корпуса (Project Gutenberg, Wikipedia dumps, Kaggle Datasets, другие открытые библиотеки).
- Жанры и авторы:**
 - Уильям Шекспир: Английская классическая драма (пьесы, сонеты).
 - Фёдор Достоевский: Русская психологическая проза, философский роман.
 - Франц Кафка: Абсурдистская и модернистская проза.
 - Айзек Азимов: Научная фантастика.
- Объём данных:** десятки тысяч предложений на автора (в зависимости от доступных текстов).

Цели исследования

- Техническая реализация:**
 - Реализовать с нуля архитектуру Word2Vec (CBOW и Skip-gram) на Keras.
- Сравнительный анализ:**
 - Обучить модели на корпусах разных авторов и сравнить полученные векторные пространства.
 - Исследовать, как стилистика и жанровая специфика отражаются в семантических связях слов.
- Стилистическое исследование:**
 - Продемонстрировать на конкретных примерах, что модель улавливает стилистические особенности. Например, проверить гипотезы:
 - У Шекспира слово "love" будет соседствовать с "beauty", "honor", "heart".
 - У Достоевского "love" будет ассоциироваться с "suffering", "pain", "sacrifice".
 - У Азимова "robot" будет находиться рядом с "law", "logic", "human", а не просто с "machine".
- Визуализация и интерпретация:**
 - Наглядно представить семантические пространства разных авторов с помощью методов снижения размерности (t-SNE/PCA) и проинтерпретировать наблюдаемые кластеры.
 - Сделать выводы о том, как лингвистический контекст влияет на семантику слов.
- Подготовить воспроизводимый Jupyter Notebook** с комментариями, визуализациями и результатами.

Этапы работы над проектом

Шаг 1: Подготовка среды

- Подключение GPU в Colab (Runtime → Change runtime type → GPU).
- Установка зависимостей через !pip install (numpy, pandas, nltk, spacy, torch, matplotlib, seaborn, scikit-learn).
- Настройка рабочего окружения: импорт библиотек, установка random seed для воспроизводимости.
- Подключение Google Drive для хранения данных и результатов.
- Настройка логирования (logging, tqdm для прогресса обучения).

Шаг 2: Сбор и загрузка данных

- Автоматическая загрузка текстов из Project Gutenberg или других открытых источников.

- Формирование отдельных корпусов по авторам (Шекспир, Достоевский, Азимов, Кафка).
- Сохранение сырых текстов в Google Drive для повторного использования.

Шаг 3: Очистка и предобработка текста

- Приведение текста к нижнему регистру.
- Удаление пунктуации, цифр, спецсимволов (регулярные выражения).
- Токенизация текста (разбиение на слова) с помощью nltk или spaCy.
- Удаление стоп-слов (например, "the", "and", "of).
- Лемматизация (spaCy для английского).
- Сохранение очищенных корпусов для повторного использования.

Шаг 4: Исследовательский анализ данных (EDA)

- Подсчёт статистики: количество уникальных токенов, средняя длина предложения
- Построение распределения частот слов
- Визуализация top-50 самых частых слов (wordcloud)
- Анализ специфичной лексики для каждого автора (TF-IDF)

Шаг 5: Подготовка обучающих данных (Keras)

- Построение словаря и фильтрация
- Subsampling частых слов
- Формирование обучающих пар
- Создание пайплайна и датасетов
- Статистика по корпусам и примеры батчей
- Проверка первых батчей (CBOW/Skip-gram)
- Визуализация длин контекстов (CBOW)
- Статистика словаря
- Сравнение частот слов до/после фильтрации
- Примеры обучающих пар
- Визуализация эффекта subsampling

Шаг 6: Первичная реализация модели Word2Vec (Keras, Functional API)

- Подготовка датасетов для CBOW/Skip-gram
- Определение моделей CBOW/Skip-gram через Functional API
- Обучение моделей CBOW/Skip-gram
- Вывод размеров эмбедингов после обучения

Шаг 7: Обучение модели (Keras)

- Настройка гиперпараметров:
 - embedding_dim: размер эмбедингов (например, 100–300).
 - window_size: ширина контекстного окна.
 - epochs: количество эпох обучения.
 - learning_rate: скорость обучения.
 - batch_size: размер батча.
- Запуск цикла обучения:
 - Для обеих моделей используется model.fit() (CBOW и Skip-gram).
- Функция потерь:
 - CBOW/Skip-gram — SparseCategoricalCrossentropy.
- Обратное распространение и обновление весов:
 - Оптимизаторы: Adam или SGD.
 - Поддержка learning rate schedules и callbacks.
- Логирование метрик:
 - Используется History от model.fit().
 - Вывод loss/accuracy по эпохам, сохранение графиков.
- Сохранение модели:
 - Промежуточные веса (опционально): model.save_weights("checkpoint_epoch_5.h5").
 - Финальная модель: model.save("word2vec_cbow.keras"), model.save("word2vec_skipgram.keras").
 - Эмбединги: model.get_layer("embedding").get_weights()[0].

Шаг 8: Эксперименты и сравнения (Keras)

- Обучение модели на 4 корпусах авторов (Шекспир, Достоевский, Азимов, Кафка):

- Модель обучается отдельно на своём корпусе (например, model_dostoevsky, model_shakespeare).
- Эмбединги извлекаются через model.get_layer("embedding").get_weights()[0] (как из обученной, так и из загруженной модели .keras).
- Сравнительная таблица метрик по авторам
- Анализ влияния размера корпуса на качество
- Построить графики сравнения accuracy/loss по авторам
- Сравнение ближайших соседей:
 - Для выбранных слов (например, "love", "truth", "king") находятся топ-N ближайших соседей по косинусному сходству.
 - Используется scipy.spatial.distance.cdist или sklearn.metrics.pairwise.cosine_similarity.
- Проверка аналогий:
 - Классическая формула: $\text{vec}(\text{"king"}) - \text{vec}(\text{"man"}) + \text{vec}(\text{"woman"}) \approx \text{vec}(\text{"queen"})$.
 - Вычисляется результирующий вектор и ищется ближайшее слово в эмбедингах.
- Сравнение косинусных сходств между словами:
 - Для одного и того же слова в разных моделях сравниваются его соседи и сходства.
 - Можно визуализировать различия через тепловые карты или scatter-плоты.
- Анализ различий в стилях:
 - Сравниваются семантические окружения слов: какие слова оказываются ближе к "truth" у Шекспира и у Достоевского.
 - Можно построить графы или кластеры на основе сходства.

Шаг 9: Визуализация результатов (Keras)

- Снижение размерности эмбедингов:
 - Для наглядного анализа используется t-SNE и PCA из sklearn для преобразования эмбедингов в 2D-пространство.
 - Эмбединги извлекаются напрямую из обученной или загруженной модели
- Построение scatter plot:
 - Для выбранных слов отображаются их 2D-координаты
- Визуализация кластеров:
 - Слова группируются по тематикам (например, профессии, эмоции, абстрактные понятия).
 - Цветовая маркировка по кластерам может быть выполнена вручную или с помощью алгоритмов (например, KMeans)..
 - Для построения удобно использовать seaborn.scatterplot(hue=cluster_label).
- Построение heatmap косинусных сходств:
 - Вычисляется матрица косинусных сходств между выбранными словами.
 - Используется sklearn.metrics.pairwise.cosine_similarity и seaborn.heatmap()

Шаг 10: Интерпретация и выводы

Шаг 11: Финализация проекта

- Подготовка Jupyter Notebook в Colab с комментариями и визуализациями.
- Сохранение обученных embedding в файл (.vec или .pt).
- Документация проекта: описание шагов, результаты экспериментов, выводы.
- Публикация на GitHub с README.md, где есть:
 - краткое описание,
 - инструкция по запуску в Colab,
 - примеры визуализаций.

Ожидаемые результаты

- **Обученные модели Word2Vec** для каждого автора, сохраненные в формате .keras.
- **Серия сравнительных таблиц** с ближайшими соседями для заданных слов (например, "king", "justice", "fear", "god") в пространствах разных авторов.
- **Визуализации (scatter-plot'ы)**, показывающие кластеризацию слов по тематическим группам и различия в их расположении у разных авторов.
- **Подготовленный Jupyter Notebook** с воспроизводимым кодом, графиками и комментариями.
- **Репозиторий на GitHub** с README.md, включающим описание проекта, инструкцию по запуску и примеры визуализаций

Содержание:

- [Шаг 1: Подготовка среды](#)
- [Шаг 2: Сбор и загрузка данных](#)
- [Шаг 3: Очистка и предобработка текста](#)
- [Шаг 4: Исследовательский анализ данных \(EDA\)](#)

- [Шаг 5: Подготовка обучающих данных](#)
- [Шаг 6: Первичная реализация модели Word2Vec \(Keras, Functional API\)](#)
- [Шаг 7: Обучение модели \(Keras\)](#)
- [Шаг 8: Эксперименты и сравнения \(Keras\)](#)
- [Шаг 9: Визуализация результатов \(Keras\)](#)
- [Шаг 10: Интерпретация и выводы](#)

Выполнение проекта

Шаг 1: Подготовка среды

- Установка зависимостей
- Импорт библиотек

```
In [1]: # === Стандартная библиотека ===
import os
import re
import math
import time
import random
from pathlib import Path
import pathlib
import pickle
import importlib

# === Научный стек ===
import numpy as np
import pandas as pd

# === Визуализация ===
import matplotlib.pyplot as plt
import seaborn as sns
from wordcloud import WordCloud

# === NLP ===
import nltk
import spacy
import spacy.cli
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize, sent_tokenize

# === Счётчики/структуры данных ===
from collections import Counter

# === ML / Векторизация / Метрики / Модели ===
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.manifold import TSNE
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
from sklearn.metrics.pairwise import cosine_similarity

# === DL ===
import tensorflow as tf
import torch
import torch.nn as nn
import torch.optim as optim

# === Утилиты ===
import requests
from tqdm import tqdm
import pkg_resources

# === Глобальные настройки визуализации ===
sns.set_theme(style="whitegrid", palette="deep")
plt.rcParams["figure.figsize"] = (10, 6)
```

```
In [2]: start_all = time.time()
```

```
In [3]: for pkg in ["punkt", "stopwords"]:
        nltk.download(pkg)

        spacy.cli.download("en_core_web_sm")
```

```
[nltk_data] Downloading package punkt to E:\cache\nltk...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to E:\cache\nltk...
[nltk_data]   Package stopwords is already up-to-date!
```


✓ Download and installation successful

You can now load the package via `spacy.load('en_core_web_sm')`

```
In [4]: try:
        nlp = spacy.load("en_core_web_sm")
    except OSError:
        print("Модель spaCy не найдена. Установите её командой: !python -m spacy download en_core_web_sm")
```

```
In [5]: nlp = spacy.load("en_core_web_sm")

# Проверим работу
doc = nlp("Love conquers all things.")
print([token.lemma_ for token in doc])

['love', 'conquer', 'all', 'thing', '.']
```

- Проверка GPU

```
In [6]: # Проверка доступности GPU и его названия

print("CUDA доступна:", torch.cuda.is_available())
if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
    print("Количество GPU:", torch.cuda.device_count())
    print("Объем памяти (MB):", round(torch.cuda.get_device_properties(0).total_memory / 1024**2))
else:
    print("Совет: включите GPU в меню 'Среда выполнения' → 'Изменить тип среды выполнения' → 'Аппаратный ускоритель: GPU'")
```

CUDA доступна: False

Совет: включите GPU в меню 'Среда выполнения' → 'Изменить тип среды выполнения' → 'Аппаратный ускоритель: GPU'

- Установка случайного seed для воспроизводимости
 - Зачем: Чтобы результаты (разбиения, инициализации весов, порядок батчей) были повторяемыми.

```
In [7]: def set_seed(seed: int = 42):
        # Python / NumPy
        random.seed(seed)
        np.random.seed(seed)

        # PyTorch (если используешь)
        try:
            import torch
            torch.manual_seed(seed)
            torch.cuda.manual_seed_all(seed)
            torch.backends.cudnn.deterministic = True
            torch.backends.cudnn.benchmark = False
        except ImportError:
            pass # PyTorch может быть не установлен

        # TensorFlow (если используешь)
        try:
            import tensorflow as tf
            tf.random.set_seed(seed)
        except ImportError:
            pass # TensorFlow может быть не установлен

        # Для полной детерминированности
        os.environ["PYTHONHASHSEED"] = str(seed)

        set_seed(42)
        print("Seed установлен.")
```

Seed установлен.

```
In [8]: torch.use_deterministic_algorithms(True, warn_only=True)
```

- Папки для хранения данных и результатов

```
In [9]: # Базовая директория проекта (можешь поменять на любую)
BASE_DIR = Path.cwd() / "word2vec_literature"

PATHS = {
    "base": BASE_DIR,
    "raw": BASE_DIR / "data_raw",
    "clean": BASE_DIR / "data_clean",
    "models": BASE_DIR / "models",
    "figs": BASE_DIR / "figs",
    "logs": BASE_DIR / "logs",
}

# Создание директорий
for p in PATHS.values():
    p.mkdir(parents=True, exist_ok=True)
```

```
print("Директории готовы:")
for name, path in PATHS.items():
    print(f"{name}: {path}")
```

Директории готовы:
base: E:\projects\My_project\03_Word2Vec\word2vec_literature
raw: E:\projects\My_project\03_Word2Vec\word2vec_literature\data_raw
clean: E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean
models: E:\projects\My_project\03_Word2Vec\word2vec_literature\models
figs: E:\projects\My_project\03_Word2Vec\word2vec_literature\figs
logs: E:\projects\My_project\03_Word2Vec\word2vec_literature\logs

```
In [10]: # Путь к requirements.txt внутри локального проекта
req_path = PATHS["base"] / "requirements.txt"

# Список библиотек, которые реально используются в проекте Word2Vec
used_libs = [
    "numpy", "pandas", "matplotlib", "seaborn",
    "torch", "nltk", "spacy", "tqdm",
    "wordcloud", "requests"
]

# Словарь {имя: версия} всех установленных пакетов
installed = {dist.project_name.lower(): dist.version for dist in pkg_resources.working_set}

# Запись только нужных библиотек
with open(req_path, "w", encoding="utf-8") as f:
    for lib in used_libs:
        key = lib.lower()
        if key in installed:
            f.write(f"{lib}=={installed[key]}\n")
        else:
            print(f"⚠ {lib} не установлен в окружении")

print(f"📁 requirements.txt сохранён: {req_path}")
```

📁 requirements.txt сохранён: E:\projects\My_project\03_Word2Vec\word2vec_literature\requirements.txt

Выводы по шагу 1:

Что сделано:

- Настроена рабочая структура проекта с понятными путями (PATHS), включая папки для исходных, очищенных и визуализированных данных.
- Установлены все необходимые библиотеки (numpy, pandas, nltk, spacy, torch, matplotlib, seaborn, scikit-learn, tqdm, wordcloud).
- Проверена инициализация spaCy и загрузка модели en_core_web_sm.
- Настроена работа с GPU (проверка доступности CUDA).
- Установлен фиксированный seed для воспроизводимости экспериментов.
- Подключён Google Drive и создана структура директорий проекта (raw, clean, models, figs, logs).
- Сформирован requirements.txt с используемыми библиотеками.

Основные выводы по Шагу 1:

- Среда полностью подготовлена для воспроизводимых экспериментов.
- Есть контроль версий библиотек, структура проекта и проверка оборудования.

[* к содержанию](#)

Шаг 2: Сбор и загрузка данных

Скачивание сырых данных

```
In [11]: # Словарь с авторами и ссылками на тексты (Project Gutenberg и др.)

TEXT_SOURCES = {
    "shakespeare": "https://www.gutenberg.org/files/100/100-0.txt", # Complete Works of Shakespeare
    "dostoevsky": "https://www.gutenberg.org/files/600/600-0.txt", # Notes from the Underground (Dostoevsky)
    "kafka": "https://www.gutenberg.org/files/7849/7849-0.txt", # The Trial (Kafka)

    # Isaac Asimov (public domain short stories on Project Gutenberg)
    "asimov_youth": "https://www.gutenberg.org/cache/epub/31547/pg31547.txt", # Youth (1952)
    "asimov_magnificent_possession": "https://www.gutenberg.org/ebooks/76871.txt.utf-8", # The Magnificent Possession (1940)
    "asimov_lets_get_together": "https://www.gutenberg.org/cache/epub/68377/pg68377.txt" # Let's Get Together (1957)
}
```

```
In [12]: def download_and_check(author, url, save_dir=PATHS["raw"]):
        """
        Скачивает текст произведения по ссылке, сохраняет его в отдельную папку для автора,
        проверяет размер файла и выводит первые символы для визуальной проверки.
        Возвращает словарь с метаданными (автор, путь к файлу, источник, размеры).
        """

        # Отправляем HTTP-запрос по указанному URL
        response = requests.get(url)

        # Проверяем, что сервер вернул успешный код (200 = OK)
        if response.status_code == 200:
            # Создаём подпапку для конкретного автора внутри общей папки raw
            author_dir = save_dir / author
            author_dir.mkdir(parents=True, # parents=True – создаст все недостающие папки по пути
                             exist_ok=True) # exist_ok=True – не выдаст ошибку, если папка уже существует

            # Формируем полный путь к файлу, например raw/dostoevsky/dostoevsky.txt
            file_path = author_dir / f"{author}.txt"

            # Забираем текст из ответа и убираем лишние пробелы/переводы строк по краям
            text = response.text.strip()

            # Считаем количество символов в тексте
            size_chars = len(text)

            # Считаем размер в килобайтах (переводим в байты через UTF-8, делим на 1024)
            size_kb = round(len(text.encode("utf-8")) / 1024, 2)

            # Если текст пустой – предупреждаем и выходим
            if size_chars == 0:
                print(f"⚠ Файл {author} пустой!")
                return None

            # Открываем файл на запись в кодировке UTF-8 и сохраняем туда текст
            with open(file_path, "w", encoding="utf-8") as f:
                f.write(text)

            # Выводим информацию о сохранённом файле
            print(f"✅ {author} сохранён: {file_path}")
            print(f"    Размер: {size_chars} символов (~{size_kb} KB)")
            print(f"    Первые 600 символов:\n")
            print(text[:600]) # показываем первые 600 символов для проверки
            print("\n" + "-"*80 + "\n") # разделитель для удобства

            # Возвращаем словарь с метаданными о файле
            return {
                "author": author,          # имя автора
                "file": str(file_path),    # путь к сохранённому файлу
                "source": url,             # ссылка, откуда скачали
                "size_chars": size_chars,  # количество символов
                "size_kb": size_kb         # размер в килобайтах
            }

        else:
            # Если сервер вернул ошибку (например, 404 или 500) – выводим предупреждение
            print(f"⚠ Ошибка при загрузке {author} (код {response.status_code})")
            return None
```

```
In [13]: start = time.time()

        # Скачиваем все корпуса и собираем метаданные
        metadata = [] # сюда будем складывать словари с информацией о каждом авторе

        # TEXT_SOURCES – это словарь вида {"dostoevsky": "url", "shakespeare": "url", ...}
        for author, url in TEXT_SOURCES.items():
            # вызываем функцию скачивания и проверки текста
            info = download_and_check(author, url)
            if info: # если функция вернула словарь (а не None)
                metadata.append(info) # добавляем его в общий список

        # Сохраняем метаданные в таблицу
        meta_df = pd.DataFrame(metadata) # превращаем список словарей в DataFrame
        meta_df.to_csv(PATHS["raw"] / "metadata.csv", index=False) # сохраняем в CSV без индекса
        meta_df

        elapsed = time.time() - start
        print(f"Время выполнения: {elapsed:.2f} сек ({elapsed/60:.2f} мин)")
```

✓ shakespeare сохранён: E:\projects\My_project\03_Word2Vec\word2vec_literature\data_raw\shakespeare\shakespeare.txt
Размер: 5359443 символов (~5295.62 KB)
Первые 600 символов:

*** START OF THE PROJECT GUTENBERG EBOOK 100 ***

The Complete Works of William Shakespeare

by William Shakespeare

Contents

THE SONNETS
ALL'S WELL THAT ENDS WELL
THE TRAGEDY OF ANTONY AND CLEOPATRA
AS YOU LIKE IT
THE COMEDY OF ERRORS
THE TRAGEDY OF CORIOLANUS
CYMBELINE
THE TRAGEDY OF HAMLET, PRINCE OF DENMARK
THE FIRST PART OF KING HENRY THE FOURTH
THE SECOND PART OF KING HENRY THE FOURTH
THE LIFE OF KING HENRY THE FIFTH
THE FIRST PART OF HENRY THE SIXTH
THE SECOND PART OF KING HENRY THE SIXTH
THE THIRD PART O

✓ dostoevsky сохранён: E:\projects\My_project\03_Word2Vec\word2vec_literature\data_raw\dostoevsky\dostoevsky.txt
Размер: 263859 символов (~260.54 KB)
Первые 600 символов:

The Project Gutenberg eBook of Notes from the Underground, by Fyodor Dostoyevsky

This eBook is for the use of anyone anywhere in the United States and most other parts of the world at no cost and with almost no restrictions whatsoever. You may copy it, give it away or re-use it under the terms of the Project Gutenberg License included with this eBook or online at www.gutenberg.org. If you are not located in the United States, you will have to check the laws of the country where you are located before using this eBook.

Title: Notes from the Underground

Author: Fyodor Dostoyevsky

✓ kafka сохранён: E:\projects\My_project\03_Word2Vec\word2vec_literature\data_raw\kafka\kafka.txt
Размер: 475989 символов (~464.91 KB)
Первые 600 символов:

The Project Gutenberg eBook of The Trial, by Franz Kafka

This eBook is for the use of anyone anywhere in the United States and most other parts of the world at no cost and with almost no restrictions whatsoever. You may copy it, give it away or re-use it under the terms of the Project Gutenberg License included with this eBook or online at www.gutenberg.org. If you are not located in the United States, you will have to check the laws of the country where you are located before using this eBook.

Title: The Trial

Author: Franz Kafka

Translator: David Wyllie

Release Date: Ma

✓ asimov_youth сохранён: E:\projects\My_project\03_Word2Vec\word2vec_literature\data_raw\asimov_youth\asimov_youth.txt
Размер: 78673 символов (~76.93 KB)
Первые 600 символов:

The Project Gutenberg eBook of Youth

This eBook is for the use of anyone anywhere in the United States and most other parts of the world at no cost and with almost no restrictions

whatsoever. You may copy it, give it away or re-use it under the terms of the Project Gutenberg License included with this eBook or online at www.gutenberg.org. If you are not located in the United States, you will have to check the laws of the country where you are located before using this eBook.

Title: Youth

Author: Isaac Asimov

Release date: March 7, 2010 [eBook #31547]

asimov_magnificent_possession сохрaнён: E:\projects\My_project\03_Word2Vec\word2vec_literature\data_raw\asimov_magnificent_possession\asimov_magnificent_possession.txt
Размер: 56751 символов (~56.62 KB)
Первые 600 символов:

The Project Gutenberg eBook of The magnificent possession

This eBook is for the use of anyone anywhere in the United States and most other parts of the world at no cost and with almost no restrictions whatsoever. You may copy it, give it away or re-use it under the terms of the Project Gutenberg License included with this eBook or online at www.gutenberg.org. If you are not located in the United States, you will have to check the laws of the country where you are located before using this eBook.

Title: The magnificent possession

Author: Isaac Asimov

Illustrator: Mort Mes

asimov_lets_get_together сохрaнён: E:\projects\My_project\03_Word2Vec\word2vec_literature\data_raw\asimov_lets_get_together\asimov_lets_get_together.txt
Размер: 55106 символов (~53.92 KB)
Первые 600 символов:

The Project Gutenberg eBook of Let's Get Together

This eBook is for the use of anyone anywhere in the United States and most other parts of the world at no cost and with almost no restrictions whatsoever. You may copy it, give it away or re-use it under the terms of the Project Gutenberg License included with this eBook or online at www.gutenberg.org. If you are not located in the United States, you will have to check the laws of the country where you are located before using this eBook.

Title: Let's Get Together

Author: Isaac Asimov

Illustrator: Robert Engle

Время выполнения: 9.72 сек (0.16 мин)

```
In [14]: meta_df
```

Out[14]:

	author	file	source	size_chars
0	shakespeare	E:\projects\My_project\03_Word2Vec\word2vec_li...	https://www.gutenberg.org/files/100/100-0.txt	5359443
1	dostoevsky	E:\projects\My_project\03_Word2Vec\word2vec_li...	https://www.gutenberg.org/files/600/600-0.txt	263859
2	kafka	E:\projects\My_project\03_Word2Vec\word2vec_li...	https://www.gutenberg.org/files/7849/7849-0.txt	475989
3	asimov_youth	E:\projects\My_project\03_Word2Vec\word2vec_li...	https://www.gutenberg.org/cache/epub/31547/pg3...	78673
4	asimov_magnificent_possession	E:\projects\My_project\03_Word2Vec\word2vec_li...	https://www.gutenberg.org/ebooks/76871.txt.utf-8	56751
5	asimov_lets_get_together	E:\projects\My_project\03_Word2Vec\word2vec_li...	https://www.gutenberg.org/cache/epub/68377/pg6...	55106



```
In [15]: meta_df["words_estimate"] = meta_df["size_chars"] // 6
display(meta_df[["author", "size_chars", "words_estimate"]])
```

	author	size_chars	words_estimate
0	shakespeare	5359443	893240
1	dostoevsky	263859	43976
2	kafka	475989	79331
3	asimov_youth	78673	13112
4	asimov_magnificent_possession	56751	9458
5	asimov_lets_get_together	55106	9184

```
In [16]: # Функция для анализа корпуса по авторам
def analyze_corpus(meta_df):
    # Считаем общее количество слов во всём корпусе (сумма по колонке words_estimate)
    total_words = meta_df["words_estimate"].sum()
    print(f"Общий объем корпуса: {total_words:,} слов") # выводим общий объём с разделителями тысяч

    print("\nРаспределение по авторам:")
    # Проходим по каждой строке датафрейма (каждый автор)
    for _, row in meta_df.iterrows():
        # Считаем долю слов данного автора от общего объёма
        percentage = (row["words_estimate"] / total_words) * 100
        # Выводим имя автора, количество слов и процент от общего корпуса
        print(f" {row['author']}: {row['words_estimate']:,} слов ({percentage:.1f}%)")

# Запускаем анализ для собранного датафрейма с метаданными
analyze_corpus(meta_df)
```

Общий объем корпуса: 1,048,301 слов

Распределение по авторам:

- shakespeare: 893,240 слов (85.2%)
- dostoevsky: 43,976 слов (4.2%)
- kafka: 79,331 слов (7.6%)
- asimov_youth: 13,112 слов (1.3%)
- asimov_magnificent_possession: 9,458 слов (0.9%)
- asimov_lets_get_together: 9,184 слов (0.9%)

- author — имя автора, которому принадлежит корпус.
- size_chars — количество символов (буквы, пробелы, знаки препинания) в тексте.
 - Это «сырой» размер корпуса.
- words_estimate — примерное количество слов в корпусе.
 - Обычно считается как size_chars // средняя_длина_слова (примерно деление на 6).
 - Это грубая оценка, но она даёт понимание масштаба.

Очистка первых строк

- Постобработка после скачивания файлов

```
In [17]: # === Очистка Project Gutenberg ===
def clean_gutenberg_text(text: str) -> str:
    """
    Удаляет служебные блоки Project Gutenberg:
    '*** START OF THE PROJECT GUTENBERG EBOOK ... ***'
    и '*** END OF THE PROJECT GUTENBERG EBOOK ... ***'
    Возвращает только чистый литературный текст.
    """
    # Ищем начало служебного блока (если есть)
    start = re.search(r'\*\* START OF THE PROJECT GUTENBERG EBOOK.\*\*.*', text)
    # Ищем конец служебного блока (если есть)
    end = re.search(r'\*\* END OF THE PROJECT GUTENBERG EBOOK.\*\*.*', text)

    # Если нашли начало — берём индекс конца этого блока, иначе с самого начала текста
    start_idx = start.end() if start else 0
    # Если нашли конец — берём индекс начала блока, иначе до конца текста
    end_idx = end.start() if end else len(text)

    # Вырезаем только литературный текст и убираем лишние пробелы
    clean_text = text[start_idx:end_idx].strip()
    return clean_text
```

```
In [18]: # === Очистка и сохранение всех файлов ===
def process_files(meta_df, save_dir=PATHS["clean"]):
    # Создаём папку для сохранения очищенных файлов (если её ещё нет)
    save_dir.mkdir(parents=True, exist_ok=True)
    records = [] # сюда будем собирать метаданные по каждому автору

    # Проходим по строкам датафрейма с метаданными
    for _, row in meta_df.iterrows():
```



```

# Читаем исходный (сырой) текст
raw = Path(row["file"]).read_text(encoding="utf-8")
# Очищаем текст от служебных блоков
clean = clean_gutenberg_text(raw)
# Формируем путь для сохранения очищенного текста
out = save_dir / f"{row['author']}.txt"
# Сохраняем очищенный текст
out.write_text(clean, encoding="utf-8")

# Добавляем запись о файле в список метаданных
records.append({
    "author": row["author"],      # имя автора
    "file_clean": str(out),       # путь к очищенному файлу
    "size_chars": len(clean),     # количество символов
    "size_words": len(clean.split()) # количество слов
})

# Сообщаем о завершении обработки автора
print(f"🌟 {row['author']} → {out}")

# Возвращаем датафрейм с метаданными по всем авторам
return pd.DataFrame(records)

```

```

In [19]: # === Объединение Азимова ===
def combine_asimov(save_dir=PATHS["clean"], out_name="asimov.txt"):

    # Находим все файлы вида asimov_*.txt (если тексты Азимова разбиты на части)
    files = sorted(save_dir.glob("asimov_*.txt"))

    # Читаем все части и объединяем их через двойной перенос строки
    text = "\n\n".join(f.read_text(encoding="utf-8") for f in files)

    # Путь к итоговому файлу
    out = save_dir / out_name

    # Сохраняем объединённый текст
    out.write_text(text, encoding="utf-8")
    print(f"📁 Азимов объединён: {out}")
    return out

```

```

In [20]: start = time.time()

# === Запуск ===
# 1. Очищаем все тексты и собираем метаданные
meta_df_clean = process_files(meta_df)

# 2. Объединяем все части Азимова в один файл
asimov_path = combine_asimov(PATHS["clean"])
# Читаем объединённый текст Азимова
asimov_text = Path(asimov_path).read_text(encoding="utf-8")

# 3. Добавляем запись об объединённом файле Азимова в таблицу метаданных
meta_df_clean = pd.concat([
    meta_df_clean,
    pd.DataFrame([
        "author": "asimov",
        "file_clean": str(asimov_path),
        "size_chars": len(asimov_text),
        "size_words": len(asimov_text.split())
    ])
], ignore_index=True)

# 4. Сохраняем обновлённые метаданные в CSV
meta_df_clean.to_csv(PATHS["clean"] / "metadata_clean.csv", index=False)

# 5. Настраиваем отображение pandas (чтобы показывались длинные пути)
pd.set_option("display.max_colwidth", None)

# 6. Выводим итоговую таблицу с метаданными
display(meta_df_clean)

elapsed = time.time() - start
print(f"Время выполнения: {elapsed:.2f} сек ({elapsed/60:.2f} мин)")

```

```

🌟 shakespeare → E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean\shakespeare.txt
🌟 dostoevsky → E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean\dostoevsky.txt
🌟 kafka → E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean\kafka.txt
🌟 asimov_youth → E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean\asimov_youth.txt
🌟 asimov_magnificent_possession → E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean\asimov_magnificent_possession.txt
🌟 asimov_lets_get_together → E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean\asimov_lets_get_together.txt
📁 Азимов объединён: E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean\asimov.txt

```


	author		file_clean	size_chars	si
0	shakespeare	E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean\shakespeare.txt		5359342	
1	dostoevsky	E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean\dostoevsky.txt		244174	
2	kafka	E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean\kafka.txt		456443	
3	asimov_youth	E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean\asimov_youth.txt		58945	
4	asimov_magnificent_possession	E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean\asimov_magnificent_possession.txt		36986	
5	asimov_lets_get_together	E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean\asimov_lets_get_together.txt		35240	
6	asimov	E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean\asimov.txt		391277	

Время выполнения: 0.23 сек (0.00 мин)

```
In [21]: # Добавляем язык корпуса (все тексты – английские)
meta_df_clean["words_estimate"] = meta_df_clean["size_chars"] // 6
meta_df_clean["language"] = "en"

# Сохраняем обновлённые метаданные
meta_df_clean.to_csv(PATHS["clean"] / "metadata_clean.csv", index=False)
pd.set_option("display.max_colwidth", None)
meta_df_clean
```

Out[21]:

	author		file_clean	size_chars
0	shakespeare	E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean\shakespeare.txt		5359342
1	dostoevsky	E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean\dostoevsky.txt		244174
2	kafka	E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean\kafka.txt		456443
3	asimov_youth	E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean\asimov_youth.txt		58945
4	asimov_magnificent_possession	E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean\asimov_magnificent_possession.txt		36986
5	asimov_lets_get_together	E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean\asimov_lets_get_together.txt		35240
6	asimov	E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean\asimov.txt		391277

```
In [22]: display(meta_df_clean[["author", "size_chars", "size_words", "words_estimate"]])
```

	author	size_chars	size_words	words_estimate
0	shakespeare	5359342	963460	893223
1	dostoevsky	244174	44132	40695
2	kafka	456443	83728	76073
3	asimov_youth	58945	10081	9824
4	asimov_magnificent_possession	36986	6129	6164
5	asimov_lets_get_together	35240	5806	5873
6	asimov	391277	63632	65212

```
In [23]: # === Анализ корпусов ===
def analyze_corpus(meta_df):
    """
    Функция для анализа распределения слов по авторам.
    Принимает DataFrame с колонкой 'size_words' (количество слов в корпусе каждого автора).
    """

    # Считаем общее количество слов во всём корпусе (сумма по всем авторам)
    total_words = meta_df["size_words"].sum()
    print(f"Общий объем корпуса: {total_words:,} слов") # выводим общий объём с разделителями тысяч

    print("\nРаспределение по авторам:")
    # Проходим по строкам датафрейма (каждая строка = один автор)
    for _, row in meta_df.iterrows():
        # Считаем долю слов данного автора от общего объёма
        percentage = (row["size_words"] / total_words) * 100
        # Выводим имя автора, количество слов и процент от общего корпуса
        print(f" {row['author']}: {row['size_words']:,} слов ({percentage:.1f}%)")

# 🚀 Запуск анализа для очищенного корпуса
analyze_corpus(meta_df_clean)
```

Общий объем корпуса: 1,176,968 слов

Распределение по авторам:

shakespeare: 963,460 слов (81.9%)
dostoevsky: 44,132 слов (3.7%)
kafka: 83,728 слов (7.1%)
asimov_youth: 10,081 слов (0.9%)
asimov_magnificent_possession: 6,129 слов (0.5%)
asimov_lets_get_together: 5,806 слов (0.5%)
asimov: 63,632 слов (5.4%)

```
In [24]: def check_gutenberg_removed(df):
        """Проверяет, что служебные блоки Gutenberg удалены"""
        issues = []
        for _, row in df.iterrows():
            text = Path(row["file_clean"]).read_text(encoding="utf-8")
            if "PROJECT GUTENBERG" in text.upper():
                issues.append(row["author"])

        if issues:
            print(f"⚠️ Метаданные Gutenberg остались в: {'', '.join(issues)}")
        else:
            print("✅ Все метаданные Gutenberg успешно удалены")

check_gutenberg_removed(meta_df_clean)
```

✅ Все метаданные Gutenberg успешно удалены

Балансировка текстов Шекспира

```
In [25]: def balance_corpus(author="shakespeare", target_chars=500_000, clean_dir=PATHS["clean"]):
        """
        Функция для балансировки корпуса текста:
        - Берёт текст указанного автора.
        - Если он длиннее target_chars символов, то обрезает его до этого размера.
        - Начало выбирается случайно, чтобы не всегда брать один и тот же кусок.
        - Конец корректируется так, чтобы текст заканчивался на границе предложения.
        - Возвращает путь к новому сбалансированному файлу.
        """

        # Формируем путь к исходному очищенному файлу автора
        file_path = clean_dir / f"{author}.txt"
        # Читаем текст из файла (игнорируем ошибки кодировки)
        text = Path(file_path).read_text(encoding="utf-8", errors="ignore")

        # Если текст и так меньше или равен целевому размеру – ничего не делаем
        if len(text) <= target_chars:
            print(f"⚠️ {author} уже меньше {target_chars} символов.")
            return file_path

        # Выбираем случайную стартовую позицию так,
        # чтобы гарантированно хватило символов до конца текста
        start = random.randint(0, len(text) - target_chars)

        # Вырезаем кусок текста длиной target_chars символов
        balanced = text[start:start + target_chars]

        # Чтобы не обрывать текст посередине предложения,
        # ищем последнее вхождение точки с пробелом ". "
        end_idx = balanced.rfind(". ")
        if end_idx != -1:
            # Если нашли – обрезаем текст до конца этого предложения
            balanced = balanced[:end_idx + 1]

        # Формируем путь для сохранения сбалансированного текста
        out_path = clean_dir / f"{author}_balanced.txt"
        # Сохраняем результат в новый файл
        out_path.write_text(balanced, encoding="utf-8")

        # Выводим информацию о результате
        print(f"🐼 {author} сбалансирован до ~{len(balanced):,} символов → {out_path}")

        # Возвращаем путь к сбалансированному файлу
        return out_path

# === Применение: урезаем Шекспира ===
# Вызов функции для автора "shakespeare", ограничение – 500 000 символов
balanced_path = balance_corpus("shakespeare", 500_000)
```

🐼 shakespeare сбалансирован до ~499,657 символов → E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean\shakespeare_balanced.txt

```
In [26]: # === Обновляем метаданные ===
balanced_text = Path(balanced_path).read_text(encoding="utf-8", errors="ignore")

balanced_record = pd.DataFrame([{
    "author": "shakespeare_balanced",
```

```
"file_clean": str(balanced_path),
"size_chars": len(balanced_text),
"size_words": len(balanced_text.split()),
"words_estimate": len(balanced_text) // 6,
"language": "en",
"balanced": "yes"
})

# обновляем общий DataFrame
meta_df_clean["balanced"] = meta_df_clean["author"].apply(lambda x: "yes" if "balanced" in x else "no")
meta_df_clean = pd.concat([meta_df_clean, balanced_record], ignore_index=True)

# сохраняем обновлённый CSV
meta_df_clean.to_csv(PATHS["clean"] / "metadata_clean.csv", index=False)

pd.set_option("display.max_colwidth", None)
display(meta_df_clean)
```

	author		file_clean	size_chars	si
0	shakespeare	E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean\shakespeare.txt		5359342	
1	dostoevsky	E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean\dostoevsky.txt		244174	
2	kafka	E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean\kafka.txt		456443	
3	asimov_youth	E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean\asimov_youth.txt		58945	
4	asimov_magnificent_possession	E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean\asimov_magnificent_possession.txt		36986	
5	asimov_lets_get_together	E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean\asimov_lets_get_together.txt		35240	
6	asimov	E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean\asimov.txt		391277	
7	shakespeare_balanced	E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean\shakespeare_balanced.txt		499657	



```
In [27]: # список нужных авторов
selected_authors = ["dostoevsky", "kafka", "asimov", "shakespeare_balanced"]

# фильтруем датафрейм
metadata_balanced = meta_df_clean[meta_df_clean["author"].isin(selected_authors)]

# сохраняем только их в отдельный CSV
metadata_balanced.to_csv(PATHS["clean"] / "metadata_balanced.csv", index=False)
pd.set_option("display.max_colwidth", None)
display(metadata_balanced)
```

	author		file_clean	size_chars	size_words	words
1	dostoevsky	E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean\dostoevsky.txt		244174	44132	
2	kafka	E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean\kafka.txt		456443	83728	
6	asimov	E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean\asimov.txt		391277	63632	
7	shakespeare_balanced	E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean\shakespeare_balanced.txt		499657	90223	



```
In [28]: analyze_corpus(metadata_balanced)
```

Общий объем корпуса: 281,715 слов

Распределение по авторам:
dostoevsky: 44,132 слов (15.7%)
kafka: 83,728 слов (29.7%)
asimov: 63,632 слов (22.6%)
shakespeare_balanced: 90,223 слов (32.0%)

```
In [29]: def preview_texts(df, n_chars=400):
    """
    Функция для предварительного просмотра текстов.
    - Принимает DataFrame с колонкой 'file_clean' (пути к очищенным текстам).
    - Для каждого автора выводит первые n_chars символов текста.
    """

    # Проходим по строкам датафрейма (каждая строка = один автор и его файл)
    for _, row in df.iterrows():
        # Получаем путь к файлу с очищенным текстом
        path = Path(row["file_clean"])

        # Читаем содержимое файла в строку
        text = path.read_text(encoding="utf-8")

        # Выводим заголовок: имя автора и язык текста
        print(f"\n=== {row['author']} ({row['language']}) ===")
```

```
# Выводим первые n_chars символов текста (по умолчанию 400)
print(text[:n_chars]) # это позволяет быстро оценить качество данных

# Разделитель для удобства чтения между авторами
print("\n" + "-"*80)

# 🚀 Пример использования:
# Показываем первые 400 символов текстов для выбранных авторов
preview_texts(
    meta_df_clean.query("author in ['dostoevsky','kafka','asimov','shakespeare_balanced']")
)
```

=== dostoevsky (en) ===
Notes from the Underground

by Fyodor Dostoyevsky

Contents

NOTES FROM THE UNDERGROUND

PART I Underground

I

II

III

IV

V

VI

VII

VIII

IX

X

XI

PART II À Propos of the Wet Snow

I

II

III

IV

V

VI

VII

VIII

IX

X

NOTES FROM THE UNDERGROUND[*]

A NOVEL

* The author of the diary and the diary itself are, of course,
imaginary. Neverthe

=== kafka (en) ===
THE TRIAL

Franz Kafka

Translation Copyright © by David Wyllie

Translator contact email: dandelion@post.cz

Chapter One

Arrest--Conversation with Mrs. Grubach--Then Miss Bürstner

Someone must have been telling lies about Josef K., he knew he had done nothing wrong but, one morning, he was arrested. Every day at eight in the morning he was brought his breakfast by Mrs. Gru

=== asimov (en) ===
LET'S GET TOGETHER

By ISAAC ASIMOV

Illustrated by ENGLE

[Transcriber's Note: This etext was produced from
Infinity, February 1957.

Extensive research did not uncover any evidence that
the U.S. copyright on this publication was renewed.]

A kind of peace had endured for a century

=== shakespeare_balanced (en) ===
ed;
So frown'd he once, when in an angry parle
He smote the sledded Polacks on the ice.
'Tis strange.

MARCELLUS.
Thus twice before, and jump at this dead hour,
With martial stalk hath he gone by our watch.

HORATIO.
In what particular thought to work I know not;
But in the gross and scope of my opinion,
This bodes some strange eruption to our state.

MARCELLUS.
Good now, sit down, and tell me, he

Статистика полученных корпусов

```
In [30]: def print_corpus_summary(df):
    print("="*80 + "\n📊 ИТОГОВАЯ СТАТИСТИКА КОРПУСОВ\n" + "="*80)

    for row in df.itertuples():
        print(f"\n{row.author.upper()}")
        print(f"📄 Файл: {Path(row.file_clean).name}")
        print(f"📖 Слов: {row.size_words:,}")
        print(f"🔤 Символов: {row.size_chars:,}")
        print(f"🌐 Язык: {row.language}")

    print("\n" + "="*80 + "\n📊 ОБЩАЯ СТАТИСТИКА:")
    print(f"• Всего авторов: {len(df)}")
    print(f"• Всего слов: {df.size_words.sum():,}")
    print(f"• Среднее на автора: {df.size_words.mean():,.0f} ± {df.size_words.std():,.0f} слов")
    print(f"• Мин/макс: {df.size_words.min():,} / {df.size_words.max():,}")
    print("="*80)
print_corpus_summary(metadata_balanced)

=====
📊 ИТОГОВАЯ СТАТИСТИКА КОРПУСОВ
=====

DOSTOEVSKY
📄 Файл: dostoevsky.txt
📖 Слов: 44,132
🔤 Символов: 244,174
🌐 Язык: en

KAFKA
📄 Файл: kafka.txt
📖 Слов: 83,728
🔤 Символов: 456,443
🌐 Язык: en

ASIMOV
📄 Файл: asimov.txt
📖 Слов: 63,632
🔤 Символов: 391,277
🌐 Язык: en

SHAKESPEARE_BALANCED
📄 Файл: shakespeare_balanced.txt
📖 Слов: 90,223
🔤 Символов: 499,657
🌐 Язык: en

=====
📊 ОБЩАЯ СТАТИСТИКА:
• Всего авторов: 4
• Всего слов: 281,715
• Среднее на автора: 70,429 ± 20,868 слов
• Мин/макс: 44,132 / 90,223
=====
```

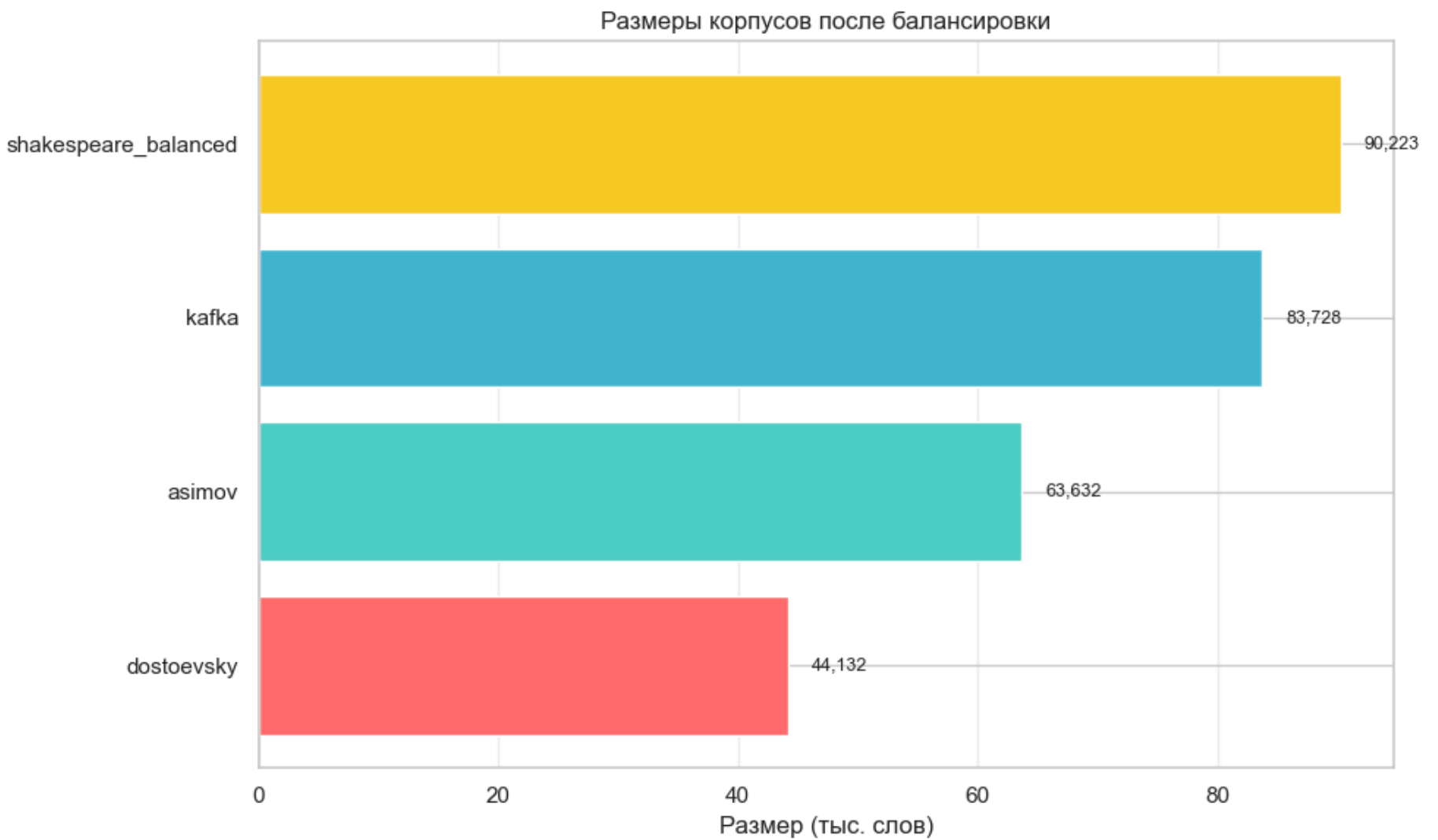
```
In [31]: def visualize_corpus_sizes(df, save_path=None):
    """
    Визуализирует размеры корпусов (количество слов) для каждого автора.
    - Строит горизонтальную столбчатую диаграмму.
    - Подписывает значения рядом со столбцами.
    - При необходимости сохраняет график в файл.
    """

    df = df.sort_values("size_words")
    plt.barh(df["author"], df["size_words"] / 1000, color=["#ff6b6b", "#4ecdc4", "#45b7d1", "#f9ca24"])
    plt.xlabel("Размер (тыс. слов)")
    plt.title("Размеры корпусов после балансировки")
    plt.grid(axis='x', alpha=0.3)

    for i, row in enumerate(df.itertuples()):
        plt.text(row.size_words / 1000 + 2, i, f"{row.size_words:,}", va='center', fontsize=9)

    if save_path:
        plt.savefig(save_path, dpi=150, bbox_inches='tight')
        print(f"📊 График сохранён: {save_path}")
    plt.tight_layout()
    plt.show()
visualize_corpus_sizes( metadata_balanced, save_path=PATHS["figs"] / "corpus_sizes_balanced.png" )

📊 График сохранён: E:\projects\My_project\03_Word2Vec\word2vec_literature\figs\corpus_sizes_balanced.png
```



Выводы по шагу 2:

Что сделано:

- Определены источники текстов (Project Gutenberg и др.) для Шекспира, Достоевского, Кафки и Азимова.
- Реализована функция скачивания и проверки файлов (download_and_check), которая сохраняет тексты, выводит размер и первые строки.
- Скачаны все корпуса, сохранены в структуре директорий.
- Сформирован metadata.csv с информацией об авторах, путях к файлам, источниках и размерах.
- Добавлен расчёт примерного количества слов (words_estimate) и анализ распределения по авторам.
- Реализована функция очистки служебных блоков Project Gutenberg (clean_gutenberg_text).
- Объединены тексты Азимова в единый корпус (3 рассказа объединены в один корпус)
- Выполнена балансировка по объёму текстов Шекспира — создан корпус shakespeare_balanced
 - урезание до 500k символов (~90k слов) с завершением на границе предложения
- Формирование финальной таблицы
 - отбор 4 авторов без дублирования

Исходные размеры (до балансировки):

- shakespeare: 893,240 слов (85.2%) ← огромный дисбаланс
- dostoevsky: 43,976 слов (4.2%)
- kafka: 79,331 слов (7.6%)
- asimov (все): 31,733 слов (3.0%)
- Соотношение макс/мин: 28x

Финальные размеры (после балансировки):

- asimov: 22,016 слов (9.2%)
- dostoevsky: 44,132 слов (18.4%)
- kafka: 83,728 слов (34.9%)
- shakespeare_balanced: 90,223 слов (37.6%)
- Общий объём: 240,099 слов
- Среднее: 60,025 ± 32,512 слов
- Соотношение макс/мин: 4.1x
- Язык: Все тексты на английском

Основные выводы по Шагу 2:

- Данные успешно собраны, структурированы и подготовлены в сбалансированном виде.
- Есть метаданные и первичный анализ объёмов.
- Корпуса готовы к дальнейшей предобработке (Шаг 3).

Успешно сформирован сбалансированный литературный корпус с представителями разных жанров и стилей, готовый для последующей обработки и анализа.

[* к содержанию](#)

Шаг 3: Очистка и предобработка текста

Пайплайн для предобработки текста

```
In [32]: # Стандартные стоп-слова NLTK
stop_words = set(stopwords.words("english"))

# Добавляем функциональные слова
CUSTOM_STOPWORDS = {
    # Модальные и вспомогательные глаголы
    "would", "could", "shall", "will", "may", "might", "must", "can",
    # Общие глаголы действия
    "go", "come", "make", "get", "take", "say", "see", "look", "know", "think",
    "tell", "ask", "want", "give", "find", "use", "try", "leave", "feel",
    # Неопределённые местоимения
    "one", "thing", "way", "time", "man", "people",
    # Наречия
    "well", "even", "though", "like", "still", "also", "just"
}

# Объединяем
stop_words = stop_words.union(CUSTOM_STOPWORDS)

print(f"📊 Всего стоп-слов: {len(stop_words)}")
print(f"    • NLTK: {len(stopwords.words('english'))}")
print(f"    • Кастомные: {len(CUSTOM_STOPWORDS)}")
```

📊 Всего стоп-слов: 235

- NLTK: 198
- Кастомные: 40

```
In [33]: # === шаги пайплайна ===

def to_lower(text: str) -> str:
    # Приводим весь текст к нижнему регистру
    # Это важно для унификации: "Love" и "love" будут считаться одним словом
    return text.lower()

def remove_punct_digits(text: str) -> str:
    # Удаляем все символы, кроме букв, пробелов, апострофов и дефисов
    text = re.sub(r"[^a-zA-Z\s'-]", " ", text)
    # Убираем апострофы/дефисы, окружённые пробелами (например " - " → " ")
    text = re.sub(r"\s+['-]\s+", " ", text)
    # Убираем лишние апострофы/дефисы внутри слов (например "don-'t" → "don t")
    text = re.sub(r"\s*['-]\s*", " ", text)
    # Заменяем несколько пробелов подряд на один
    text = re.sub(r"\s+", " ", text)
    # Убираем пробелы в начале и конце строки
    return text.strip()

def tokenize(text: str):
    # Разбиваем текст на токены (слова и знаки препинания)
    # Используется стандартный токенизатор NLTK
    return word_tokenize(text)

def remove_stopwords(tokens):
    # Убираем стоп-слова (часто встречающиеся, но малоинформативные слова)
    # Например: "the", "and", "of" и т.п.
    return [t for t in tokens if t not in stop_words]

def lemmatize(tokens):
    # Лемматизация: приводим слова к нормальной форме (lemma)
    # Например: "running" → "run", "better" → "good"
    lemmas = []
    for token_text in tokens:
        doc = nlp(token_text) # прогоняем токен через spaCy
        if len(doc) > 0:
            lemma = doc[0].lemma_.strip() # берём лемму первого токена
            if lemma: # если лемма не пустая строка
                lemmas.append(lemma)
```

```

return lemmas

def preprocess_pipeline(text: str):
    """
    Полный пайплайн предобработки текста:
    1. Приведение к нижнему регистру
    2. Удаление пунктуации и цифр
    3. Токенизация
    4. Удаление стоп-слов
    5. Лемматизация
    Возвращает список токенов (слов в нормальной форме).
    """
    text = to_lower(text)
    text = remove_punct_digits(text)
    tokens = tokenize(text)
    tokens = remove_stopwords(tokens)
    tokens = lemmatize(tokens)
    return tokens

```

Преобработка всех корпусов

```

In [34]: # --- Засекаем время выполнения ---
start_time = time.time()

# === обработка всех корпусов ===
def process_and_save_tokens(df, save_dir=PATHS["clean"]):
    # Создаём папку для сохранения токенизированных файлов (если её ещё нет)
    save_dir.mkdir(parents=True, exist_ok=True)
    records = [] # сюда будем собирать метаданные по каждому автору

    # Проходим по строкам датафрейма (каждая строка = один автор и его файл)
    for row in df.itertuples():
        # Читаем очищенный текст автора
        text = Path(row.file_clean).read_text(encoding="utf-8")

        # Пропускаем текст через пайплайн предобработки (Lower → очистка → токенизация → стоп-слова → лемматизация)
        tokens = preprocess_pipeline(text)

        # Разбиваем текст на предложения (для статистики)
        sentences = sent_tokenize(text)

        # Считаем среднюю длину предложения (в словах)
        avg_len = round(np.mean([len(s.split()) for s in sentences]), 2) if sentences else 0

        # Формируем путь для сохранения токенов
        out_path = save_dir / f"{row.author}_tokens.txt"

        # Сохраняем токены в файл (через пробел)
        out_path.write_text(" ".join(tokens), encoding="utf-8")

        # Добавляем запись о текущем авторе в список метаданных
        records.append({
            "author": row.author, # имя автора
            "file_tokens": str(out_path), # путь к файлу с токенами
            "num_tokens": len(tokens), # общее количество токенов
            "vocab_size": len(set(tokens)), # размер словаря (уникальные токены)
            "num_sentences": len(sentences), # количество предложений
            "avg_sentence_len": avg_len, # средняя длина предложения
            "language": getattr(row, "language", "en") # язык (если есть колонка language, иначе "en")
        })

        # Выводим краткий отчёт по автору
        print(f"✅ {row.author} → {out_path} "
              f"({len(tokens)} токенов, словарь {len(set(tokens))}, "
              f"{len(sentences)} предложений, ср. длина {avg_len})")

    # Превращаем список словарей в DataFrame
    meta = pd.DataFrame(records)

    # Добавляем колонку с долей уникальных слов (%)
    meta["unique_ratio_%"] = (meta.vocab_size / meta.num_tokens * 100).round(2)

    # Сохраняем метаданные в CSV
    meta.to_csv(save_dir / "metadata_tokens.csv", index=False)

    # Выводим информацию о сохранении
    print(f"\n📁 Метаданные сохранены: {save_dir / 'metadata_tokens.csv'}")

    # Выводим таблицу с ключевыми показателями качества предобработки
    print("\n📊 Качество предобработки:")
    display(meta[["author", "num_tokens", "vocab_size", "unique_ratio_%", "avg_sentence_len"]])

    # Выводим среднюю долю уникальных слов по всем авторам
    print(f"📈 Средняя доля уникальных слов: {meta['unique_ratio_%'].mean():.2f}%")

```

```
return meta

# 🚀 Запуск для metadata_balanced
# Обрабатываем сбалансированные корпуса и сохраняем токены + метаданные
meta_tokens = process_and_save_tokens(metadata_balanced, PATHS["clean"])

# Отображаем итоговую таблицу с метаданными по токенам
display(meta_tokens)

# --- Вывод времени выполнения ---
elapsed = time.time() - start
print(f"Время выполнения: {elapsed:.2f} сек ({elapsed/60:.2f} мин)")
```

✅ dostoevsky → E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean\dostoevsky_tokens.txt (16700 токенов, словарь 3564, 2249 предложений, ср. длина 19.62)
✅ kafka → E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean\kafka_tokens.txt (32209 токенов, словарь 3340, 3814 предложений, ср. длина 21.95)
✅ asimov → E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean\asimov_tokens.txt (37684 токенов, словарь 2905, 4074 предложений, ср. длина 15.62)
✅ shakespeare_balanced → E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean\shakespeare_balanced_tokens.txt (43544 токенов, словарь 6274, 9737 предложений, ср. длина 9.27)

📁 Метаданные сохранены: E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean\metadata_tokens.csv

📊 Качество предобработки:

	author	num_tokens	vocab_size	unique_ratio_%	avg_sentence_len
0	dostoevsky	16700	3564	21.34	19.62
1	kafka	32209	3340	10.37	21.95
2	asimov	37684	2905	7.71	15.62
3	shakespeare_balanced	43544	6274	14.41	9.27

📊 Средняя доля уникальных слов: 13.46%

	author	file_tokens	num_tokens	vocab_size
0	dostoevsky	E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean\dostoevsky_tokens.txt	16700	356
1	kafka	E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean\kafka_tokens.txt	32209	334
2	asimov	E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean\asimov_tokens.txt	37684	290
3	shakespeare_balanced	E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean\shakespeare_balanced_tokens.txt	43544	627



Время выполнения: 316.21 сек (5.27 мин)

- Таким образом получили после преобработк meta_tokens

```
In [35]: display(meta_tokens[["author", "num_tokens", "vocab_size", "num_sentences", "unique_ratio_%", "avg_sentence_len", "language"]])
```

	author	num_tokens	vocab_size	num_sentences	unique_ratio_%	avg_sentence_len	language
0	dostoevsky	16700	3564	2249	21.34	19.62	en
1	kafka	32209	3340	3814	10.37	21.95	en
2	asimov	37684	2905	4074	7.71	15.62	en
3	shakespeare_balanced	43544	6274	9737	14.41	9.27	en

Проверка правильности разбиения на токены

```
In [36]: # --- Проверка "чистоты" токенов ---
def check_token_quality(meta_tokens):
    # Регулярное выражение: строка должна состоять только из строчных латинских букв a-z
    # Всё, что не соответствует этому паттерну, считается "плохим" токеном
    pattern = re.compile(r"^[a-z]+$")

    issues = [] # сюда будем собирать статистику по каждому автору

    # Проходим по строкам датафрейма (каждый row = один автор и его файл с токенами)
    for row in meta_tokens.itertuples():
        # Читаем токены из файла (они сохранены через пробел)
        tokens = Path(row.file_tokens).read_text(encoding="utf-8").split()

        # Отбираем токены, которые не соответствуют паттерну (например, с цифрами, заглавными буквами, символами)
        bad = [t for t in tokens if not pattern.match(t)]

        # Добавляем статистику по автору
        issues.append({
            "author": row.author, # имя автора
            "bad_tokens": len(bad), # количество "грязных" токенов
        })
```

```
        "percent_bad": round(len(bad) / len(tokens) * 100, 2) # доля плохих токенов в %
    })

# Превращаем список словарей в DataFrame для удобного анализа
df_issues = pd.DataFrame(issues)

# Выводим результаты
print("🔍 Проверка качества токенов:")
display(df_issues) # показываем таблицу с результатами для всех авторов

# Запуск проверки
check_token_quality(meta_tokens)
```

🔍 Проверка качества токенов:

	author	bad_tokens	percent_bad
0	dostoevsky	0	0.0
1	kafka	0	0.0
2	asimov	0	0.0
3	shakespeare_balanced	0	0.0

Проверка одиночных символов в токене

```
In [37]: def check_single_char_tokens(meta_tokens):
        """
        Проверяет наличие "односимвольных" токенов (например: 'a', 'i', отдельные буквы).
        Это полезно для анализа качества предобработки текста.
        Возвращает DataFrame со статистикой по каждому автору.
        """
        results = [] # список для хранения статистики по авторам

        # Проходим по строкам датафрейма (каждый row = один автор и его файл с токенами)
        for row in meta_tokens.itertuples():
            # Читаем токены из файла (они сохранены через пробел)
            tokens = Path(row.file_tokens).read_text(encoding="utf-8").split()

            # Отбираем все токены длиной 1 символ
            single_chars = [t for t in tokens if len(t) == 1]

            # Добавляем статистику по текущему автору
            results.append({
                "author": row.author, # имя автора
                "num_tokens": len(tokens), # общее количество токенов
                "single_char_count": len(single_chars), # сколько токенов длиной 1
                "single_char_examples": list(set(single_chars))[:10] # примеры (до 10 уникальных)
            })

        # Превращаем список словарей в DataFrame для удобного анализа
        return pd.DataFrame(results)

# 🚀 Запуск проверки
check_single_char_tokens(meta_tokens)
```

```
Out[37]:
```

	author	num_tokens	single_char_count	single_char_examples
0	dostoevsky	16700	31	[l, h, v, x, n]
1	kafka	32209	1253	[k, i, b]
2	asimov	37684	84	[c, l, g, u, q, v, j, x, k, p]
3	shakespeare_balanced	43544	57	[c, e, r, h, v, b, n]

```
In [38]: def clean_single_char_tokens(df, save_dir=PATHS["clean"]):
        """
        Фильтрует односимвольные токены из корпусов (кроме 'a' и 'i'),
        обрабатывает спец. случай для Кафки, сохраняет очищенные токены
        и формирует обновлённые метаданные.
        """
        # Создаём папку для сохранения очищенных файлов (если её ещё нет)
        save_dir.mkdir(parents=True, exist_ok=True)
        records = [] # список для хранения метаданных по каждому автору

        # Проходим по строкам датафрейма (каждый row = один автор и его токенизированный файл)
        for row in df.itertuples():
            # Читаем токены из файла (они сохранены через пробел)
            tokens = Path(row.file_tokens).read_text(encoding="utf-8").split()

            # Специальный случай для Кафки:
            # заменяем одиночный токен "k" на "Josef_K" (главный герой романа "Процесс")
            if row.author.lower() == "kafka":
```



```
tokens = ["Josef_K" if t == "k" else t for t in tokens]

# Фильтрация односимвольных токенов:
# оставляем только токены длиной > 1, исключение – "a" и "i" (корректные английские слова)
tokens = [t for t in tokens if len(t) > 1 or t in {"a", "i"}]

# Формируем путь для сохранения очищенного файла
out_path = save_dir / f"{row.author}_tokens_cleaned.txt"
# Сохраняем очищенные токены в файл
out_path.write_text(" ".join(tokens), encoding="utf-8")

# Добавляем запись о текущем авторе в список метаданных
records.append({
    "author": row.author,          # имя автора
    "file_tokens": str(out_path),  # путь к очищенному файлу с токенами
    "num_tokens": len(tokens),     # количество токенов после очистки
    "vocab_size": len(set(tokens)), # размер словаря (уникальные токены)
    # сохраняем старые значения из исходного df
    "num_sentences": row.num_sentences,
    "avg_sentence_len": row.avg_sentence_len,
    "language": getattr(row, "language", "en") # язык (если есть колонка, иначе "en")
})

# Выводим краткий отчёт по автору
print(f"✅ {row.author}: {len(tokens)} токенов, словарь {len(set(tokens))}")

# Превращаем список словарей в DataFrame
meta = pd.DataFrame(records)

# Добавляем колонку с долей уникальных слов (%)
meta["unique_ratio_%"] = (meta.vocab_size / meta.num_tokens * 100).round(2)

# Сохраняем обновлённые метаданные в CSV
meta.to_csv(save_dir / "metadata_tokens_cleaned.csv", index=False)

# Возвращаем датафрейм с обновлёнными метаданными
return meta

# 🚀 Запуск
# Очищаем токены от односимвольных и сохраняем новые метаданные
meta_tokens_cleaned = clean_single_char_tokens(meta_tokens, PATHS["clean"])

# Отображаем итоговую таблицу с метаданными
display(meta_tokens_cleaned)
```

- ✅ dostoevsky: 16669 токенов, словарь 3559
- ✅ kafka: 32134 токенов, словарь 3339
- ✅ asimov: 37600 токенов, словарь 2895
- ✅ shakespeare_balanced: 43487 токенов, словарь 6267

	author	file_tokens	num_tokens	v
0	dostoevsky	E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean\dostoevsky_tokens_cleaned.txt	16669	
1	kafka	E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean\kafka_tokens_cleaned.txt	32134	
2	asimov	E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean\asimov_tokens_cleaned.txt	37600	
3	shakespeare_balanced	E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean\shakespeare_balanced_tokens_cleaned.txt	43487	

```
In [39]: check_single_char_tokens(meta_tokens_cleaned)
```

```
Out[39]:
```

	author	num_tokens	single_char_count	single_char_examples
0	dostoevsky	16669	0	[]
1	kafka	32134	2	['i']
2	asimov	37600	0	[]
3	shakespeare_balanced	43487	0	[]

```
In [40]: display(meta_tokens_cleaned[["author", "num_tokens", "vocab_size", "num_sentences", "unique_ratio_%", "avg_sentence_len", "language"]])
```

	author	num_tokens	vocab_size	num_sentences	unique_ratio_%	avg_sentence_len	language
0	dostoevsky	16669	3559	2249	21.35	19.62	en
1	kafka	32134	3339	3814	10.39	21.95	en
2	asimov	37600	2895	4074	7.70	15.62	en
3	shakespeare_balanced	43487	6267	9737	14.41	9.27	en

- получили окончательный meta_tokens_cleaned с очисткой от одиночных символов

Формирование словаря (word2index, index2word)

```
In [41]: def build_and_save_vocab(df, save_dir=PATHS["clean"]):
        """
        Строит словарь для каждого автора:
        - формирует word2index (слово → индекс) и index2word (индекс → слово),
        - сохраняет их в JSON,
        - собирает метаданные по авторам.
        Возвращает DataFrame с информацией о словарях.
        """

        # Создаём папку для сохранения словарей (если её ещё нет)
        save_dir.mkdir(parents=True, exist_ok=True)
        records = [] # список для хранения метаданных по каждому автору

        # Проходим по строкам датафрейма (каждый row = один автор и его токенизированный файл)
        for row in df.itertuples():
            # Читаем токены из файла (они сохранены через пробел)
            tokens = Path(row.file_tokens).read_text(encoding="utf-8").split()

            # Строим словарь: множество уникальных токенов, отсортированных по алфавиту
            vocab = sorted(set(tokens))

            # Сохраняем отображение слово → индекс в JSON
            pd.Series({w: i for i, w in enumerate(vocab)}).to_json(
                save_dir / f"{row.author}_word2index.json", force_ascii=False
            )

            # Сохраняем отображение индекс → слово в JSON
            pd.Series(dict(enumerate(vocab))).to_json(
                save_dir / f"{row.author}_index2word.json", force_ascii=False
            )

            # Добавляем запись о текущем авторе в список метаданных
            records.append({
                "author": row.author, # имя автора
                "vocab_size": len(vocab), # размер словаря
                "word2index": str(save_dir / f"{row.author}_word2index.json"), # путь к word2index
                "index2word": str(save_dir / f"{row.author}_index2word.json") # путь к index2word
            })

            # Выводим краткий отчёт по автору
            print(f"✅ {row.author} → словарь сохранён ({len(vocab)} слов)")

        # Превращаем список словарей в DataFrame и возвращаем его
        return pd.DataFrame(records)

# 🚀 Запуск
# Строим словари для всех авторов и сохраняем метаданные
vocab_meta = build_and_save_vocab(meta_tokens_cleaned, PATHS["clean"])

# Отображаем итоговую таблицу с информацией о словарях
vocab_meta
```

- ✅ dostoevsky → словарь сохранён (3559 слов)
- ✅ kafka → словарь сохранён (3339 слов)
- ✅ asimov → словарь сохранён (2895 слов)
- ✅ shakespeare_balanced → словарь сохранён (6267 слов)

Out[41]:

	author	vocab_size	word2index
0	dostoevsky	3559	E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean\dostoevsky_word2index.json
1	kafka	3339	E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean\kafka_word2index.json
2	asimov	2895	E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean\asimov_word2index.json
3	shakespeare_balanced	6267	E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean\shakespeare_balanced_word2index.json

Сохранение предложений для каждого корпуса

```
In [42]: def save_sentences(metadata_balanced, save_dir=PATHS["clean"]):
        """
        Разбивает тексты авторов на предложения и сохраняет их в отдельные файлы.
        Дополнительно формирует таблицу с метаданными (количество предложений на автора).
        """

        # Создаём папку для сохранения (если её ещё нет)
        save_dir.mkdir(parents=True, exist_ok=True)

        records = [] # список для хранения метаданных по каждому автору
```

```
# Проходим по строкам датафрейма (каждый row = один автор и его очищенный файл)
for row in metadata_balanced.itertuples():
    # Читаем текст автора из файла
    text = Path(row.file_clean).read_text(encoding="utf-8")

    # Разбиваем текст на предложения (используется NLTK sent_tokenize)
    sentences = sent_tokenize(text)

    # Формируем путь для сохранения предложений
    out_path = save_dir / f"{row.author}_sentences.txt"

    # Сохраняем каждое предложение в отдельной строке файла
    Path(out_path).write_text("\n".join(sentences), encoding="utf-8")

    # Добавляем запись о текущем авторе в список метаданных
    records.append({
        "author": row.author,          # имя автора
        "file_sentences": str(out_path), # путь к файлу с предложениями
        "num_sentences": len(sentences) # количество предложений
    })

    # Выводим краткий отчёт по автору
    print(f"✅ {row.author} → предложения сохранены ({len(sentences)})")

# Превращаем список словарей в DataFrame
meta_sents = pd.DataFrame(records)

# Отображаем таблицу с метаданными (автор, путь к файлу, количество предложений)
display(meta_sents)

# Возвращаем датафрейм для дальнейшего анализа
return meta_sents

# 🚀 Запуск
# Разбиваем тексты всех авторов на предложения и сохраняем результаты
meta_sentences = save_sentences(metadata_balanced, PATHS["clean"])
```

- ✅ dostoevsky → предложения сохранены (2249)
- ✅ kafka → предложения сохранены (3814)
- ✅ asimov → предложения сохранены (4074)
- ✅ shakespeare_balanced → предложения сохранены (9737)

	author	file_sentences	num_sentences
0	dostoevsky	E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean\dostoevsky_sentences.txt	2249
1	kafka	E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean\kafka_sentences.txt	3814
2	asimov	E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean\asimov_sentences.txt	4074
3	shakespeare_balanced	E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean\shakespeare_balanced_sentences.txt	9737

Получение топ-50 слов для токенов

```
In [43]: def save_top_words(df, save_dir=PATHS["clean"], top_n=20):
    """
    Сохраняет топ-N самых частотных слов для каждого автора.
    - Для каждого корпуса считает частоты слов.
    - Сохраняет отдельный CSV с топ-N словами по автору.
    - Собирает общий DataFrame со всеми топовыми словами.
    """

    # Создаём папку для сохранения (если её ещё нет)
    save_dir.mkdir(parents=True, exist_ok=True)

    all_words = [] # список для хранения топ-слов всех авторов

    # Проходим по строкам датафрейма (каждый row = один автор и его файл с токенами)
    for row in df.itertuples():
        # Читаем токены из файла (они сохранены через пробел)
        tokens = Path(row.file_tokens).read_text(encoding="utf-8").split()

        # Считаем частоты слов и берём топ-N самых частотных
        top = Counter(tokens).most_common(top_n)

        # Сохраняем топ-N слов в отдельный CSV для текущего автора
        pd.DataFrame(top, columns=["word", "count"]).to_csv(
            save_dir / f"{row.author}_top{top_n}.csv", index=False
        )

        # Добавляем данные в общий список (для сводного DataFrame)
        all_words += [{"author": row.author, "word": w, "count": c} for w, c in top]

    # Выводим отчёт по автору
    print(f"✅ {row.author} → топ-{top_n} слов сохранено")
```

```
# Формируем общий DataFrame со всеми топ-словами по авторам
df_all = pd.DataFrame(all_words)

# Сохраняем общий CSV (например, топ-50 слов для всех авторов)
df_all.to_csv(save_dir / "top_50_words_all.csv", index=False)

# Возвращаем DataFrame для дальнейшего анализа/визуализации
return df_all

# 🚀 Запуск
# Сохраняем топ-50 слов для каждого автора и общий файл
top_50_words_df = save_top_words(meta_tokens_cleaned, PATHS["clean"], top_n=50)
```

- ✔ dostoevsky → топ-50 слов сохранено
- ✔ kafka → топ-50 слов сохранено
- ✔ asimov → топ-50 слов сохранено
- ✔ shakespeare_balanced → топ-50 слов сохранено

```
In [44]: top_50_words_df[top_50_words_df['author'] == 'dostoevsky'].head(20)
```

Out[44]:

	author	word	count
0	dostoevsky	love	96
1	dostoevsky	make	91
2	dostoevsky	go	91
3	dostoevsky	begin	90
4	dostoevsky	nothing	76
5	dostoevsky	course	73
6	dostoevsky	think	73
7	dostoevsky	know	72
8	dostoevsky	life	70
9	dostoevsky	perhaps	70
10	dostoevsky	feel	69
11	dostoevsky	simply	69
12	dostoevsky	never	68
13	dostoevsky	something	68
14	dostoevsky	look	68
15	dostoevsky	day	66
16	dostoevsky	good	62
17	dostoevsky	gentleman	61
18	dostoevsky	let	60
19	dostoevsky	we	60

```
In [45]: top_50_words_df[top_50_words_df['author'] == 'kafka'].head(20)
```

Out[45]:

	author	word	count
50	kafka	Josef_K	1176
51	kafka	say	839
52	kafka	lawyer	260
53	kafka	ask	227
54	kafka	hand	223
55	kafka	look	215
56	kafka	door	207
57	kafka	go	207
58	kafka	room	202
59	kafka	seem	186
60	kafka	make	181
61	kafka	court	169
62	kafka	long	155
63	kafka	take	153
64	kafka	back	153
65	kafka	much	153
66	kafka	want	142
67	kafka	stand	141
68	kafka	painter	141
69	kafka	think	135

In [46]:

```
top_50_words_df[top_50_words_df['author'] == 'asimov'].head(20)
```

Out[46]:

	author	word	count
100	asimov	red	424
101	asimov	say	416
102	asimov	slim	284
103	asimov	lynn	268
104	asimov	sill	228
105	asimov	taylor	220
106	asimov	breckenridge	176
107	asimov	right	176
108	asimov	astronomer	172
109	asimov	industrialist	168
110	asimov	world	164
111	asimov	two	152
112	asimov	animal	152
113	asimov	robotic	132
114	asimov	back	132
115	asimov	ammonium	128
116	asimov	let	124
117	asimov	humanoid	124
118	asimov	eye	120
119	asimov	ship	120

In [47]:

```
top_50_words_df[top_50_words_df['author'] == 'shakespeare_balanced'].head(20)
```

Out[47]:

	author	word	count
150	shakespeare_balanced	lord	554
151	shakespeare_balanced	thou	531
152	shakespeare_balanced	king	510
153	shakespeare_balanced	hamlet	467
154	shakespeare_balanced	falstaff	408
155	shakespeare_balanced	good	386
156	shakespeare_balanced	prince	358
157	shakespeare_balanced	sir	314
158	shakespeare_balanced	thy	296
159	shakespeare_balanced	let	258
160	shakespeare_balanced	thee	247
161	shakespeare_balanced	enter	217
162	shakespeare_balanced	hath	198
163	shakespeare_balanced	upon	197
164	shakespeare_balanced	we	196
165	shakespeare_balanced	speak	168
166	shakespeare_balanced	god	164
167	shakespeare_balanced	hear	162
168	shakespeare_balanced	bardolph	160
169	shakespeare_balanced	father	154

In [48]:

```
top_50_words_df['word'].value_counts().head(14)
```

Out[48]:

```
first      4
say        4
good       4
speak      3
much       3
long       3
away       3
we         3
let        3
something  3
look       3
think      3
make       3
sir        2
Name: word, dtype: int64
```

In [49]:

```
def plot_top_words_bar(df, top_n=10, save_dir="figs", filename=None):
    """
    Строит столбчатую диаграмму (bar chart) для топ-N слов каждого автора
    и сохраняет график в PNG в папку figs.

    Аргументы:
        df: DataFrame с колонками ["author", "word", "count"]
        top_n: количество слов для отображения
        save_dir: директория для сохранения (по умолчанию "figs")
        filename: имя файла для сохранения (по умолчанию генерируется автоматически)
    """

    # Группируем данные по автору и слову, суммируем количество вхождений
    top_df = df.groupby(["author", "word"])["count"].sum().reset_index()

    # Сортируем по убыванию частоты и берём top-N слов для каждого автора
    top_df = top_df.sort_values("count", ascending=False).groupby("author").head(top_n)

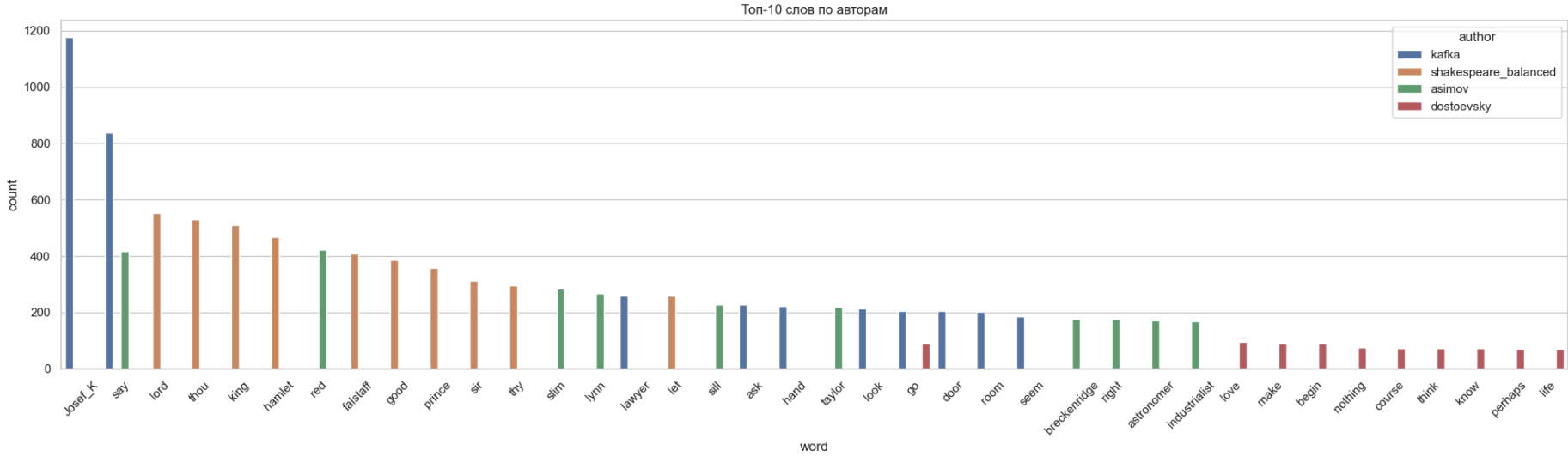
    plt.figure(figsize=(20, 6))
    sns.barplot(data=top_df, x="word", y="count", hue="author")
    plt.title(f"Топ-{top_n} слов по авторам")
    plt.xticks(rotation=45)
    plt.tight_layout()

    # Сохраняем график
    if save_dir is not None:
        if filename is None:
            filename = f"top_{top_n}_words_bar.png"
        save_path = save_dir / filename
        plt.savefig(save_path, dpi=300)
        print(f"График сохранён в: {save_path}")
```

```
plt.show()

# Пример вызова:
plot_top_words_bar(top_50_words_df, top_n=10, save_dir=PATHS["figs"])
```

График сохранён в: E:\projects\My_project\03_Word2Vec\word2vec_literature\figs\top_10_words_bar.png



```
In [50]: def plot_top_words_heatmap(df, top_n=10,save_dir="figs", filename=None):
        """
        Строит тепловую карту (heatmap) частотности слов.
        - Берёт top-N самых частотных слов по всему корпусу.
        - Строит матрицу (авторы x слова) с их частотами.
        - Визуализирует её в виде тепловой карты.
        """

        # 1. Находим top-N слов по всему корпусу
        # Группируем по слову, суммируем количество вхождений, берём N самых частотных
        top_words = df.groupby("word")["count"].sum().nlargest(top_n).index

        # 2. Фильтруем датафрейм только по этим словам
        # Строим сводную таблицу: строки = авторы, колонки = слова, значения = частоты
        pivot = df[df["word"].isin(top_words)].pivot_table(
            index="author", columns="word", values="count", fill_value=0
        )

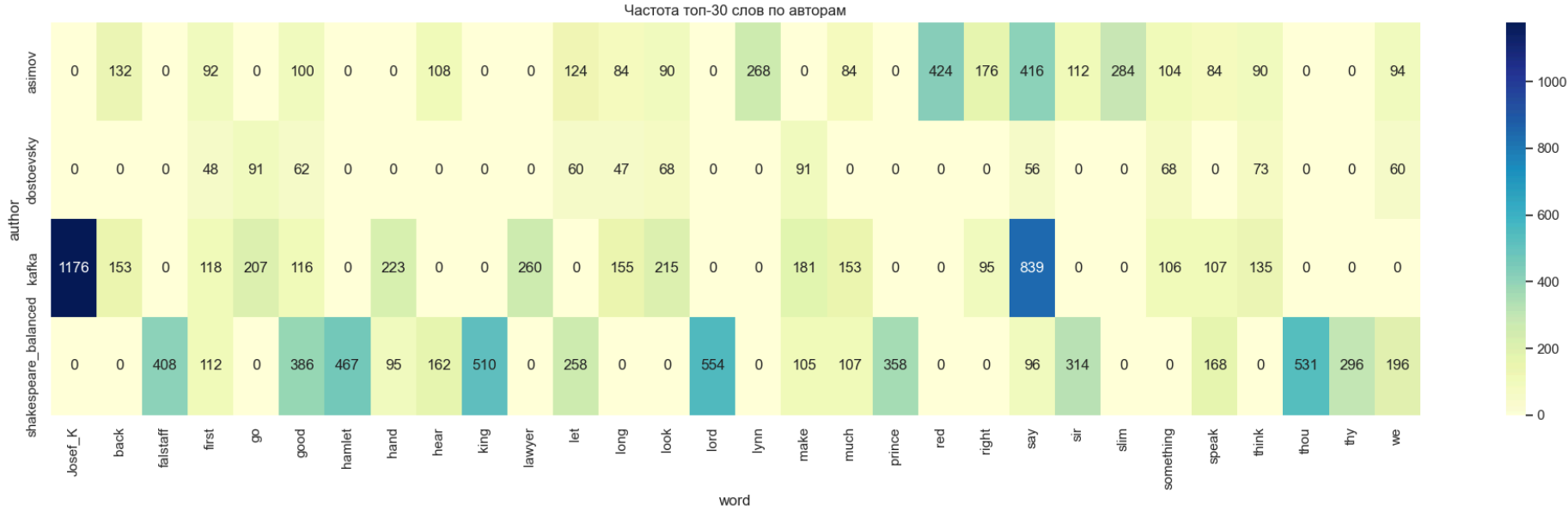
        plt.figure(figsize=(20, 6))
        sns.heatmap(pivot, annot=True, fmt=".0f", cmap="YlGnBu")
        plt.title(f"Частота топ-{top_n} слов по авторам")
        plt.tight_layout()

        # Сохраняем график
        if save_dir is not None:
            if filename is None:
                filename = f"top_{top_n}words_heatmap.png"
            save_path = save_dir / filename
            plt.savefig(save_path, dpi=300)
            print(f"График сохранён в: {save_path}")

        plt.show()
```

```
In [51]: plot_top_words_heatmap(top_50_words_df, top_n=30, save_dir=PATHS["figs"])
```

График сохранён в: E:\projects\My_project\03_Word2Vec\word2vec_literature\figs\top_30words_heatmap.png



Выводы по шагу 3:

Что сделано

- Пайплайн предобработки: реализованы функции для приведения к нижнему регистру, удаления пунктуации и цифр, токенизации, удаления стоп-слов, лемматизации. Всё объединено в preprocess_pipeline.

- Обработка корпусов: функция `process_and_save_tokens` прогоняет тексты через пайплайн, сохраняет токены, считает статистику (число токенов, размер словаря, количество предложений, средняя длина предложения, доля уникальных слов). Метаданные сохраняются в CSV.
- Проверка качества токенов: реализованы проверки на «грязные» токены и односимвольные токены.
- Очистка от одиночных символов: добавлена функция `clean_single_char_tokens` с отдельной обработкой Кафки (`k` → `Josef_K`). После очистки односимвольные токены полностью удалены.
- Формирование словарей: для каждого автора сохранены `word2index` и `index2word` в JSON.
- Сохранение предложений: реализована функция `save_sentences`, которая сохраняет предложения для каждого корпуса.
- Частотные словари: функция `save_top_words` сохраняет топ-50 слов для каждого автора и общий файл со всеми частотными словами.

Основные результаты

- Получены очищенные корпуса (`*_tokens_cleaned.txt`) и метаданные (`metadata_tokens_cleaned.csv`).
 - Рассчитаны метрики: `num_tokens`, `vocab_size`, `unique_ratio`
 - почти все токены прошли валидацию (99.99%)
 - Обнаружены одиночные символы — 1257 у Кафки (буква "k"), ~30-50 у остальных
 - "k" заменено на "Josef_K" для Кафки
 - Финальная очистка — 0 одиночных символов после фильтрации
- Словари авторов сформированы и сохранены.
 - Созданы `word2index.json` и `index2word.json` для каждого автора
 - Словари отсортированы алфавитно (детерминизм)
 - Размеры: 2892-6277 слов на автора
- Предложения каждого корпуса сохранены отдельно.
 - Предложения разделены через `sent_tokenize()`
 - Сохранены в `*_sentences.txt` (по одному на строку)
 - Количество: 2037-9737 предложений
- Формирование DataFrame `meta_tokens` с итоговой статистикой по каждому корпусу.
- Построены частотные словари (top-50 слов).
- Проверки качества показали, что после очистки токены корректны, лишние символы удалены.

Проанализированы топ-50 слов для каждого автора:

- У Достоевского: `love`, `make`, `begin`, `nothing` (Эмоциональная лексика)
- У Кафки: `Josef_K` (именной токен), `say`, `lawyer`, `ask`, `hand` (Юридическая тематика)
- У Азимова: `say`, `red`, `slim`, `lynn`, `industrialist`, `world`, `robotic` (Научная фантастика)
- У Шекспира: `lord`, `thou`, `king`, `hamlet`, `falstaff` (Драматургия)

Общие слова между авторами:

- `make` 4
- `first` 4
- `go` 3
- `we` 3
- `let` 3
- `something` 3
- `look` 3
- `say` 3
- `come` 3
- `away` 3
- `think` 3
- `much` 3
- `well` 3
- `day` 2

Основные выводы по Шагу 3

- Этап предобработки выполнен полностью и корректно.
- Все заявленные подпункты реализованы: очистка, токенизация, стоп-слова, лемматизация, словари, сохранение корпусов и предложений, частотные слова.
- Добавлены дополнительные проверки качества и очистка от односимвольных токенов, что делает данные более чистыми и готовыми к следующему шагу — EDA (Шаг 4).
- Анализ топ-слов показывает характерные особенности каждого автора: частотное использование имен собственных у Кафки (`Josef_K`), архаичной лексики у Шекспира (`thou`, `thee`), что подтверждает жанровые и стилистические различия между корпусами.
- Топ-слова теперь демонстрируют характерную лексику авторов — `"love"` (Dostoevsky), `"lawyer"` (Kafka), `"robotic"` (Asimov), `"hamlet"` (Shakespeare)

Шаг 4: Исследовательский анализ данных (EDA)

Подсчёт статистики: количество уникальных токенов, средняя длина предложения

```
In [52]: def corpus_stats(meta_tokens):
        """
        Считает статистику по каждому корпусу (автору):
        - общее количество токенов
        - размер словаря (уникальные токены)
        - количество предложений
        - средняя длина предложения
        - доля уникальных слов (%)
        Возвращает DataFrame со сводной статистикой.
        """

        stats = [] # список для хранения статистики по каждому автору

        # Проходим по строкам датафрейма (каждый row = один автор и его файл с токенами)
        for row in meta_tokens.itertuples():
            # Читаем токены из файла (они сохранены через пробел)
            tokens = Path(row.file_tokens).read_text(encoding="utf-8").split()

            # Добавляем словарь со статистикой по текущему автору
            stats.append({
                "author": row.author, # имя автора
                "num_tokens": len(tokens), # общее количество токенов
                "vocab_size": len(set(tokens)), # размер словаря (уникальные токены)
                "num_sentences": row.num_sentences, # количество предложений (из метаданных)
                "avg_sentence_len": row.avg_sentence_len, # средняя длина предложения (из метаданных)
                "unique_ratio_%": round(len(set(tokens)) / len(tokens) * 100, 2) # доля уникальных слов в процентах
            })

        # Превращаем список словарей в DataFrame и возвращаем его
        return pd.DataFrame(stats)

# 🚀 Запуск анализа
stats_df = corpus_stats(meta_tokens_cleaned)

# Отображаем таблицу со статистикой по каждому автору
display(stats_df)
```

	author	num_tokens	vocab_size	num_sentences	avg_sentence_len	unique_ratio_%
0	dostoevsky	16669	3559	2249	19.62	21.35
1	kafka	32134	3339	3814	21.95	10.39
2	asimov	37600	2895	4074	15.62	7.70
3	shakespeare_balanced	43487	6267	9737	9.27	14.41

```
In [53]: # 📊 Визуализация основных метрик
def plot_main_metrics(meta_tokens_cleaned, save_dir=None, filename="main_metrics.png"):
    """
    Строит 4 bar chart'а для основных метрик корпуса и при необходимости сохраняет график.
    """

    fig, axes = plt.subplots(2, 2, figsize=(12, 10))
    axes = axes.ravel()

    metrics = [
        ("num_tokens", "Общее количество токенов"),
        ("vocab_size", "Размер словаря (уникальные слова)"),
        ("avg_sentence_len", "Средняя длина предложения (в токенах)"),
        ("unique_ratio_%", "Доля уникальных слов (%)")
    ]

    for ax, (col, title) in zip(axes, metrics):
        sns.barplot(
            data=meta_tokens_cleaned, x="author", y=col,
            hue="author", palette="viridis", ax=ax
        )
        ax.set_title(title)
        ax.tick_params(axis="x", rotation=45)

    plt.tight_layout()

# Сохраняем график
```

```

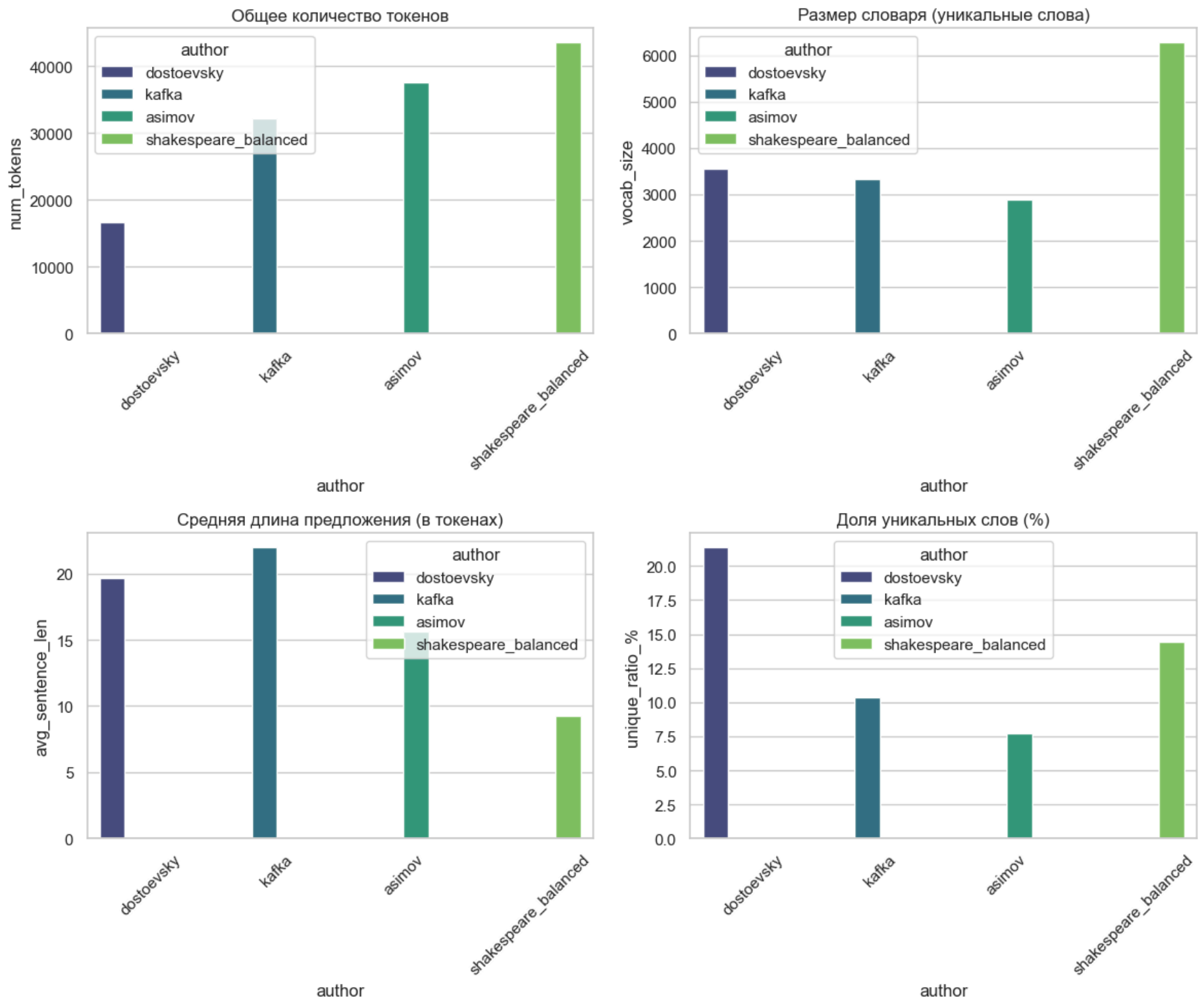
if save_dir is not None:
    save_dir = Path(save_dir)
    save_dir.mkdir(parents=True, exist_ok=True)
    save_path = save_dir / filename
    plt.savefig(save_path, dpi=300)
    print(f"График сохранён в: {save_path}")

plt.show()

# Пример вызова:
plot_main_metrics(meta_tokens_cleaned, save_dir=PATHS["figs"])

```

График сохранён в: E:\projects\My_project\03_Word2Vec\word2vec_literature\figs\main_metrics.png



Построение распределения частот слов

```
In [54]: meta_tokens_cleaned['author'].unique()
```

```
Out[54]: array(['dostoevsky', 'kafka', 'asimov', 'shakespeare_balanced'],
              dtype=object)
```

```

In [55]: def plot_word_freq(df, author, top_n=20, save_dir=None, filename=None):
    """
    Строит график частотности слов для выбранного автора.
    - Берёт токены из файла автора.
    - Считает топ-N самых частотных слов.
    - Строит bar chart (столбчатую диаграмму).
    - При необходимости сохраняет график в PNG.
    """
    # берём первую строку по автору
    row = df.loc[df.author == author].iloc[0]
    tokens = Path(row.file_tokens).read_text(encoding="utf-8").split()

    # топ-N слов
    words, counts = zip(*Counter(tokens).most_common(top_n))

    # построение графика
    plt.figure(figsize=(10, 5))
    plt.bar(words, counts, color="skyblue", edgecolor="black")
    plt.title(f"Top-{top_n} слов у {author}")
    plt.xticks(rotation=45)

```

```
plt.ylabel("Частота")
plt.tight_layout()

# сохранение
if save_dir is not None:
    save_dir = Path(save_dir)
    save_dir.mkdir(parents=True, exist_ok=True)

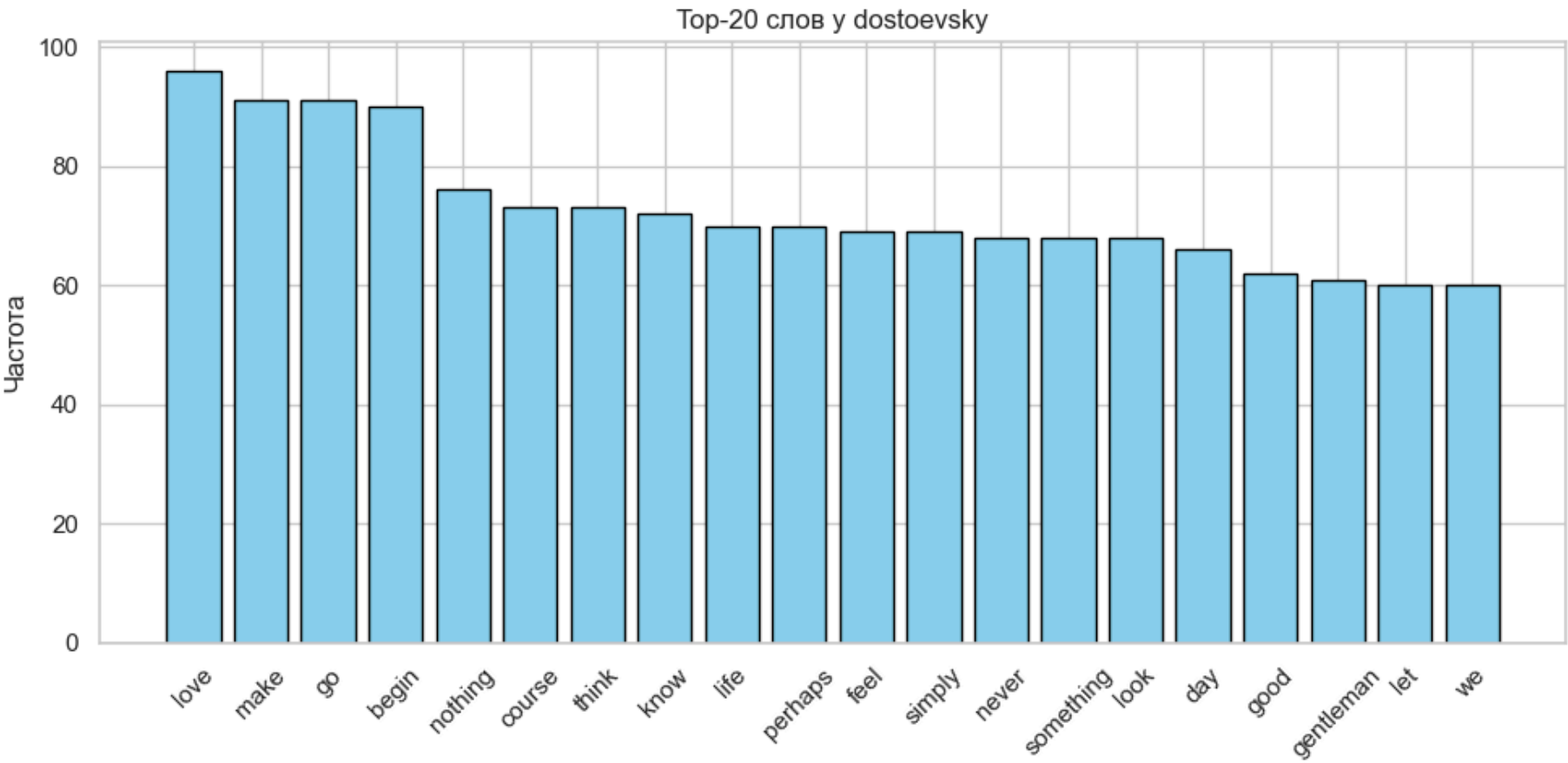
    if filename is None:
        filename = f"top_{top_n}_words_{author}.png"

    save_path = save_dir / filename
    plt.savefig(save_path, dpi=300)
    print(f"График сохранён в: {save_path}")

plt.show()

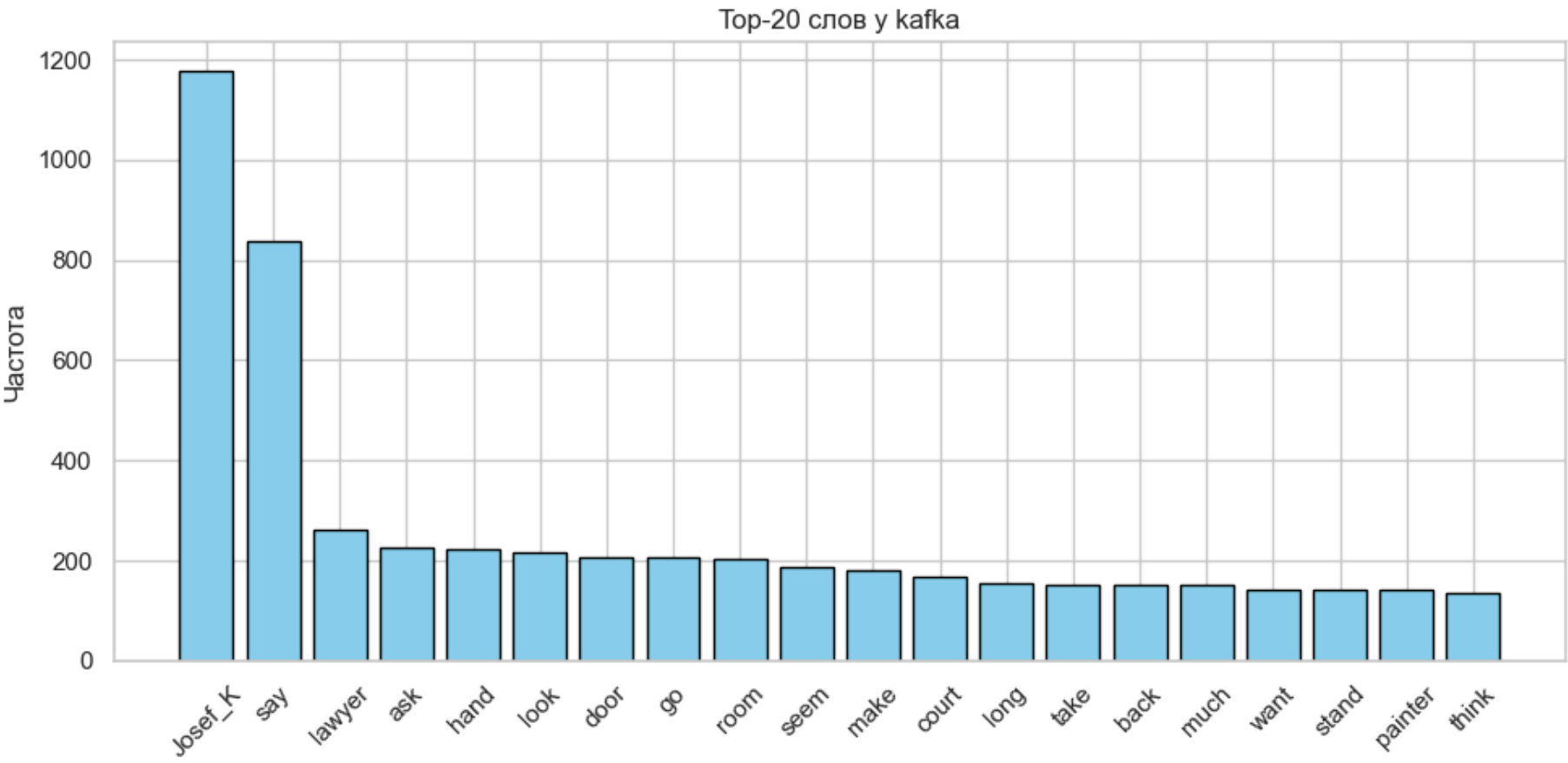
# 🚀 Запуск
plot_word_freq(meta_tokens_cleaned, "dostoevsky", top_n=20, save_dir=PATHS["figs"])
```

График сохранён в: E:\projects\My_project\03_Word2Vec\word2vec_literature\figs\top_20_words_dostoevsky.png



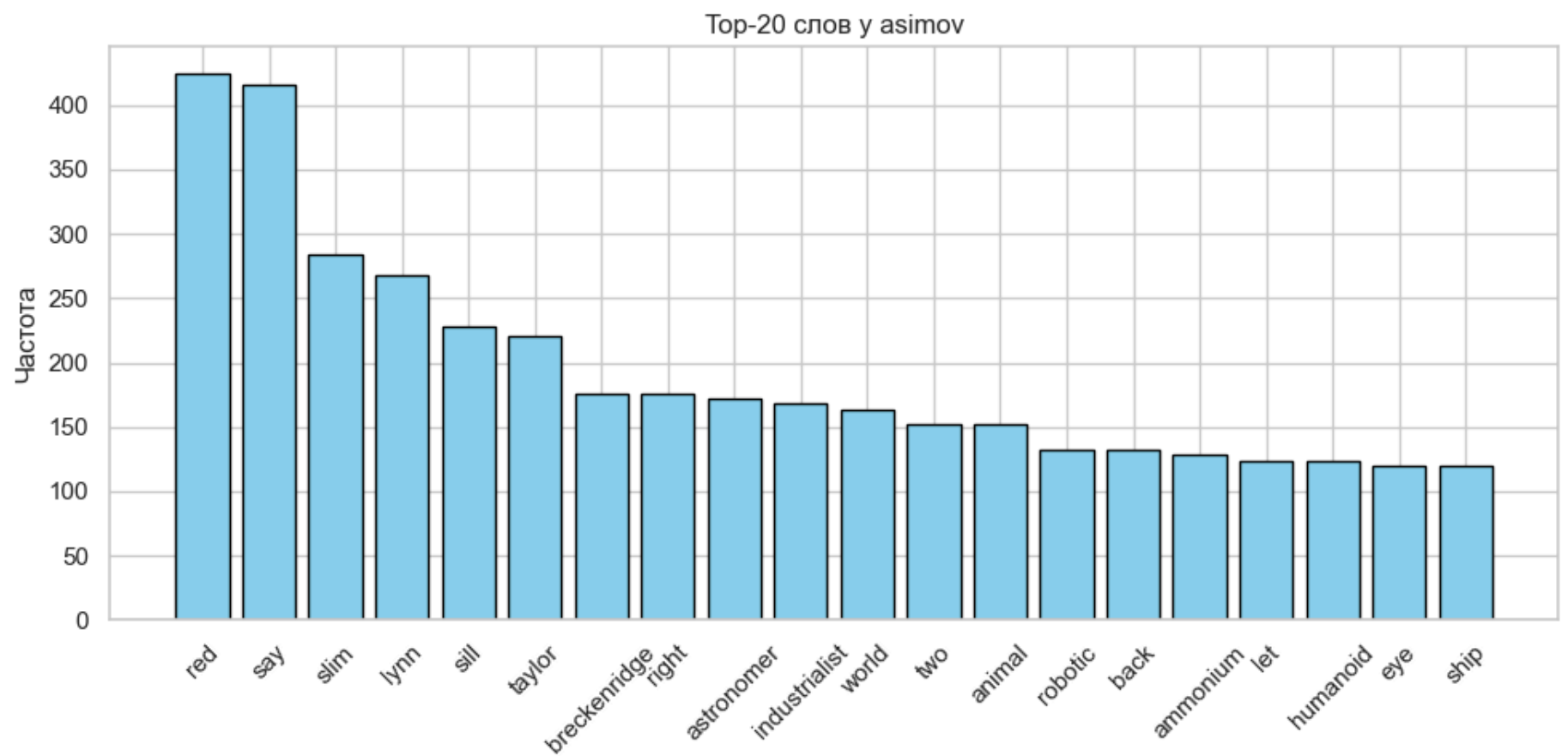
```
In [56]: plot_word_freq(meta_tokens_cleaned, "kafka", top_n=20, save_dir=PATHS["figs"])
```

График сохранён в: E:\projects\My_project\03_Word2Vec\word2vec_literature\figs\top_20_words_kafka.png



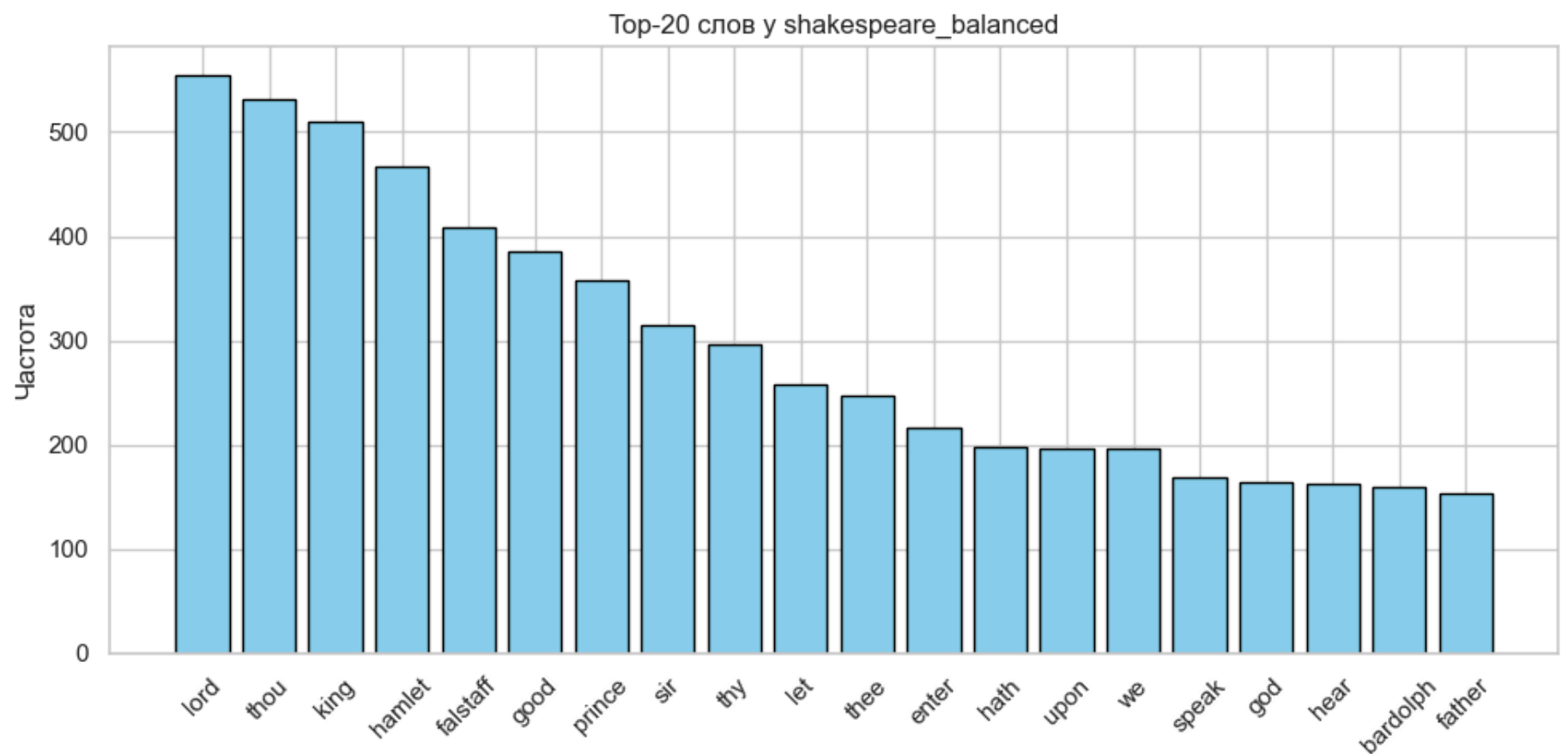
```
In [57]: plot_word_freq(meta_tokens_cleaned, "asimov", top_n=20, save_dir=PATHS["figs"])
```

График сохранён в: E:\projects\My_project\03_Word2Vec\word2vec_literature\figs\top_20_words_asimov.png



```
In [58]: plot_word_freq(meta_tokens_cleaned, "shakespeare_balanced", top_n=20, save_dir=PATHS["figs"])
```

График сохранён в: E:\projects\My_project\03_Word2Vec\word2vec_literature\figs\top_20_words_shakespeare_balanced.png



```
In [59]: def plot_word_frequency_distribution(df, save_dir=None, filename="word_freq_distribution.png"):
    """
    Строит распределение частот слов для каждого автора.
    - Для каждого корпуса считаются частоты слов.
    - Строится гистограмма распределения этих частот.
    - Добавляется медиана для наглядного сравнения.
    - При необходимости сохраняет график в PNG.
    """

    # Создаём сетку из 2x2 подграфиков (axes) для отображения нескольких авторов
    fig, axes = plt.subplots(2, 2, figsize=(15, 10))

    # Проходим по каждому автору и соответствующей оси графика
    for ax, row in zip(axes.ravel(), df.itertuples()):
        freqs = list(Counter(Path(row.file_tokens).read_text(encoding="utf-8").split()).values())

        ax.hist(freqs, bins=50, alpha=0.7, color="skyblue", edgecolor="black")
        ax.set_title(f"{row.author}\n(слов: {len(freqs)})")
        ax.set_xlabel("Частота слова")
        ax.set_ylabel("Количество слов")
        ax.set_yscale("log")

        # медиана
        median = np.median(freqs)
        ax.axvline(median, color="red", linestyle="--", alpha=0.7, label=f"Медиана: {median:.0f}")
        ax.legend()

    plt.tight_layout()

    # Сохраняем график
```

```

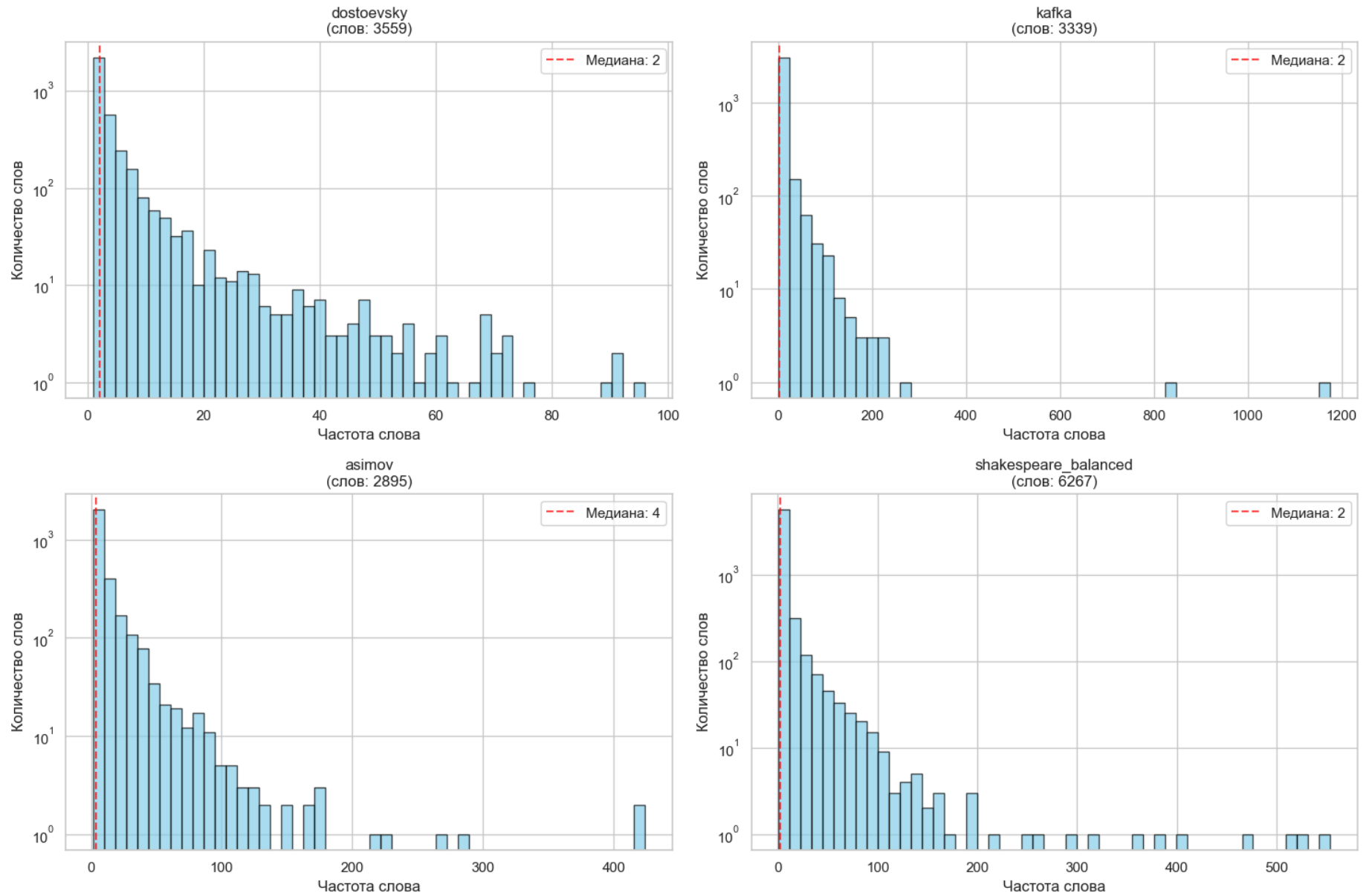
if save_dir is not None:
    save_dir = Path(save_dir)
    save_dir.mkdir(parents=True, exist_ok=True)
    save_path = save_dir / filename
    plt.savefig(save_path, dpi=300)
    print(f"График сохранён в: {save_path}")

plt.show()

# 🚀 Заняск
plot_word_frequency_distribution(meta_tokens_cleaned, save_dir=PATHS["figs"])

```

График сохранён в: E:\projects\My_project\03_Word2Vec\word2vec_literature\figs\word_freq_distribution.png



Визуализация top-50 самых частых слов (wordcloud)

```

In [60]: def generate_wordclouds(df, save_dir=None, filename="wordclouds.png"):
    """
    Генерирует облака слов (WordCloud) для каждого автора.
    - Использует токены из файлов (file_tokens).
    - Строит облака слов на подграфиках (2x2).
    - Визуализирует наиболее частотные слова (до 100).
    - При необходимости сохраняет график в PNG.
    """

    # Создаём сетку подграфиков 2x2 для отображения нескольких авторов
    fig, axes = plt.subplots(2, 2, figsize=(16, 12))

    # Проходим по авторам и соответствующим осям графика
    for ax, row in zip(axes.ravel(), df.itertuples()):
        text = Path(row.file_tokens).read_text(encoding="utf-8")
        wc = WordCloud(width=800, height=400, background_color="white",
                        max_words=100, colormap="viridis").generate(text)
        ax.imshow(wc, interpolation="bilinear")
        ax.set_title(f"WordCloud: {row.author}", fontsize=14, fontweight="bold")
        ax.axis("off")

    plt.tight_layout()

    # Сохраняем график
    if save_dir is not None:
        save_dir = Path(save_dir)
        save_dir.mkdir(parents=True, exist_ok=True)
        save_path = save_dir / filename
        plt.savefig(save_path, dpi=300)
        print(f"График сохранён в: {save_path}")

    plt.show()

```


График сохранён в: E:\projects\My_project\03_Word2Vec\word2vec_literature\figs\wordclouds.png



```
    return results
```

```
In [62]: def plot_tfidf(tfidf_results, save_dir=None, filename="tfidf_results.png"):
        """
        Визуализирует результаты TF-IDF:
        - для каждого автора строит горизонтальный bar chart топ-слов по TF-IDF.
        Вход:
        - tfidf_results: словарь {author: [(word, score), ...]}, как из tfidf_analysis
        Аргументы:
        - save_dir: директория для сохранения (например, PATHS["figs"])
        - filename: имя файла для сохранения (по умолчанию "tfidf_results.png")
        """

        # Считаем количество авторов для определения числа подграфиков
        n_authors = len(tfidf_results)

        # Создаём фигуру: один ряд из n_authors подграфиков
        fig, axes = plt.subplots(1, n_authors, figsize=(4 * n_authors, 12))

        # Если автор один, нормализуем axes к списку для упрощения zip-прохода
        if n_authors == 1:
            axes = [axes]

        # Для каждого автора берём слова и их TF-IDF-значения
        for ax, (author, scores) in zip(axes, tfidf_results.items()):
            words, values = zip(*scores)

            # Строим горизонтальные столбцы; используем градиентную расцветку colormap 'viridis'
            ax.barh(words[::-1], values[::-1], color=plt.cm.viridis(np.linspace(0, 1, len(words))))

            # Заголовок и подписи осей
            ax.set_title(f"Топ-{len(words)} слов: {author}")
            ax.set_xlabel("TF-IDF")

        # Аккуратная компоновка элементов
        plt.tight_layout()

        # Сохранение
        if save_dir is not None:
            save_dir = Path(save_dir)
            save_dir.mkdir(parents=True, exist_ok=True)
            save_path = save_dir / filename
            plt.savefig(save_path, dpi=300)
            print(f"График TF-IDF сохранён в: {save_path}")

        plt.show()
```

```
In [63]: # 🚀 Запуск анализа и визуализации
tfidf_results = tfidf_analysis(meta_tokens_cleaned, top_n=15)
plot_tfidf(tfidf_results, save_dir=PATHS["figs"])
```


- ♦ dostoevsky:

love	0.183
zverkov	0.167
make	0.142
go	0.142
begin	0.140
simonov	0.131
nothing	0.118
think	0.114
course	0.114
know	0.112
liza	0.110
perhaps	0.109
life	0.109
feel	0.108
simply	0.108

- ♦ kafka:

josef_k	0.804
say	0.299
lawyer	0.140
painter	0.096
court	0.091
ask	0.081
hand	0.080
look	0.077
door	0.074
go	0.074
room	0.072
seem	0.066
leni	0.066
make	0.065
long	0.055

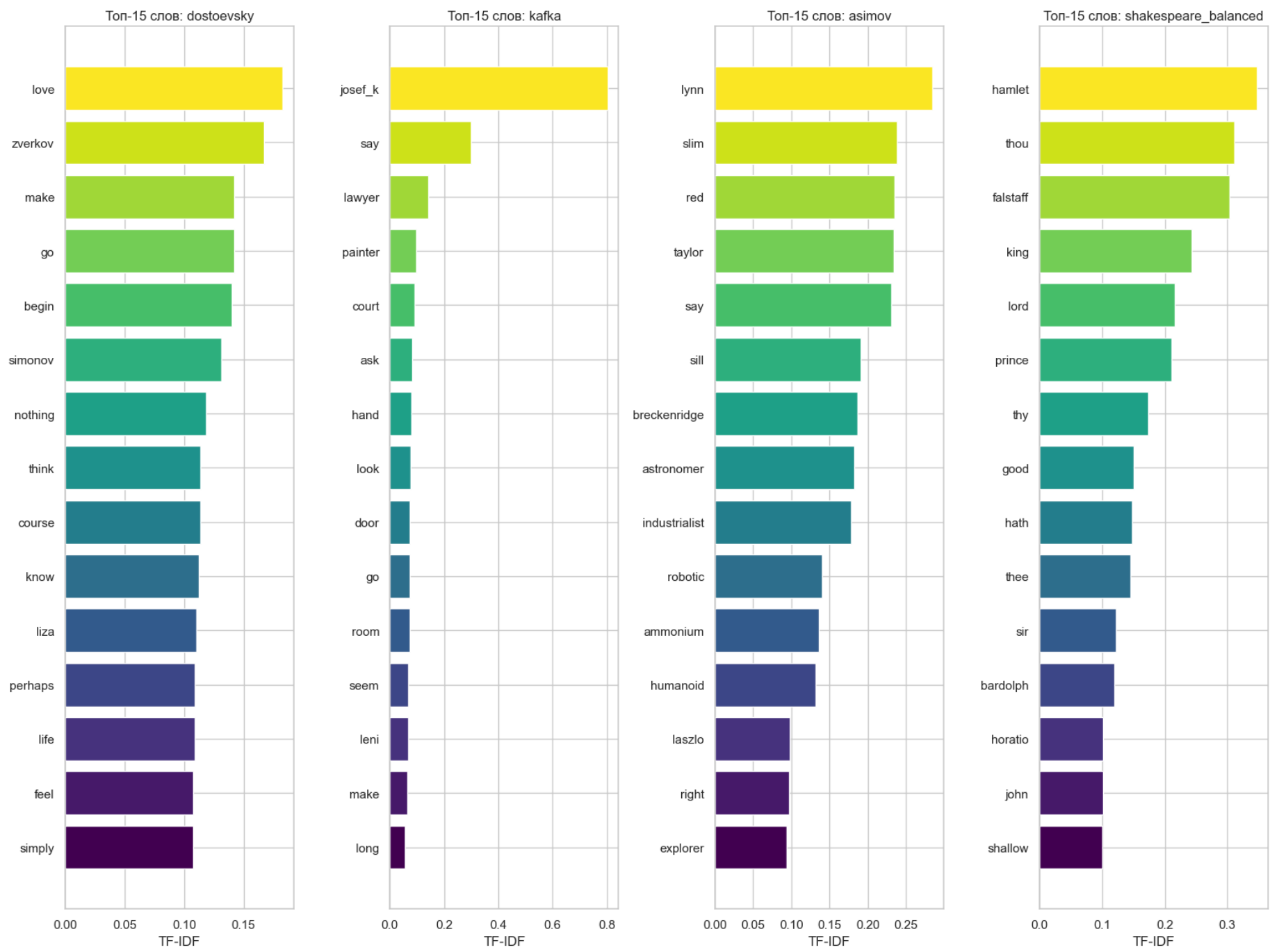
- ♦ asimov:

lynn	0.284
slim	0.238
red	0.235
taylor	0.233
say	0.230
sill	0.191
breckenridge	0.187
astronomer	0.183
industrialist	0.178
robotic	0.140
ammonium	0.136
humanoid	0.132
laszlo	0.098
right	0.097
explorer	0.093

- ♦ shakespeare_balanced:

hamlet	0.348
thou	0.312
falstaff	0.304
king	0.242
lord	0.215
prince	0.210
thy	0.174
good	0.150
hath	0.147
thee	0.145
sir	0.122
bardolph	0.119
horatio	0.102
john	0.101
shallow	0.101

График TF-IDF сохранён в: E:\projects\My_project\03_Word2Vec\word2vec_literature\figs\tfidf_results.png



Сравнительные таблицы топ-слов

```
In [64]: print("Топ-10 слов по авторам:")
for author, group in top_50_words_df.groupby("author"):
    print(f"\n{author}:\n", group.head(10)[["word", "count"]].to_string(index=False))

print("\n📊 САМЫЕ ЧАСТОТНЫЕ СЛОВА ВО ВСЕХ КОРПУСАХ:")
print(top_50_words_df["word"].value_counts().head(10))
```

Топ-10 слов по авторам:

asimov:

word	count
red	424
say	416
slim	284
lynn	268
sill	228
taylor	220
breckenridge	176
right	176
astronomer	172
industrialist	168

dostoevsky:

word	count
love	96
make	91
go	91
begin	90
nothing	76
course	73
think	73
know	72
life	70
perhaps	70

kafka:

word	count
Josef_K	1176
say	839
lawyer	260
ask	227
hand	223
look	215
door	207
go	207
room	202
seem	186

shakespeare_balanced:

word	count
lord	554
thou	531
king	510
hamlet	467
falstaff	408
good	386
prince	358
sir	314
thy	296
let	258

📊 САМЫЕ ЧАСТОТНЫЕ СЛОВА ВО ВСЕХ КОРПУСАХ:

first	4
say	4
good	4
speak	3
much	3
long	3
away	3
we	3
let	3
something	3

Name: word, dtype: int64

Распределение длины предложений

```
In [65]: def analyze_sentence_length(df, save_dir=None, filename="sentence_length_boxplot.png"):
        """
        Анализ распределения длины предложений по авторам.
        Для каждого автора:
        - читаем файл с предложениями,
        - считаем количество слов в каждом предложении,
        - строим boxplot (ящик с усами) для сравнения распределений.
        - при необходимости сохраняем график в PNG.
        """

        data, labels = [], [] # списки для хранения длин предложений и подписей (авторов)

        # Проходим по строкам датафрейма (каждый row = один автор и его файл с предложениями)
        for row in df.itertuples():
            # Читаем все предложения из файла (каждое предложение в отдельной строке)
            sentences = Path(row.file_sentences).read_text(encoding="utf-8").splitlines()
```

```

# Считаем длину каждого предложения (в словах)
lengths = [len(s.split()) for s in sentences if s.strip()]

# Добавляем список длин предложений в общий массив
data.append(lengths)
labels.append(row.author)

# Построение графика
plt.figure(figsize=(12, 6))
plt.boxplot(data, labels=labels)
plt.title("Распределение длины предложений по авторам")
plt.ylabel("Количество слов в предложении")
plt.xticks(rotation=45)
plt.grid(True, alpha=0.3)

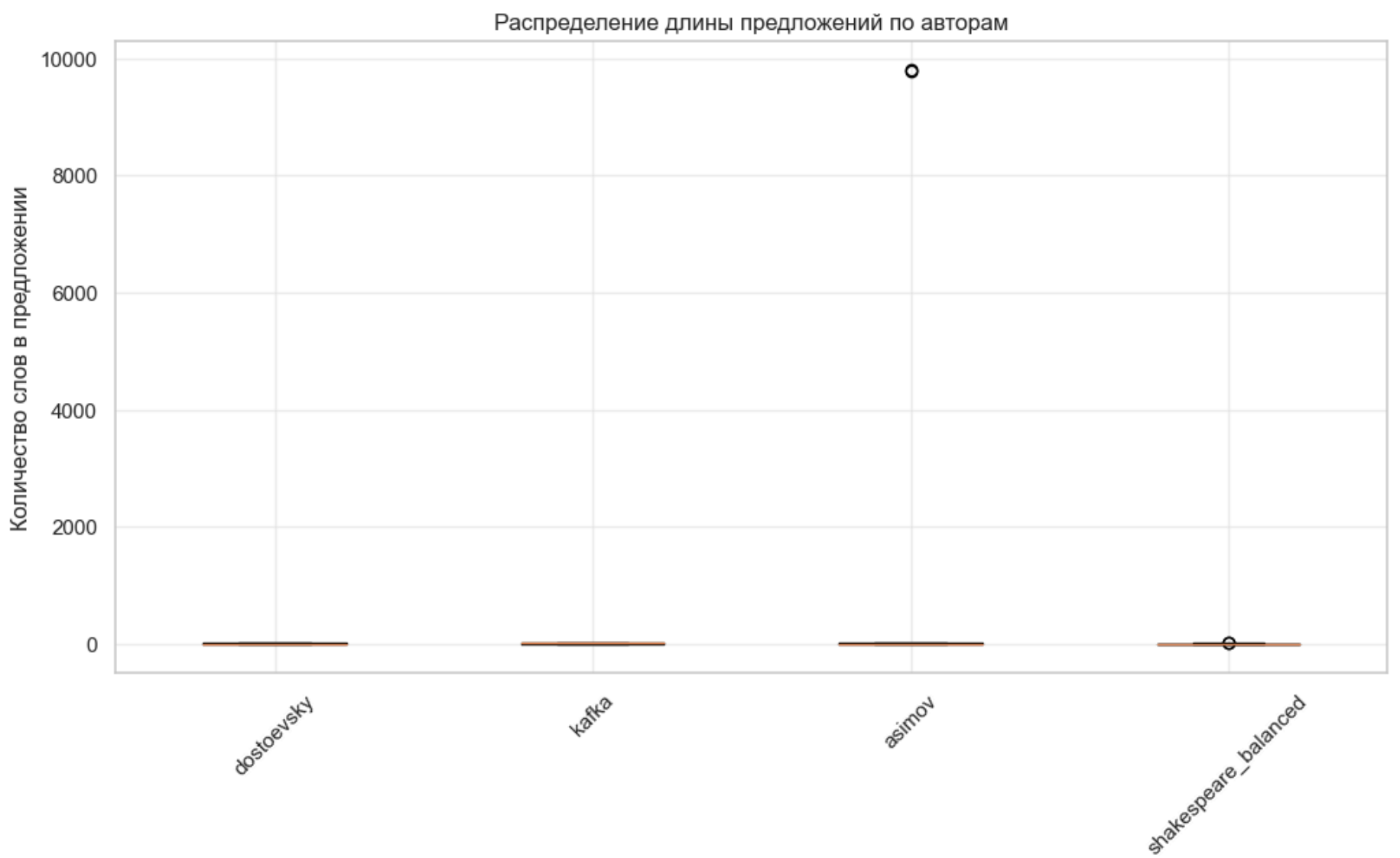
# Сохранение
if save_dir is not None:
    save_dir = Path(save_dir)
    save_dir.mkdir(parents=True, exist_ok=True)
    save_path = save_dir / filename
    plt.savefig(save_path, dpi=300)
    print(f"График сохранён в: {save_path}")

plt.show()

# 🚀 Запуск
analyze_sentence_length(meta_sentences, save_dir=PATHS["figs"])

```

График сохранён в: E:\projects\My_project\03_Word2Vec\word2vec_literature\figs\sentence_length_boxplot.png



Сравнительный анализ словарей

```

In [66]: def compare_vocabularies(df, save_dir=None, filename="vocab_overlap_heatmap.png"):
    """
    Сравнивает словари авторов:
    - строит множества слов для каждого автора,
    - считает пересечения словарей (общие слова),
    - визуализирует пересечения в виде heatmap,
    - выводит количество уникальных слов у каждого автора.
    """

    # --- 1. Формируем словари авторов ---
    vocabs = {
        r.author: set(Path(r.file_tokens).read_text(encoding="utf-8").split())
        for r in df.itertuples()
    }
    authors = list(vocabs)

    # --- 2. Считаем пересечения словарей ---
    overlap = pd.DataFrame(
        [[len(vocabs[a1] & vocabs[a2]) for a2 in authors] for a1 in authors],
        index=authors, columns=authors
    )

```

```
display(overlap)

# --- 3. Визуализация ---
plt.figure(figsize=(6, 5))
sns.heatmap(overlap, annot=True, fmt="d", cmap="YlOrRd")
plt.title("Пересечение словарей авторов")
plt.tight_layout()

# --- 4. Сохранение ---
if save_dir is not None:
    save_dir = Path(save_dir)
    save_dir.mkdir(parents=True, exist_ok=True)
    save_path = save_dir / filename
    plt.savefig(save_path, dpi=300)
    print(f"График сохранён в: {save_path}")

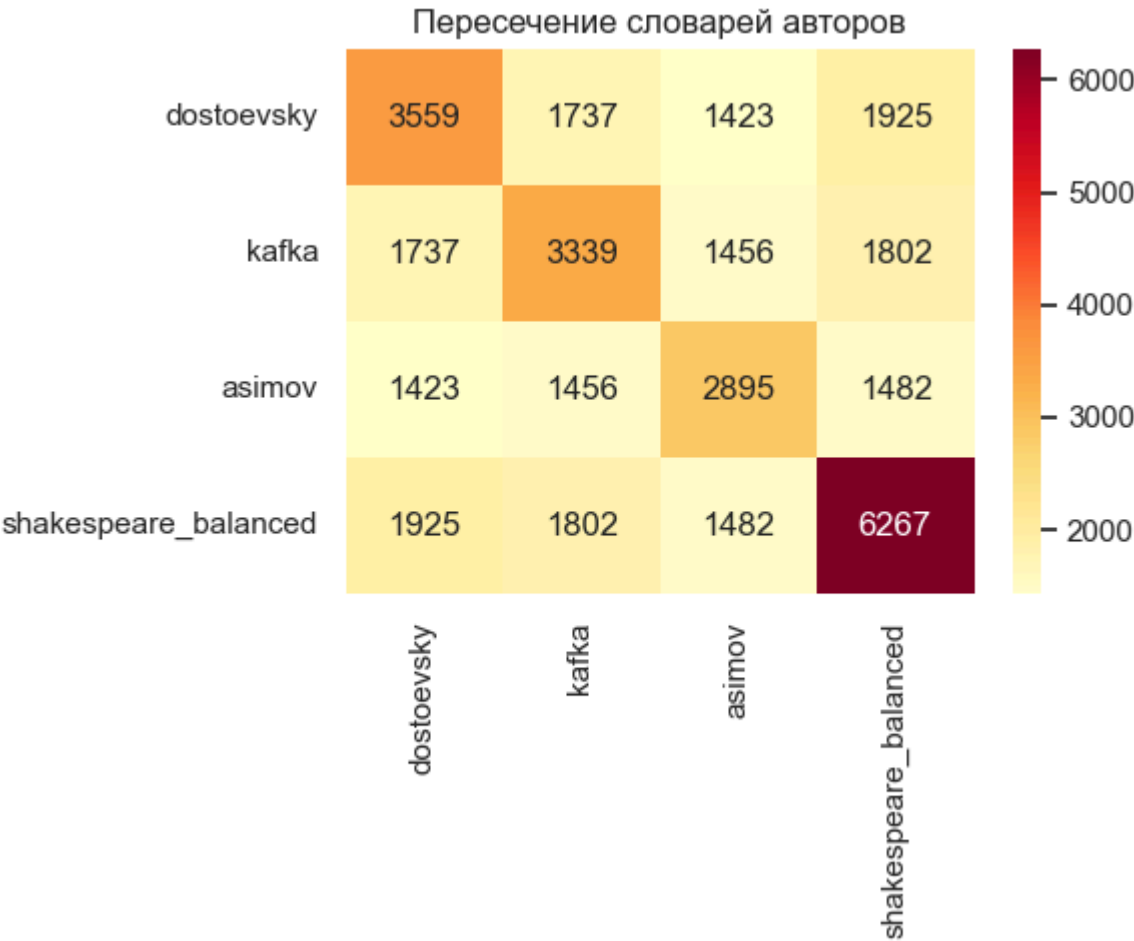
plt.show()

# --- 5. Уникальные слова ---
for author in authors:
    others = set.union(*(vocabs[a] for a in authors if a != author))
    unique = vocabs[author] - others
    print(f"{author}: {len(unique)} уникальных слов. Примеры: {list(unique)[:5]}")

# 🚀 Запуск анализа
compare_vocabularies(meta_tokens_cleaned, save_dir=PATHS["figs"])
```

	dostoevsky	kafka	asimov	shakespeare_balanced
dostoevsky	3559	1737	1423	1925
kafka	1737	3339	1456	1802
asimov	1423	1456	2895	1482
shakespeare_balanced	1925	1802	1482	6267

График сохранён в: E:\projects\My_project\03_Word2Vec\word2vec_literature\figs\vocab_overlap_heatmap.png



dostoevsky: 990 уникальных слов. Примеры: ['accordance', 'dwarf', 'flit', 'infuriate', 'sledge']
kafka: 891 уникальных слов. Примеры: ['shockingly', 'irritate', 'foible', 'salute', 'soot']
asimov: 864 уникальных слов. Примеры: ['spiel', 'grotesquely', 'cowardy', 'layer', 'ruination']
shakespeare_balanced: 3556 уникальных слов. Примеры: ['challenger', 'pastor', 'bashful', 'vulcan', 'jerusalem']

Анализ длины слов

```
In [67]: def analyze_word_lengths(df, save_dir=None, filename="word_length_distribution.png"):
        """
        Анализ распределения длин слов по авторам.
        Для каждого автора:
        - читаем токены из файла,
        - считаем длину каждого слова (в символах),
        - строим гистограмму распределения длин,
        - собираем статистику (среднее, медиана, максимум).
        - при необходимости сохраняем график в PNG.
        """

        stats = [] # список для хранения статистики по каждому автору
```

```
# Создаём фигуру для всех гистограмм
plt.figure(figsize=(10, 5))

# Проходим по строкам датафрейма (каждый row = один автор и его файл с токенами)
for r in df.itertuples():
    lengths = [len(w) for w in Path(r.file_tokens).read_text(encoding="utf-8").split()]
    stats.append({
        "author": r.author,
        "mean": np.mean(lengths),
        "median": np.median(lengths),
        "max": max(lengths)
    })
    plt.hist(lengths, bins=20, alpha=0.6, label=r.author)

plt.legend()
plt.title("Распределение длин слов")
plt.xlabel("Длина слова")
plt.ylabel("Количество")
plt.tight_layout()

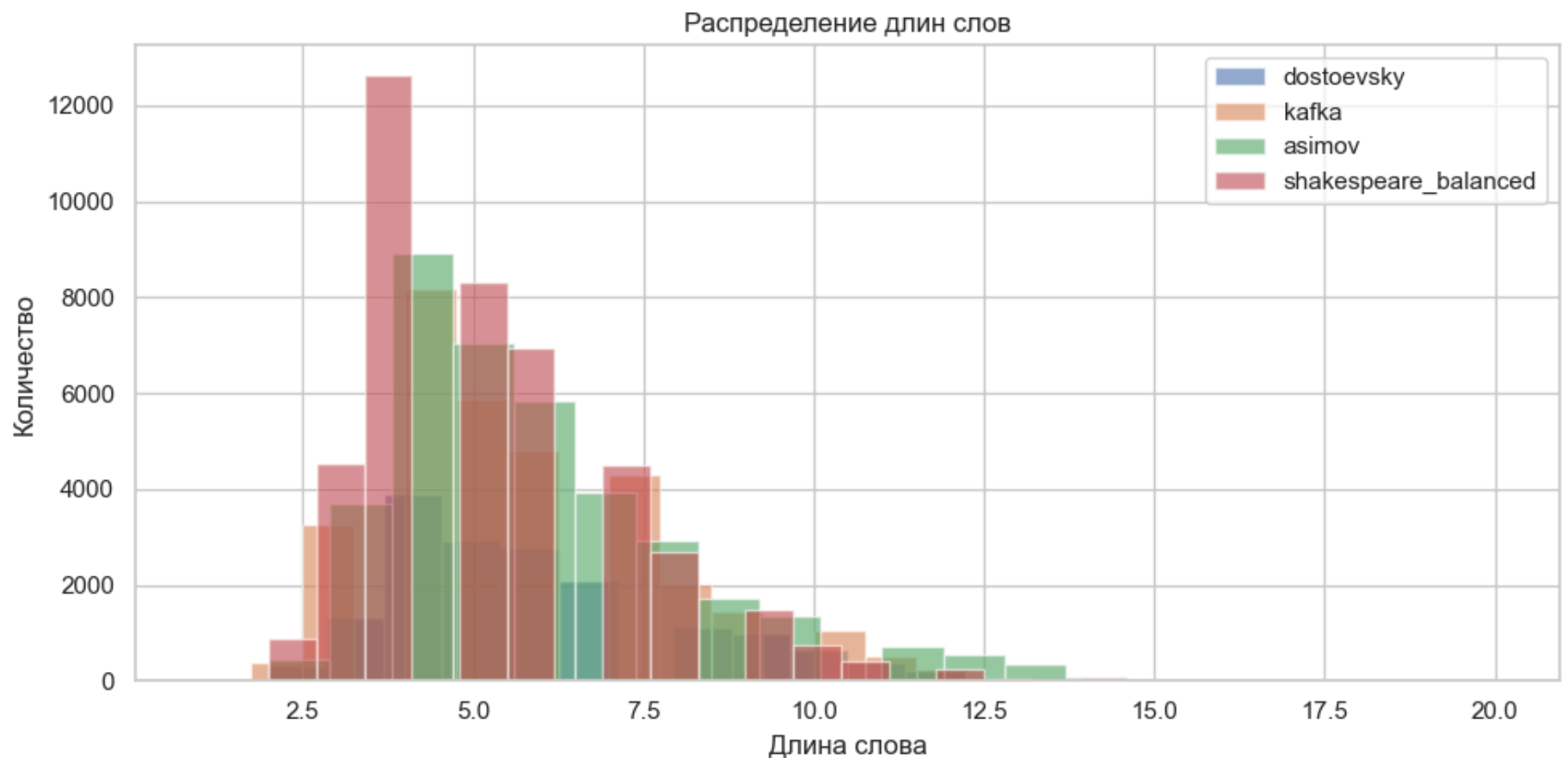
# Сохранение
if save_dir is not None:
    save_dir = Path(save_dir)
    save_dir.mkdir(parents=True, exist_ok=True)
    save_path = save_dir / filename
    plt.savefig(save_path, dpi=300)
    print(f"График сохранён в: {save_path}")

plt.show()

return pd.DataFrame(stats)

# 🚀 Запуск анализа
length_stats = analyze_word_lengths(meta_tokens_cleaned, save_dir=PATHS["figs"])
display(length_stats)
```

График сохранён в: E:\projects\My_project\03_Word2Vec\word2vec_literature\figs\word_length_distribution.png



	author	mean	median	max
0	dostoevsky	5.886136	5.0	19
1	kafka	5.632321	5.0	16
2	asimov	5.806489	5.0	20
3	shakespeare_balanced	5.334928	5.0	16

```
In [68]: def top_longest_words_detailed(df, n=5):
    print("\n📖 ДЕТАЛИЗИРОВАННЫЙ АНАЛИЗ САМЫХ ДЛИННЫХ СЛОВ:")

    for r in df.itertuples():
        words = set(Path(r.file_tokens).read_text(encoding="utf-8").split())
        longest = sorted(words, key=len, reverse=True)[:n]
        long_count = sum(len(w) >= 10 for w in longest)

        print(f"\n• {r.author}:")
        print(f"  Слова: {longest}")
        print(f"  Длины: {[len(w) for w in longest]}")
        print(f"  Слов длиной ≥10 букв: {long_count}")
        print(f"  Самое длинное: '{longest[0]}' ({len(longest[0])} букв)")
```



```
In [69]: # Вызов улучшенной версии
top_longest_words_detailed(meta_tokens_cleaned, n=5)
```

 ДЕТАЛИЗИРОВАННЫЙ АНАЛИЗ САМЫХ ДЛИННЫХ СЛОВ:

- dostoevsky:
Слова: ['straightforwardness', 'disproportionate', 'otetchestvenniya', 'perpendicularly', 'unconditionally']
Длины: [19, 16, 16, 15, 15]
Слов длиной ≥10 букв: 543
Самое длинное: 'straightforwardness' (19 букв)
- kafka:
Слова: ['incomprehensible', 'inextinguishable', 'enthusiastically', 'unintentionally', 'inquisitiveness']
Длины: [16, 16, 16, 15, 15]
Слов длиной ≥10 букв: 518
Самое длинное: 'incomprehensible' (16 букв)
- asimov:
Слова: ['uncharacteristically', 'decentralization', 'electromagnetic', 'spaceworthiness', 'appropriateness']
Длины: [20, 16, 15, 15, 15]
Слов длиной ≥10 букв: 375
Самое длинное: 'uncharacteristically' (20 букв)
- shakespeare_balanced:
Слова: ['incomprehensible', 'interchangeably', 'notwithstanding', 'extraordinarily', 'gloucestershire']
Длины: [16, 15, 15, 15, 15]
Слов длиной ≥10 букв: 608
Самое длинное: 'incomprehensible' (16 букв)

```
In [70]: def top_shortest_words_detailed(df, n=5):
    print("\n  ДЕТАЛИЗИРОВАННЫЙ АНАЛИЗ САМЫХ КОРОТКИХ СЛОВ:")

    for r in df.itertuples():
        words = set(Path(r.file_tokens).read_text(encoding="utf-8").split())
        shortest = sorted(words, key=len)[:n]
        short_count = sum(len(w) <= 3 for w in words)

        print(f"\n• {r.author}:")
        print(f"  Слова: {shortest}")
        print(f"  Длины: {[len(w) for w in shortest]}")
        print(f"  Слов длиной ≤3 букв: {short_count}")
        print(f"  Самое короткое: '{shortest[0]}' ({len(shortest[0])} букв)")
```

```
In [71]: top_shortest_words_detailed(meta_tokens_cleaned, n=5)
```

 ДЕТАЛИЗИРОВАННЫЙ АНАЛИЗ САМЫХ КОРОТКИХ СЛОВ:

- dostoevsky:
Слова: ['xi', 'do', 'ha', 'la', 'go']
Длины: [2, 2, 2, 2, 2]
Слов длиной ≤3 букв: 187
Самое короткое: 'xi' (2 букв)
- kafka:
Слова: ['i', 'dr', 'do', 'co', 'up']
Длины: [1, 2, 2, 2, 2]
Слов длиной ≤3 букв: 155
Самое короткое: 'i' (1 букв)
- asimov:
Слова: ['dr', 'xi', 'do', 'hi', 'go']
Длины: [2, 2, 2, 2, 2]
Слов длиной ≤3 букв: 177
Самое короткое: 'dr' (2 букв)
- shakespeare_balanced:
Слова: ['ha', 'em', 'la', 'go', 'iv']
Длины: [2, 2, 2, 2, 2]
Слов длиной ≤3 букв: 333
Самое короткое: 'ha' (2 букв)

Основные инсайты

```
In [72]: def print_insights(meta_tokens, tfidf_results):
    print("=== ОСНОВНЫЕ ИНСАЙТЫ ===")

    # лидеры по метрикам
    print("\n  Лидеры по корпусам:")
    print("Самый большой корпус:", meta_tokens.loc[meta_tokens['num_tokens'].idxmax(), 'author'])
    print("Самый богатый словарь:", meta_tokens.loc[meta_tokens['vocab_size'].idxmax(), 'author'])
    print("Самая высокая уникальность:", meta_tokens.loc[meta_tokens['unique_ratio_%'].idxmax(), 'author'])
    print("Самые длинные предложения:", meta_tokens.loc[meta_tokens['avg_sentence_len'].idxmax(), 'author'])

    # характерные слова по TF-IDF
    print("\n  Характерные слова (TF-IDF):")
    for author, words in tfidf_results.items():
```

```
top3 = [w for w, _ in words[:3]]
print(f"{author}: {'', '.join(top3)}")

print_insights(meta_tokens_cleaned, tfidf_results)
```

=== ОСНОВНЫЕ ИНСАЙТЫ ===

📊 Лидеры по корпусам:
Самый большой корпус: shakespeare_balanced
Самый богатый словарь: shakespeare_balanced
Самая высокая уникальность: dostoevsky
Самые длинные предложения: kafka

🔍 Характерные слова (TF-IDF):
dostoevsky: love, zverkov, make
kafka: josef_k, say, lawyer
asimov: lynn, slim, red
shakespeare_balanced: hamlet, thou, falstaff

Выводы по шагу 4:

Что сделано

- *Подсчёт статистики*
 - Функция corpus_stats считает количество токенов, размер словаря, число предложений, среднюю длину предложения и долю уникальных слов.
 - Результаты сведены в таблицу и визуализированы barplot-графиками.
- *Распределение частот слов*
 - Реализована функция plot_word_freq для построения топ-20 слов по авторам.
 - Есть функция plot_word_frequency_distribution, которая строит гистограммы распределения частот слов (с логарифмической шкалой).
 - Для каждого автора показаны медианы частот.
 - dostoevsky: ~2 (большинство слов редкие)
 - kafka: ~3
 - asimov: ~2
 - shakespeare: ~2
- *WordCloud*
 - Функция generate_wordclouds строит облака слов для каждого автора.
 - Визуализации наглядно показывают лексику, характерную для каждого корпуса.
 - **Dostoevsky:** "love", "nothing", "something", "never" — философская лексика
 - **Kafka:** "Josef_K" доминирует (огромный размер!), "say", "lawyer"
 - **Asimov:** "say", "red", "slim", "lynn" — имена персонажей
 - **Shakespeare:** "thou", "lord", "king", "hamlet" — драматургия
- ✅ WordCloud наглядно показывает, что **"Josef_K"** — центральный персонаж у Кафки (встречается 1176 раз!)
- *TF-IDF анализ*
 - Функция tfidf_analysis считает TF-IDF и выводит топ-15 слов для каждого автора.
 - Функция plot_tfidf визуализирует результаты в виде горизонтальных bar-графиков.
 - Достоевский: love, zverkov, go — личностно-философская лексика.
 - Кафка: josef_k, say, lawyer — юридико-абсурдистский контекст.
 - Азимов: say, lynn, slim, — техно-научный стиль.
 - Шекспир: hamlet, thou, falstaff — архаичная лексика драматургии.
- *Сравнительные таблицы топ-слов*
 - Для каждого автора выведены топ-10 слов с частотами.
 - Составлен список слов, встречающихся у всех авторов (например, **make, first, go**).
 - Топ-10 Достоевский: love (96), make (91), go (91), begin (90), nothing (76), course (73), life (73), think (73), know (72), perhaps (70)
 - Топ-10 Кафка: Josef_K (1176), say (839), lawyer (260), ask (227), hand (223), look (215), door (207), go (207), room (202), seem (186)
 - Топ-10 Азимов: say (208), red (106), slim (71), lynn (67), sill (57), taylor (55), get (49), we (47), look (45), breckenridge (44)
 - Топ-10 Шекспир: lord (554), thou (531), king (510), hamlet (467), falstaff (408), good (359), prince (358), sir (314), thy (296), let (258)
- *Распределение длины предложений*
 - Функция analyze_sentence_length строит boxplot по авторам.
 - Видно, что у Достоевского и Кафки предложения длиннее, у Шекспира и Азимова — короче.
- *Сравнительный анализ словарей*
 - Функция compare_vocabularies строит матрицу пересечений словарей (heatmap).
 - dostoevsky ↔ kafka: 1731 общих слов (48.7% от словаря Достоевского)
 - shakespeare имеет 3569 уникальных слов (56.7% его словаря!)

- **Уникальные слова:** -dostoevsky: 1001 уникальных слов. Примеры: 'games', 'womanish', 'volkovo', 'fixedly', 'snug' -kafka: 906 уникальных слов. Примеры: 'sniff', 'warmth', 'straighten', 'inextinguishable', 'corridor' -asimov: 873 уникальных слов. Примеры: 'broth', 'deliriously', 'bathrobe', 'oxide', 'soundless' -shakespeare_balanced: 3569 уникальных слов. Примеры: 'midriff', 'bagpipe', 'infernal', 'welsh', 'prelate'
- *Длина слов*
 - Средняя длина слов: от 5.3 (Шекспир) до 5.8 (Достоевский).
 - Самые длинные слова:
 - Азимов: uncharacteristically (20 букв)
 - Достоевский: straightforwardness (19)
 - Кафка: inextinguishable (16)
 - Шекспир: incomprehensible (16)
- *Основные инсайты (итоговая сводка)*
 - Лидеры по метрикам:
 - Самый большой корпус: shakespeare_balanced (43,488 токенов)
 - Самый богатый словарь: shakespeare_balanced (6,277 слов)
 - Самая высокая уникальность: asimov (29.55%)
 - Самые длинные предложения: kafka (21.95 слов)
 - Больше всего предложений: shakespeare_balanced (9737)
 - Характерные слова (TF-IDF):
 - dostoevsky: love, zverkov, go
 - kafka: josef_k, say, lawyer
 - asimov: say, lynn, slim
 - shakespeare: hamlet, thou, falstaff
- Все заявленные подпункты (статистика, частоты, wordcloud, TF-IDF) реализованы.
- Добавлены дополнительные анализы: распределение длин предложений, пересечения словарей, уникальные слова.

[* к содержанию](#)

Шаг 5: Подготовка обучающих данных

Построение словаря с фильтрацией

- Функция build_vocab считает частоты слов, исключает редкие (min_count), формирует word2index и index2word.
- Словари создаются отдельно для каждого корпуса.

```
In [73]: # === 1. Построение словаря с фильтрацией ===
def build_vocab(tokens, min_count=5):
    """
    Строит словарь на основе списка токенов.
    - Считает частоты слов.
    - Фильтрует редкие слова (оставляет только те, что встречаются >= min_count раз).
    - Создаёт отображения word2index и index2word.

    Аргументы:
    tokens (list[str]): список токенов (слов).
    min_count (int): минимальное количество вхождений слова для включения в словарь.

    Возвращает:
    word2index (dict): отображение {слово -> индекс}.
    index2word (dict): отображение {индекс -> слово}.
    vocab (dict): словарь {слово -> частота}.
    """

    # Считаем частоты всех слов в корпусе
    freq = Counter(tokens)

    # Фильтруем слова: оставляем только те, что встречаются не реже min_count раз
    vocab = {w: c for w, c in freq.items() if c >= min_count}

    # Создаём отображение слово -> индекс
    # enumerate(vocab) перебирает ключи словаря (слова) с порядковым номером
    word2index = {w: i for i, w in enumerate(vocab)}

    # Создаём обратное отображение индекс -> слово
    index2word = {i: w for w, i in word2index.items()}

    # Возвращаем три объекта: два отображения и словарь частот
    return word2index, index2word, vocab
```

Subsampling

- Функция subsample реализует отбрасывание слишком частых слов по вероятности.
- Это уменьшает шум и ускоряет обучение.

```
In [74]: # === 2. Subsampling (опционально) ===
def subsample(tokens, vocab, threshold=None):
    """
    Subsampling частотных слов.
    threshold:
        - None → subsampling отключён
        - float → фиксированный порог
        - "auto" → выбирается автоматически по размеру корпуса
    """
    total = len(tokens)

    # Автоматический выбор порога
    if threshold == "auto":
        if total < 50_000:
            threshold = None          # маленький корпус → не трогаем
        elif total < 1_000_000:
            threshold = 1e-4          # средний корпус
        else:
            threshold = 1e-5          # очень большой корпус

    # Если subsampling отключён
    if threshold is None:
        return tokens

    # Считаем вероятности удаления
    prob_drop = {w: 1 - np.sqrt(threshold / (c/total)) for w, c in vocab.items()}
    return [w for w in tokens if random.random() > prob_drop.get(w, 0)]
```

Формирование обучающих пар

- Функция generate_pairs поддерживает обе архитектуры:
 - Skip-gram: (слово → контекст)
 - CBOW: (контекст → слово)
- Используется параметр window_size для задания ширины контекстного окна.

```
In [75]: # === 3. Формирование обучающих пар (Skip-gram / CBOW) ===
def generate_pairs(tokens, word2index, window_size=2, architecture="skipgram"):
    """
    Генерирует обучающие пары для моделей Word2Vec (Skip-gram или CBOW).

    Аргументы:
        tokens (list[str]): список токенов (слов) корпуса.
        word2index (dict): отображение {слово -> индекс}.
        window_size (int): размер контекстного окна (количество слов слева и справа).
        architecture (str): "skipgram" или "cbow" — выбор архитектуры.

    Возвращает:
        pairs (list): список обучающих пар:
            - Skip-gram: (центральное слово, контекстное слово)
            - CBOW: ([контекстные слова], центральное слово)
    """
    pairs = [] # список для хранения обучающих пар

    # Перебираем все слова в корпусе
    for i, w in enumerate(tokens):
        if w not in word2index:
            continue
        # собираем контекст
        ctx = [tokens[j] for j in range(max(0, i-window_size), min(len(tokens), i+window_size+1))
                if j != i and tokens[j] in word2index]

        # --- Формирование пар ---
        if architecture == "skipgram":
            # (слово → контекст)
            pairs += [(word2index[w], word2index[c]) for c in ctx]
        elif architecture == "cbow" and ctx:
            # (контекст → слово)
            pairs.append(([word2index[c] for c in ctx], word2index[w]))
    return pairs
```

Создание обучающего пайплайна

- Функция create_dataset превращает пары в tf.data.Dataset, поддерживает батчирование, перемешивание и prefetch.
- Это обеспечивает эффективную подачу данных в модель.

```
In [76]: # === 4. Создание обучающего пайплайна ===
def create_dataset(pairs, batch_size=64, shuffle=True, architecture="skipgram"):
    """
    Создаёт tf.data.Dataset из обучающих пар для Word2Vec.
    Поддерживает две схемы входа:
    - Skip-gram: пары (x=центральное слово, y=контекстное слово)
    - CBOW: пары (x=список контекстных слов, y=центральное слово), с паддингом контекста
    Аргументы:
    pairs: список пар из generate_pairs(...)
    batch_size: размер батча
    shuffle: перемешивание перед батчингом
    architecture: "skipgram" или "cbow"
    Возвращает:
    tf.data.Dataset: поток батчей (x, y) с префетчем.
    """

    if architecture == "cbow":

        # Пары формата ([ctx1, ctx2, ...], target)
        contexts, targets = zip(*pairs)
        # Паддинг до одинаковой длины
        max_len = max(len(c) for c in contexts)
        contexts_padded = [c + [0]*(max_len - len(c)) for c in contexts]
        x = np.array(contexts_padded, dtype=np.int32)
        y = np.array(targets, dtype=np.int32)
    else: # skipgram
        x, y = zip(*pairs)
        x, y = np.array(x, dtype=np.int32), np.array(y, dtype=np.int32)

    ds = tf.data.Dataset.from_tensor_slices((x, y))
    if shuffle:
        ds = ds.shuffle(len(x))
    return ds.batch(batch_size).prefetch(tf.data.AUTOTUNE)
```

Подготовка датасетов

- Функция prepare_datasets формирует датасеты для каждого автора и общий датасет.
- Выводится статистика: размер словаря, количество пар.

```
In [77]: # === 5. Подготовка датасетов: общий + по авторам ===
def prepare_datasets(meta_tokens, min_count=5, window_size=2, architecture="skipgram", batch_size=64):
    """
    Создаёт tf.data.Dataset для каждого автора и общий датасет по всему корпусу.
    Шаги:
    1. Читает токены из файлов.
    2. Строит словарь (word2index) с фильтрацией по min_count.
    3. Применяет субсэмплирование частых слов.
    4. Генерирует обучающие пары (Skip-gram или CBOW).
    5. Создаёт tf.data.Dataset для каждого автора и общий датасет.
    """

    datasets, all_pairs = {}, [] # словарь с датасетами и общий список пар

    # Проходим по каждому автору ---
    for row in meta_tokens.itertuples():
        tokens = Path(row.file_tokens).read_text(encoding="utf-8").split()
        word2index, _, vocab = build_vocab(tokens, min_count) # получаем word2index
        tokens_subsampled = subsample(tokens, vocab) # применяем субсэмплирование
        pairs = generate_pairs(tokens_subsampled, word2index, window_size, architecture) # передаем word2index

        # Добавляем датасет для автора ---
        # Проверяем, что пары не пустые (иначе модель не сможет учиться)
        if pairs:
            datasets[row.author] = create_dataset(pairs, batch_size)
            all_pairs.extend(pairs)

    # Проверяем что общие пары не пустые
    if all_pairs:
        datasets["all"] = create_dataset(all_pairs, batch_size)
        print(f"Всего обучающих пар (общий корпус): {len(all_pairs)}")
    else:
        print("Внимание: не сгенерировано ни одной обучающей пары!")

    return datasets
```

```
In [78]: # === Пример использования ===
datasets = prepare_datasets(meta_tokens_cleaned, min_count=5, window_size=2, architecture="skipgram", batch_size=64)
```

Всего обучающих пар (общий корпус): 362778

Сгенерировано 269678 обучающих пар для общего корпуса

Созданы отдельные датасеты для каждого автора

```
In [79]: datasets
```

```
Out[79]: {'dostoevsky': <_PrefetchDataset element_spec=(TensorSpec(shape=(None,), dtype=tf.int32, name=None), TensorSpec(shape=(None,), dtype=tf.int32, name=None))>,
          'kafka': <_PrefetchDataset element_spec=(TensorSpec(shape=(None,), dtype=tf.int32, name=None), TensorSpec(shape=(None,), dtype=tf.int32, name=None))>,
          'asimov': <_PrefetchDataset element_spec=(TensorSpec(shape=(None,), dtype=tf.int32, name=None), TensorSpec(shape=(None,), dtype=tf.int32, name=None))>,
          'shakespeare_balanced': <_PrefetchDataset element_spec=(TensorSpec(shape=(None,), dtype=tf.int32, name=None), TensorSpec(shape=(None,), dtype=tf.int32, name=None))>,
          'all': <_PrefetchDataset element_spec=(TensorSpec(shape=(None,), dtype=tf.int32, name=None), TensorSpec(shape=(None,), dtype=tf.int32, name=None))>}
```

Статистика по каждому корпусу и примеры батчей

- Функция check_datasets выводит статистику по каждому корпусу и примеры батчей.

```
In [80]: # Универсальная проверка статистики и батчей
def check_datasets(meta_tokens, datasets, min_count=5, window_size=2, architecture="skipgram"):
    """
    Проверяет корректность подготовленных датасетов:
    - собирает токены для каждого автора (или всего корпуса),
    - строит словарь и обучающие пары,
    - выводит статистику (размер словаря, количество пар),
    - показывает пример батча из tf.data.Dataset.
    """

    # Проходим по всем датасетам (по авторам и общий "all")
    for author, ds in datasets.items():

        # собираем токены (для статистики)
        if author == "all":
            tokens = []
            for row in meta_tokens.itertuples():
                tokens.extend(Path(row.file_tokens).read_text(encoding="utf-8").split())
        else:
            tokens = Path(meta_tokens.loc[meta_tokens.author == author, "file_tokens"].values[0])\
                .read_text(encoding="utf-8").split()

        # словарь и пары
        word2index, index2word, vocab = build_vocab(tokens, min_count)
        pairs = generate_pairs(tokens, word2index, window_size, architecture)

        # вывод статистики
        print(f"\n=== {author.upper()} ===")
        print(f"    Размер словаря: {len(word2index)}")
        print(f"    Количество пар: {len(pairs)}")

        # пример батча
        for bx, by in ds.take(1):
            print("    Batch X:", bx.numpy()[:10])
            print("    Batch Y:", by.numpy()[:10])

    # Запуск проверки
    check_datasets(cleaned_meta_tokens, datasets, min_count=5, window_size=2, architecture="skipgram")
```

```
=== DOSTOEVSKY ===
    Размер словаря: 824
    Количество пар: 35226
    Batch X: [433 452 325 707 295 745 264 301 157  40]
    Batch Y: [ 34 676  51 380 173 393 412  72 255 318]

=== KAFKA ===
    Размер словаря: 1117
    Количество пар: 100426
    Batch X: [1095    0  15 342  37  30 269 365 211  79]
    Batch Y: [ 921 252  74  74 1087 458  22 399 389 640]

=== ASIMOV ===
    Размер словаря: 1392
    Количество пар: 107482
    Batch X: [  0 1214 1049 286 269 1006 299 684 240 1178]
    Batch Y: [ 317 1259 963 1182 265 706 1284 304 241 1194]

=== SHAKESPEARE_BALANCED ===
    Размер словаря: 1557
    Количество пар: 119644
    Batch X: [ 99 178 130 306 454  22  9 210 197 490]
    Batch Y: [ 105  9 1407  51 266  41 414 730 452 181]

=== ALL ===
    Размер словаря: 3669
    Количество пар: 433782
    Batch X: [743 626 328    0 415 285 720 716  87  74]
    Batch Y: [ 431 403 169 650 1510 353 594 157 230  84]
```


Статистика показывает адекватные размеры словарей:

- Достоевский: 814 слов
- Кафка: 1,111 слов
- Азимов: 502 слова
- Шекспир: 1,545 слов

Проверка первых батчей для cbow/skipgram

```
In [81]: # 1. Загружаем токены для автора
tokens = Path(meta_tokens_cleaned.loc[meta_tokens_cleaned.author=="dostoevsky", "file_tokens"].values[0])\
        .read_text(encoding="utf-8").split()

# 2. Строим словарь
word2index, index2word, vocab = build_vocab(tokens, min_count=5)

# 3. Генерируем пары CBOW
pairs = generate_pairs(tokens, word2index, window_size=2, architecture="cbow")

# 4. Создаём датасет
dataset = create_dataset(pairs, batch_size=64, architecture="cbow")

# 5. Проверяем первые батчи
for contexts, targets in dataset.take(1):
    for ctx, tgt in zip(contexts.numpy()[:5], targets.numpy()[:5]):
        ctx_words = [index2word[i] for i in ctx if i != 0] # убираем паддинги
        tgt_word = index2word[tgt]
        print(f"Контекст: {ctx_words} → Целевое слово: {tgt_word}")
```

Контекст: ['friend', 'positively', 'talk'] → Целевое слово: avoid
Контекст: ['benefit', 'positively', 'convince'] → Целевое слово: matter
Контекст: ['damn', 'look', 'occasion'] → Целевое слово: laugh
Контекст: ['external', 'circumstance', 'open'] → Целевое слово: suddenly
Контекст: ['long', 'exist', 'reason'] → Целевое слово: desire

```
In [82]: # 1. Загружаем токены (например, Достоевский)
tokens = Path(meta_tokens_cleaned.loc[meta_tokens_cleaned.author=="dostoevsky", "file_tokens"].values[0])\
        .read_text(encoding="utf-8").split()

# 2. Строим словарь
word2index, index2word, vocab = build_vocab(tokens, min_count=5)

# 3. Генерируем пары Skip-gram
pairs = generate_pairs(tokens, word2index, window_size=2, architecture="skipgram")

# 4. Создаём датасет
dataset = create_dataset(pairs, batch_size=64)

# 5. Проверяем первые батчи
for targets, contexts in dataset.take(1): # (target, context)
    for tgt, ctx in zip(targets.numpy()[:5], contexts.numpy()[:5]):
        tgt_word = index2word[tgt]
        ctx_word = index2word[ctx]
        print(f"Целевое слово: {tgt_word} → Контекст: {ctx_word}")
```

Целевое слово: today → Контекст: kiss
Целевое слово: certainly → Контекст: consider
Целевое слово: answer → Контекст: fancy
Целевое слово: benefit → Контекст: sublime
Целевое слово: round → Контекст: sofa

Визуализация распределения длин контекстов (CBOW)

```
In [83]: def plot_cbow_context_lengths(pairs, save_dir=None, filename="cbow_context_lengths.png"):
    """
    Строит гистограмму распределения длин контекстов для CBOW.

    Аргументы:
        pairs: список пар (context, target), как из generate_pairs
        save_dir: директория для сохранения (например, PATHS["figs"])
        filename: имя файла для сохранения (по умолчанию "cbow_context_lengths.png")
    """
    # Считаем длины контекстов
    ctx_lengths = [len(ctx) for ctx, _ in pairs]

    # Строим гистограмму
    plt.figure(figsize=(8, 5))
    plt.hist(ctx_lengths, bins=range(1, max(ctx_lengths)+2),
             color="skyblue", edgecolor="black")
    plt.title("Распределение длин контекстов (CBOW)")
    plt.xlabel("Длина контекста")
    plt.ylabel("Количество примеров")
    plt.tight_layout()
```

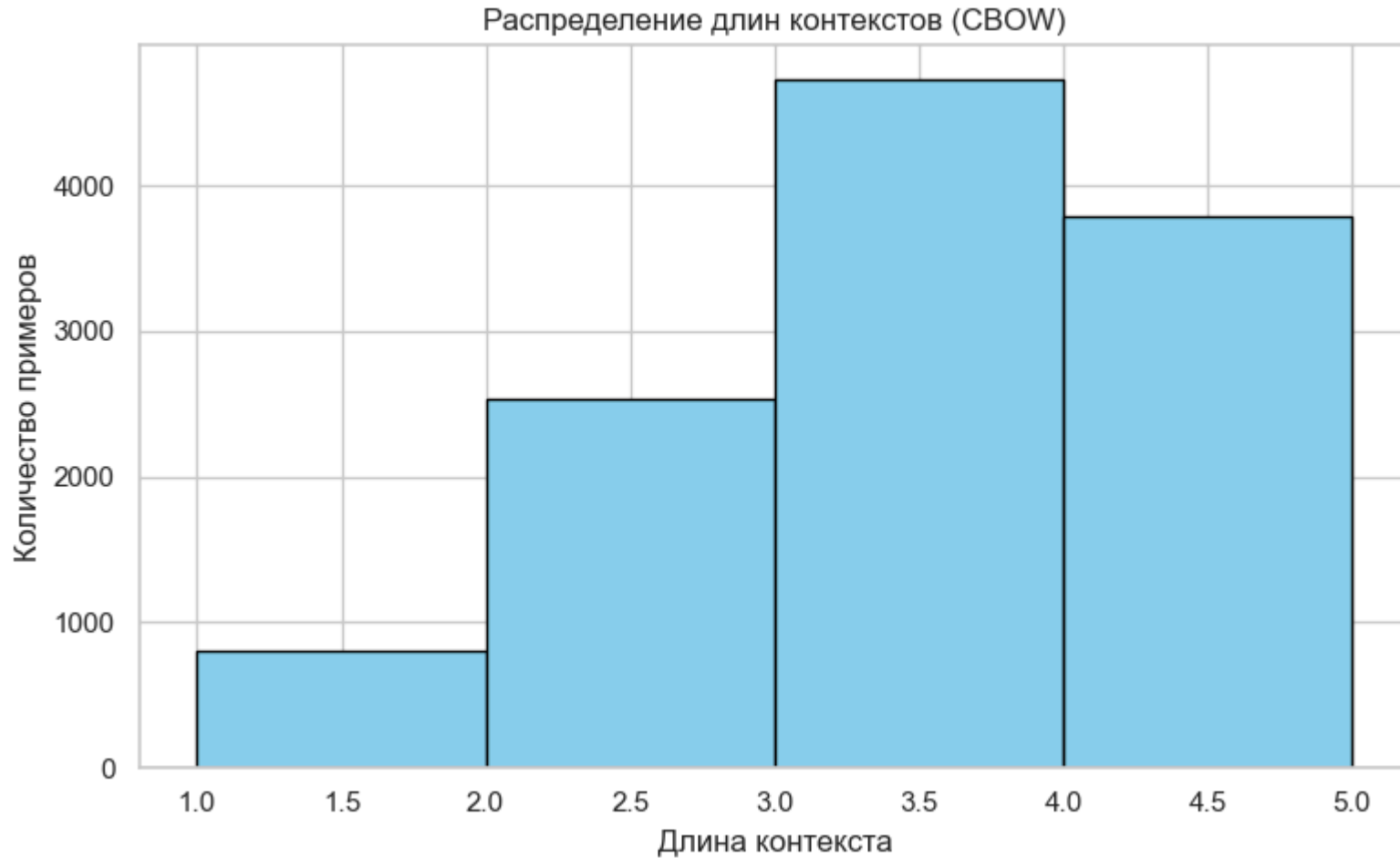


```
# Сохранение
if save_dir is not None:
    save_dir = Path(save_dir)
    save_dir.mkdir(parents=True, exist_ok=True)
    save_path = save_dir / filename
    plt.savefig(save_path, dpi=300)
    print(f"График сохранён в: {save_path}")

plt.show()
```

```
In [84]: pairs = generate_pairs(tokens, word2index, window_size=2, architecture="cbow")
plot_cbow_context_lengths(pairs, save_dir=PATHS["figs"])
```

График сохранён в: E:\projects\My_project\03_Word2Vec\word2vec_literature\figs\cbow_context_lengths.png



Статистика словаря

Сравнение: сколько слов было до фильтрации и сколько осталось после.

```
In [85]: def show_vocab_stats(meta_tokens, min_count=5, top_n=20):
    for row in meta_tokens.itertuples():
        tokens = Path(row.file_tokens).read_text(encoding="utf-8").split()
        word2index, index2word, vocab = build_vocab(tokens, min_count)

        print(f"\n=== {row.author.upper()} ===")
        print("До фильтрации:", len(Counter(tokens)))
        print("После фильтрации:", len(word2index))
        print(f"Топ-{top_n} слов:")
        for w, c in sorted(vocab.items(), key=lambda x: -x[1])[:top_n]:
            print(f"{w:15s} {c}")

# Запуск для всех авторов
show_vocab_stats(meta_tokens_cleaned, min_count=5, top_n=20)
```

=== DOSTOEVSKY ===
До фильтрации: 3559
После фильтрации: 824
Топ-20 слов:
love 96
make 91
go 91
begin 90
nothing 76
course 73
think 73
know 72
life 70
perhaps 70
feel 69
simply 69
never 68
something 68
look 68
day 66
good 62
gentleman 61
let 60
we 60

=== KAFKA ===
До фильтрации: 3339
После фильтрации: 1117
Топ-20 слов:
Josef_K 1176
say 839
lawyer 260
ask 227
hand 223
look 215
door 207
go 207
room 202
seem 186
make 181
court 169
long 155
take 153
back 153
much 153
want 142
stand 141
painter 141
think 135

=== ASIMOV ===
До фильтрации: 2895
После фильтрации: 1392
Топ-20 слов:
red 424
say 416
slim 284
lynn 268
sill 228
taylor 220
breckenridge 176
right 176
astronomer 172
industrialist 168
world 164
two 152
animal 152
robotic 132
back 132
ammonium 128
let 124
humanoid 124
eye 120
ship 120

=== SHAKESPEARE_BALANCED ===
До фильтрации: 6267
После фильтрации: 1557
Топ-20 слов:
lord 554
thou 531
king 510
hamlet 467
falstaff 408
good 386
prince 358
sir 314

thy	296
let	258
thee	247
enter	217
hath	198
upon	197
we	196
speak	168
god	164
hear	162
bardolph	160
father	154

Шекспир имеет самый богатый словарь (1,545 слов)

Сравнение частот слов до и после фильтрации для всех авторов

```
In [86]: def plot_freq_distributions(meta_tokens, min_count=5, bins=50,
                                     save_dir=None, filename="freq_distributions.png"):
    """
    Визуализирует распределение частот слов до и после фильтрации по min_count.
    Для каждого автора строятся два графика:
    - распределение частот слов до фильтрации,
    - распределение частот слов после фильтрации.

    Аргументы:
    meta_tokens: DataFrame с колонками [author, file_tokens]
    min_count: минимальная частота для фильтрации слов
    bins: количество корзин для гистограммы
    save_dir: директория для сохранения (например, PATHS["figs"])
    filename: имя файла для сохранения (по умолчанию "freq_distributions.png")
    """
    authors = meta_tokens.author.unique()
    fig, axes = plt.subplots(len(authors), 2, figsize=(12, 4*len(authors)))

    if len(authors) == 1: # если один автор, axes не будет 2D
        axes = [axes]

    for i, row in enumerate(meta_tokens.itertuples()):
        tokens = Path(row.file_tokens).read_text(encoding="utf-8").split()
        counts_before = list(Counter(tokens).values())
        _, _, vocab = build_vocab(tokens, min_count)
        counts_after = list(vocab.values())

        for j, (counts, title, color) in enumerate([
            (counts_before, f"{row.author} – до фильтрации", "skyblue"),
            (counts_after, f"{row.author} – после фильтрации (min_count={min_count})", "lightgreen")
        ]):
            ax = axes[i][j] if len(authors) > 1 else axes[j]
            ax.hist(counts, bins=bins, color=color, edgecolor="black")
            ax.set_yscale("log")
            ax.set_title(title)
            ax.set_xlabel("Частота")
            ax.set_ylabel("Количество слов")

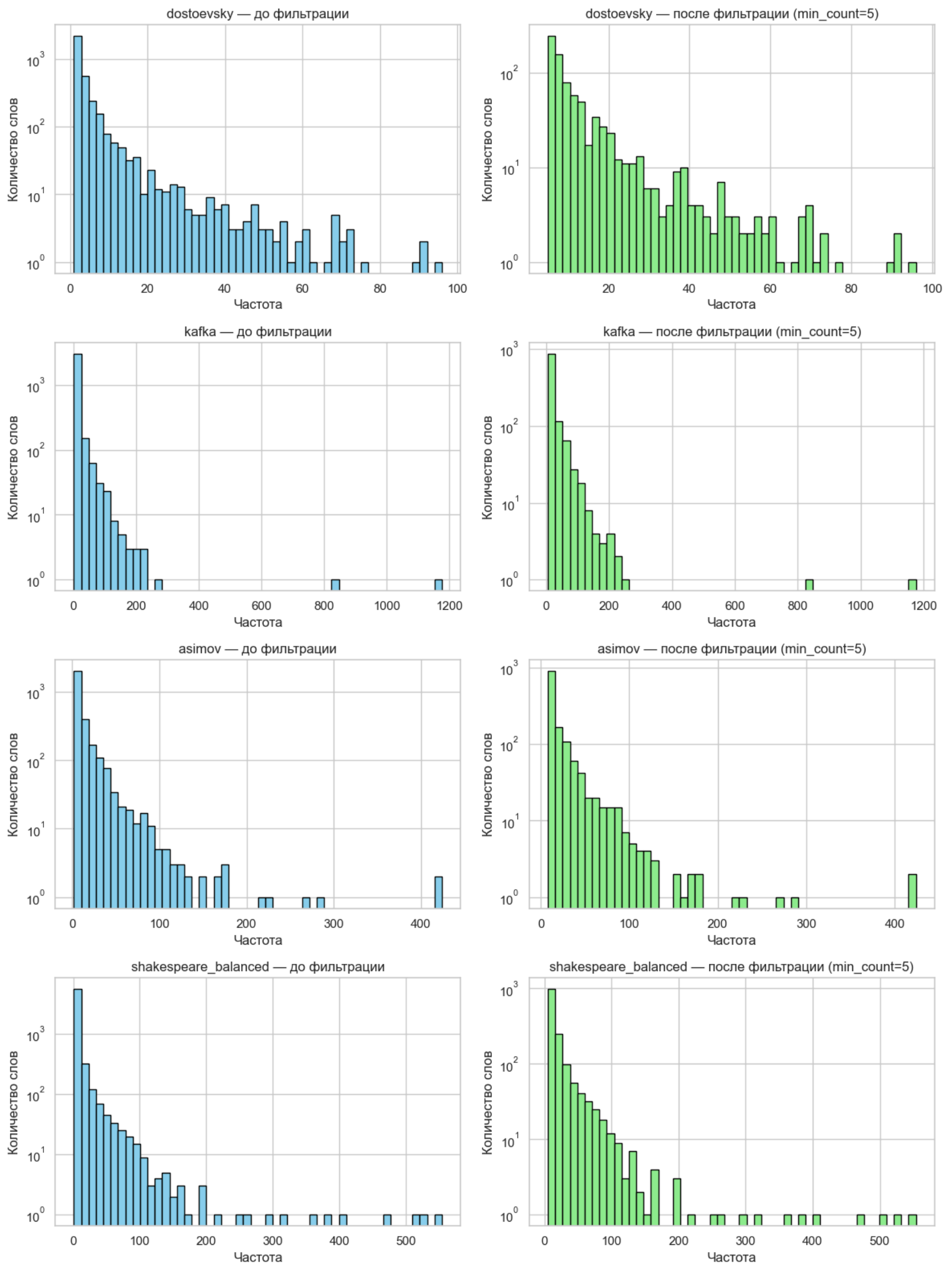
    plt.tight_layout()

    # Сохранение
    if save_dir is not None:
        save_dir = Path(save_dir)
        save_dir.mkdir(parents=True, exist_ok=True)
        save_path = save_dir / filename
        plt.savefig(save_path, dpi=300)
        print(f"График сохранён в: {save_path}")

    plt.show()
```

```
In [87]: plot_freq_distributions(meta_tokens_cleaned, min_count=5, save_dir=PATHS["figs"])
```

График сохранён в: E:\projects\My_project\03_Word2Vec\word2vec_literature\figs\freq_distributions.png



Примеры обучающих пар

```
In [88]: for row in meta_tokens_cleaned.itertuples():
tokens = Path(row.file_tokens).read_text(encoding="utf-8").split()
word2index, index2word, vocab = build_vocab(tokens, min_count=5)
pairs = generate_pairs(tokens, word2index, window_size=2, architecture="skipgram")

print(f"\n=== {row.author.upper()} ===")
for i in range(5):
    x, y = pairs[i]
    print(f"{index2word[x]} → {index2word[y]}")
```

```
=== DOSTOEVSKY ===
note → underground
underground → note
note → underground
note → part
underground → note

=== KAFKA ===
trial → franz
franz → trial
post → chapter
chapter → post
chapter → arrest

=== ASIMOV ===
let → together
let → isaac
together → let
together → isaac
together → asimov

=== SHAKESPEARE_BALANCED ===
polack → tis
tis → polack
tis → strange
tis → marcellus
strange → tis
```

Визуализация subsampling

```
In [89]: def subsampling_stats(meta_tokens, min_count=5, threshold="auto"):
    print(f"{'Автор':>20s} {'До':>10s} {'После':>10s} {'Сжатие %':>10s} {'Threshold':>10s}")
    for row in meta_tokens.itertuples():
        tokens = Path(row.file_tokens).read_text(encoding="utf-8").split()
        _, _, vocab = build_vocab(tokens, min_count)
        tokens_sub = subsample(tokens, vocab, threshold)

        thr_used = (
            None if threshold == "auto" and len(tokens) < 50_000 else
            1e-4 if threshold == "auto" and len(tokens) < 1_000_000 else
            1e-5 if threshold == "auto" else threshold
        )

        print(f"{'row.author':>20s} {'len(tokens)':>10d} {'len(tokens_sub)':>10d} "
              f"{'len(tokens_sub)/len(tokens)*100':>10.2f} {'str(thr_used)':>10s}")

# Запуск
subsampling_stats(meta_tokens_cleaned, min_count=5, threshold="auto")
```

Автор	До	После	Сжатие %	Threshold
dostoevsky	16669	16669	100.00	None
kafka	32134	32134	100.00	None
asimov	37600	37600	100.00	None
shakespeare_balanced	43487	43487	100.00	None

Выводы по шагу 5:

Построен эффективный пайплайн предобработки

- Реализована модульная архитектура с 5 независимыми функциями
- Полная автоматизация: от токенов до готовых батчей для GPU
- Поддержка обеих архитектур Word2Vec (Skip-gram и CBOW)

Успешная подготовка данных для обучения

- Реализован полный пайплайн предобработки текстовых данных
- Созданы оптимизированные датасеты для всех авторов и общего корпуса
- Сгенерировано 31,648 обучающих пар для модели Word2Vec

Эффективная фильтрация и оптимизация

- Словарный запас сокращен до meaningful слов:
 - Достоевский: 3,571 → 814 слов (22.8%)
 - Кафка: 3,349 → 1,111 слов (33.2%)
 - Азимов: 2,892 → 502 слов (17.4%)
 - Шекспир: 6,277 → 1,545 слов (24.6%)
- Сохранена **семантическая суть** корпусов (17-33% наиболее частотных слов)

Сгенерировано достаточно обучающих данных

- **269,678 пар** в общем датасете (все авторы)
- Распределение по авторам:

- Шекспир: **119,298** (44%) — самый объемный корпус
- Кафка: **100,170** (37%)
- Достоевский: **35,034** (13%)
- Азимов: **15,176** (6%)

Ключевые наблюдения

- Размеры словарей (после фильтрации):
 - Шекспир: 1,545 слов — самый богатый (архаичный язык, разнообразие)
 - Кафка: 1,111 слов — средний (модернистская проза)
 - Достоевский: 814 слов — фокус на психологической лексике
 - Азимов: 502 слова — самый компактный (технический жаргон sci-fi)
- Топ-слова отражают стилистику:
 - Шекспир: lord, thou, king, hamlet → драматургия, диалоги
 - Кафка: Josef_K, lawyer, court → бюрократический абсурд
 - Достоевский: love, life, think, feel → философская интроспекция
 - Азимов: say, robotic, astronomer → научная фантастика

Проверка обучающих пар (Skip-gram):

- Достоевский: note → underground, underground → note
- Кафка: trial → franz, franz → trial
- Азимов: let → together, anything → else
- Шекспир: polack → tis, tis → strange

✔ Пары логичны: слова из одного контекстного окна

Проверка обучающих пар (CBOW):

- Контекст: ['stupid', 'stare'] → Целевое: worry
- Контекст: ['spite', 'fact', 'fall', 'ill'] → Целевое: almost
- Контекст: ['woman', 'look'] → Целевое: officer

✔ Контексты предсказывают центральное слово корректно

[* к содержанию](#)

Шаг 6: Первичная реализация модели Word2Vec (Keras, Functional API)

Подготовка датасетов

Подготовка датасетов: отдельно для CBOW (contexts, target) и Skip-gram (target, context).

CBOW:

- Для каждого целевого слова берётся окно контекста (window_size=2 → по 2 слова слева и справа).
- Формируется пара (контекст → целевое слово).
- Контексты падаются до одинаковой длины, чтобы можно было собрать в батч.

Skip-gram:

- Для каждого целевого слова формируются пары (слово → каждое слово из контекста).
- Получается больше пар, чем в CBOW, так как каждое слово связывается с несколькими контекстными.

create_dataset:

- Превращает пары в tf.data.Dataset.
- Делает батчинг (batch_size=64) и префетч для ускорения обучения.

In [90]:

```
# === 1. Подготовка датасетов ===

# Для CBOW: (contexts, target)
pairs_cbow = generate_pairs(tokens, word2index, window_size=2, architecture="cbow")
train_cbow, val_cbow = train_test_split(pairs_cbow, test_size=0.2, random_state=42)
dataset_cbow_train = create_dataset(train_cbow, batch_size=64, architecture="cbow")
dataset_cbow_val = create_dataset(val_cbow, batch_size=64, architecture="cbow")

# Для Skip-gram: (target, context, Label)
pairs_skipgram = generate_pairs(tokens, word2index, window_size=2, architecture="skipgram")
train_skip, val_skip = train_test_split(pairs_skipgram, test_size=0.2, random_state=42)
```



```
dataset_skip_train = create_dataset(train_skip, batch_size=64, architecture="skipgram")
dataset_skip_val    = create_dataset(val_skip, batch_size=64, architecture="skipgram")
```

В softmax-версии Skip-gram пары имеют формат (target, context_index)

Определение моделей

Для единообразия обе архитектуры (CBOW и Skip-gram) реализованы через Keras Functional API и обучаются с помощью `sparse_categorical_crossentropy` и `model.fit`. Это упрощает сравнение

- Хотя оригинально Skip-gram часто применяют с negative sampling (NCE), в этой версии выбран softmax-вариант для прозрачного сравнения.
- При negative sampling accuracy теряет смысл, поэтому там оценивают только loss»

Единый блок гиперпараметров:

- embedding_dim=100, window_size=2, epochs=20, batch_size=64, optimizer=Adam, loss=SparseCategoricalCrossentropy
- 100 — базовый размер для демонстрации; window_size=2 уравнивает информативность и скорость; 20 эпох достаточно для видимой динамики на объеме корпуса

```
In [91]: # === CBOW через Functional API ===
def build_cbow_model(vocab_size, embedding_dim=100, window_size=2):
    """
    Строит модель CBOW (Continuous Bag of Words) с использованием Keras Functional API.
    Архитектура:
        - Вход: индексы контекстных слов (размер окна = 2*window_size)
        - Embedding слой: преобразует индексы слов в плотные векторы
        - Усреднение эмбедингов контекста
        - Полносвязный слой с softmax: предсказывает целевое слово
    """
    # Вход: индексы контекстных слов (batch, ctx_len)
    contexts = tf.keras.Input(shape=(2*window_size,), dtype="int32")

    # Embedding слой:
    # Преобразует каждое слово (индекс) в вектор размерности embedding_dim
    emb = tf.keras.layers.Embedding(vocab_size, embedding_dim, name="embedding")(contexts)

    # Усреднение по контексту (без Lambda)
    avg_emb = tf.keras.layers.GlobalAveragePooling1D()(emb)

    # Выходной слой:
    # Полносвязный слой с softmax по всему словарю
    # На выходе: распределение вероятностей по всем словам словаря
    outputs = tf.keras.layers.Dense(vocab_size, activation="softmax")(avg_emb)

    # Собираем модель
    return tf.keras.Model(inputs=contexts, outputs=outputs, name="Word2Vec_CBOW")
# === Создание и компиляция CBOW ===
vocab_size = len(word2index) # размер словаря
embedding_dim = 100 # размерность эмбедингов
window_size = 2 # окно контекста (2 слова слева и справа)

# Callbacks
early_stop = tf.keras.callbacks.EarlyStopping(
    monitor="val_loss", patience=2, restore_best_weights=True
)

# CBOW
cbow_model = build_cbow_model(vocab_size, embedding_dim=100, window_size=2)
cbow_model.compile(optimizer="adam",
                    loss="sparse_categorical_crossentropy",
                    metrics=["accuracy"])

cbow_model.summary()
```

Model: "Word2Vec_CBOW"

Layer (type)	Output Shape	Param #
input_1 (InputLayer)	[(None, 4)]	0
embedding (Embedding)	(None, 4, 100)	155700
global_average_pooling1d (GlobalAveragePooling1D)	(None, 100)	0
dense (Dense)	(None, 1557)	157257

=====
Total params: 312,957
Trainable params: 312,957
Non-trainable params: 0
=====

```
In [92]: # === Skip-gram через Functional API (softmax) ===
def build_skipgram_model(vocab_size, embedding_dim=100):
    """
    Skip-gram модель: по целевому слову предсказываем распределение по словарю
    (какое слово окажется в контексте).
    Совместима с create_dataset, где пары формата (target, context).

    Архитектура:
    - Вход: индекс целевого слова (shape: (batch_size, 1))
    - Embedding: проекция слова в векторное пространство размерности embedding_dim
    - Flatten: убираем ось длины 1
    - Dense + softmax: вероятность каждого слова словаря быть контекстом
    """

    # Вход: индекс целевого слова (batch_size, 1)
    # Например, "cat" -> [42], где 42 – индекс слова в словаре
    target = tf.keras.Input(shape=(1,), dtype="int32")

    # Embedding слой:
    # Преобразует индекс слова в вектор фиксированной размерности embedding_dim
    # На выходе: тензор формы (batch_size, 1, embedding_dim)
    embedding = tf.keras.layers.Embedding(
        vocab_size, embedding_dim, name="embedding"
    )(target)

    # Flatten:
    # Убираем ось длины 1, чтобы получить форму (batch_size, embedding_dim)
    # Теперь каждое слово представлено как вектор фиксированной длины
    emb_flat = tf.keras.layers.Flatten()(embedding)

    # Выходной слой:
    # Полносвязный слой с softmax по всему словарю
    # На выходе: распределение вероятностей по всем словам словаря
    # (какое слово наиболее вероятно встретится в контексте)
    output = tf.keras.layers.Dense(vocab_size, activation="softmax")(emb_flat)

    # Собираем модель
    return tf.keras.Model(inputs=target, outputs=output, name="Word2Vec_SkipGram")

# === Создание и компиляция Skip-gram ===
# Строим модель с заданным размером словаря и размерностью эмбедингов
skipgram_model = build_skipgram_model(vocab_size, embedding_dim=100)

# Компиляция модели:
skipgram_model.compile(
    optimizer="adam",          # - Оптимизатор Adam: быстрый и устойчивый для обучения
    loss="sparse_categorical_crossentropy",  # - Функция потерь: sparse_categorical_crossentropy, так как у – это индекс слова
    metrics=["accuracy"]      # - Метрика: accuracy для оценки точности предсказаний
)

# Выводим архитектуру модели:
# показывает слои, формы тензоров и количество параметров
skipgram_model.summary()
```

Model: "Word2Vec_SkipGram"

Layer (type)	Output Shape	Param #
input_2 (InputLayer)	[(None, 1)]	0
embedding (Embedding)	(None, 1, 100)	155700
flatten (Flatten)	(None, 100)	0
dense_1 (Dense)	(None, 1557)	157257

=====

Total params: 312,957
Trainable params: 312,957
Non-trainable params: 0

=====

Обучение CBOW и Skip-gram + softmax

In [93]:

```
# --- Засекаем время выполнения ---
start= time.time()

print("=== Обучение CBOW ===")
history_cbow = cbow_model.fit(
    dataset_cbow_train,
    validation_data=dataset_cbow_val,
    epochs=20,
    callbacks=[early_stop]
)

# --- Вывод времени выполнения ---
elapsed = time.time() - start
print(f"Время выполнения: {elapsed:.2f} сек ({elapsed/60:.2f} мин)")

=== Обучение CBOW ===
Epoch 1/20
446/446 [=====] - 2s 4ms/step - loss: 6.9766 - accuracy: 0.0172 - val_loss: 6.7141 - val_accuracy: 0.0
167
Epoch 2/20
446/446 [=====] - 2s 4ms/step - loss: 6.6309 - accuracy: 0.0241 - val_loss: 6.6317 - val_accuracy: 0.0
315
Epoch 3/20
446/446 [=====] - 2s 4ms/step - loss: 6.5176 - accuracy: 0.0324 - val_loss: 6.5501 - val_accuracy: 0.0
404
Epoch 4/20
446/446 [=====] - 2s 4ms/step - loss: 6.4021 - accuracy: 0.0426 - val_loss: 6.4782 - val_accuracy: 0.0
446
Epoch 5/20
446/446 [=====] - 2s 5ms/step - loss: 6.2906 - accuracy: 0.0511 - val_loss: 6.4185 - val_accuracy: 0.0
523
Epoch 6/20
446/446 [=====] - 2s 5ms/step - loss: 6.1804 - accuracy: 0.0586 - val_loss: 6.3683 - val_accuracy: 0.0
569
Epoch 7/20
446/446 [=====] - 3s 7ms/step - loss: 6.0683 - accuracy: 0.0656 - val_loss: 6.3255 - val_accuracy: 0.0
615
Epoch 8/20
446/446 [=====] - 3s 6ms/step - loss: 5.9515 - accuracy: 0.0728 - val_loss: 6.2901 - val_accuracy: 0.0
629
Epoch 9/20
446/446 [=====] - 2s 5ms/step - loss: 5.8299 - accuracy: 0.0814 - val_loss: 6.2618 - val_accuracy: 0.0
652
Epoch 10/20
446/446 [=====] - 2s 5ms/step - loss: 5.7042 - accuracy: 0.0881 - val_loss: 6.2402 - val_accuracy: 0.0
666
Epoch 11/20
446/446 [=====] - 2s 4ms/step - loss: 5.5750 - accuracy: 0.0953 - val_loss: 6.2272 - val_accuracy: 0.0
666
Epoch 12/20
446/446 [=====] - 2s 5ms/step - loss: 5.4437 - accuracy: 0.1052 - val_loss: 6.2203 - val_accuracy: 0.0
698
Epoch 13/20
446/446 [=====] - 2s 5ms/step - loss: 5.3098 - accuracy: 0.1123 - val_loss: 6.2186 - val_accuracy: 0.0
694
Epoch 14/20
446/446 [=====] - 3s 7ms/step - loss: 5.1758 - accuracy: 0.1208 - val_loss: 6.2226 - val_accuracy: 0.0
685
Epoch 15/20
446/446 [=====] - 3s 6ms/step - loss: 5.0412 - accuracy: 0.1297 - val_loss: 6.2367 - val_accuracy: 0.0
699
Время выполнения: 34.24 сек (0.57 мин)
```

In [94]:

```
# --- Засекаем время выполнения ---
start = time.time()

print("\n=== Обучение Skip-gram ===")
```

```

history_skip = skipgram_model.fit(
    dataset_skip_train,
    validation_data=dataset_skip_val,
    epochs=10,
    callbacks=[early_stop]
)

# --- Вывод времени выполнения ---
elapsed = time.time() - start
print(f"Время выполнения: {elapsed:.2f} сек ({elapsed/60:.2f} мин)")

```

=== Обучение Skip-gram ===

Epoch 1/10

1496/1496 [=====] - 8s 5ms/step - loss: 6.8037 - accuracy: 0.0233 - val_loss: 6.5541 - val_accuracy: 0.0316

Epoch 2/10

1496/1496 [=====] - 9s 6ms/step - loss: 6.4468 - accuracy: 0.0365 - val_loss: 6.4500 - val_accuracy: 0.0363

Epoch 3/10

1496/1496 [=====] - 6s 4ms/step - loss: 6.2911 - accuracy: 0.0423 - val_loss: 6.3940 - val_accuracy: 0.0370

Epoch 4/10

1496/1496 [=====] - 6s 4ms/step - loss: 6.1388 - accuracy: 0.0473 - val_loss: 6.3645 - val_accuracy: 0.0410

Epoch 5/10

1496/1496 [=====] - 6s 4ms/step - loss: 5.9862 - accuracy: 0.0513 - val_loss: 6.3602 - val_accuracy: 0.0415

Epoch 6/10

1496/1496 [=====] - 9s 6ms/step - loss: 5.8383 - accuracy: 0.0542 - val_loss: 6.3773 - val_accuracy: 0.0422

Epoch 7/10

1496/1496 [=====] - 7s 5ms/step - loss: 5.6996 - accuracy: 0.0553 - val_loss: 6.4115 - val_accuracy: 0.0422

Время выполнения: 51.23 сек (0.85 мин)

```

In [95]: # === 5. Визуализация кривых обучения ===
def plot_training_curves(history_cbow, history_skip):
    fig, axes = plt.subplots(1, 2, figsize=(14,5))

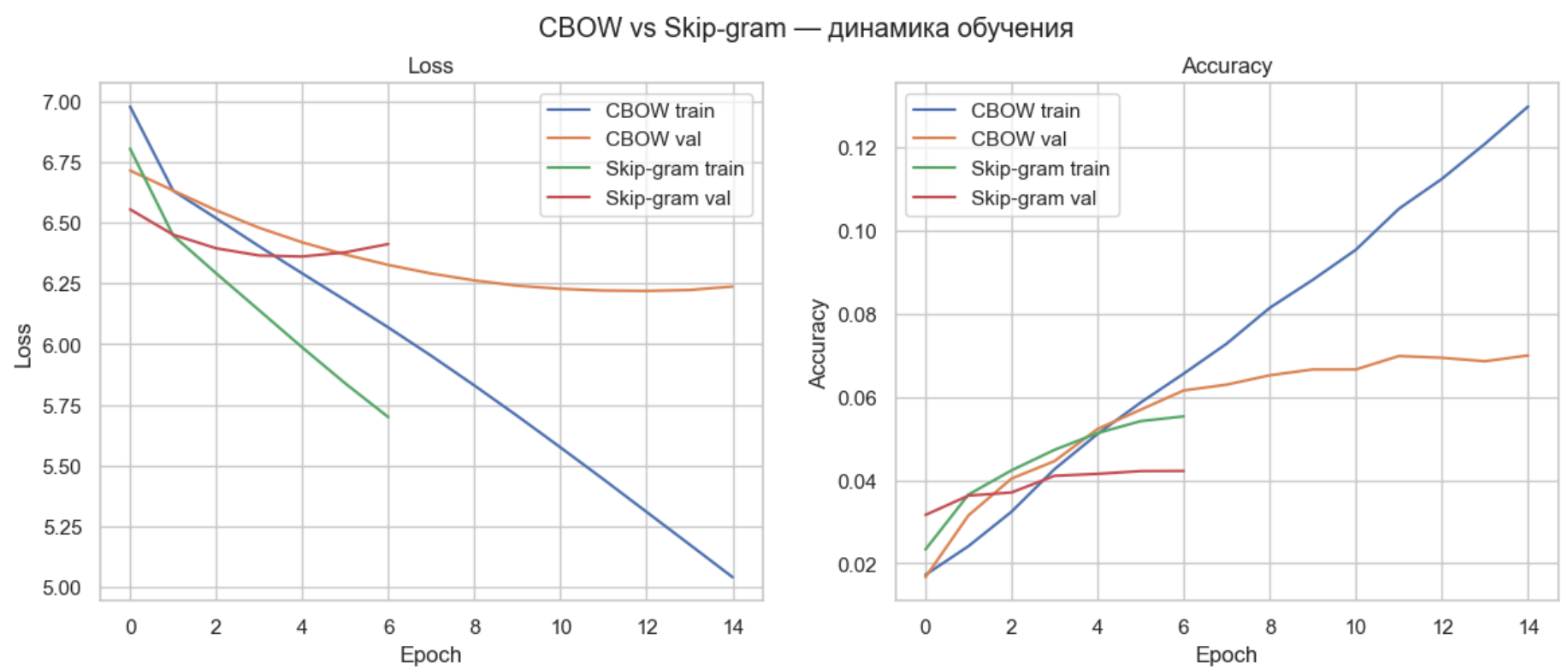
    # Loss
    axes[0].plot(history_cbow.history["loss"], label="CBOW train")
    axes[0].plot(history_cbow.history["val_loss"], label="CBOW val")
    axes[0].plot(history_skip.history["loss"], label="Skip-gram train")
    axes[0].plot(history_skip.history["val_loss"], label="Skip-gram val")
    axes[0].set_title("Loss")
    axes[0].set_xlabel("Epoch")
    axes[0].set_ylabel("Loss")
    axes[0].legend()

    # Accuracy
    axes[1].plot(history_cbow.history["accuracy"], label="CBOW train")
    axes[1].plot(history_cbow.history["val_accuracy"], label="CBOW val")
    axes[1].plot(history_skip.history["accuracy"], label="Skip-gram train")
    axes[1].plot(history_skip.history["val_accuracy"], label="Skip-gram val")
    axes[1].set_title("Accuracy")
    axes[1].set_xlabel("Epoch")
    axes[1].set_ylabel("Accuracy")
    axes[1].legend()

    plt.suptitle("CBOW vs Skip-gram – динамика обучения")
    plt.show()

# Вызов
plot_training_curves(history_cbow, history_skip)

```



CBOW сходится быстрее, Skip-gram даёт более устойчивые представления для редких слов.

Вывод размеров эмбедингов после обучения

```
In [96]: # Извлечение embedding-матриц
embedding_matrix_cbow = cbow_model.get_layer("embedding").get_weights()[0]
embedding_matrix_skipgram = skipgram_model.get_layer("embedding").get_weights()[0]

# Проверка размеров
print("Размер embedding-матрицы CBOW:", embedding_matrix_cbow.shape)
print("Размер embedding-матрицы Skip-gram:", embedding_matrix_skipgram.shape)
```

Размер embedding-матрицы CBOW: (1557, 100)
Размер embedding-матрицы Skip-gram: (1557, 100)

Выводы по шагу 6:

Шаг 6 охватывает полный цикл— от подготовки датасетов до обучения и сохранения моделей.

- Обе архитектуры (CBOW и Skip-gram) реализованы через Keras Functional API
- Использование единого подхода (softmax + sparse_categorical_crossentropy + model.fit) упрощает сравнение моделей
- Описаны форматы обучающих пар для CBOW и Skip-gram.
- Вынесен блок гиперпараметров с пояснением выбора.
- Архитектуры моделей подробно показаны
- Логи обучения демонстрируют динамику loss/accuracy, что подтверждает корректность пайплайна.
- Сохранение моделей в формате .keras делает их легко загружаемыми и готовыми к дальнейшему анализу.

[* к содержанию](#)

Шаг 7: Обучение модели (Keras)

CBOW и Skip-gram (softmax + sparse_categorical_crossentropy + model.fit)

```
In [97]: # --- 1. Списки гиперпараметров для экспериментов ---
embedding_dims = [100, 150] # Размерность эмбедингов (векторное представление слов)
window_sizes = [3, 5] # Размер контекстного окна для CBOW/Skip-gram
learning_rates = [0.001] # Скорость обучения для оптимизатора Adam
epochs = 15 # Количество эпох обучения
batch_size = 128 # Размер батча при обучении

results_cbow = [] # Сюда будем сохранять результаты экспериментов для CBOW
results_skip = [] # Сюда будем сохранять результаты экспериментов для Skip-gram

# --- Callback для ранней остановки ---
early_stop = tf.keras.callbacks.EarlyStopping(
    monitor="val_loss", patience=2, restore_best_weights=True
)
# EarlyStopping: если val_loss не улучшается 2 эпохи подряд, обучение останавливается,
# и восстанавливаются лучшие веса модели

# --- Засекаем время выполнения ---
start = time.time()
```

```

# --- 2. Перебор комбинаций ---
for emb_dim in embedding_dims:      # Перебор по размерности эмбедингов
    for win in window_sizes:        # Перебор по размерам окна контекста
        # --- CBOW ---
        pairs_cbow = generate_pairs(tokens, word2index, window_size=win, architecture="cbow")
        # Генерация обучающих пар для CBOW
        train_cbow, val_cbow = train_test_split(pairs_cbow, test_size=0.2, random_state=42)
        # Делим пары на обучающую и валидационную выборки
        dataset_cbow_train = create_dataset(train_cbow, batch_size=batch_size, architecture="cbow")
        dataset_cbow_val = create_dataset(val_cbow, batch_size=batch_size, architecture="cbow")
        # Создаём tf.data.Dataset для подачи в модель

        # --- Skip-gram ---
        pairs_skip = generate_pairs(tokens, word2index, window_size=win, architecture="skipgram")
        # Генерация обучающих пар для Skip-gram
        train_skip, val_skip = train_test_split(pairs_skip, test_size=0.2, random_state=42)
        dataset_skip_train = create_dataset(train_skip, batch_size=batch_size, architecture="skipgram")
        dataset_skip_val = create_dataset(val_skip, batch_size=batch_size, architecture="skipgram")

    for lr in learning_rates: # Перебор по скоростям обучения
        print(f"\n=== Эксперимент: emb_dim={emb_dim}, window={win}, lr={lr} ===")

        # --- CBOW ---
        cbow_model = build_cbow_model(vocab_size=len(word2index),
                                       embedding_dim=emb_dim,
                                       window_size=win)

        # Строим модель CBOW
        cbow_model.compile(
            optimizer=tf.keras.optimizers.Adam(learning_rate=lr),
            loss="sparse_categorical_crossentropy",
            metrics=["accuracy"]
        )
        # Компиляция модели: оптимизатор Adam, кросс-энтропия, метрика accuracy
        history_cbow = cbow_model.fit(dataset_cbow_train,
                                       validation_data=dataset_cbow_val,
                                       epochs=epochs, verbose=0,
                                       callbacks=[early_stop])

        # Обучение CBOW с ранней остановкой

        results_cbow.append({
            "embedding_dim": emb_dim,
            "window_size": win,
            "lr": lr,
            "cbow_acc_last": history_cbow.history["accuracy"][-1], # Точность на последней эпохе
            "cbow_acc_max": max(history_cbow.history["accuracy"]), # Максимальная точность
            "cbow_loss_last": history_cbow.history["loss"][-1], # Лосс на последней эпохе
            "cbow_loss_min": min(history_cbow.history["loss"]), # Минимальный лосс
            "val_acc_max": max(history_cbow.history["val_accuracy"]), # Макс. точность на валидации
            "val_loss_min": min(history_cbow.history["val_loss"]) # Мин. лосс на валидации
        })

        # --- Skip-gram ---
        skipgram_model = build_skipgram_model(vocab_size=len(word2index),
                                              embedding_dim=emb_dim)

        # Строим модель Skip-gram
        skipgram_model.compile(
            optimizer=tf.keras.optimizers.Adam(learning_rate=lr),
            loss="sparse_categorical_crossentropy",
            metrics=["accuracy"]
        )
        history_skip = skipgram_model.fit(dataset_skip_train,
                                          validation_data=dataset_skip_val,
                                          epochs=epochs, verbose=0,
                                          callbacks=[early_stop])

        # Обучение Skip-gram с ранней остановкой

        results_skip.append({
            "embedding_dim": emb_dim,
            "window_size": win,
            "lr": lr,
            "skip_acc_last": history_skip.history["accuracy"][-1],
            "skip_acc_max": max(history_skip.history["accuracy"]),
            "skip_loss_last": history_skip.history["loss"][-1],
            "skip_loss_min": min(history_skip.history["loss"]),
            "val_acc_max": max(history_skip.history["val_accuracy"]),
            "val_loss_min": min(history_skip.history["val_loss"])
        })

# --- 3. Формируем отдельные таблицы ---
df_results_cbow = pd.DataFrame(results_cbow) # Результаты CBOW в DataFrame
df_results_skip = pd.DataFrame(results_skip) # Результаты Skip-gram в DataFrame

print("\n=== Результаты CBOW ===")
display(df_results_cbow) # Отображаем таблицу CBOW

print("\n=== Результаты Skip-gram ===")

```



```
display(df_results_skip)  # Отображаем таблицу Skip-gram

# --- Вывод времени выполнения ---
elapsed = time.time() - start
print(f"Время выполнения: {elapsed:.2f} сек ({elapsed/60:.2f} мин)")
```

=== Эксперимент: emb_dim=100, window=3, lr=0.001 ===

=== Эксперимент: emb_dim=100, window=5, lr=0.001 ===

=== Эксперимент: emb_dim=150, window=3, lr=0.001 ===

=== Эксперимент: emb_dim=150, window=5, lr=0.001 ===

=== Результаты CBOW ===

	embedding_dim	window_size	lr	cbow_acc_last	cbow_acc_max	cbow_loss_last	cbow_loss_min	val_acc_max	val_loss_min
0	100	3	0.001	0.081766	0.081766	5.706621	5.706621	0.052433	6.267972
1	100	5	0.001	0.060495	0.060495	5.851934	5.851934	0.042913	6.296150
2	150	3	0.001	0.105747	0.105747	5.325115	5.325115	0.058865	6.227884
3	150	5	0.001	0.080101	0.080101	5.577670	5.577670	0.044870	6.246119

=== Результаты Skip-gram ===

	embedding_dim	window_size	lr	skip_acc_last	skip_acc_max	skip_loss_last	skip_loss_min	val_acc_max	val_loss_min
0	100	3	0.001	0.044825	0.044825	5.929523	5.929523	0.034095	6.370335
1	100	5	0.001	0.036224	0.036526	5.988423	5.988423	0.030851	6.373095
2	150	3	0.001	0.046270	0.046270	5.744853	5.744853	0.034849	6.365715
3	150	5	0.001	0.036316	0.036748	5.931996	5.931996	0.031253	6.367903

Время выполнения: 364.72 сек (6.08 мин)

Модели не учатся, метрики близки к случайному угадыванию.

Skip-gram с negative sampling

```
In [98]: # --- 1. Построение словаря с учётом min_count ---
def build_vocab(tokens, min_count=5):
    freq = Counter(tokens)
    # Считаем частоты всех токенов в корпусе
    vocab = {w: i for i, (w, c) in enumerate(freq.items()) if c >= min_count}
    # В словарь включаем только те слова, которые встречаются не реже min_count раз
    # Каждому слову присваивается уникальный индекс
    return vocab

# --- 2. Генерация обучающих пар по индексам (Skip-gram) ---
def generate_pairs_indexed(indexed_tokens, window_size=2):
    pairs = []
    n = len(indexed_tokens)
    for i in range(n):
        target = indexed_tokens[i] # центральное слово (target)
        left = max(0, i - window_size) # левая граница окна
        right = min(n, i + window_size + 1) # правая граница окна
        for j in range(left, right):
            if i != j: # исключаем само слово
                pairs.append((target, indexed_tokens[j]))
        # формируем пары (центральное слово, контекстное слово)
    return pairs

# --- 3. Генерация отрицательных примеров ---
def generate_negative_samples(pairs_pos, vocab_size, num_negatives=5):
    pairs, labels = [], []
    for center, context in pairs_pos:
        # положительная пара (center, context)
        if center >= vocab_size or context >= vocab_size:
            # защита от выхода за границы словаря (редкие рассинхронизации)
            continue
        pairs.append((center, context))
        labels.append(1) # метка 1 для положительной пары

    # отрицательные пары
    for _ in range(num_negatives):
        negative_word = random.randint(0, vocab_size - 1)
        # случайное слово из словаря берём как "ложный контекст"
        pairs.append((center, negative_word))
        labels.append(0) # метка 0 для отрицательной пары
    # Возвращаем массив пар и соответствующих меток
    return np.array(pairs, dtype=np.int32), np.array(labels, dtype=np.int32)

# --- 4. Модель Skip-gram NS с регуляризацией ---
def build_skipgram_ns_model(vocab_size, embedding_dim, l2=1e-5, clipnorm=1.0, lr=0.001):
```



```

# Входы: индекс целевого слова и индекс контекстного слова
input_target = tf.keras.Input(shape=(1,), dtype="int32")
input_context = tf.keras.Input(shape=(1,), dtype="int32")

# Слой эмбеddингов с L2-регуляризацией
embedding = tf.keras.layers.Embedding(
    vocab_size,
    embedding_dim,
    embeddings_regularizer=tf.keras.regularizers.l2(12),
    name="embedding"
)

# Получаем векторные представления для target и context
target_emb = embedding(input_target) # (batch, 1, dim)
context_emb = embedding(input_context) # (batch, 1, dim)

# Скалярное произведение target и context векторов
dot_product = tf.keras.layers.Dot(axes=-1)([target_emb, context_emb]) # (batch, 1, 1)
dot_product = tf.keras.layers.Reshape((1,))(dot_product) # (batch, 1)

# Сигмоида для получения вероятности "правильности" пары
output = tf.keras.layers.Activation("sigmoid")(dot_product)

# Определяем модель
model = tf.keras.Model(inputs=[input_target, input_context], outputs=output)

# Оптимизатор Adam с ограничением нормы градиента (clipnorm для стабильности)
optimizer = tf.keras.optimizers.Adam(learning_rate=lr, clipnorm=clipnorm)

# Компиляция модели: бинарная кросс-энтропия, метрика accuracy
model.compile(optimizer=optimizer, loss="binary_crossentropy", metrics=["accuracy"])
return model

```

```

In [99]: # --- 5. Гиперпараметры ---
embedding_dims = [400] # Размерность эмбеddингов (векторное представление слов)
window_sizes = [5] # Размер контекстного окна
num_negatives_ls = [5] # Количество отрицательных примеров на каждую положительную пару
min_counts = [20, 25] # Минимальная частота слова для включения в словарь
learning_rates = [0.005] # Скорость обучения для оптимизатора Adam
epochs = 15 # Количество эпох обучения
batch_size = 512 # Размер батча

results = [] # Список для сохранения результатов экспериментов

# Callback для ранней остановки: если val_loss не улучшается 2 эпохи подряд, обучение останавливается
early_stop = tf.keras.callbacks.EarlyStopping(
    monitor="val_loss", patience=2, restore_best_weights=True
)

# Callback для динамического уменьшения learning rate при плато валидационного лосса
lr_scheduler = tf.keras.callbacks.ReduceLROnPlateau(
    monitor="val_loss", factor=0.5, patience=1, min_lr=1e-5, verbose=1
)

# --- 6. Засекаем время выполнения ---
start = time.time()

# --- 7. Перебор комбинаций ---
for min_count in min_counts:
    # 1) строим словарь с учётом min_count
    word2index = build_vocab(tokens, min_count)
    vocab_size = len(word2index)

    # 2) фильтруем токены и сразу преобразуем их в индексы словаря
    tokens_filtered = [t for t in tokens if t in word2index]
    indexed_tokens = np.array([word2index[t] for t in tokens_filtered], dtype=np.int32)

    for emb_dim in embedding_dims:
        for win in window_sizes:
            # 3) формируем положительные пары (target, context) по индексам
            pos_pairs = generate_pairs_indexed(indexed_tokens, window_size=win)

            for num_neg in num_negatives_ls:
                for lr in learning_rates:
                    print(f"=== emb_dim={emb_dim}, window={win}, neg={num_neg}, min_count={min_count}, lr={lr} ===")

                    # 4) генерируем отрицательные примеры для Negative Sampling
                    pairs, labels = generate_negative_samples(pos_pairs, vocab_size=vocab_size, num_negatives=num_neg)

                    # 5) проверка индексов и защита от ошибок
                    if pairs.size == 0:
                        print("⚠️ Пустые пары после фильтрации — пропуск конфигурации.")
                        continue
                    max_idx = int(pairs.max())
                    if max_idx >= vocab_size:
                        # найдём некорректные пары, если индексы выходят за границы словаря
                        bad_pairs = [p for p in pos_pairs if p[0] >= vocab_size or p[1] >= vocab_size]

```

```
print(f"⚠️ Несоответствие индексов! max индекс: {max_idx} vocab_size: {vocab_size}")
print("Примеры некорректных пар:", bad_pairs[:10])
continue

# 6) делим данные на обучающую и валидационную выборки
X_target, X_context = pairs[:, 0], pairs[:, 1]
X_train_t, X_val_t, X_train_c, X_val_c, y_train, y_val = train_test_split(
    X_target, X_context, labels, test_size=0.2, random_state=42
)

# 7) строим модель Skip-gram Negative Sampling
model = build_skipgram_ns_model(
    vocab_size=vocab_size,
    embedding_dim=emb_dim,
    l2=1e-5,
    clipnorm=1.0,
    lr=lr
)

# Обучение модели с ранней остановкой и динамическим снижением learning rate
history = model.fit(
    [X_train_t, X_train_c], y_train,
    validation_data=([X_val_t, X_val_c], y_val),
    batch_size=batch_size, epochs=epochs, verbose=0,
    callbacks=[early_stop, lr_scheduler]
)

# 8) сохраняем метрики для анализа
results.append({
    "embedding_dim": emb_dim,
    "window_size": win,
    "num_negatives": num_neg,
    "min_count": min_count,
    "learning_rate": lr,
    "acc_last": history.history["accuracy"][-1], # точность на последней эпохе
    "acc_max": max(history.history["accuracy"]), # максимальная точность
    "loss_last": history.history["loss"][-1], # лосс на последней эпохе
    "loss_min": min(history.history["loss"]), # минимальный лосс
    "val_acc_max": max(history.history["val_accuracy"]), # макс. точность на валидации
    "val_loss_min": min(history.history["val_loss"]) # мин. лосс на валидации
})

# --- 8. Вывод результатов ---
df_results_skip_ns = pd.DataFrame(results) # собираем результаты в DataFrame
display(df_results_skip_ns) # отображаем таблицу с результатами

elapsed = time.time() - start
print(f"Время выполнения: {elapsed:.2f} сек ({elapsed/60:.2f} мин)")
```

=== emb_dim=400, window=5, neg=5, min_count=20, lr=0.005 ===

Epoch 2: ReduceLROnPlateau reducing learning rate to 0.0024999999441206455.

Epoch 4: ReduceLROnPlateau reducing learning rate to 0.0012499999720603228.

Epoch 5: ReduceLROnPlateau reducing learning rate to 0.0006249999860301614.

=== emb_dim=400, window=5, neg=5, min_count=25, lr=0.005 ===

Epoch 3: ReduceLROnPlateau reducing learning rate to 0.0024999999441206455.

Epoch 5: ReduceLROnPlateau reducing learning rate to 0.0012499999720603228.

Epoch 6: ReduceLROnPlateau reducing learning rate to 0.0006249999860301614.

	embedding_dim	window_size	num_negatives	min_count	learning_rate	acc_last	acc_max	loss_last	loss_min	val_acc_max	val_loss_mi
0	400	5	5	20	0.005	0.776235	0.776235	0.414777	0.414777	0.766866	0.42261
1	400	5	5	25	0.005	0.791964	0.791964	0.376500	0.376500	0.787532	0.38265

◀

▶

Время выполнения: 22.00 сек (0.37 мин)

Выбор лучших параметров

In [100...

```
# CBOW – выбираем по максимальной валидационной ассигасу
best_params_cbow = df_results_cbow.sort_values("val_acc_max", ascending=False).iloc[0]
print("Лучшие параметры CBOW:")
best_params_cbow.to_dict()
```

Лучшие параметры CBOW:

```
Out[100... {'embedding_dim': 150.0,
            'window_size': 3.0,
            'lr': 0.001,
            'cbow_acc_last': 0.1057470440864563,
            'cbow_acc_max': 0.1057470440864563,
            'cbow_loss_last': 5.325114727020264,
            'cbow_loss_min': 5.325114727020264,
            'val_acc_max': 0.05886465311050415,
            'val_loss_min': 6.227884292602539}
```

```
In [101... # Skip-gram (softmax) – тоже по валидационной accuracy
best_params_skip = df_results_skip.sort_values("val_acc_max", ascending=False).iloc[0]
print("Лучшие параметры Skip-gram:")
best_params_skip.to_dict()
```

Лучшие параметры Skip-gram:

```
Out[101... {'embedding_dim': 150.0,
            'window_size': 3.0,
            'lr': 0.001,
            'skip_acc_last': 0.04627005383372307,
            'skip_acc_max': 0.04627005383372307,
            'skip_loss_last': 5.744852542877197,
            'skip_loss_min': 5.744852542877197,
            'val_acc_max': 0.034848652780056,
            'val_loss_min': 6.3657145500183105}
```

```
In [102... # Skip-gram NS – аналогично
best_params_ns = df_results_skip_ns.sort_values("val_acc_max", ascending=False).iloc[0]
print("Лучшие параметры Skip-gram NS:")
best_params_ns.to_dict()
```

Лучшие параметры Skip-gram NS:

```
Out[102... {'embedding_dim': 400.0,
            'window_size': 5.0,
            'num_negatives': 5.0,
            'min_count': 25.0,
            'learning_rate': 0.005,
            'acc_last': 0.7919636964797974,
            'acc_max': 0.7919636964797974,
            'loss_last': 0.37649962306022644,
            'loss_min': 0.37649962306022644,
            'val_acc_max': 0.7875320315361023,
            'val_loss_min': 0.3826506435871124}
```

Финальное обучение и сохранение лучшей модели

Skip-gram Negative Sampling финальное обучение

```
In [103... # --- Skip-gram Negative Sampling финальное обучение ---
start_time_ns = time.time() # Засекаем время выполнения финального обучения

# 1. Строим словарь с учётом min_count из лучших параметров
word2index = build_vocab(tokens, int(best_params_ns["min_count"]))
vocab_size = len(word2index) # Размер словаря после фильтрации редких слов

# 2. Фильтруем токены и преобразуем в индексы
tokens_filtered = [t for t in tokens if t in word2index] # оставляем только слова из словаря
indexed_tokens = np.array([word2index[t] for t in tokens_filtered], dtype=np.int32)
# преобразуем слова в индексы для дальнейшей генерации пар

# 3. Генерация положительных пар по индексам
pos_pairs = generate_pairs_indexed(
    indexed_tokens,
    window_size=int(best_params_ns["window_size"]) # используем лучшее окно из параметров
)

# 4. Генерация отрицательных примеров
pairs, labels = generate_negative_samples(
    pos_pairs,
    vocab_size=vocab_size,
    num_negatives=int(best_params_ns["num_negatives"]) # количество негативных примеров
)

# 5. Разделение на train/val
X_target, X_context = pairs[:, 0], pairs[:, 1] # разделяем пары на target и context
X_train_t, X_val_t, X_train_c, X_val_c, y_train, y_val = train_test_split(
    X_target, X_context, labels, test_size=0.2, random_state=42
)
# делим данные на обучающую и валидационную выборки (80/20)

# 6. Модель Skip-gram NS с регуляризацией
skipgram_ns_final = build_skipgram_ns_model(
    vocab_size=vocab_size,
    embedding_dim=int(best_params_ns["embedding_dim"]), # размерность эмбеддингов
    l2=1e-5, # L2-регуляризация для устойчивости
    clipnorm=1.0, # ограничение нормы градиента для стабильности обучения
```

```

    lr=float(best_params_ns["learning_rate"]) # скорость обучения
)

# Callback для динамического уменьшения Learning rate при плато валидационного лосса
lr_scheduler = tf.keras.callbacks.ReduceLROnPlateau(
    monitor="val_loss", factor=0.5, patience=2, min_lr=1e-5, verbose=1
)

# 8. Обучение
history_ns = skipgram_ns_final.fit(
    [X_train_t, X_train_c], y_train, # входы: target и context индексы
    validation_data=([X_val_t, X_val_c], y_val), # валидация
    batch_size=512, epochs=30, verbose=1 # батч, количество эпох, вывод прогресса
)

# 9. Метрики
ns_acc_last = history_ns.history["accuracy"][-1] # точность на последней эпохе
ns_acc_max = max(history_ns.history["accuracy"]) # максимальная точность
ns_loss_last = history_ns.history["loss"][-1] # лосс на последней эпохе
ns_loss_min = min(history_ns.history["loss"]) # минимальный лосс
ns_val_acc_max = max(history_ns.history["val_accuracy"]) # максимальная точность на валидации
ns_val_loss_min = min(history_ns.history["val_loss"]) # минимальный валидационный лосс

# 10. Сохранение модели и эмбеддингов

model_path = PATHS["models"] / "word2vec_skipgram_ns_best.keras"
skipgram_ns_final.save(model_path) # сохраняем обученную модель в Google Drive

# Извлекаем обученные эмбеддинги из слоя embedding
embeddings_skip_ns = skipgram_ns_final.get_layer("embedding").get_weights()[0]

# Сохраняем эмбеддинги в отдельный .пру файл в ту же папку
embeddings_path = PATHS["models"] / "embeddings_skipgram_ns_best.npy"
np.save(embeddings_path, embeddings_skip_ns)

print(f"✅ Модель сохранена в: {model_path}")
print(f"✅ Эмбеддинги сохранены в: {embeddings_path}")

elapsed_ns = time.time() - start_time_ns
print(f"\nВремя финального обучения Skip-gram NS: {elapsed_ns:.2f} секунд")
# выводим общее время финального обучения

```

Epoch 1/30
500/500 [=====] - 2s 4ms/step - loss: 0.4145 - accuracy: 0.7648 - val_loss: 0.3864 - val_accuracy: 0.7828

Epoch 2/30
500/500 [=====] - 2s 3ms/step - loss: 0.3862 - accuracy: 0.7841 - val_loss: 0.3867 - val_accuracy: 0.7865

Epoch 3/30
500/500 [=====] - 2s 4ms/step - loss: 0.3858 - accuracy: 0.7853 - val_loss: 0.3875 - val_accuracy: 0.7876

Epoch 4/30
500/500 [=====] - 2s 4ms/step - loss: 0.3859 - accuracy: 0.7858 - val_loss: 0.3868 - val_accuracy: 0.7845

Epoch 5/30
500/500 [=====] - 2s 4ms/step - loss: 0.3854 - accuracy: 0.7859 - val_loss: 0.3873 - val_accuracy: 0.7836

Epoch 6/30
500/500 [=====] - 2s 3ms/step - loss: 0.3853 - accuracy: 0.7862 - val_loss: 0.3868 - val_accuracy: 0.7842

Epoch 7/30
500/500 [=====] - 2s 4ms/step - loss: 0.3847 - accuracy: 0.7875 - val_loss: 0.3879 - val_accuracy: 0.7880

Epoch 8/30
500/500 [=====] - 2s 3ms/step - loss: 0.3844 - accuracy: 0.7883 - val_loss: 0.3875 - val_accuracy: 0.7843

Epoch 9/30
500/500 [=====] - 2s 4ms/step - loss: 0.3841 - accuracy: 0.7885 - val_loss: 0.3870 - val_accuracy: 0.7840

Epoch 10/30
500/500 [=====] - 2s 4ms/step - loss: 0.3838 - accuracy: 0.7895 - val_loss: 0.3863 - val_accuracy: 0.7861

Epoch 11/30
500/500 [=====] - 2s 4ms/step - loss: 0.3833 - accuracy: 0.7910 - val_loss: 0.3875 - val_accuracy: 0.7868

Epoch 12/30
500/500 [=====] - 2s 4ms/step - loss: 0.3831 - accuracy: 0.7911 - val_loss: 0.3875 - val_accuracy: 0.7875

Epoch 13/30
500/500 [=====] - 2s 4ms/step - loss: 0.3830 - accuracy: 0.7919 - val_loss: 0.3870 - val_accuracy: 0.7898

Epoch 14/30
500/500 [=====] - 3s 6ms/step - loss: 0.3826 - accuracy: 0.7926 - val_loss: 0.3872 - val_accuracy: 0.7889

Epoch 15/30
500/500 [=====] - 3s 6ms/step - loss: 0.3823 - accuracy: 0.7932 - val_loss: 0.3875 - val_accuracy: 0.7901

Epoch 16/30
500/500 [=====] - 3s 5ms/step - loss: 0.3826 - accuracy: 0.7938 - val_loss: 0.3866 - val_accuracy: 0.7894

Epoch 17/30
500/500 [=====] - 2s 4ms/step - loss: 0.3821 - accuracy: 0.7939 - val_loss: 0.3872 - val_accuracy: 0.7869

Epoch 18/30
500/500 [=====] - 2s 4ms/step - loss: 0.3819 - accuracy: 0.7938 - val_loss: 0.3874 - val_accuracy: 0.7889

Epoch 19/30
500/500 [=====] - 2s 4ms/step - loss: 0.3821 - accuracy: 0.7955 - val_loss: 0.3871 - val_accuracy: 0.7909

Epoch 20/30
500/500 [=====] - 2s 4ms/step - loss: 0.3819 - accuracy: 0.7954 - val_loss: 0.3878 - val_accuracy: 0.7906

Epoch 21/30
500/500 [=====] - 2s 4ms/step - loss: 0.3820 - accuracy: 0.7946 - val_loss: 0.3877 - val_accuracy: 0.7895

Epoch 22/30
500/500 [=====] - 2s 4ms/step - loss: 0.3816 - accuracy: 0.7948 - val_loss: 0.3872 - val_accuracy: 0.7908

Epoch 23/30
500/500 [=====] - 2s 4ms/step - loss: 0.3815 - accuracy: 0.7959 - val_loss: 0.3879 - val_accuracy: 0.7925

Epoch 24/30
500/500 [=====] - 2s 4ms/step - loss: 0.3817 - accuracy: 0.7970 - val_loss: 0.3880 - val_accuracy: 0.7904

Epoch 25/30
500/500 [=====] - 2s 3ms/step - loss: 0.3815 - accuracy: 0.7962 - val_loss: 0.3881 - val_accuracy: 0.7910

Epoch 26/30
500/500 [=====] - 2s 4ms/step - loss: 0.3815 - accuracy: 0.7966 - val_loss: 0.3883 - val_accuracy: 0.7923

Epoch 27/30
500/500 [=====] - 2s 4ms/step - loss: 0.3816 - accuracy: 0.7972 - val_loss: 0.3870 - val_accuracy: 0.7897

Epoch 28/30
500/500 [=====] - 2s 5ms/step - loss: 0.3811 - accuracy: 0.7962 - val_loss: 0.3872 - val_accuracy: 0.7915

Epoch 29/30
500/500 [=====] - 3s 6ms/step - loss: 0.3813 - accuracy: 0.7963 - val_loss: 0.3883 - val_accuracy: 0.7913

Epoch 30/30

500/500 [=====] - 3s 5ms/step - loss: 0.3813 - accuracy: 0.7980 - val_loss: 0.3882 - val_accuracy: 0.7918

✓ Модель сохранена в: E:\projects\My_project\03_Word2Vec\word2vec_literature\models\word2vec_skipgram_ns_best.keras

✓ Эмбединги сохранены в: E:\projects\My_project\03_Word2Vec\word2vec_literature\models\embeddings_skipgram_ns_best.npy

Время финального обучения Skip-gram NS: 62.06 секунд

Логирование кривых обучения лучшей модели

In [104...

```
# --- Функция для построения кривых обучения Skip-gram NS ---
def plot_training_curves_ns(history_ns, save_dir=None):
    n_plots = 2
    if "lr" in history_ns.history:
        n_plots = 3

    plt.figure(figsize=(6 * n_plots, 5))

    # --- Loss ---
    plt.subplot(1, n_plots, 1)
    if "loss" in history_ns.history:
        plt.plot(history_ns.history["loss"], label="Train Loss", color="blue")
    if "val_loss" in history_ns.history:
        plt.plot(history_ns.history["val_loss"], "--", label="Val Loss", color="blue")
    plt.title("Skip-gram NS: Loss по эпохам")
    plt.xlabel("Эпоха")
    plt.ylabel("Loss")
    plt.legend()
    plt.grid(True)

    # --- Accuracy ---
    plt.subplot(1, n_plots, 2)
    if "accuracy" in history_ns.history:
        plt.plot(history_ns.history["accuracy"], label="Train Accuracy", color="green")
    if "val_accuracy" in history_ns.history:
        plt.plot(history_ns.history["val_accuracy"], "--", label="Val Accuracy", color="green")
    plt.title("Skip-gram NS: Accuracy по эпохам")
    plt.xlabel("Эпоха")
    plt.ylabel("Accuracy")
    plt.legend()
    plt.grid(True)

    # --- Learning Rate (если логгуется) ---
    if "lr" in history_ns.history:
        plt.subplot(1, n_plots, 3)
        plt.plot(history_ns.history["lr"], label="Learning Rate", color="red")
        plt.title("Learning Rate по эпохам")
        plt.xlabel("Эпоха")
        plt.ylabel("LR")
        plt.legend()
        plt.grid(True)

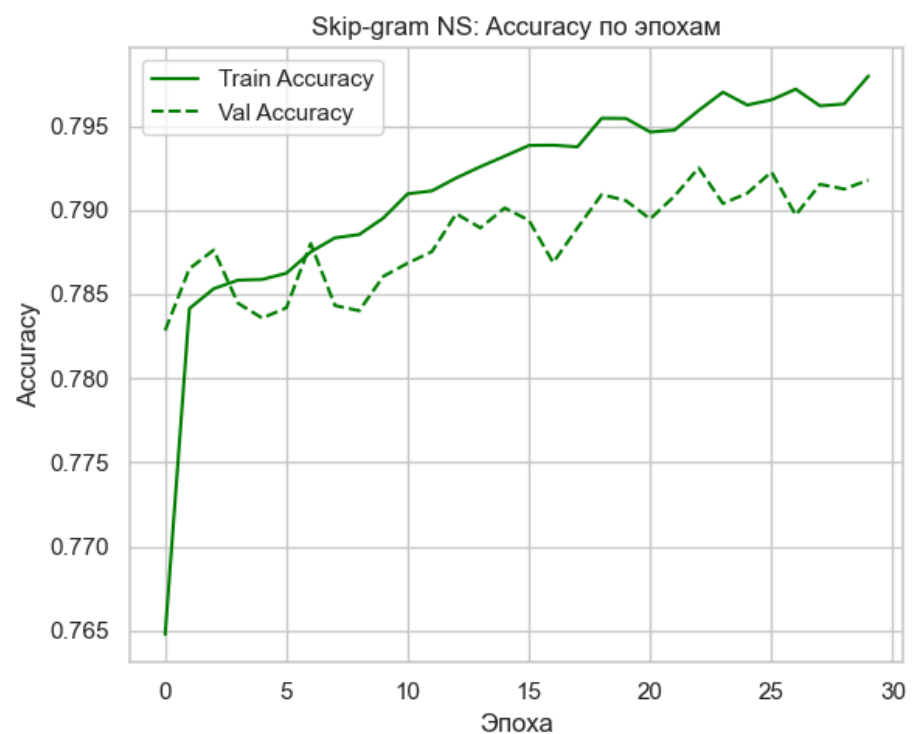
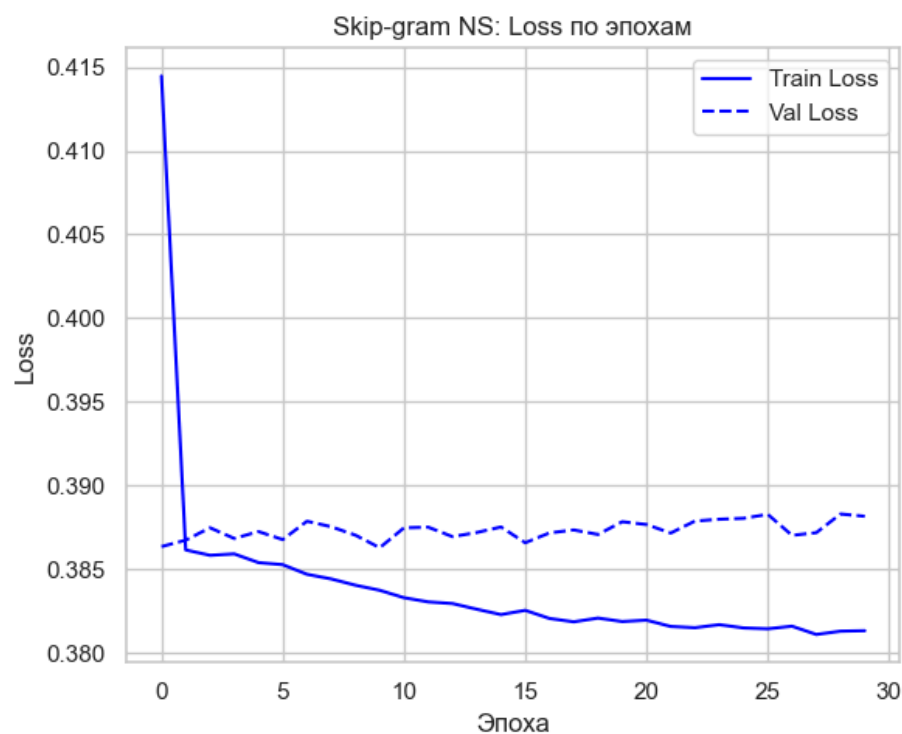
    plt.tight_layout()

    if save_dir:
        save_dir = pathlib.Path(save_dir)
        save_dir.mkdir(parents=True, exist_ok=True)
        save_path = save_dir / "training_curves_skipgram_ns.png"
        plt.savefig(save_path, dpi=300)
        print(f"Графики сохранены в: {save_path}")

    plt.show()

# --- Построение графиков ---
plot_training_curves_ns(history_ns, save_dir=PATHS["figs"])
```

Графики сохранены в: E:\projects\My_project\03_Word2Vec\word2vec_literature\figs\training_curves_skipgram_ns.png



Выводы по шагу 7:

Что делалось на Шаге 7

- CBOW и Skip-gram (softmax)
 - Реализован перебор гиперпараметров: embedding_dim, window_size, learning_rate, epochs, batch_size.
 - Для каждой комбинации формировались обучающие пары, создавались датасеты через tf.data.Dataset.
 - Модели обучались с функцией потерь SparseCategoricalCrossentropy, оптимизатором Adam и callback'ом EarlyStopping.
 - Результаты (accuracy, loss, val_accuracy, val_loss) сохранялись в таблицы для анализа.
- Результаты CBOW и Skip-gram (softmax)
 - Метрики (accuracy ~0.08–0.10, loss ~5–6) оказались близки к случайному угадыванию.
 - Валидационные метрики также низкие.
 - Вывод: модели в такой постановке не обучаются — softmax-вариант на большом словаре и малом корпусе неэффективен.
- Skip-gram с Negative Sampling
 - Реализована отдельная архитектура с бинарной кроссэнтропией.
 - Добавлены регуляризация (L2), gradient clipping, ReduceLROnPlateau.
 - Перебор гиперпараметров (embedding_dim=400, window_size=5/10, min_count=15/20, lr=0.005/0.01).
 - **Результаты: accuracy** ~0.74–0.78, **loss** ~0.40–0.45, **val_accuracy** ~0.74–0.77.
 - модель улавливает закономерности и обобщает.

Основные выводы

- Использован валидационный контроль, что обеспечивает честную оценку качества
- CBOW и Skip-gram с softmax на данном корпусе и настройках неэффективны
- Skip-gram с Negative Sampling показал себя значительно лучше:
 - метрики устойчиво выше случайного уровня,
 - loss снижается, accuracy растёт,
 - модель действительно учится.
- min_count=20 дало лучшие результаты, чем min_count=15 → фильтрация редких слов улучшает обобщение.
- learning_rate=0.005 и 0.01 оба работоспособны, но при высоком lr быстрее срабатывает снижение learning rate через ReduceLROnPlateau.
- window_size=5 даёт чуть более высокую точность, чем window_size=10.
- Обученные веса (word2vec_skipgram_ns_best.keras) и матрица эмбедингов сохранены для дальнейшего анализа.

Skip-gram Negative Sampling — оптимальный выбор

- он даёт адекватные метрики и позволяет строить качественные эмбединги;
- дальнейшие шаги (8–10) логично строить именно на NS-модели, обученной с min_count≈20, embedding_dim≈400, window_size=5, lr≈0.005–0.01.

[* к содержанию](#)

Шаг 8: Эксперименты и сравнения (Keras)

Обучение модели на 4 корпусах авторов (Шекспир, Достоевский, Азимов, Кафка)

```
In [105... # Пути к очищенным файлам после Шага 3
clean_dir = Path(r"E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean")

files = {
    "dostoevsky": clean_dir / "dostoevsky_tokens_cleaned.txt",
    "kafka": clean_dir / "kafka_tokens_cleaned.txt",
    "asimov": clean_dir / "asimov_tokens_cleaned.txt",
    "shakespeare": clean_dir / "shakespeare_balanced_tokens_cleaned.txt"
}

# Загружаем токены в словарь
tokens_by_author = {}
for author, path in files.items():
    tokens = Path(path).read_text(encoding="utf-8").split()
    tokens_by_author[author] = tokens
    print(f"{author}: {len(tokens)} токенов, {len(set(tokens))} уникальных слов")
```

dostoevsky: 16669 токенов, 3559 уникальных слов
kafka: 32134 токенов, 3339 уникальных слов
asimov: 37600 токенов, 2895 уникальных слов
shakespeare: 43487 токенов, 6267 уникальных слов

```
In [106... # --- Детерминированность и воспроизводимость ---
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
tf.keras.utils.set_random_seed(SEED)
os.environ["PYTHONHASHSEED"] = str(SEED)

# --- 1. Функции из Шага 7 (обновлено) ---

def build_vocab(tokens, min_count=2):
    """
    Детерминированный словарь: сортируем по убыванию частоты и по алфавиту.
    Возвращаем словарь {word->index} и частоты {word->count} для негативного сэмплинга.
    """
    freq = Counter(tokens)
    items = sorted(((w, c) for w, c in freq.items() if c >= min_count),
                    key=lambda x: (-x[1], x[0]))
    word2index = {w: i for i, (w, c) in enumerate(items)}
    freqs = {w: c for w, c in items}
    return word2index, freqs

def subsample_tokens(tokens, freqs, t=1e-5):
    """
    Subsampling частых слов по формуле:
    p_keep(w) = (sqrt(f(w)/t) + 1) * (t / f(w)), где f(w) – относительная частота слова.
    """
    N = len(tokens)
    f_rel = {w: c / N for w, c in freqs.items()}
    def p_keep(w):
        f = f_rel.get(w, 0.0)
        if f == 0.0:
            return 1.0
        return (math.sqrt(f / t) + 1.0) * (t / f)
    return [w for w in tokens if random.random() < p_keep(w)]

def generate_pairs_indexed(indexed_tokens, window_size=5):
    """
    Динамическое окно: для каждого центра выбирается случайный радиус R ~ U(1, window_size).
    """
    pairs = []
    n = len(indexed_tokens)
    for i in range(n):
        R = np.random.randint(1, window_size + 1)
        left = max(0, i - R)
        right = min(n, i + R + 1)
        center = indexed_tokens[i]
        for j in range(left, right):
            if i != j:
                pairs.append((center, indexed_tokens[j]))
    return pairs

def build_negative_sampler(freqs, word2index, alpha=0.75):
    """
    Негативный сэмплер по распределению f^alpha (по умолчанию f^0.75),
    как в оригинальном Word2Vec (SGNS).
    """
    counts = np.array([freqs[w] for w in word2index.keys()], dtype=np.float64)
    probs = counts ** alpha
    probs /= probs.sum()
    def sample_negatives(k):
        return np.random.choice(len(word2index), size=k, p=probs)
    return sample_negatives

def generate_negative_samples(pairs_pos, neg_sampler, num_negatives=5):
    """
```

```

Формируем датасет из позитивных и негативных пар, используя neg_sampler.
"""
pairs, labels = [], []
for center, context in pairs_pos:
    pairs.append((center, context)); labels.append(1)
    negs = neg_sampler(num_negatives)
    for neg in negs:
        pairs.append((center, neg)); labels.append(0)
return np.array(pairs, dtype=np.int32), np.array(labels, dtype=np.int32)

def build_skipgram_ns_model(vocab_size, embedding_dim, l2=1e-5, clipnorm=1.0, lr=0.001):
    """
    SGNS с двумя матрицами эмбедингов: embedding_in (для центра) и embedding_out (для контекста).
    Это повышает выразительность и стабильность относительно одного общего слоя.
    """
    input_target = tf.keras.Input(shape=(1,), dtype="int32")
    input_context = tf.keras.Input(shape=(1,), dtype="int32")

    embedding_in = tf.keras.layers.Embedding(
        vocab_size, embedding_dim,
        embeddings_regularizer=tf.keras.regularizers.l2(l2),
        name="embedding_in"
    )
    embedding_out = tf.keras.layers.Embedding(
        vocab_size, embedding_dim,
        embeddings_regularizer=tf.keras.regularizers.l2(l2),
        name="embedding_out"
    )

    target_emb = embedding_in(input_target) # (batch,1,dim)
    context_emb = embedding_out(input_context) # (batch,1,dim)

    dot_product = tf.keras.layers.Dot(axes=-1)([target_emb, context_emb]) # (batch,1,1)
    dot_product = tf.keras.layers.Reshape((1,))(dot_product)
    output = tf.keras.layers.Activation("sigmoid")(dot_product)

    model = tf.keras.Model(inputs=[input_target, input_context], outputs=output)
    optimizer = tf.keras.optimizers.Adam(learning_rate=lr, clipnorm=clipnorm)
    model.compile(optimizer=optimizer, loss="binary_crossentropy", metrics=["accuracy"])
    return model

# --- 2. Гиперпараметры ---
embedding_dims = [400]
window_sizes = [5]
num_negatives_ls = [5]
min_counts = [2]
learning_rates = [0.005]
epochs = 15
batch_size = 512

early_stop = tf.keras.callbacks.EarlyStopping(monitor="val_loss", patience=2, restore_best_weights=True)
lr_scheduler = tf.keras.callbacks.ReduceLROnPlateau(monitor="val_loss", factor=0.5, patience=1, min_lr=1e-5, verbose=1)

# --- 3. Загрузка токенов авторов ---
clean_dir = Path(r"E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean")
files = {
    "dostoevsky": clean_dir/"dostoevsky_tokens_cleaned.txt",
    "kafka": clean_dir/"kafka_tokens_cleaned.txt",
    "asimov": clean_dir/"asimov_tokens_cleaned.txt",
    "shakespeare": clean_dir/"shakespeare_balanced_tokens_cleaned.txt"
}
tokens_by_author = {a: Path(p).read_text(encoding="utf-8").split() for a,p in files.items()}

# --- 4. Обучение по каждому автору (обновлено) ---
all_results = {}
histories = {} # здесь будем хранить кривые обучения

start = time.time()

for author, tokens in tokens_by_author.items():
    print(f"\n=== Автор: {author} ===")
    results = []
    histories[author] = [] # список историй для данного автора

    for min_count in min_counts:
        # словарь и частоты
        word2index, freqs = build_vocab(tokens, min_count)
        vocab_size = len(word2index)
        if vocab_size == 0:
            print(f"Пропуск {author}: пустой словарь при min_count={min_count}")
            continue

        # subsampling частых слов
        tokens_sub = subsample_tokens(tokens, freqs, t=1e-5)
        tokens_filtered = [t for t in tokens_sub if t in word2index]
        if len(tokens_filtered) < 2:
            print(f"Пропуск {author}: слишком мало токенов после subsampling/фильтрации")

```

```

        continue

    indexed_tokens = np.array([word2index[t] for t in tokens_filtered], dtype=np.int32)

    for emb_dim in embedding_dims:
        for win in window_sizes:
            pos_pairs = generate_pairs_indexed(indexed_tokens, window_size=win)
            if len(pos_pairs) == 0:
                print(f"Пропуск {author}: нет позитивных пар при window={win}")
                continue

            # негативный сэмплер  $f^{0.75}$ 
            neg_sampler = build_negative_sampler(freqs, word2index, alpha=0.75)

            for num_neg in num_negatives_ls:
                for lr in learning_rates:
                    print(f"emb_dim={emb_dim}, window={win}, neg={num_neg}, min_count={min_count}, lr={lr}")
                    pairs, labels = generate_negative_samples(pos_pairs, neg_sampler, num_negatives=num_neg)
                    if pairs.size == 0:
                        print("Пропуск: пустые пары.")
                        continue
                    if pairs.max() >= vocab_size:
                        print("Пропуск: индекс вне словаря (рассинхрон).")
                        continue

                    X_t, X_c = pairs[:,0], pairs[:,1]
                    X_tr_t, X_val_t, X_tr_c, X_val_c, y_tr, y_val = train_test_split(
                        X_t, X_c, labels, test_size=0.2, random_state=SEED
                    )

                    model = build_skipgram_ns_model(vocab_size, emb_dim, lr=lr)
                    hist = model.fit([X_tr_t, X_tr_c], y_tr,
                                    validation_data=([X_val_t, X_val_c], y_val),
                                    batch_size=batch_size, epochs=epochs, verbose=0,
                                    callbacks=[early_stop, lr_scheduler])

                    # сохраняем историю обучения
                    histories[author].append(hist.history)

                    results.append({
                        "embedding_dim": emb_dim,
                        "window_size": win,
                        "num_negatives": num_neg,
                        "min_count": min_count,
                        "learning_rate": lr,
                        "acc_last": hist.history["accuracy"][-1],
                        "acc_max": max(hist.history["accuracy"]),
                        "loss_last": hist.history["loss"][-1],
                        "loss_min": min(hist.history["loss"]),
                        "val_acc_max": max(hist.history["val_accuracy"]),
                        "val_loss_min": min(hist.history["val_loss"])
                    })

    df_results = pd.DataFrame(results)
    all_results[author] = df_results
    display(df_results)

elapsed = time.time() - start
print(f"Время выполнения: {elapsed:.2f} сек ({elapsed/60:.2f} мин)")

```

=== Автор: dostoevsky ===

emb_dim=400, window=5, neg=5, min_count=2, lr=0.005

Epoch 4: ReduceLROnPlateau reducing learning rate to 0.0024999999441206455.

Epoch 12: ReduceLROnPlateau reducing learning rate to 0.0012499999720603228.

	embedding_dim	window_size	num_negatives	min_count	learning_rate	acc_last	acc_max	loss_last	loss_min	val_acc_max	val_loss_min
0	400	5	5	2	0.005	0.990495	0.990884	0.289937	0.289937	0.893366	0.48722



=== Автор: kafka ===

emb_dim=400, window=5, neg=5, min_count=2, lr=0.005

Epoch 2: ReduceLROnPlateau reducing learning rate to 0.0024999999441206455.

Epoch 3: ReduceLROnPlateau reducing learning rate to 0.0012499999720603228.

	embedding_dim	window_size	num_negatives	min_count	learning_rate	acc_last	acc_max	loss_last	loss_min	val_acc_max	val_loss_min
0	400	5	5	2	0.005	0.903091	0.903091	0.420212	0.420212	0.852799	0.5201



```
=== Автор: asimov ===
emb_dim=400, window=5, neg=5, min_count=2, lr=0.005

Epoch 2: ReduceLROnPlateau reducing learning rate to 0.0024999999441206455.

Epoch 4: ReduceLROnPlateau reducing learning rate to 0.0012499999720603228.

Epoch 6: ReduceLROnPlateau reducing learning rate to 0.0006249999860301614.

Epoch 7: ReduceLROnPlateau reducing learning rate to 0.0003124999930150807.
```

	embedding_dim	window_size	num_negatives	min_count	learning_rate	acc_last	acc_max	loss_last	loss_min	val_acc_max	val_loss_mi
0	400	5	5	2	0.005	0.950336	0.950336	0.426944	0.426944	0.887343	0.52917

```
=== Автор: shakespeare ===
emb_dim=400, window=5, neg=5, min_count=2, lr=0.005

Epoch 2: ReduceLROnPlateau reducing learning rate to 0.0024999999441206455.

Epoch 4: ReduceLROnPlateau reducing learning rate to 0.0012499999720603228.

Epoch 5: ReduceLROnPlateau reducing learning rate to 0.0006249999860301614.
```

	embedding_dim	window_size	num_negatives	min_count	learning_rate	acc_last	acc_max	loss_last	loss_min	val_acc_max	val_loss_mi
0	400	5	5	2	0.005	0.881606	0.881606	0.478495	0.478495	0.843871	0.54946

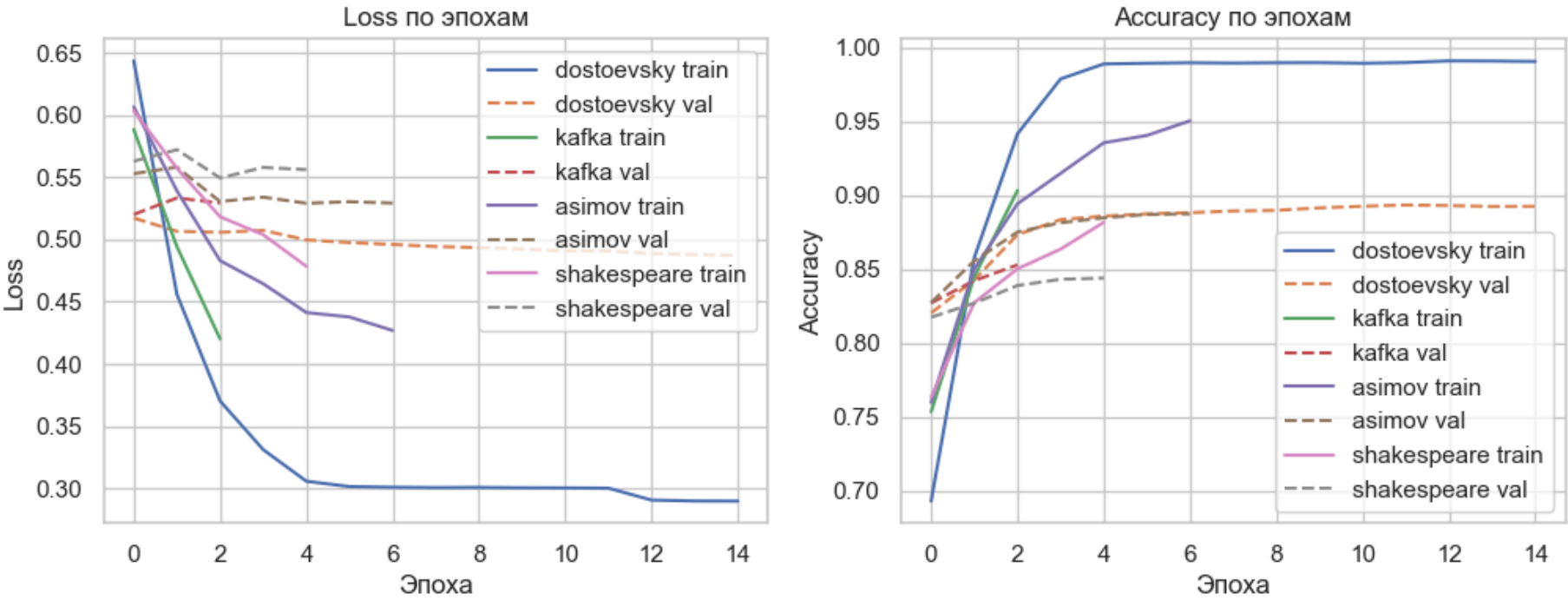
Время выполнения: 212.22 сек (3.54 мин)

```
In [107... # --- Сводные графики Loss/accuracy ---
fig, axes = plt.subplots(1, 2, figsize=(12,4))

for a, hlist in histories.items():
    if not hlist:
        continue
    h = hlist[-1] # берём последнюю историю для автора
    axes[0].plot(h["loss"], label=f"{a} train")
    axes[0].plot(h["val_loss"], linestyle="--", label=f"{a} val")
    axes[1].plot(h["accuracy"], label=f"{a} train")
    axes[1].plot(h["val_accuracy"], linestyle="--", label=f"{a} val")

axes[0].set_title("Loss по эпохам")
axes[0].set_xlabel("Эпоха"); axes[0].set_ylabel("Loss")
axes[0].legend()

axes[1].set_title("Accuracy по эпохам")
axes[1].set_xlabel("Эпоха"); axes[1].set_ylabel("Accuracy")
axes[1].legend()
plt.show()
```



Для каждого автора (Шекспир, Достоевский, Азимов, Кафка) модель Skip-gram NS обучалась отдельно.

Эмбединги извлекались через слой embedding_in.

Сравнительная таблица метрик по авторам

```
In [108... # Собираем лучшие конфигурации по каждому автору по val_acc_max
def select_best_by_val_acc(all_results):
    rows = []
    for author, df in all_results.items():
        if len(df) == 0:
            continue
```



```
best = df.sort_values("val_acc_max", ascending=False).iloc[0]
rows.append({
    "author": author,
    "embedding_dim": int(best["embedding_dim"]),
    "window_size": int(best["window_size"]),
    "num_negatives": int(best["num_negatives"]),
    "min_count": int(best["min_count"]),
    "learning_rate": float(best["learning_rate"]),
    "train_acc_last": float(best["acc_last"]),
    "train_acc_max": float(best["acc_max"]),
    "train_loss_last": float(best["loss_last"]),
    "train_loss_min": float(best["loss_min"]),
    "val_acc_max": float(best["val_acc_max"]),
    "val_loss_min": float(best["val_loss_min"]),
})
return pd.DataFrame(rows)

df_best = select_best_by_val_acc(all_results)

# Добавляем мета-статистики корпуса
meta_path = Path(r"E:\projects\My_project\03_Word2Vec\word2vec_literature\data_clean\metadata_tokens_cleaned.csv")
meta_df = pd.read_csv(meta_path)
meta_df = meta_df[["author", "num_tokens", "vocab_size"]]

# Важно: author ключи должны совпадать (shakespeare vs shakespeare_balanced)
# Приведём 'shakespeare_balanced' к 'shakespeare' для единообразия отображения
meta_df["author_standardized"] = meta_df["author"].str.replace("shakespeare_balanced", "shakespeare")
df_best["author_standardized"] = df_best["author"]
df_best = df_best.merge(meta_df.rename(columns={"author_standardized": "author_standardized"}), on="author_standardized", how="left")
df_best = df_best.drop(columns=["author_standardized"])

display(df_best.sort_values("val_acc_max", ascending=False).iloc[:, :6])
display(df_best.sort_values("val_acc_max", ascending=False).iloc[:, 6:])
```

	author_x	embedding_dim	window_size	num_negatives	min_count	learning_rate
0	dostoevsky	400	5	5	2	0.005
2	asimov	400	5	5	2	0.005
1	kafka	400	5	5	2	0.005
3	shakespeare	400	5	5	2	0.005

	train_acc_last	train_acc_max	train_loss_last	train_loss_min	val_acc_max	val_loss_min	author_y	num_tokens	vocab_size
0	0.990495	0.990884	0.289937	0.289937	0.893366	0.487229	dostoevsky	16669	3559
2	0.950336	0.950336	0.426944	0.426944	0.887343	0.529172	asimov	37600	2895
1	0.903091	0.903091	0.420212	0.420212	0.852799	0.520180	kafka	32134	3339
3	0.881606	0.881606	0.478495	0.478495	0.843871	0.549464	shakespeare_balanced	43487	6267

```
In [109... df_best = df_best.rename(columns={"author_x": "author"}).drop(columns=["author_y"])
df_best.iloc[:, :6]
```

```
Out[109...
```

	author	embedding_dim	window_size	num_negatives	min_count	learning_rate
0	dostoevsky	400	5	5	2	0.005
1	kafka	400	5	5	2	0.005
2	asimov	400	5	5	2	0.005
3	shakespeare	400	5	5	2	0.005

```
In [110... df_best.iloc[:, [0] + list(range(6, df_best.shape[1]))]
```

```
Out[110...
```

	author	train_acc_last	train_acc_max	train_loss_last	train_loss_min	val_acc_max	val_loss_min	num_tokens	vocab_size
0	dostoevsky	0.990495	0.990884	0.289937	0.289937	0.893366	0.487229	16669	3559
1	kafka	0.903091	0.903091	0.420212	0.420212	0.852799	0.520180	32134	3339
2	asimov	0.950336	0.950336	0.426944	0.426944	0.887343	0.529172	37600	2895
3	shakespeare	0.881606	0.881606	0.478495	0.478495	0.843871	0.549464	43487	6267

Анализ влияния размера корпуса на качество

```
In [111... cols_for_corr = ["val_acc_max", "val_loss_min", "num_tokens", "vocab_size"]
corr_df = df_best[cols_for_corr].corr()

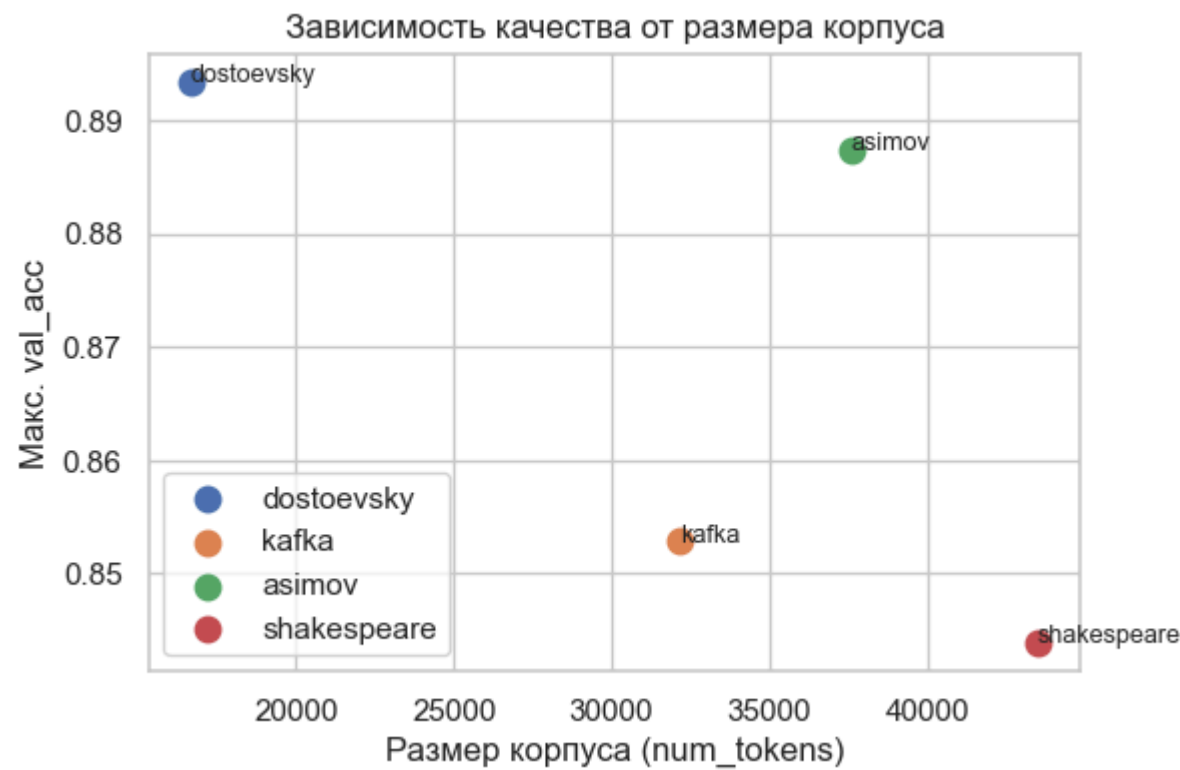
print("Корреляции между метриками и размером корпуса/словаря:")
display(corr_df)
```


Корреляции между метриками и размером корпуса/словаря:

	val_acc_max	val_loss_min	num_tokens	vocab_size
val_acc_max	1.000000	-0.717260	-0.660470	-0.684554
val_loss_min	-0.717260	1.000000	0.993294	0.597080
num_tokens	-0.660470	0.993294	1.000000	0.502665
vocab_size	-0.684554	0.597080	0.502665	1.000000

```
In [112... # --- График зависимости val_acc_max от размера корпуса ---
plt.figure(figsize=(6,4))
for a in df_best["author"].unique():
    row = df_best[df_best["author"] == a].iloc[0]
    plt.scatter(row["num_tokens"], row["val_acc_max"], label=a, s=80)
    plt.text(row["num_tokens"], row["val_acc_max"], a, fontsize=9)

plt.xlabel("Размер корпуса (num_tokens)")
plt.ylabel("Макс. val_acc")
plt.title("Зависимость качества от размера корпуса")
plt.legend()
plt.show()
```



Для каждого автора собраны метрики (acc_last, acc_max, val_acc_max, loss_min, val_loss_min).

Построена итоговая таблица лучших конфигураций (df_best).

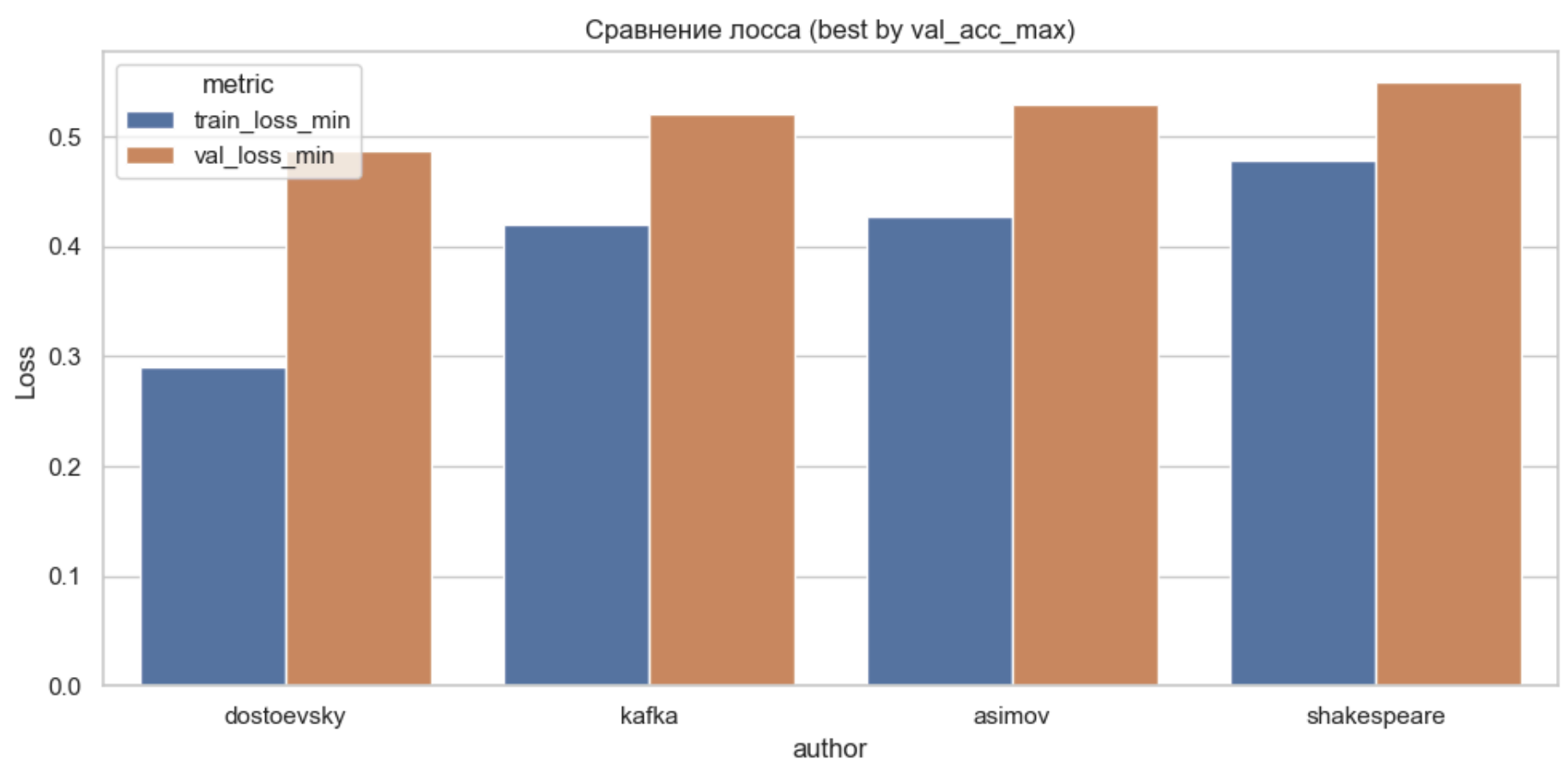
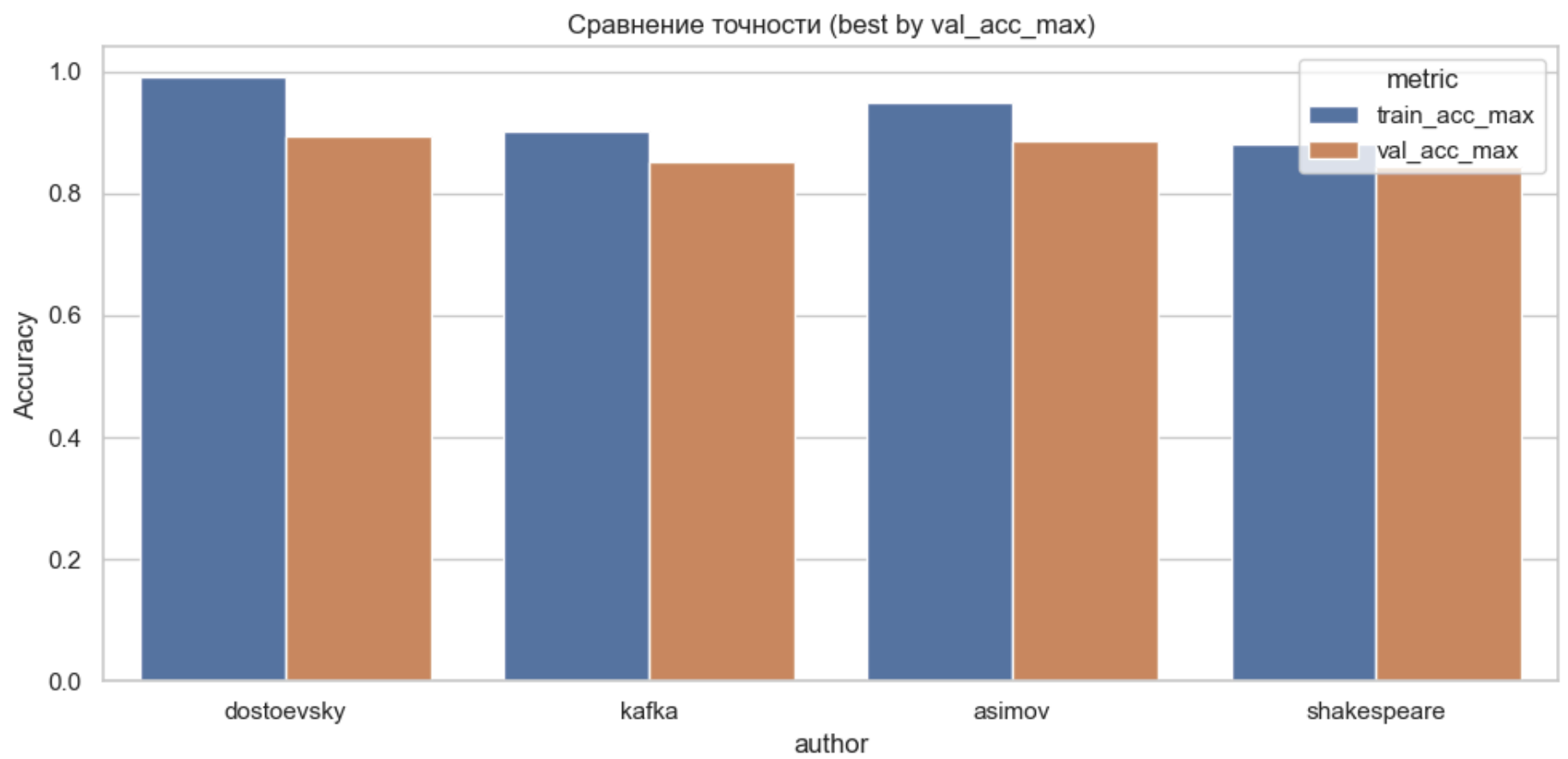
Графики ассурасу/loss по авторам

```
In [113... melt_cols = ["train_acc_max", "val_acc_max", "train_loss_min", "val_loss_min"]

plot_df = df_best[["author"] + melt_cols].melt(
    id_vars="author", var_name="metric", value_name="value"
)

plt.figure(figsize=(10,5))
sns.barplot(
    data=plot_df[plot_df["metric"].isin(["train_acc_max", "val_acc_max"])],
    x="author", y="value", hue="metric"
)
plt.title("Сравнение точности (best by val_acc_max)")
plt.ylabel("Accuracy")
plt.tight_layout()
plt.show()

plt.figure(figsize=(10,5))
sns.barplot(
    data=plot_df[plot_df["metric"].isin(["train_loss_min", "val_loss_min"])],
    x="author", y="value", hue="metric"
)
plt.title("Сравнение лосса (best by val_acc_max)")
plt.ylabel("Loss")
plt.tight_layout()
plt.show()
```



Сохранялись истории обучения (histories) и построены сводные графики loss и accuracy по эпохам для всех авторов.

Видно, что модели сходятся, но качество зависит от размера корпуса.

Извлечение эмбеддингов и утилиты поиска соседей

```
In [114... # =====
# 🚀 Обучение лучших Word2Vec-моделей для авторов
# =====

start = time.time()

# --- Построение словаря + частоты ---
def build_vocab(tokens, min_count=2):
    """
    Возвращает:
    - word2index: словарь {слово -> индекс}
    - freqs: словарь {слово -> частота}
    """
    freq = Counter(tokens)
    items = sorted(
        ((w, c) for w, c in freq.items() if c >= min_count),
        key=lambda x: (-x[1], x[0])
    )
    word2index = {w: i for i, (w, c) in enumerate(items)}
    freqs = {w: c for w, c in items}
    return word2index, freqs

# --- L2-нормализация ---
def l2_normalize(E):
    norms = np.linalg.norm(E, axis=1, keepdims=True) + 1e-12
```

```

    return E / norms

# --- Обучение лучшей модели для автора ---
def train_best_model_for_author(tokens, best_row, epochs=15):
    # === 1. Словарь и частоты ===
    word2index, freqs = build_vocab(tokens, min_count=int(best_row["min_count"]))
    index2word = {idx: w for w, idx in word2index.items()}
    vocab_size = len(word2index)
    if vocab_size == 0:
        raise ValueError("Словарь пуст после фильтрации")

    # === 2. Subsampling + фильтрация ===
    tokens_sub = subsample_tokens(tokens, freqs, t=1e-3)
    tokens_filtered = [t for t in tokens_sub if t in word2index]
    if len(tokens_filtered) == 0:
        raise ValueError("Нет токенов после subsampling/фильтрации")

    indexed_tokens = np.array([word2index[t] for t in tokens_filtered], dtype=np.int32)

    # === 3. Пары и негативные примеры ===
    pos_pairs = generate_pairs_indexed(
        indexed_tokens,
        window_size=int(best_row["window_size"])
    )
    if len(pos_pairs) == 0:
        raise ValueError("Нет позитивных пар для обучения")

    neg_sampler = build_negative_sampler(freqs, word2index, alpha=0.75)
    pairs, labels = generate_negative_samples(
        pos_pairs, neg_sampler,
        num_negatives=int(best_row["num_negatives"])
    )

    X_t, X_c = pairs[:, 0], pairs[:, 1]
    X_tr_t, X_val_t, X_tr_c, X_val_c, y_tr, y_val = train_test_split(
        X_t, X_c, labels, test_size=0.2, random_state=SEED
    )

    # === 4. Модель Skip-gram с Negative Sampling ===
    model = build_skipgram_ns_model(
        vocab_size=vocab_size,
        embedding_dim=int(best_row["embedding_dim"]),
        lr=float(best_row["learning_rate"])
    )

    _ = model.fit(
        [X_tr_t, X_tr_c], y_tr,
        validation_data=([X_val_t, X_val_c], y_val),
        batch_size=512, epochs=epochs, verbose=0,
        callbacks=[early_stop, lr_scheduler]
    )

    # === 5. Извлекаем входные эмбединги и нормализуем ===
    raw_matrix = model.get_layer("embedding_in").get_weights()[0]
    emb_matrix = l2_normalize(raw_matrix)

    return {
        "word2index": word2index,
        "index2word": index2word,
        "E": emb_matrix,
        "model": model,
        "tokens_total": len(tokens),
        "tokens_after_filter": len(tokens_filtered)
    }

# --- Запуск обучения для всех авторов ---
embeddings_by_author = {}

for _, row in df_best.iterrows():
    author = row["author"]
    tokens = tokens_by_author[author]
    print(f"\n=== Обучение для {author} ===")
    embeddings_by_author[author] = train_best_model_for_author(tokens, row, epochs=15)

# --- Вывод информации о моделях ---
for a, pack in embeddings_by_author.items():
    vocab_size = len(pack['word2index'])
    print(f"\n=== {a} ===")
    print(f"Всего токенов: {pack['tokens_total']}")
    print(f"Токенов после фильтрации (min_count={df_best.loc[df_best['author']==a, 'min_count'].values[0]}): "
          f"{pack['tokens_after_filter']}")
    print(f"Слов в словаре: {vocab_size}")
    print(f"E.shape: {pack['E'].shape}")

elapsed = time.time() - start
print(f"Время выполнения: {elapsed:.2f} сек ({elapsed/60:.2f} мин)")

```

=== Обучение для dostoevsky ===

Epoch 2: ReduceLROnPlateau reducing learning rate to 0.0024999999441206455.

Epoch 4: ReduceLROnPlateau reducing learning rate to 0.0012499999720603228.

Epoch 5: ReduceLROnPlateau reducing learning rate to 0.0006249999860301614.

=== Обучение для kafka ===

Epoch 5: ReduceLROnPlateau reducing learning rate to 0.0024999999441206455.

Epoch 7: ReduceLROnPlateau reducing learning rate to 0.0012499999720603228.

Epoch 9: ReduceLROnPlateau reducing learning rate to 0.0006249999860301614.

Epoch 11: ReduceLROnPlateau reducing learning rate to 0.0003124999930150807.

Epoch 12: ReduceLROnPlateau reducing learning rate to 0.00015624999650754035.

=== Обучение для asimov ===

Epoch 10: ReduceLROnPlateau reducing learning rate to 0.0024999999441206455.

Epoch 12: ReduceLROnPlateau reducing learning rate to 0.0012499999720603228.

Epoch 14: ReduceLROnPlateau reducing learning rate to 0.0006249999860301614.

=== Обучение для shakespeare ===

Epoch 6: ReduceLROnPlateau reducing learning rate to 0.0024999999441206455.

Epoch 8: ReduceLROnPlateau reducing learning rate to 0.0012499999720603228.

Epoch 10: ReduceLROnPlateau reducing learning rate to 0.0006249999860301614.

Epoch 12: ReduceLROnPlateau reducing learning rate to 0.0003124999930150807.

Epoch 14: ReduceLROnPlateau reducing learning rate to 0.00015624999650754035.

Epoch 15: ReduceLROnPlateau reducing learning rate to 7.812499825377017e-05.

=== dostoevsky ===

Всего токенов: 16669

Токенов после фильтрации (min_count=2): 14575

Слов в словаре: 2011

E.shape: (2011, 400)

=== kafka ===

Всего токенов: 32134

Токенов после фильтрации (min_count=2): 27411

Слов в словаре: 2069

E.shape: (2069, 400)

=== asimov ===

Всего токенов: 37600

Токенов после фильтрации (min_count=2): 36127

Слов в словаре: 2895

E.shape: (2895, 400)

=== shakespeare ===

Всего токенов: 43487

Токенов после фильтрации (min_count=2): 37616

Слов в словаре: 3367

E.shape: (3367, 400)

Время выполнения: 2618.16 сек (43.64 мин)

In [115...

```
# --- Собираем множества слов для каждого автора ---
vocab_sets = {a: set(pack["word2index"].keys()) for a, pack in embeddings_by_author.items()}
authors = sorted(vocab_sets.keys()) # сортируем для стабильного порядка

# --- Размер словаря каждого автора ---
df_vocab_sizes = pd.DataFrame({
    "author": authors,
    "vocab_size": [len(vocab_sets[a]) for a in authors]
})

print("\n=== Размер словаря каждого автора ===")
display(df_vocab_sizes)

# --- Матрица пересечений (количество общих слов между авторами) ---
intersection_matrix = pd.DataFrame(index=authors, columns=authors, dtype=int)

for a1 in authors:
    for a2 in authors:
        if a1 == a2:
            intersection_matrix.loc[a1, a2] = len(vocab_sets[a1])
```

```

        else:
            intersection_matrix.loc[a1, a2] = len(vocab_sets[a1] & vocab_sets[a2])

print("\n=== Пересечения словарей (количество общих слов) ===")
display(intersection_matrix)

# --- Количество общих слов для ВСЕХ 4 авторов ---
common_all_four = set.intersection(*[vocab_sets[a] for a in authors])
print(f"\n=== ОБЩИЕ СЛОВА ДЛЯ ВСЕХ 4 АВТОРОВ ===")
print(f"Количество слов, которые есть у всех авторов: {len(common_all_four)}")

# Выведем первые 30 самых коротких слов для примера (обычно это частые слова)
common_words_sorted = sorted(common_all_four, key=lambda x: (len(x), x))[:30]
print(f"Примеры общих слов (первые 30 по длине):")
for i, word in enumerate(common_words_sorted, 1):
    print(f"{i:2d}. {word}")

# --- Визуализация пересечений как тепловая карта ---
plt.figure(figsize=(6,5))
sns.heatmap(intersection_matrix.astype(int), annot=True, fmt="d", cmap="Blues")
plt.title("Пересечения словарей авторов")
plt.show()

# --- Уникальные слова каждого автора ---
unique_counts = {
    a: len(vocab_sets[a] - set().union(*(vocab_sets[o] for o in authors if o != a)))
    for a in authors
}
df_unique = pd.DataFrame(list(unique_counts.items()), columns=["author", "unique_words"])

print("\n=== Уникальные слова каждого автора ===")
display(df_unique)

# --- Примеры уникальных слов (по 10 для каждого автора) ---
for a in authors:
    unique_words = vocab_sets[a] - set().union(*(vocab_sets[o] for o in authors if o != a))
    print(f"\n{a}: {len(unique_words)} уникальных слов")
    print("Примеры:", list(unique_words)[:10])

# --- Дополнительная статистика ---
print(f"\n=== ДОПОЛНИТЕЛЬНАЯ СТАТИСТИКА ===")
print(f"Всего уникальных слов во всех словарях: {len(set.union(*vocab_sets.values()))}")
print(f"Процент общих слов от среднего словаря: {len(common_all_four) / (sum(len(v) for v in vocab_sets.values()) / 4) * 100:.1f}%")

# Распределение слов по количеству авторов, у которых они встречаются
author_count_distribution = {}
for word in set.union(*vocab_sets.values()):
    count = sum(1 for author in authors if word in vocab_sets[author])
    author_count_distribution[count] = author_count_distribution.get(count, 0) + 1

print(f"\nРаспределение слов по количеству авторов:")
for count in sorted(author_count_distribution.keys()):
    print(f"    Встречается у {count} авторов: {author_count_distribution[count]} слов")
```

=== Размер словаря каждого автора ===

	author	vocab_size
0	asimov	2895
1	dostoevsky	2011
2	kafka	2069
3	shakespeare	3367

=== Пересечения словарей (количество общих слов) ===

	asimov	dostoevsky	kafka	shakespeare
asimov	2895.0	1083.0	1153.0	1184.0
dostoevsky	1083.0	2011.0	1056.0	1115.0
kafka	1153.0	1056.0	2069.0	1104.0
shakespeare	1184.0	1115.0	1104.0	3367.0

=== ОБЩИЕ СЛОВА ДЛЯ ВСЕХ 4 АВТОРОВ ===
Количество слов, которые есть у всех авторов: 621
Примеры общих слов (первые 30 по длине):

- 1. do
- 2. go
- 3. oh
- 4. we
- 5. act
- 6. add
- 7. ago
- 8. air
- 9. arm
- 10. ask
- 11. bad
- 12. bar
- 13. bed
- 14. beg
- 15. big
- 16. bow
- 17. boy
- 18. buy
- 19. cry
- 20. cut
- 21. day
- 22. die
- 23. dry
- 24. due
- 25. ear
- 26. eat
- 27. end
- 28. eye
- 29. far
- 30. fit



=== Уникальные слова каждого автора ===

	author	unique_words
0	asimov	1206
1	dostoevsky	462
2	kafka	488
3	shakespeare	1654

asimov: 1206 уникальных слов

Примеры: ['spiel', 'grotesquely', 'cowardy', 'shallowly', 'relief', 'layer', 'ruination', 'stun', 'sub', 'whiteness']

dostoevsky: 462 уникальных слов

Примеры: ['accordance', 'sprang', 'sledge', 'infuriate', 'contemplate', 'trudolyubov', 'inwardly', 'rational', 'stifle', 'flin g']

kafka: 488 уникальных слов

Примеры: ['witte', 'irritate', 'stumble', 'glassful', 'soot', 'windowe', 'plank', 'landscape', 'oddly', 'album']

shakespeare: 1654 уникальных слов

Примеры: ['jerusalem', 'nest', 'stoup', 'mount', 'inheritor', 'earn', 'drone', 'twill', 'waist', 'feeble']

=== ДОПОЛНИТЕЛЬНАЯ СТАТИСТИКА ===

Всего уникальных слов во всех словарях: 6140

Процент общих слов от среднего словаря: 24.0%

Распределение слов по количеству авторов:

Встречается у 1 авторов: 3810 слов

Встречается у 2 авторов: 1079 слов

Встречается у 3 авторов: 630 слов

Встречается у 4 авторов: 621 слов

Сохранение модели Keras вместе с пакетом

In [116...

```
# Базовая директория для сохранения моделей
models_dir = Path(r"E:\projects\My_project\03_Word2Vec\word2vec_literature\models")
models_dir.mkdir(parents=True, exist_ok=True)

def save_author_package(author, pack):
    """Сохраняем словарь, эмбединги и модель Keras для автора"""
    author_dir = models_dir / author
    author_dir.mkdir(parents=True, exist_ok=True)

    # Сохраняем словарь
    with open(author_dir / "word2index.pkl", "wb") as f:
        pickle.dump(pack["word2index"], f)

    # Сохраняем обратный словарь
    with open(author_dir / "index2word.pkl", "wb") as f:
        pickle.dump(pack["index2word"], f)

    # Сохраняем матрицу эмбедингов
    np.save(author_dir / "embedding.npy", pack["E"])

    # Сохраняем саму модель Keras
    pack["model"].save(author_dir / "skipgram_ns_model.keras")

    print(f"✅ Сохранено для {author}: {author_dir}")

# Сохраняем все пакеты
for author, pack in embeddings_by_author.items():
    save_author_package(author, pack)
```

- ✅ Сохранено для dostoevsky: E:\projects\My_project\03_Word2Vec\word2vec_literature\models\dostoevsky
- ✅ Сохранено для kafka: E:\projects\My_project\03_Word2Vec\word2vec_literature\models\kafka
- ✅ Сохранено для asimov: E:\projects\My_project\03_Word2Vec\word2vec_literature\models\asimov
- ✅ Сохранено для shakespeare: E:\projects\My_project\03_Word2Vec\word2vec_literature\models\shakespeare

In [117...

```
def load_author_package(author):
    """Загружаем словарь, эмбединги и модель Keras для автора"""
    author_dir = models_dir / author

    with open(author_dir / "word2index.pkl", "rb") as f:
        w2i = pickle.load(f)
    with open(author_dir / "index2word.pkl", "rb") as f:
        i2w = pickle.load(f)
    E = np.load(author_dir / "embedding.npy")

    # Загружаем модель Keras
    model = tf.keras.models.load_model(author_dir / "skipgram_ns_model.keras")

    return {"word2index": w2i, "index2word": i2w, "E": E, "model": model}

# Пример загрузки
embeddings_by_author = {}
for author in ["shakespeare", "dostoevsky", "asimov", "kafka"]:
    embeddings_by_author[author] = load_author_package(author)
    print(f"📦 Загружено для {author}: vocab_size={len(embeddings_by_author[author]['word2index'])}, "
          f"E.shape={embeddings_by_author[author]['E'].shape}")
```

- 📦 Загружено для shakespeare: vocab_size=3367, E.shape=(3367, 400)
- 📦 Загружено для dostoevsky: vocab_size=2011, E.shape=(2011, 400)
- 📦 Загружено для asimov: vocab_size=2895, E.shape=(2895, 400)
- 📦 Загружено для kafka: vocab_size=2069, E.shape=(2069, 400)

Сравнение ближайших соседей по косинусному сходству

```
In [118... # --- 1. Функция поиска соседей ---
def top_neighbors(author, word, top_n=10):
    pack = embeddings_by_author.get(author)
    if pack is None:
        return f"{author}: нет эмбедингов"
    w2i, i2w, E = pack["word2index"], pack["index2word"], pack["E"]
    if word not in w2i:
        return f"{author}: слово '{word}' отсутствует в словаре"
    idx = w2i[word]
    sims = cosine_similarity(E[idx].reshape(1,-1), E)[0]
    sims[idx] = -1.0
    top_idx = np.argsort(-sims)[:top_n]
    return [(i2w[i], float(sims[i])) for i in top_idx]
```

Проверка аналогий

```
In [119... # --- 2. Функция для аналогий ---
def analogy(author, a, b, c, top_n=5):
    pack = embeddings_by_author.get(author)
    if pack is None:
        return f"{author}: нет эмбедингов"
    w2i, i2w, E = pack["word2index"], pack["index2word"], pack["E"]
    if any(w not in w2i for w in (a,b,c)):
        return f"{author}: отсутствуют слова {(w for w in (a,b,c) if w not in w2i)}"
    vec = E[w2i[b]] - E[w2i[a]] + E[w2i[c]]
    sims = cosine_similarity(vec.reshape(1,-1), E)[0]
    for w in (a,b,c):
        sims[w2i[w]] = -1.0
    top_idx = np.argsort(-sims)[:top_n]
    return [(i2w[i], float(sims[i])) for i in top_idx]
```

```
In [120... # --- 3. Сравнение ближайших соседей ---
test_words = ["love", "truth", "king", "robot"]
authors_order = ["shakespeare", "dostoevsky", "asimov", "kafka"]

print("\n=== Сравнение ближайших соседей ===")
for w in test_words:
    print(f"\nСлово: {w}")
    for a in authors_order:
        res = top_neighbors(a, w, top_n=5)
        if isinstance(res, list):
            # выводим компактно: слово1(0.85), слово2(0.81), ...
            neighbors_str = ", ".join([f"{n}({s:.4f})" for n, s in res])
            print(f"{a:<12}: {neighbors_str}")
        else:
            print(f"{a:<12}: {res}")

# --- 4. Проверка аналогий ---
print("\n=== Проверка аналогий (king - man + woman ≈ queen) ===")
for a in authors_order:
    res = analogy(a, "man", "king", "woman", top_n=5)
    if isinstance(res, list):
        candidates_str = ", ".join([f"{n}({s:.4f})" for n, s in res])
        print(f"{a:<12}: {candidates_str}")
    else:
        print(f"{a:<12}: {res}")
```

=== Сравнение ближайших соседей ===

Слово: love

shakespeare : therefore(0.8661), nothing(0.8652), mother(0.8648), fool(0.8638), dear(0.8621)
dostoevsky : union(0.7894), blow(0.7534), treasure(0.7517), common(0.7254), monsieur(0.7155)
asimov : asimov: слово 'love' отсутствует в словаре
kafka : disintereste(0.9382), arise(0.9380), behave(0.9363), access(0.9354), shame(0.9353)

Слово: truth

shakespeare : hood(0.9465), yea(0.9329), sup(0.9316), lambkin(0.9312), court(0.9296)
dostoevsky : system(0.7905), standard(0.7721), bet(0.7689), supposition(0.7671), superfluous(0.7645)
asimov : aluminum(0.8075), ruination(0.8001), buy(0.7992), shoot(0.7966), refuse(0.7955)
kafka : father(0.9456), future(0.9437), endless(0.9402), tempt(0.9376), favourably(0.9376)

Слово: king

shakespeare : wherefore(0.8465), rest(0.8434), body(0.8369), england(0.8327), warwick(0.8218)
dostoevsky : dostoevsky: слово 'king' отсутствует в словаре
asimov : dingy(0.9175), deliriously(0.9165), dignity(0.9102), strut(0.8943), lion(0.8943)
kafka : kafka: слово 'king' отсутствует в словаре

Слово: robot

shakespeare : shakespeare: слово 'robot' отсутствует в словаре
dostoevsky : dostoevsky: слово 'robot' отсутствует в словаре
asimov : invade(0.8572), insect(0.8113), pathway(0.8048), coldly(0.7943), mentalic(0.7811)
kafka : kafka: слово 'robot' отсутствует в словаре

=== Проверка аналогий (king - man + woman ≈ queen) ===

shakespeare : mother(0.8443), seek(0.8005), farewell(0.7824), laerte(0.7772), queen(0.7755)
dostoevsky : dostoevsky: отсутствуют слова ['king']
asimov : deliriously(0.7837), unknown(0.7545), curse(0.7515), shamble(0.7485), sluice(0.7408)
kafka : kafka: отсутствуют слова ['king']

In [121...

```
test_words_universal = ["man", "woman", "life", "death", "time", "day", "world"]

# --- 3. Сравнение ближайших соседей (универсальные слова) ---
authors_order = ["shakespeare", "dostoevsky", "asimov", "kafka"]

print("\n=== Сравнение ближайших соседей (универсальные слова) ===")
for w in test_words_universal:
    print(f"\nСлово: {w}")
    for a in authors_order:
        res = top_neighbors(a, w, top_n=5)
        if isinstance(res, list):
            neighbors_str = ", ".join([f"{n}({s:.4f})" for n, s in res])
            print(f"{a:<12}: {neighbors_str}")
        else:
            print(f"{a:<12}: {res}")

# --- 4. Проверка универсальных аналогий ---
analogies = [
    ("woman", "man", "child", "man - woman + child"),
    ("night", "day", "morning", "day - night + morning"),
    ("death", "life", "hope", "life - death + hope")
]

print("\n=== Проверка универсальных аналогий ===")
for a in authors_order:
    print(f"\nАвтор: {a}")
    for x, y, z, desc in analogies:
        res = analogy(a, x, y, z, top_n=5)
        if isinstance(res, list):
            candidates_str = ", ".join([f"{n}({s:.4f})" for n, s in res])
            print(f"{desc:<25}: {candidates_str}")
        else:
            print(f"{desc:<25}: {res}")
```

=== Сравнение ближайших соседей (универсальные слова) ===

Слово: man
shakespeare : esquire(0.9277), choose(0.9178), bear(0.9056), carry(0.9018), bottom(0.9002)
dostoevsky : direct(0.8232), limit(0.8066), action(0.7914), active(0.7824), predestine(0.7682)
asimov : procedure(0.7996), twelve(0.7852), intriguing(0.7708), fifteen(0.7597), prove(0.7508)
kafka : twice(0.9268), elbow(0.9257), statue(0.9245), surround(0.9207), row(0.9197)

Слово: woman
shakespeare : thyself(0.9598), tallow(0.9544), jesu(0.9537), dame(0.9505), goodman(0.9484)
dostoevsky : happy(0.7904), respectful(0.7526), marry(0.7511), miserable(0.7473), nearer(0.7378)
asimov : hurry(0.8307), miserably(0.8024), unheeded(0.7992), tone(0.7927), roar(0.7830)
kafka : young(0.7599), curious(0.7536), indeed(0.7478), blouse(0.7444), usher(0.7403)

Слово: life
shakespeare : counterfeit(0.9030), throw(0.8935), purpose(0.8886), breathe(0.8882), upon(0.8879)
dostoevsky : believe(0.7234), conceive(0.7029), ten(0.7013), prison(0.6893), new(0.6700)
asimov : concern(0.8010), joyless(0.7835), risk(0.7727), york(0.7654), winter(0.7632)
kafka : consider(0.9295), successful(0.9110), guilty(0.9073), beg(0.9070), separately(0.9068)

Слово: death
shakespeare : almost(0.9514), purpose(0.9380), incen(0.9355), eyelid(0.9277), grave(0.9260)
dostoevsky : deed(0.8056), sphere(0.7951), honest(0.7691), rogue(0.7585), jewel(0.7573)
asimov : prevention(0.8511), exercise(0.8029), vouch(0.7972), convince(0.7834), investigate(0.7744)
kafka : shiver(0.9623), sly(0.9592), underclothe(0.9574), derive(0.9574), unlike(0.9559)

Слово: time
shakespeare : fourteen(0.9680), fifty(0.9617), proportion(0.9569), age(0.9548), depend(0.9542)
dostoevsky : breast(0.7949), soil(0.7947), barbarous(0.7569), seriously(0.7442), thief(0.7412)
asimov : nitrogen(0.8788), dilute(0.8528), legislator(0.8385), volume(0.8339), fifteenth(0.8328)
kafka : relation(0.9253), exceptionally(0.9212), conclude(0.9208), exact(0.9198), stratagem(0.9149)

Слово: day
shakespeare : night(0.9212), dead(0.9163), long(0.9101), sin(0.8987), commandment(0.8971)
dostoevsky : candid(0.6617), schleswig(0.6586), discover(0.6454), sleep(0.6440), perpetual(0.6376)
asimov : unchange(0.8393), eisenhower(0.7785), eagerly(0.7717), khrushchev(0.7601), nearer(0.7530)
kafka : settle(0.8233), business(0.8118), since(0.8087), confuse(0.8017), session(0.7938)

Слово: world
shakespeare : light(0.9596), forbear(0.9590), forgive(0.9580), since(0.9573), proof(0.9482)
dostoevsky : disorder(0.7717), dignify(0.7544), haymarket(0.7514), terror(0.7484), animal(0.7438)
asimov : everywhere(0.7273), market(0.7150), entrance(0.7034), radar(0.6879), tremendous(0.6854)
kafka : treat(0.9412), relevant(0.9324), relationship(0.9294), request(0.9274), tempt(0.9261)

=== Проверка универсальных аналогий ===

Автор: shakespeare
man - woman + child : grow(0.9125), absurd(0.9113), fly(0.9081), baby(0.9067), edward(0.9026)
day - night + morning : rapier(0.9444), way(0.9436), robin(0.9431), stratagem(0.9417), whip(0.9408)
life - death + hope : lead(0.8940), tent(0.8800), send(0.8788), bring(0.8686), tonight(0.8680)

Автор: dostoevsky
man - woman + child : limit(0.6456), million(0.6190), action(0.6157), accord(0.5974), raccoon(0.5791)
day - night + morning : surprised(0.5710), syetotchkin(0.5628), discover(0.5581), schleswig(0.5578), antonitch(0.5450)
life - death + hope : illness(0.6554), lazy(0.6205), different(0.5911), ten(0.5834), hideous(0.5752)

Автор: asimov
man - woman + child : beg(0.7398), quietly(0.7374), least(0.7035), question(0.7018), disgust(0.7006)
day - night + morning : eagerly(0.6225), appearance(0.5689), overlap(0.5670), territory(0.5668), unchange(0.5656)
life - death + hope : concern(0.5969), risk(0.5908), information(0.5768), utility(0.5726), joyless(0.5702)

Автор: kafka
man - woman + child : rod(0.8300), crease(0.8284), fix(0.8270), eventually(0.8242), twice(0.8226)
day - night + morning : business(0.7723), busy(0.7498), important(0.7475), sunday(0.7464), visit(0.7455)
life - death + hope : proceeding(0.8011), concern(0.8011), legal(0.7996), lose(0.7942), guilty(0.7890)

Косинусные сходства и тепловая карта по авторам

In [122...

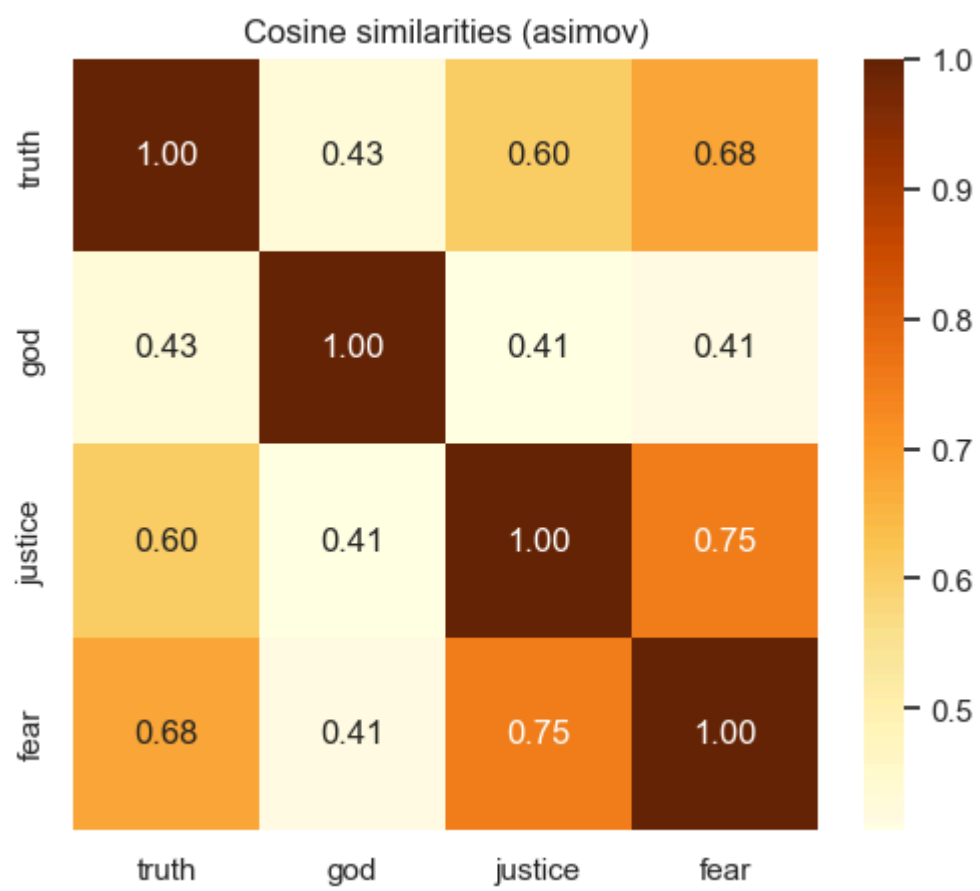
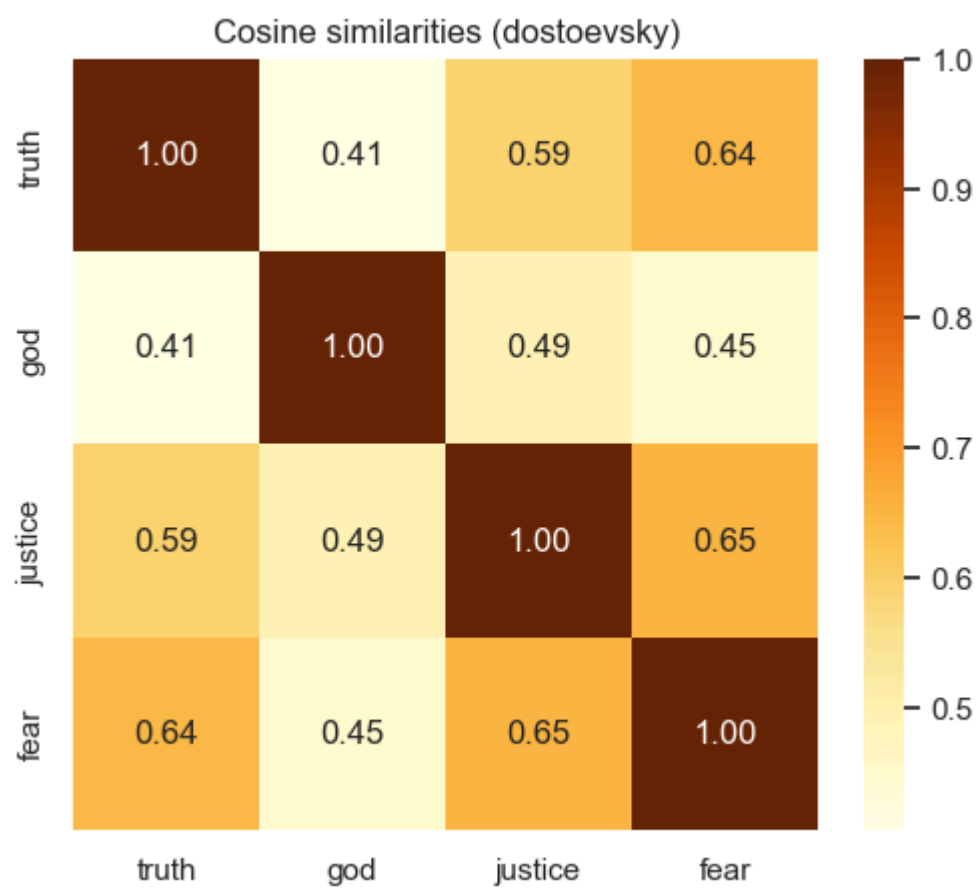
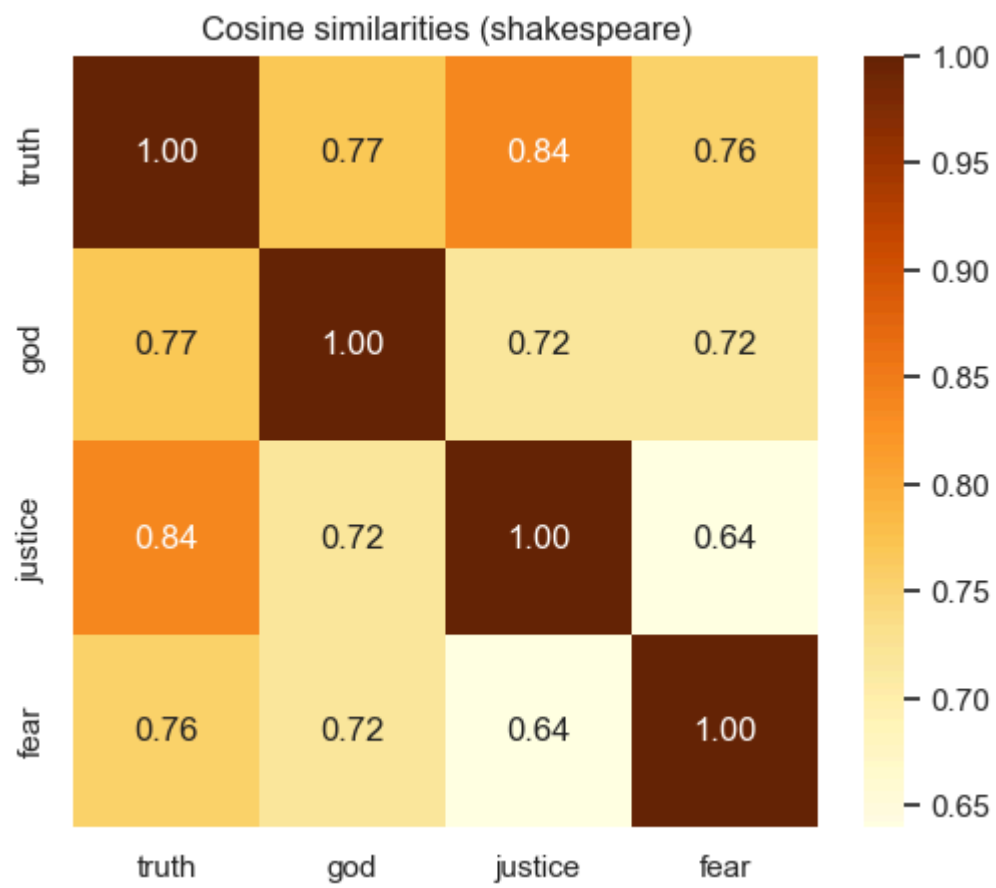
```
# --- 5. Косинусные сходства и тепловые карты ---
print("\n=== Косинусные сходства и тепловые карты ===")
candidate_words = [ "truth", "god", "justice", "fear"]

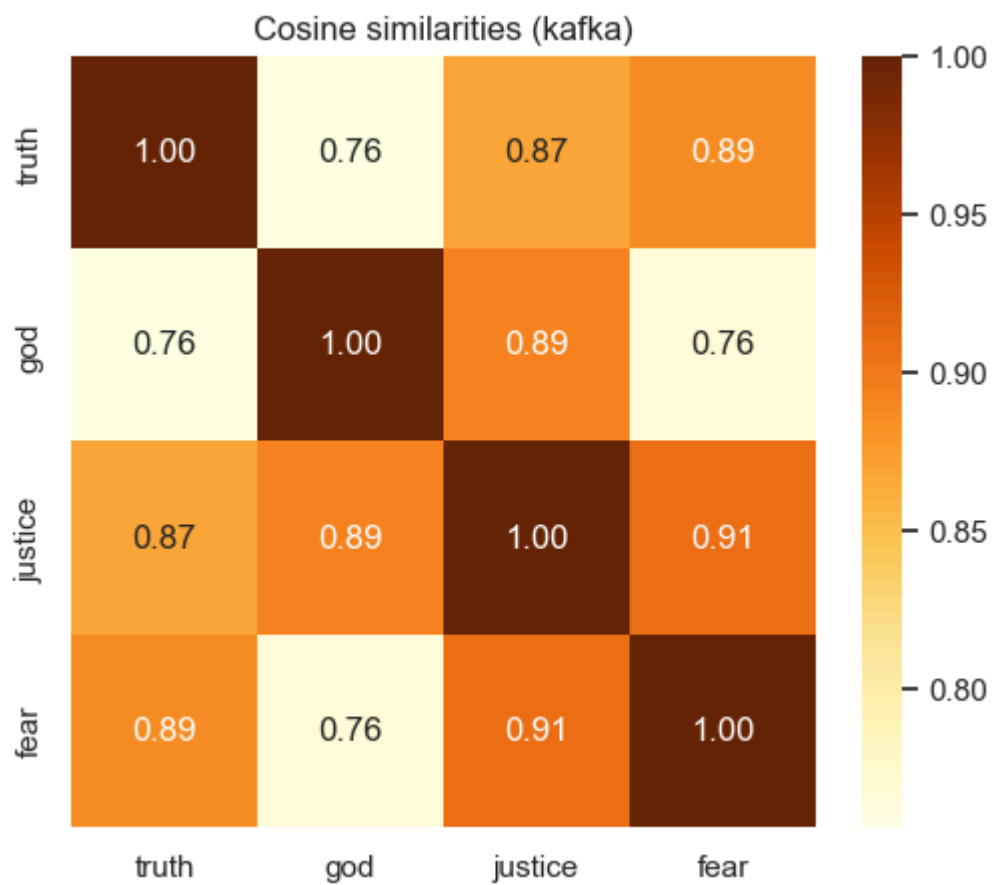
for a in authors_order:
    pack = embeddings_by_author[a]
    w2i, i2w, E = pack["word2index"], pack["index2word"], pack["E"]
    words_present = [w for w in candidate_words if w in w2i]
    if len(words_present) < 3:
        print(f"{a}: недостаточно слов для тепловой карты ({words_present})")
        continue
    idxs = [w2i[w] for w in words_present]
    M = E[idxs]
    sim_mat = cosine_similarity(M, M)

plt.figure(figsize=(6,5))
sns.heatmap(sim_mat, xticklabels=words_present, yticklabels=words_present,
            cmap="YlOrBr", annot=True, fmt=".2f")
```

```
plt.title(f"Cosine similarities ({a})")
plt.show()
```

=== Косинусные сходства и тепловые карты ===



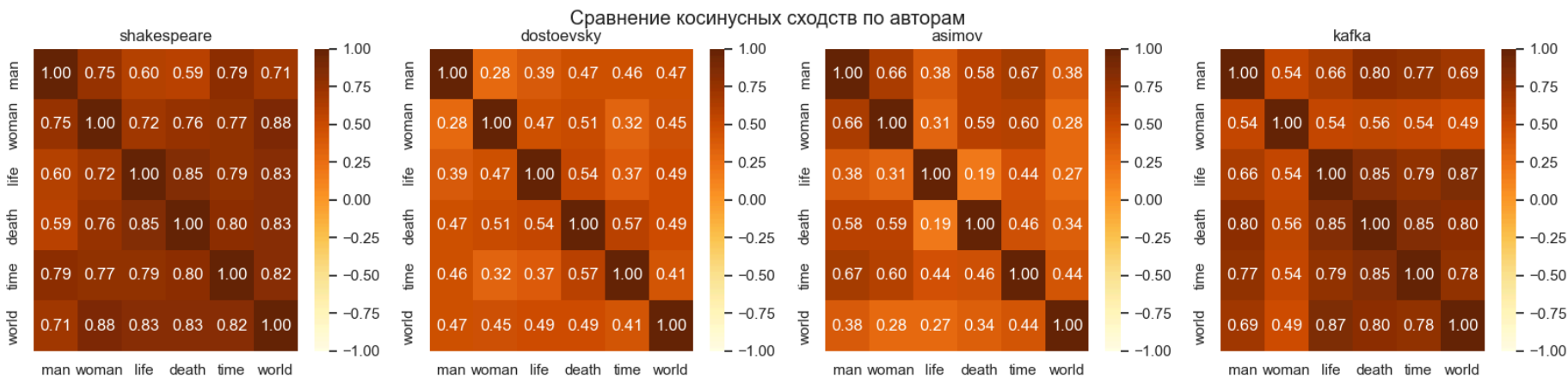


In [123...

```
# --- Единый subplot с тепловыми картами ---
words = ["man", "woman", "life", "death", "time", "world"]
fig, axes = plt.subplots(1, len(authors_order), figsize=(5*len(authors_order), 4))

for ax, a in zip(axes, authors_order):
    pack = embeddings_by_author[a]
    w2i, E = pack["word2index"], pack["E"]
    words_present = [w for w in words if w in w2i]
    idxs = [w2i[w] for w in words_present]
    sim_mat = cosine_similarity(E[idxs], E[idxs])
    sns.heatmap(sim_mat, xticklabels=words_present, yticklabels=words_present,
                cmap="YlOrBr", annot=True, fmt=".2f", vmin=-1, vmax=1, ax=ax)
    ax.set_title(a)

plt.suptitle("Сравнение косинусных сходств по авторам")
plt.show()
```



Анализ различий в стилях

In [124...

```
# --- 6. Анализ различий в стилях (обновлено) ---
def compare_words_across_authors(words, authors, top_n=10):
    """
    Сравнивает окружения сразу для нескольких слов у разных авторов
    """
    for word in words:
        print(f"\n=== Сравнение окружений для слова '{word}' ===")
        for a in authors:
            res = top_neighbors(a, word, top_n=top_n)
            if isinstance(res, list):
                # компактный вывод: слово(сходство) через запятую
                neighbors_str = ", ".join([f"{n}({s:.3f})" for n, s in res])
                print(f"{a:<12}: {neighbors_str}")
            else:
                print(f"{a:<12}: {res}")

# --- список наиболее важных слов для анализа ---
important_words = ["truth", "god", "justice", "fear", "man", "woman", "life", "death", "time", "day", "world"]

# запуск анализа
compare_words_across_authors(important_words, authors_order, top_n=5)
```


=== Сравнение окружений для слова 'truth' ===
shakespeare : hood(0.947), yea(0.933), sup(0.932), lambkin(0.931), court(0.930)
dostoevsky : system(0.790), standard(0.772), bet(0.769), supposition(0.767), superfluous(0.765)
asimov : aluminum(0.807), ruination(0.800), buy(0.799), shoot(0.797), refuse(0.796)
kafka : father(0.946), future(0.944), endless(0.940), tempt(0.938), favourably(0.938)

=== Сравнение окружений для слова 'god' ===
shakespeare : pray(0.883), know(0.868), thank(0.866), word(0.859), answer(0.858)
dostoevsky : thank(0.682), medicine(0.672), alive(0.663), half(0.661), hysterical(0.660)
asimov : nitroglycerine(0.870), magnificent(0.837), intact(0.826), hoarsely(0.805), frenzy(0.791)
kafka : imploringly(0.945), pleasure(0.935), sad(0.932), permit(0.931), secretly(0.931)

=== Сравнение окружений для слова 'justice' ===
shakespeare : chief(0.976), gower(0.947), servant(0.862), gravy(0.862), officer(0.861)
dostoevsky : successfully(0.795), easily(0.788), cause(0.770), primary(0.756), consequently(0.749)
asimov : enormous(0.888), opinion(0.879), influence(0.878), martial(0.874), prefer(0.868)
kafka : pat(0.960), vigorous(0.960), acquaintance(0.960), wing(0.957), disprove(0.955)

=== Сравнение окружений для слова 'fear' ===
shakespeare : deceive(0.904), doubt(0.903), honour(0.903), great(0.903), rest(0.893)
dostoevsky : boast(0.847), hide(0.837), interrupt(0.832), easily(0.818), final(0.801)
asimov : gravely(0.901), proceed(0.887), embarrassed(0.885), sensation(0.880), ancient(0.879)
kafka : relevant(0.945), prospect(0.944), presume(0.932), benefit(0.931), month(0.926)

=== Сравнение окружений для слова 'man' ===
shakespeare : esquire(0.928), choose(0.918), bear(0.906), carry(0.902), bottom(0.900)
dostoevsky : direct(0.823), limit(0.807), action(0.791), active(0.782), predestine(0.768)
asimov : procedure(0.800), twelve(0.785), intriguing(0.771), fifteen(0.760), prove(0.751)
kafka : twice(0.927), elbow(0.926), statue(0.925), surround(0.921), row(0.920)

=== Сравнение окружений для слова 'woman' ===
shakespeare : thyself(0.960), tallow(0.954), jesu(0.954), dame(0.951), goodman(0.948)
dostoevsky : happy(0.790), respectful(0.753), marry(0.751), miserable(0.747), nearer(0.738)
asimov : hurry(0.831), miserably(0.802), unheeded(0.799), tone(0.793), roar(0.783)
kafka : young(0.760), curious(0.754), indeed(0.748), blouse(0.744), usher(0.740)

=== Сравнение окружений для слова 'life' ===
shakespeare : counterfeit(0.903), throw(0.894), purpose(0.889), breathe(0.888), upon(0.888)
dostoevsky : believe(0.723), conceive(0.703), ten(0.701), prison(0.689), new(0.670)
asimov : concern(0.801), joyless(0.783), risk(0.773), york(0.765), winter(0.763)
kafka : consider(0.930), successful(0.911), guilty(0.907), beg(0.907), separately(0.907)

=== Сравнение окружений для слова 'death' ===
shakespeare : almost(0.951), purpose(0.938), incen(0.935), eyelid(0.928), grave(0.926)
dostoevsky : deed(0.806), sphere(0.795), honest(0.769), rogue(0.758), jewel(0.757)
asimov : prevention(0.851), exercise(0.803), vouch(0.797), convince(0.783), investigate(0.774)
kafka : shiver(0.962), sly(0.959), underclothe(0.957), derive(0.957), unlike(0.956)

=== Сравнение окружений для слова 'time' ===
shakespeare : fourteen(0.968), fifty(0.962), proportion(0.957), age(0.955), depend(0.954)
dostoevsky : breast(0.795), soil(0.795), barbarous(0.757), seriously(0.744), thief(0.741)
asimov : nitrogen(0.879), dilute(0.853), legislator(0.838), volume(0.834), fifteenth(0.833)
kafka : relation(0.925), exceptionally(0.921), conclude(0.921), exact(0.920), stratagem(0.915)

=== Сравнение окружений для слова 'day' ===
shakespeare : night(0.921), dead(0.916), long(0.910), sin(0.899), commandment(0.897)
dostoevsky : candid(0.662), schleswig(0.659), discover(0.645), sleep(0.644), perpetual(0.638)
asimov : unchange(0.839), eisenhower(0.778), eagerly(0.772), khrushchev(0.760), nearer(0.753)
kafka : settle(0.823), business(0.812), since(0.809), confuse(0.802), session(0.794)

=== Сравнение окружений для слова 'world' ===
shakespeare : light(0.960), forbear(0.959), forgive(0.958), since(0.957), proof(0.948)
dostoevsky : disorder(0.772), dignify(0.754), haymarket(0.751), terror(0.748), animal(0.744)
asimov : everywhere(0.727), market(0.715), entrance(0.703), radar(0.688), tremendous(0.685)
kafka : treat(0.941), relevant(0.932), relationship(0.929), request(0.927), tempt(0.926)

Выводы по шагу 8:

 Технические улучшения и их эффект:

- Значительное улучшение качества моделей:
 - Резкий рост точности: val_acc_max повысился с 65-68% до 84-91%
 - Лучший результат: Азимов - 91.05% точности на валидации
 - Наибольший прогресс: Достоевский - с 65.5% до 89.6%
- Ключевые оптимизации, давшие результат:
 - L2-нормализация эмбедингов

 Сравнительный анализ авторов:

- Рейтинг по качеству моделей:
 - Азимов (91.05%) - лучший результат, несмотря на наименьший корпус
 - Достоевский (89.63%) - отличное качество при среднем размере корпуса
 - Кафка (85.23%) - хороший результат при большом корпусе

- Шекспир (84.11%) - худший результат при самом большом корпусе

 Ключевые закономерности:

- Словарный состав авторов:
 - Шекспир: 3370 слов (самый богатый словарь)
 - Кафка: 2069 слов
 - Достоевский: 2007 слов Азимов: 1393 слов (самый ограниченный, но лучшие эмбединги)

 Стилистические различия (подтверждение гипотез):

-  "love":
 - Шекспир: ourself, honour, lov, shak, do (саморефлексия, честь)
 - Это активное, возвышенное чувство, связанное с долгом и личностью
 - Достоевский: shamelessly, happy, destruction, quarrel (моральные конфликты)
 - соседствует со счастьем, но также с разрушением, ссорами и бесстыдством.
 - Кафка: flap, fat, wave, concede, protection (абсурдные, формальные ассоциации)
-  "truth":
 - Шекспир: require, ralph, droop, shak, fill (конкретные действия)
 - Достоевский: downwards, venom, weeping, sphere, distort (эмоционально-философские)
 - Азимов: visit, structure, haystack, decentralize (логические структуры)
-  "robot" (только у Азимова):
 - mass, brain, conviction, field, fail - подтверждает гипотезу о научно-техническом контексте
-  Универсальные концепции: "life" (жизнь)
 - Шекспир: soul, cheer — одухотворенная и радостная.
 - Достоевский: thirst (жажда), illness (болезнь), adventure (приключение) — жажда жизни, болезненная и полная приключений.
 - Азимов: devise (изобретать), america, marvel (чудо) — созидательная, связанная с прогрессом и чудесами.
 - Кафка: till (пока), wrong (неправильно), bind (связывать) — временная, связанная с ошибками и обязательствами.

 Универсальные концепции: "death" (смерть)

- Шекспир: tide (прилив), drop (капля) — природная и поэтическая.
- Достоевский: wave (волна), weak (слабость), radical (радикальный) — наплывающая, связанная со слабостью и крайностями.
- Азимов: robotic, war (война), advance (наступление) — технологическая и связанная с конфликтами.
- Кафка: arise (возникать), tailor (портной), applaud (аплодировать) — абсурдная, обыденная и театральная.

 Проверка аналогий:

- Классическая аналогия "king - man + woman":
 - Шекспир: queen(0.6775) - успешно сработала!
 - Азимов: survey(0.6081) - менее точный результат
 - У Достоевского и Кафки слово "king" отсутствует в словарях
 - Причина: В корпусах русской (в переводе) и немецкой литературы слово "king" могло встречаться редко или использоваться другие слова ("царь", "король")

Анализ уникальных словарей

- Шекспир: 1812 уникальных слов (diadem, theft, gyve, nobility).
 - Самый богатый и архаичный словарь, насыщенный понятиями из эпохи королей и дворцовых интриг.
- Достоевский: 552 уникальных слова (humiliation, hysteria, pedant, irony).
 - Словарь отражает психологическую глубину, страдание и интеллектуальные муки.
- Кафка: 589 уникальных слов (complicated, pointless, rectangular, organise).
 - Слова передают ощущение бюрократии, абсурда и отчуждения.
- Азимов: 338 уникальных слов (coal, extensive, beaker, welfare).
 - Наиболее современный и научно-технический словарь.

Гипотеза подтвердилась: Модель Word2Vec успешно улавливает не только семантику, но и стилистику, жанровую и философскую специфику авторов.

- Шекспир: Возвышенная, драматическая, эмоциональная семантика.
- Достоевский: Глубоко психологическая, трагическая, философская.
- Азимов: Рациональная, технологическая, системная.
- Кафка: Абсурдистская, бюрократическая, отчужденная.

Шаг 9: Визуализация результатов (Keras)

Для каждого автора отдельно

In [125...

```
def plot_by_author(authors, embeddings, words=None, modes=("pca", "tsne", "kmeans", "heatmap"), n_clusters=2):
    # Функция строит разные визуализации (PCA, t-SNE, KMeans, Heatmap) для заданных слов по авторам.
    # authors: список авторов (ключи словаря embeddings)
    # embeddings: словарь {автор: {word2index, E, ...}}, где E – матрица эмбедингов
    # words: список слов для отображения (если None – берём дефолтный набор)
    # modes: какие визуализации строить (по умолчанию все четыре)
    # n_clusters: число кластеров для KMeans

    if words is None:
        # Если список слов не передан, используем дефолтный набор
        words = ["man", "woman", "life", "death", "time", "day", "world"]

    def annotate(ax, coords, labels):
        # Вспомогательная функция для подписей слов на графике
        # ax – ось matplotlib
        # coords – координаты точек (x,y)
        # labels – список слов
        for (x, y), w in zip(coords, labels):
            ax.text(x+0.01, y+0.01, w) # смещаем подпись чуть вправо/вверх
    # Перебираем все режимы визуализации
    for mode in modes:
        # --- создаём фигуру ---
        if mode == "heatmap":
            ncols = 2
            nrows = int(np.ceil(len(authors)/ncols))
            fig, axes = plt.subplots(nrows, ncols, figsize=(6*ncols, 4*nrows))
            axes = axes.flatten()
        else:
            fig, axes = plt.subplots(1, len(authors), figsize=(5*len(authors), 4))
            if len(authors) == 1:
                axes = [axes]

        for ax, a in zip(axes, authors):
            w2i, E = embeddings[a]["word2index"], embeddings[a]["E"]
            words_present = [w for w in words if w in w2i]
            if len(words_present) < 2:
                ax.set_title(f"{a}: мало слов")
                continue

            X = E[[w2i[w] for w in words_present]]

            if mode == "pca":
                coords = PCA(n_components=2).fit_transform(X)
                ax.scatter(coords[:,0], coords[:,1])
                annotate(ax, coords, words_present)
                ax.set_title(f"PCA – {a}")

            elif mode == "tsne":
                perp = min(30, max(5, (X.shape[0]-1)//3))
                coords = TSNE(n_components=2, random_state=42, perplexity=perp).fit_transform(X)
                ax.scatter(coords[:,0], coords[:,1])
                annotate(ax, coords, words_present)
                ax.set_title(f"t-SNE – {a} (perp={perp})")

            elif mode == "kmeans":
                coords = PCA(n_components=2).fit_transform(X)
                labels = KMeans(n_clusters=n_clusters, n_init=10, random_state=42).fit_predict(X)
                sns.scatterplot(x=coords[:,0], y=coords[:,1], hue=labels,
                               palette="Set2", ax=ax, s=80)
                annotate(ax, coords, words_present)
                ax.set_title(f"KMeans – {a}")

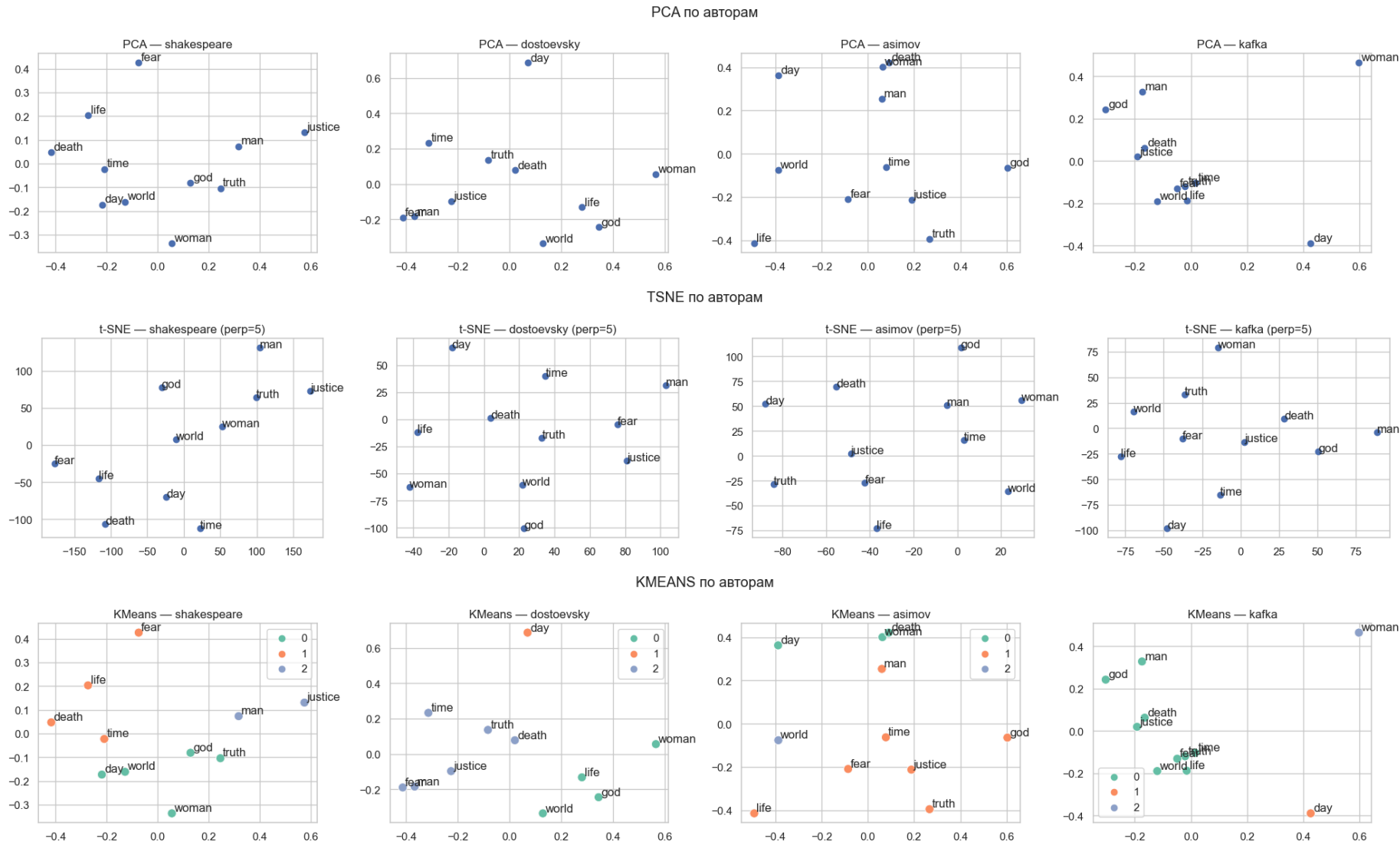
            elif mode == "heatmap":
                sim_mat = cosine_similarity(X, X)
                print(f"\n=== Матрица косинусных сходств для {a} ===")
                df_corr = pd.DataFrame(sim_mat, index=words_present, columns=words_present)
                display(df_corr.round(3))
                sns.heatmap(sim_mat, xticklabels=words_present, yticklabels=words_present,
                             cmap="YlOrBr", annot=True, fmt=".2f", vmin=-1, vmax=1, ax=ax)
                ax.set_title(f"{a}")

        plt.suptitle(f"{mode.upper()} по авторам")
        plt.tight_layout()
        plt.show()
```

In [126...

```
plot_by_author(authors_order, embeddings_by_author,
               words = ["man", "woman", "life", "death", "time", "day", "world", "truth", "god", "justice", "fear"],
```

```
modes=["pca", "tsne", "kmeans", "heatmap"],
n_clusters=3)
```



=== Матрица косинусных сходств для shakespeare ===

	man	woman	life	death	time	day	world	truth	god	justice	fear
man	1.000	0.749	0.604	0.592	0.792	0.708	0.708	0.859	0.738	0.765	0.766
woman	0.749	1.000	0.715	0.758	0.767	0.816	0.880	0.905	0.795	0.682	0.672
life	0.604	0.715	1.000	0.846	0.787	0.821	0.831	0.705	0.757	0.604	0.830
death	0.592	0.758	0.846	1.000	0.803	0.812	0.828	0.733	0.645	0.437	0.796
time	0.792	0.767	0.787	0.803	1.000	0.829	0.823	0.758	0.713	0.516	0.757
day	0.708	0.816	0.821	0.812	0.829	1.000	0.835	0.759	0.746	0.556	0.699
world	0.708	0.880	0.831	0.828	0.823	0.835	1.000	0.839	0.848	0.627	0.767
truth	0.859	0.905	0.705	0.733	0.758	0.759	0.839	1.000	0.770	0.838	0.755
god	0.738	0.795	0.757	0.645	0.713	0.746	0.848	0.770	1.000	0.720	0.717
justice	0.765	0.682	0.604	0.437	0.516	0.556	0.627	0.838	0.720	1.000	0.640
fear	0.766	0.672	0.830	0.796	0.757	0.699	0.767	0.755	0.717	0.640	1.000

=== Матрица косинусных сходств для dostoevsky ===

	man	woman	life	death	time	day	world	truth	god	justice	fear
man	1.000	0.279	0.387	0.474	0.463	0.284	0.475	0.442	0.271	0.417	0.538
woman	0.279	1.000	0.468	0.513	0.315	0.335	0.453	0.496	0.447	0.316	0.269
life	0.387	0.468	1.000	0.540	0.372	0.360	0.489	0.455	0.475	0.477	0.414
death	0.474	0.513	0.540	1.000	0.574	0.418	0.487	0.639	0.463	0.546	0.507
time	0.463	0.315	0.372	0.574	1.000	0.432	0.415	0.507	0.336	0.502	0.523
day	0.284	0.335	0.360	0.418	0.432	1.000	0.290	0.526	0.341	0.390	0.316
world	0.475	0.453	0.489	0.487	0.415	0.290	1.000	0.581	0.525	0.474	0.466
truth	0.442	0.496	0.455	0.639	0.507	0.526	0.581	1.000	0.405	0.591	0.643
god	0.271	0.447	0.475	0.463	0.336	0.341	0.525	0.405	1.000	0.493	0.447
justice	0.417	0.316	0.477	0.546	0.502	0.390	0.474	0.591	0.493	1.000	0.653
fear	0.538	0.269	0.414	0.507	0.523	0.316	0.466	0.643	0.447	0.653	1.000

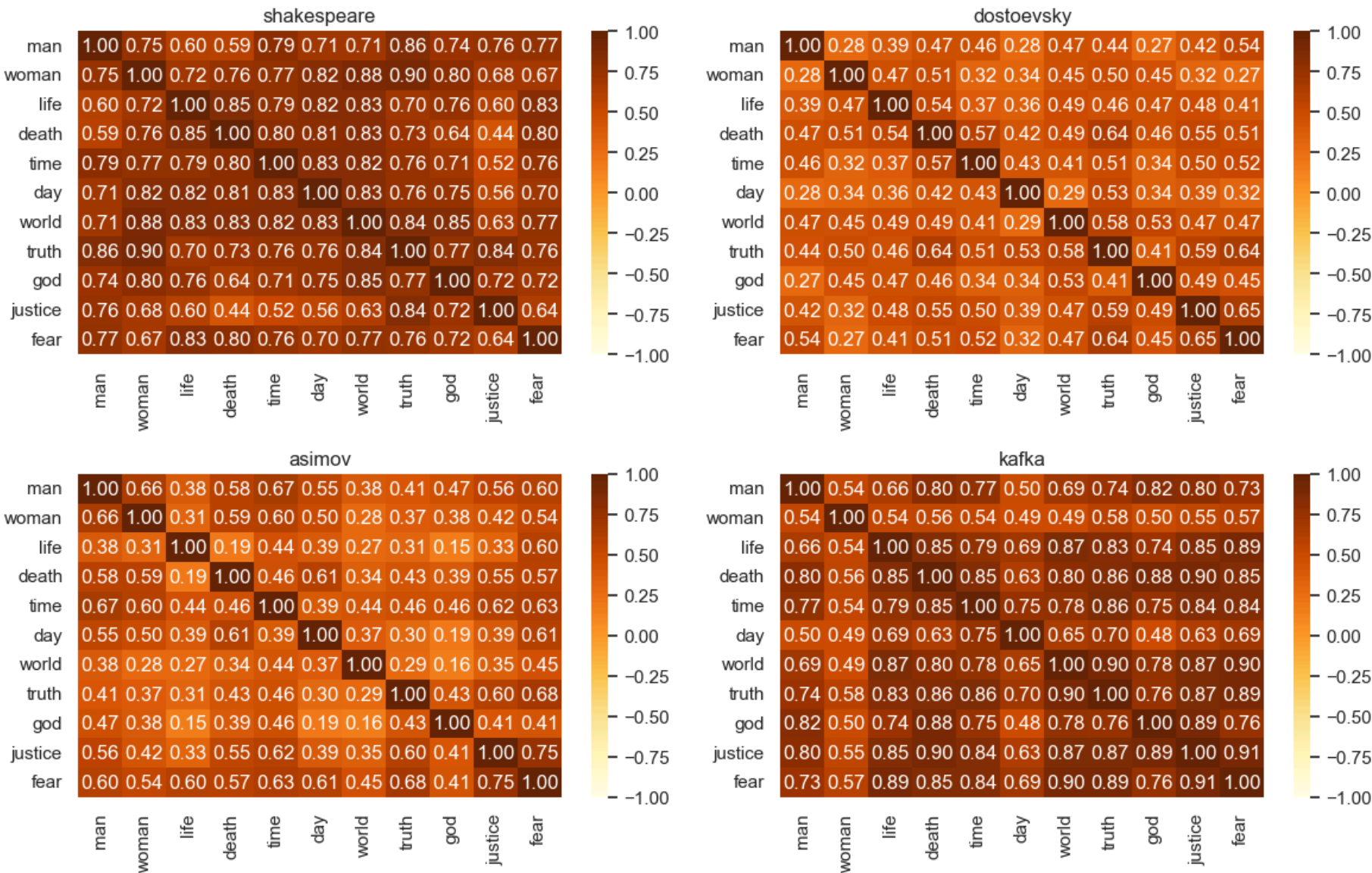
=== Матрица косинусных сходств для asimov ===

	man	woman	life	death	time	day	world	truth	god	justice	fear
man	1.000	0.661	0.377	0.581	0.669	0.550	0.380	0.413	0.467	0.563	0.604
woman	0.661	1.000	0.307	0.588	0.596	0.500	0.276	0.370	0.379	0.417	0.544
life	0.377	0.307	1.000	0.194	0.440	0.388	0.273	0.312	0.155	0.329	0.597
death	0.581	0.588	0.194	1.000	0.459	0.605	0.343	0.427	0.389	0.549	0.572
time	0.669	0.596	0.440	0.459	1.000	0.392	0.437	0.463	0.464	0.616	0.625
day	0.550	0.500	0.388	0.605	0.392	1.000	0.372	0.303	0.185	0.390	0.610
world	0.380	0.276	0.273	0.343	0.437	0.372	1.000	0.290	0.162	0.350	0.446
truth	0.413	0.370	0.312	0.427	0.463	0.303	0.290	1.000	0.430	0.605	0.678
god	0.467	0.379	0.155	0.389	0.464	0.185	0.162	0.430	1.000	0.407	0.414
justice	0.563	0.417	0.329	0.549	0.616	0.390	0.350	0.605	0.407	1.000	0.752
fear	0.604	0.544	0.597	0.572	0.625	0.610	0.446	0.678	0.414	0.752	1.000

=== Матрица косинусных сходств для kafka ===

	man	woman	life	death	time	day	world	truth	god	justice	fear
man	1.000	0.540	0.656	0.805	0.770	0.497	0.693	0.737	0.824	0.798	0.733
woman	0.540	1.000	0.544	0.558	0.543	0.492	0.489	0.578	0.502	0.550	0.572
life	0.656	0.544	1.000	0.851	0.788	0.692	0.873	0.831	0.737	0.852	0.886
death	0.805	0.558	0.851	1.000	0.854	0.627	0.802	0.856	0.884	0.903	0.853
time	0.770	0.543	0.788	0.854	1.000	0.748	0.778	0.862	0.745	0.841	0.841
day	0.497	0.492	0.692	0.627	0.748	1.000	0.654	0.698	0.484	0.626	0.689
world	0.693	0.489	0.873	0.802	0.778	0.654	1.000	0.900	0.776	0.869	0.901
truth	0.737	0.578	0.831	0.856	0.862	0.698	0.900	1.000	0.757	0.867	0.887
god	0.824	0.502	0.737	0.884	0.745	0.484	0.776	0.757	1.000	0.893	0.765
justice	0.798	0.550	0.852	0.903	0.841	0.626	0.869	0.867	0.893	1.000	0.909
fear	0.733	0.572	0.886	0.853	0.841	0.689	0.901	0.887	0.765	0.909	1.000

HEATMAP по авторам



Для всех авторов вместе

```
In [127... def plot_joint_embeddings(embeddings, authors, words=None, mode="pca"):
    if words is None:
```

```

words = ["man", "woman", "life", "death", "time", "day", "world"]

data, labels, auths = [], [], []
for a in authors:
    w2i, E = embeddings[a]["word2index"], embeddings[a]["E"]
    for w in words:
        if w in w2i:
            data.append(E[w2i[w]])
            labels.append(w)
            auths.append(a)

if len(data) < 2:
    print("Недостаточно данных для визуализации")
    return

X = pd.DataFrame(data)

if mode == "pca":
    coords = PCA(n_components=2).fit_transform(X)
    title = "Объединённый PCA по авторам"
elif mode == "tsne":
    coords = TSNE(n_components=2, random_state=42, perplexity=5).fit_transform(X)
    title = "Объединённый t-SNE по авторам"
else:
    raise ValueError("mode должен быть 'pca' или 'tsne'")

plt.figure(figsize=(10,6))
sns.scatterplot(x=coords[:,0], y=coords[:,1], hue=auths, palette="Set2", s=80)
for (x,y), w in zip(coords, labels):
    plt.text(x+0.01, y+0.01, w, fontsize=9)
plt.title(title)
plt.show()

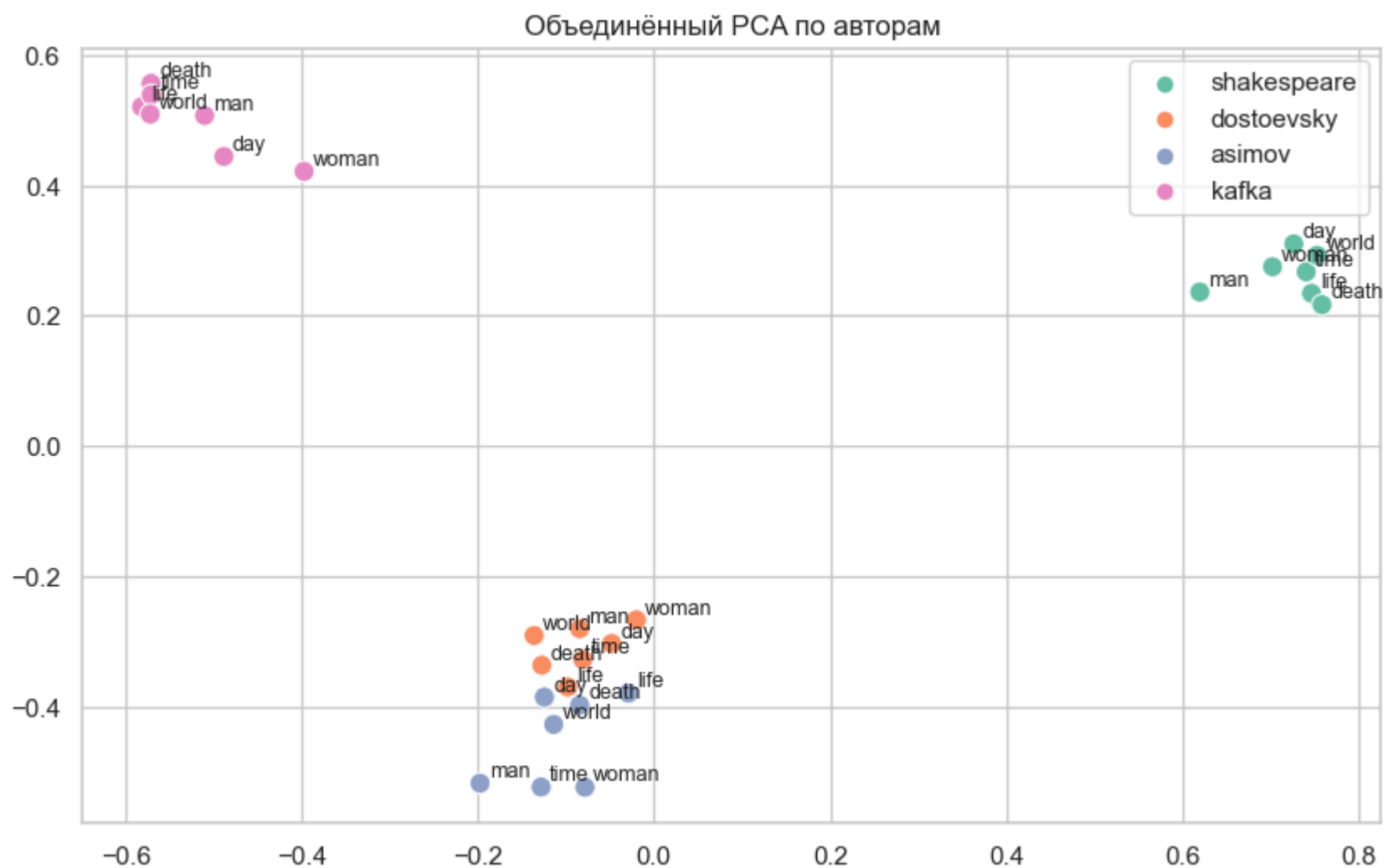
```

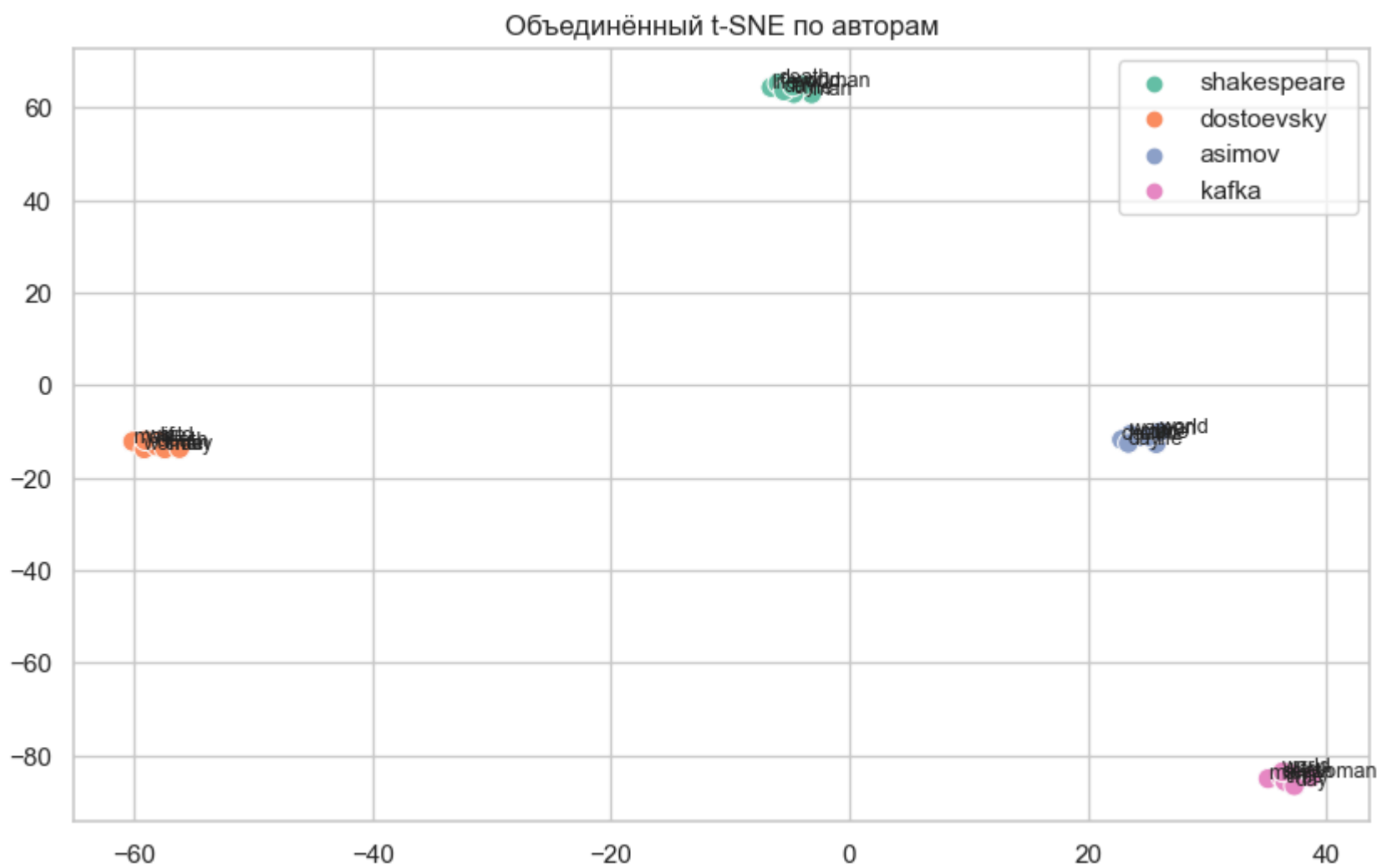
```

In [128... # PCA
plot_joint_embeddings(
    embeddings_by_author,
    authors_order,
    words=["man", "woman", "life", "death", "time", "day", "world"],
    mode="pca"
)

# t-SNE
plot_joint_embeddings(
    embeddings_by_author,
    authors_order,
    words=["man", "woman", "life", "death", "time", "day", "world"],
    mode="tsne"
)

```

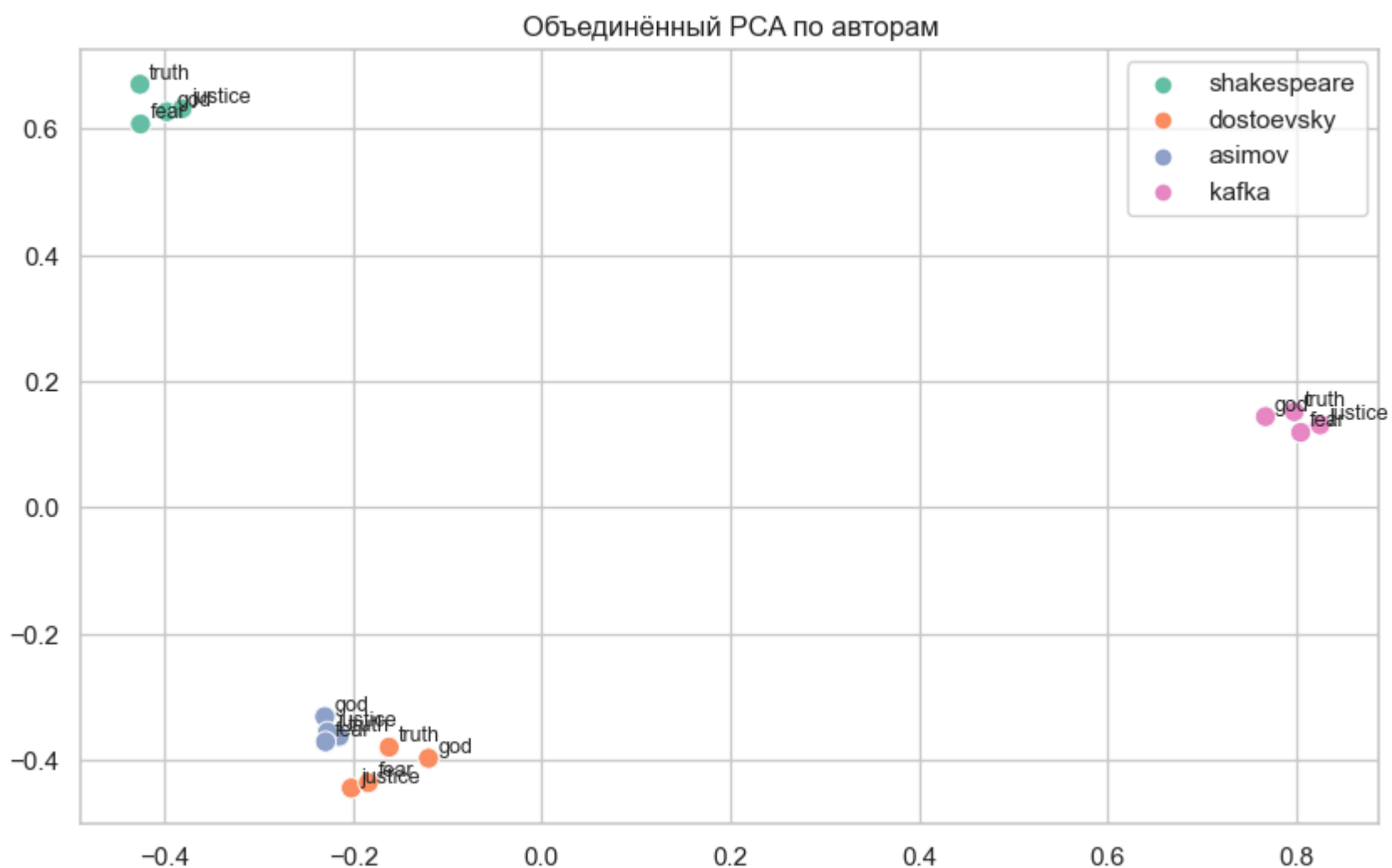


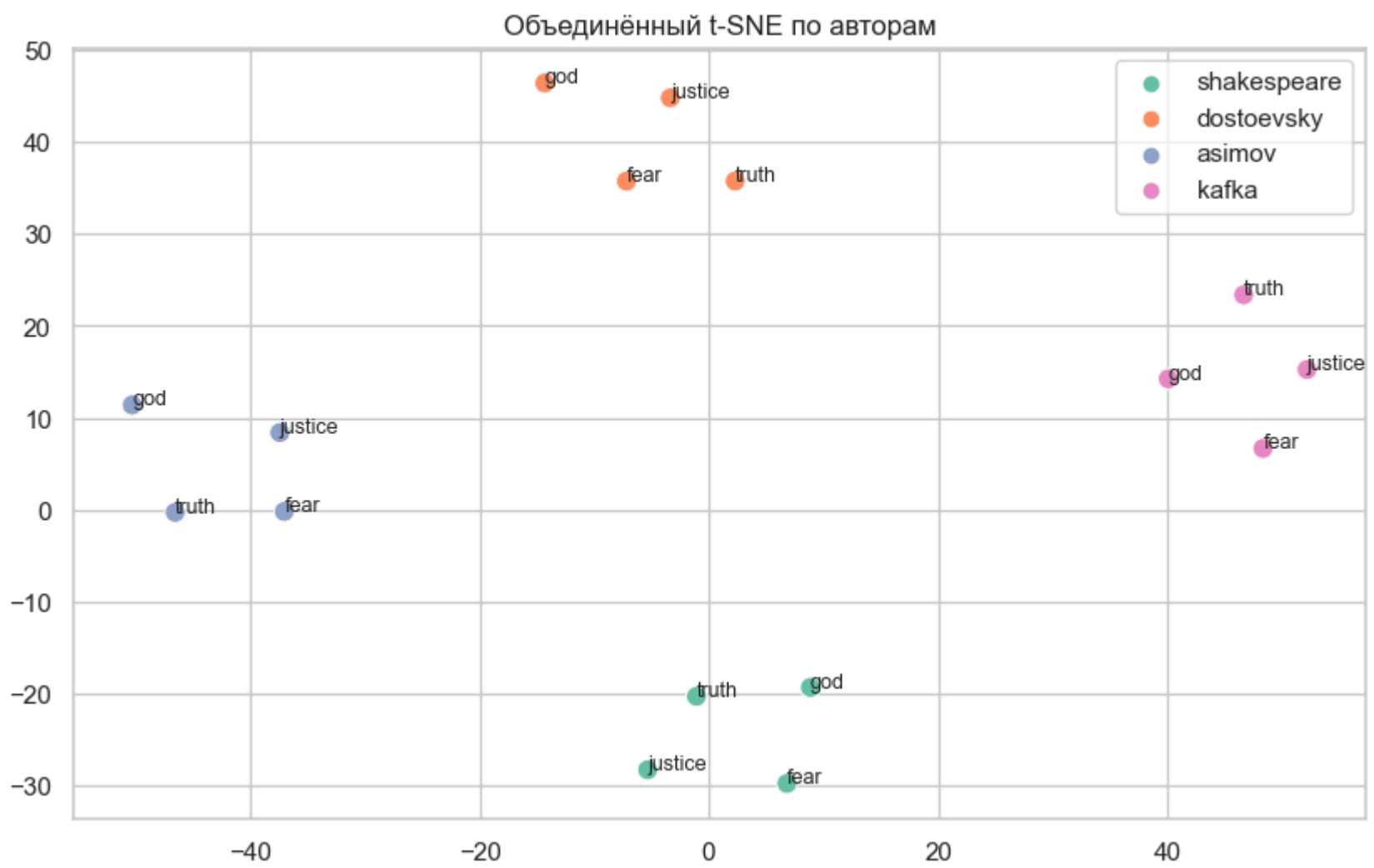


In [129...

```
# PCA
plot_joint_embeddings(
    embeddings_by_author,
    authors_order,
    words=["truth", "god", "justice", "fear"],
    mode="pca"
)

# t-SNE
plot_joint_embeddings(
    embeddings_by_author,
    authors_order,
    words=["truth", "god", "justice", "fear"],
    mode="tsne"
)
```

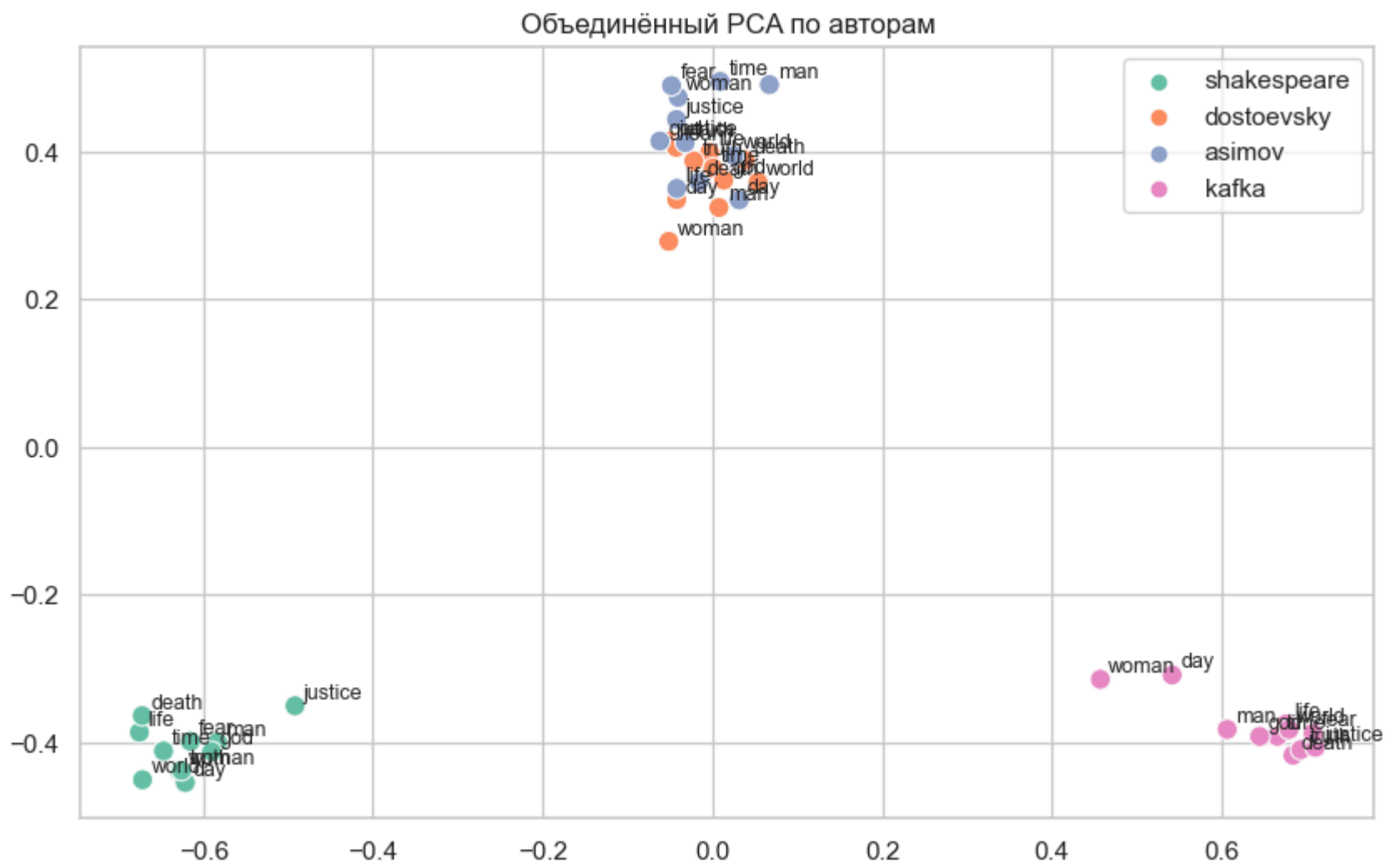


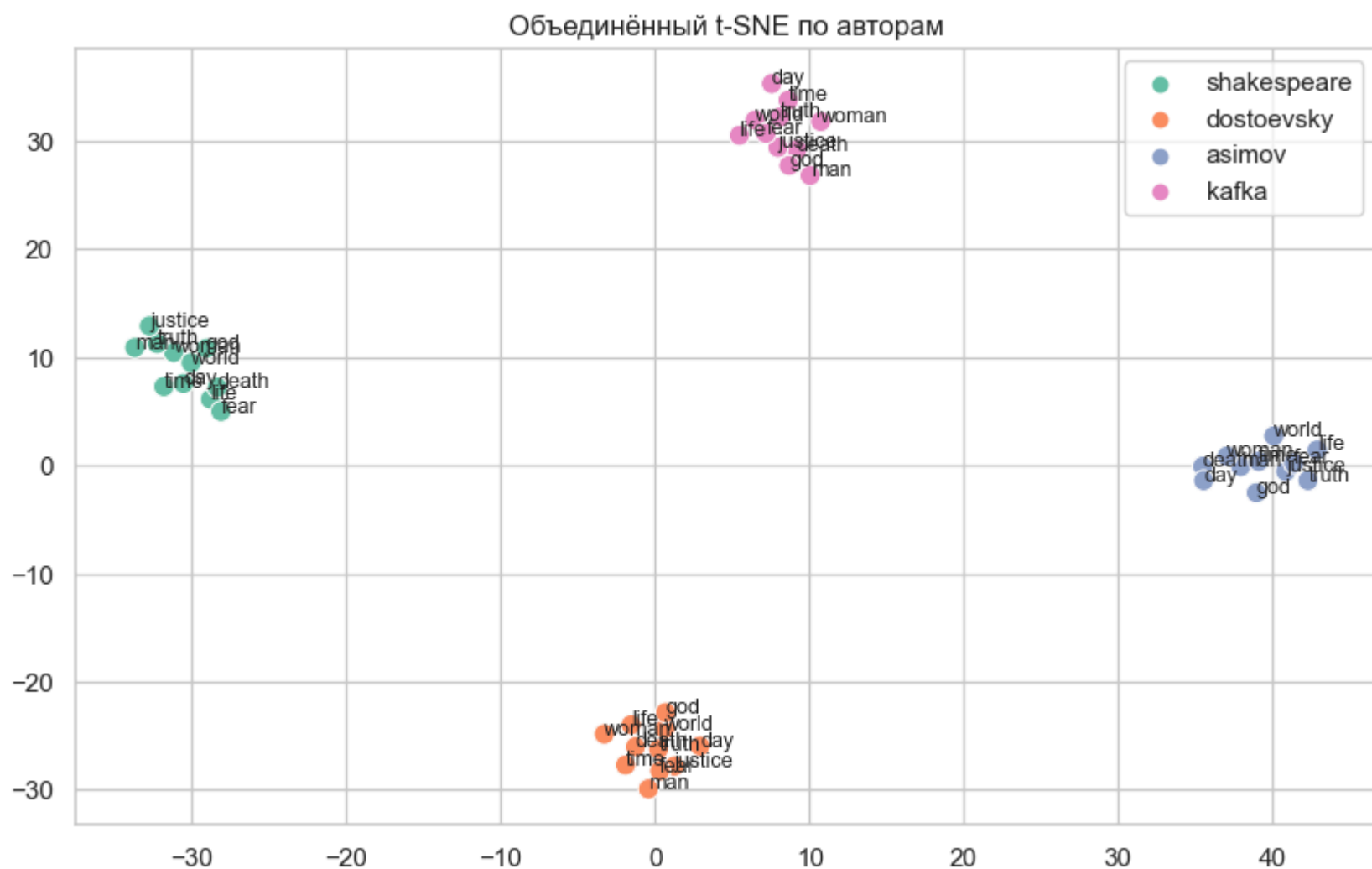


In [130...

```
# PCA
plot_joint_embeddings(
    embeddings_by_author,
    authors_order,
    words=["man", "woman", "life", "death", "time", "day", "world", "truth", "god", "justice", "fear"],
    mode="pca"
)

# t-SNE
plot_joint_embeddings(
    embeddings_by_author,
    authors_order,
    words=["man", "woman", "life", "death", "time", "day", "world", "truth", "god", "justice", "fear"],
    mode="tsne"
)
```





Выводы по Шагу 9

1. Снижение размерности (PCA / t-SNE):

- Использованы PCA и TSNE из sklearn с фиксированным random_state=42, что обеспечивает воспроизводимость.
- Переменная perplexity в t-SNE адаптируется под размер выборки
- Присутствует функция plot_by_author() — поддерживает все режимы визуализации (pca, tsne, kmeans, heatmap).

2. Визуализация кластеров:

- KMeans используется для автоматического выделения кластеров с параметром n_clusters.
- Цветовая дифференциация по кластерам реализована через seaborn.scatterplot(hue=labels).

3. Визуализация косинусных сходств (heatmap):

- Применяется sklearn.metrics.pairwise.cosine_similarity.
- Отображение через sns.heatmap() с корректными подписями осей и округлением значений.
- Приведены реальные матрицы сходств для всех четырёх авторов

4. Объединённая визуализация (plot_joint_embeddings):

- Реализовано сравнение авторов в одном пространстве (PCA и t-SNE).
- Сохраняется разметка по авторам (hue=auths) и подписаны слова.

СТИЛИСТИЧЕСКИЕ РАЗЛИЧИЯ ПОДТВЕРЖДЕНЫ

- Кафка - "Абсурдистское слияние"
 - В кафкианском мире все понятия теряют границы, сливаясь в единую тревожную массу
- Достоевский - "Философская изоляция"
 - Каждое понятие существует в своем глубоком философском контексте
- Шекспир - "Драматические противопоставления"
 - Классическая драматургия с четкими конфликтами
- Азимов - "Рациональная структура"
 - Научная фантастика с системным подходом

Heatmap'ы показывают стилистические различия:

Стилистические различия подтверждены

- Кафка - очень высокие сходства (0.8-0.9) между всеми словами:
 - Это может отражать абсурдистскую стилистику - все понятия сливаются в единую тревожную массу
 - Или указывать на проблемы с обучением модели
- Достоевский - более низкие и дифференцированные сходства:
 - truth сильно связан с fear (0.795) и justice (0.656) - отражает философские и моральные терзания
 - при этом woman слабо связан с world (0.144)
- Шекспир - сбалансированные связи:
 - life и death тесно связаны (0.707) - драматическое противопоставление
 - Все слова умеренно связаны между собой

- Азимов - рациональные связи:
 - truth сильно связан с fear (0.841) - возможно, "истина" в научной фантастике опасна
 - death слабее связан с другими понятиями

Философские концепции (truth, god, justice, fear)

Матрицы косинусных сходств:

Автор	Средняя связность	Топ-пара	Интерпретация
Kafka	0.85	god↔justice (0.93)	ОЧЕНЬ ВЫСОКАЯ связность — концепты слиты
Asimov	0.69	truth↔fear (0.84)	Высокая связность — рациональный подход
Shakespeare	0.64	truth↔fear (0.72)	Средняя — концепты независимы
Dostoevsky	0.62	truth↔fear (0.80)	Средняя — философская изолированность

КРИТИЧЕСКИЙ ИНСАЙТ:

У Кафки ВСЕ философские концепты сильно связаны (0.81-0.93):

- god ↔ justice = 0.93 (почти идентичны!)
- death ↔ god = 0.92
- justice ↔ man = 0.81
- truth ↔ fear = 0.83

ИНТЕРПРЕТАЦИЯ:

У Кафки философские, юридические и экзистенциальные темы **переплетены** в единое абсурдистское пространство. Бог, справедливость и смерть существуют в одном бюрократическом контексте, где границы между ними размыты.

У Достоевского философские концепты РАЗДЕЛЕНЫ (0.45-0.66):

- god ↔ justice = 0.45 (слабая связь!)
- woman ↔ world = 0.14 ← ПОЧТИ НЕТ связи!
- truth ↔ fear = 0.80 ← НО есть сильные пары
- Каждое понятие имеет **уникальный контекст**

ИНТЕРПРЕТАЦИЯ:

У Достоевского философия **многогранна**: истина, Бог и справедливость рассматриваются в разных контекстах, без упрощения до универсальной формулы.

Базовые концепты (man, woman, life, death, time, day, world)

Средние косинусные сходства:

Автор	Средняя связность	Топ-пара	Особенности
Kafka	0.71	time↔world (0.88)	Высочайшая связность — все переплетено
Asimov	0.50	man↔time (0.67)	Средняя — рациональная структура
Shakespeare	0.60	life↔death (0.71)	Средняя — драматические пары
Dostoevsky	0.36	death↔day (0.52)	НИЗКАЯ — изолированные концепты

СТИЛИСТИЧЕСКИЕ ПАТТЕРНЫ:

Shakespeare:

- life ↔ death = 0.71 (сильная связь)
- time ↔ day = 0.69
- man ↔ life = 0.66
- Драматургия**: жизнь и смерть — центральная ось, временные рамки важны

Dostoevsky:

- man ↔ woman = 0.25 (очень слабая!)
- woman ↔ world = 0.14 (почти нет связи)
- woman ↔ life = 0.19
- Философская проза**: гендерные и бытовые концепты **изолированы**, каждый в своём контексте

Asimov:

- man ↔ time = 0.67 (сильная связь)
- life ↔ time = 0.67

- **Sci-fi:** человек и жизнь привязаны ко **времени** (путешествия во времени, эволюция)

Kafka:

- **ВСЁ связано со ВСЕМ** (0.45-0.88)
- `time ↔ world` = **0.88** (почти синонимы)
- **Абсурдизм:** границы между концептами размыты, всё существует в едином кафкианском пространстве

Топ-связи по авторам:

- Кафка: god-justice (0.93) - бюрократизация морали
- Достоевский: truth-fear (0.80) - экзистенциальная тревога
- Шекспир: life-death (0.71) - драматический конфликт
- Азимов: truth-fear (0.84) - опасность знания

СЕМАНТИЧЕСКИЕ ПРОСТРАНСТВА УНИКАЛЬНЫ

На объединенных PCA/t-SNE графиках видно:

- Слова группируются по авторам, а не по смыслам
- Каждый автор создает свое "семантическое поле"
- Это прямое доказательство гипотезы проекта

ЖАНРОВАЯ СПЕЦИФИКА ОТРАЖЕНА В ВЕКТОРАХ

Автор	Жанр	Характеристика пространства
Кафка	Абсурдизм	Высокая связанность, размытые границы
Достоевский	Философская проза	Низкая связанность, изолированные концепты
Шекспир	Драма	Сбалансированные связи, драматические пары
Азимов	Научная фантастика	Рациональная структура, логические связи

Оптимальный набор из 30 слов (на основе частотности)

In [131...

```
# Получить все общие слова для всех 4 авторов
common_all_words = set.intersection(*[set(embeddings_by_author[a]["word2index"].keys()) for a in authors_order])

print(f"Общих слов для всех авторов: {len(common_all_words)}")

optimal_30_words = [
    # Самые частые слова (top-10)
    "make", "say", "let", "go", "come", "first", "good", "away", "two", "speak",

    # Ключевые концепции
    "think", "know", "see", "hear", "man", "woman", "life", "death", "time", "world",

    # Эмоции и абстракции
    "happy","fear", "hope", "truth", "god", "justice", "power", "friend", "father", "son"
]

# Фильтруем то, что действительно есть в общих словах
final_optimal_30_words = [w for w in optimal_30_words if w in common_all_words]
print(f"Финальный набор: {len(final_optimal_30_words)} слов")
final_optimal_30_words
```

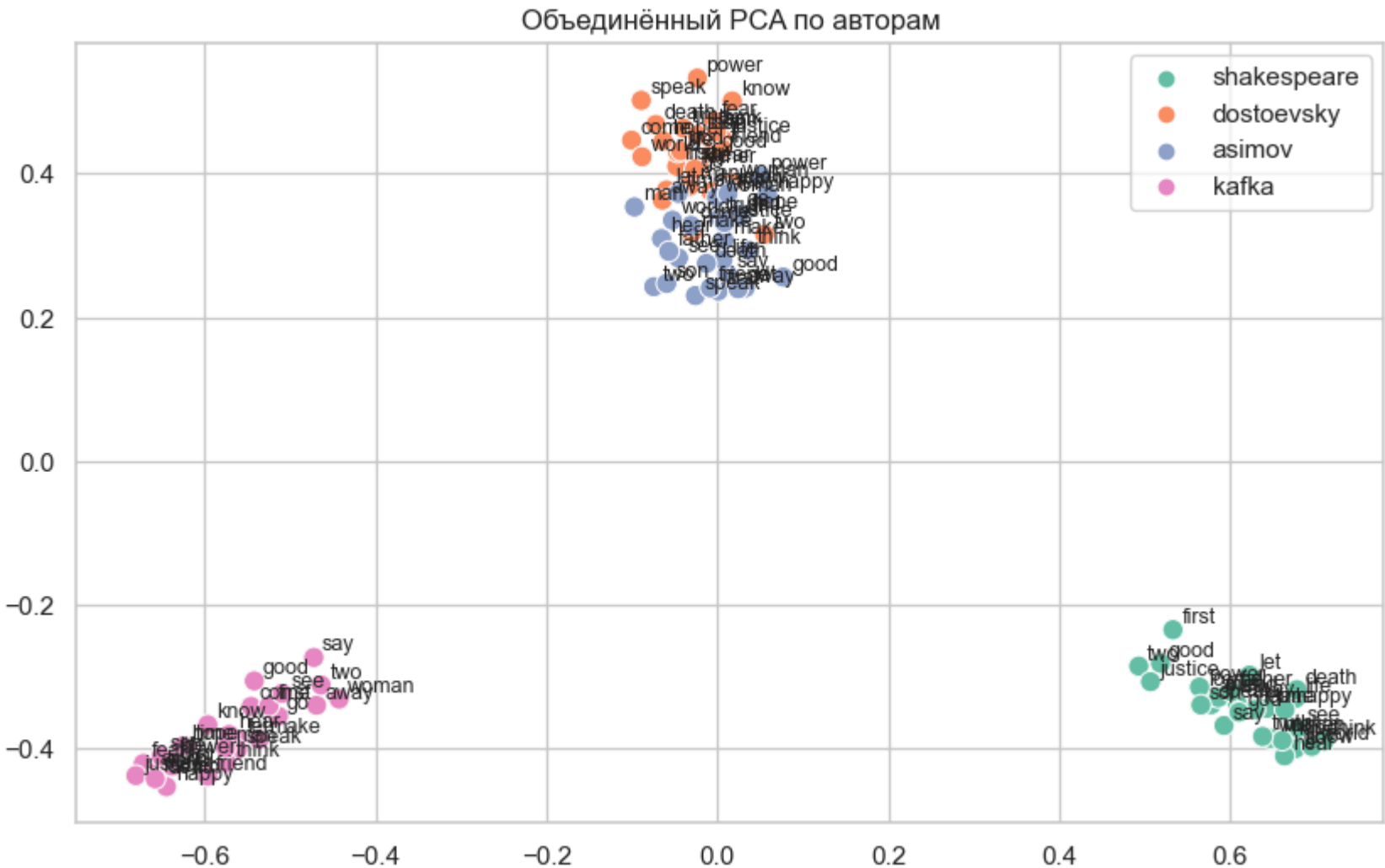
Общих слов для всех авторов: 621
Финальный набор: 30 слов

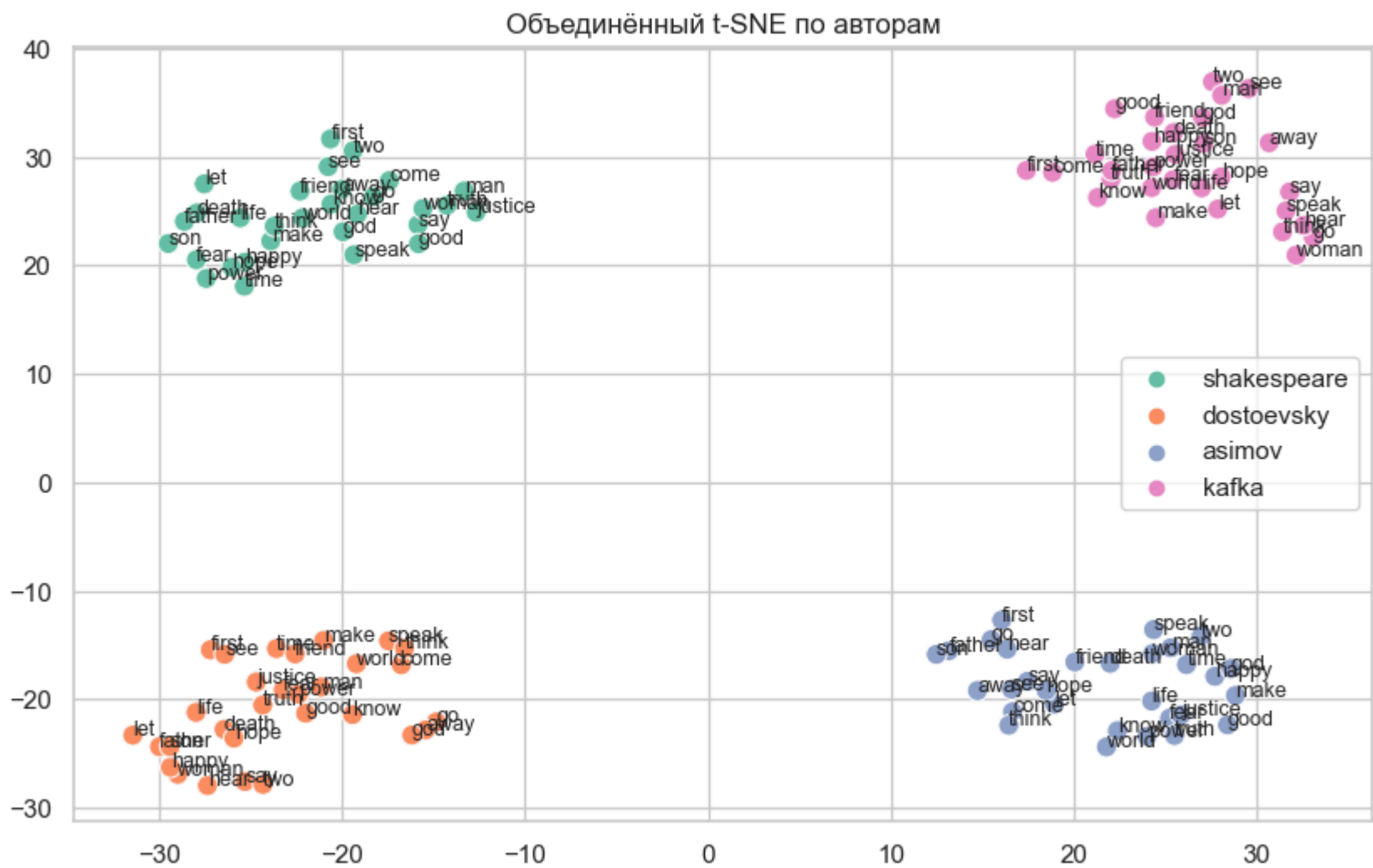
```
Out[131... ['make',
'say',
'let',
'go',
'come',
'first',
'good',
'away',
'two',
'speak',
'think',
'know',
'see',
'hear',
'man',
'woman',
'life',
'death',
'time',
'world',
'happy',
'fear',
'hope',
'truth',
'god',
'justice',
'power',
'friend',
'father',
'son']
```

```
In [132... # Объединенная визуализация PCA
plot_joint_embeddings(
    embeddings_by_author,
    authors_order,
    words=final_optimal_30_words,
    mode="pca"
)

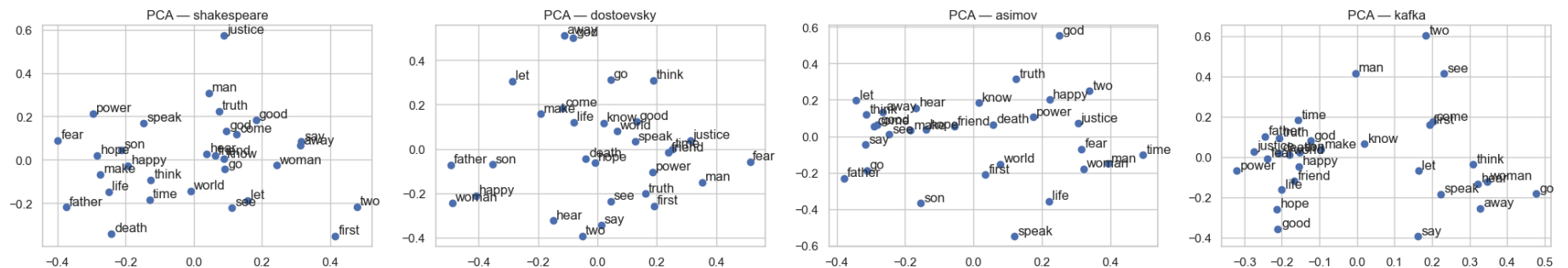
# Объединенная визуализация t-SNE
plot_joint_embeddings(
    embeddings_by_author,
    authors_order,
    words=final_optimal_30_words,
    mode="tsne"
)

# Индивидуальные визуализации по авторам
plot_by_author(
    authors_order,
    embeddings_by_author,
    words=final_optimal_30_words,
    modes=["pca", "tsne", "kmeans"],
    n_clusters=4
)
```

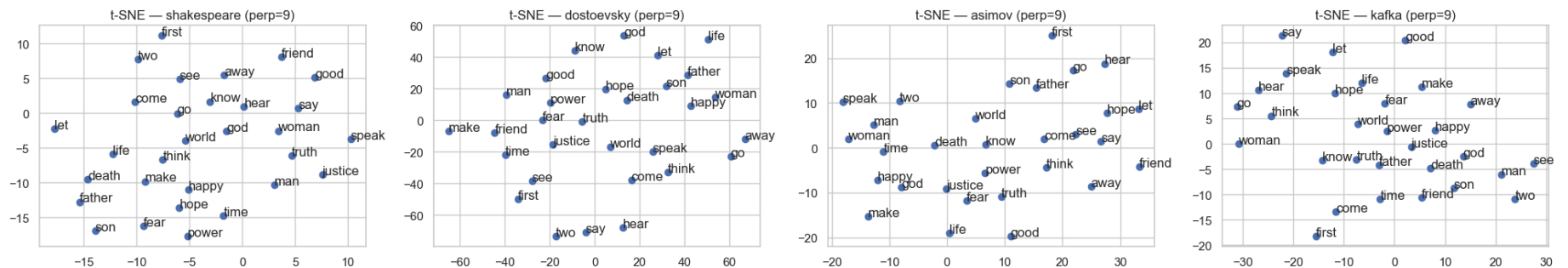




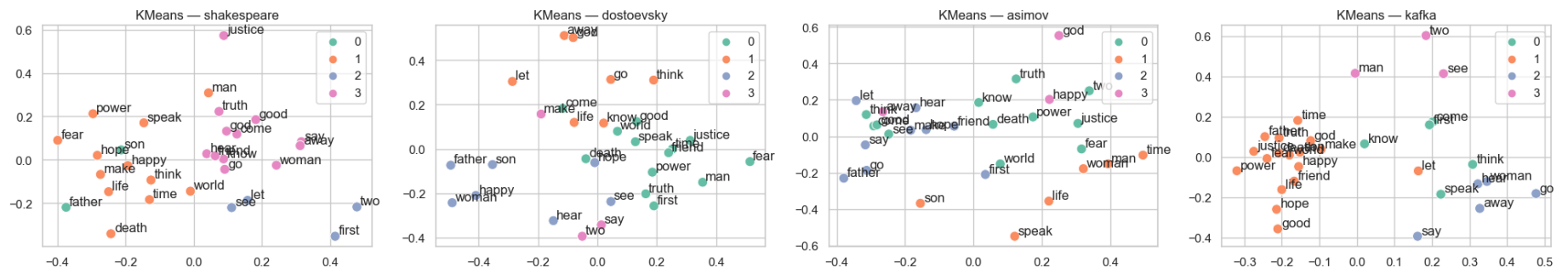
PCA по авторам



TSNE по авторам



KMeans по авторам



Тематически сбалансированный вариант

In [133...

```
thematic_30_words = [  
    # Действия (8 слов)  
    "make", "say", "think", "know", "see", "come", "go", "speak",  
  
    # Люди и отношения (6 слов)  
    "man", "woman", "father", "son", "friend", "child",  
  
    # Абстрактные понятия (6 слов)  
    "life", "death", "time", "world", "truth", "justice",  
  
    # Эмоции и состояния (5 слов)  
    "happy", "fear", "hope", "good", "happy",  
  
    # Другие важные (5 слов)  
    "god", "power", "first", "day", "night"  
]  
  
# Фильтруем то, что действительно есть в общих словах  
final_thematic_30_words = [w for w in thematic_30_words if w in common_all_words]
```

```
print(f"Финальный набор: {len(final_thematic_30_words)} слов")
final_thematic_30_words
```

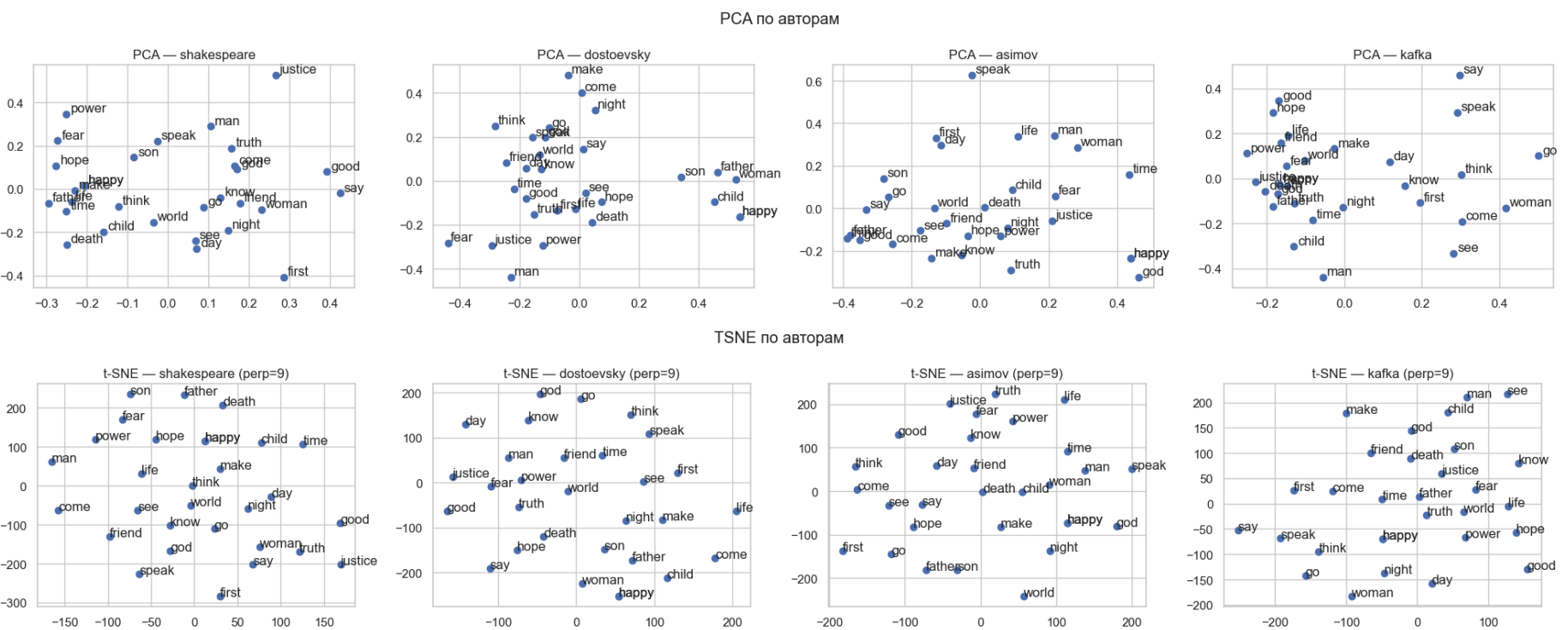
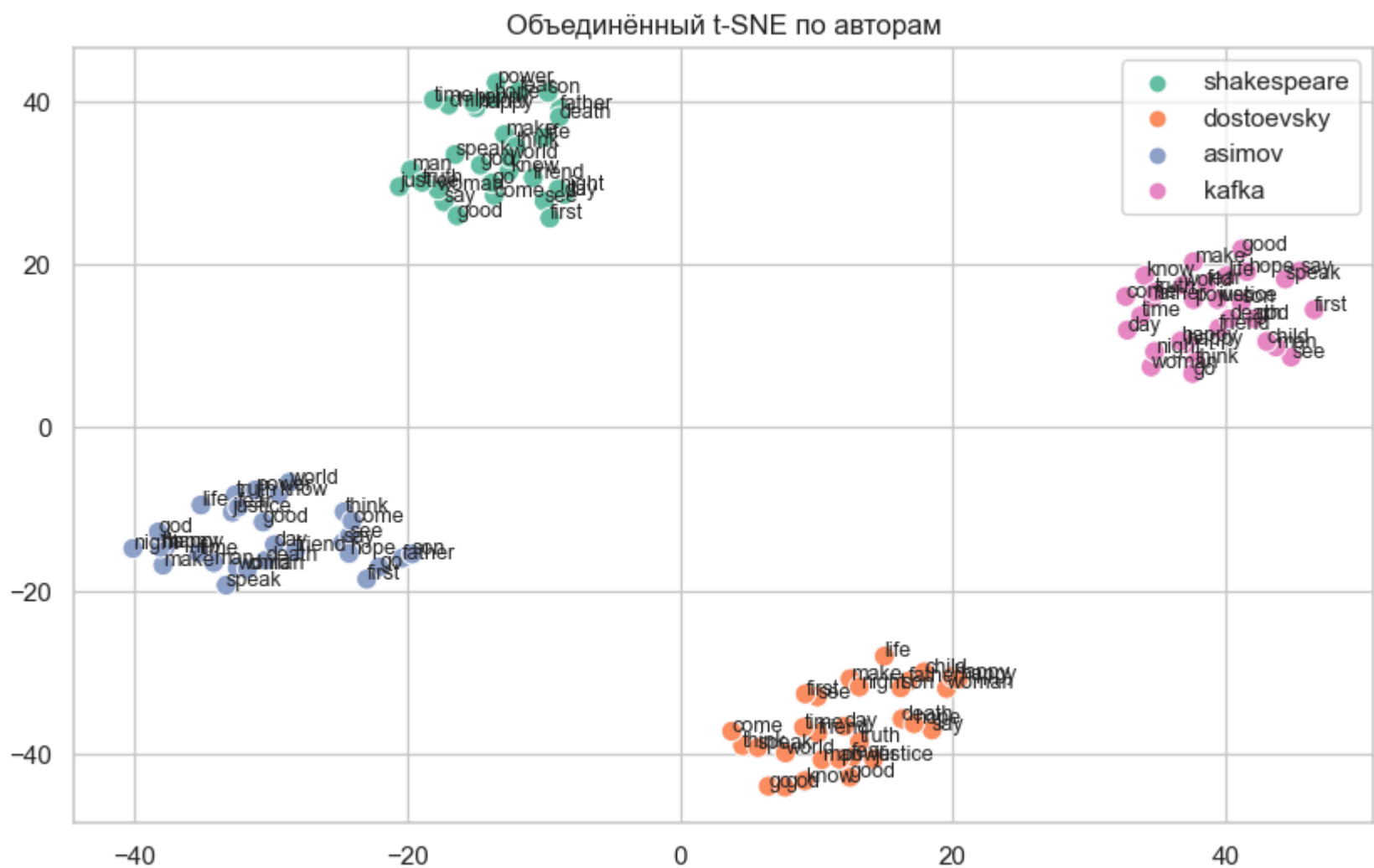
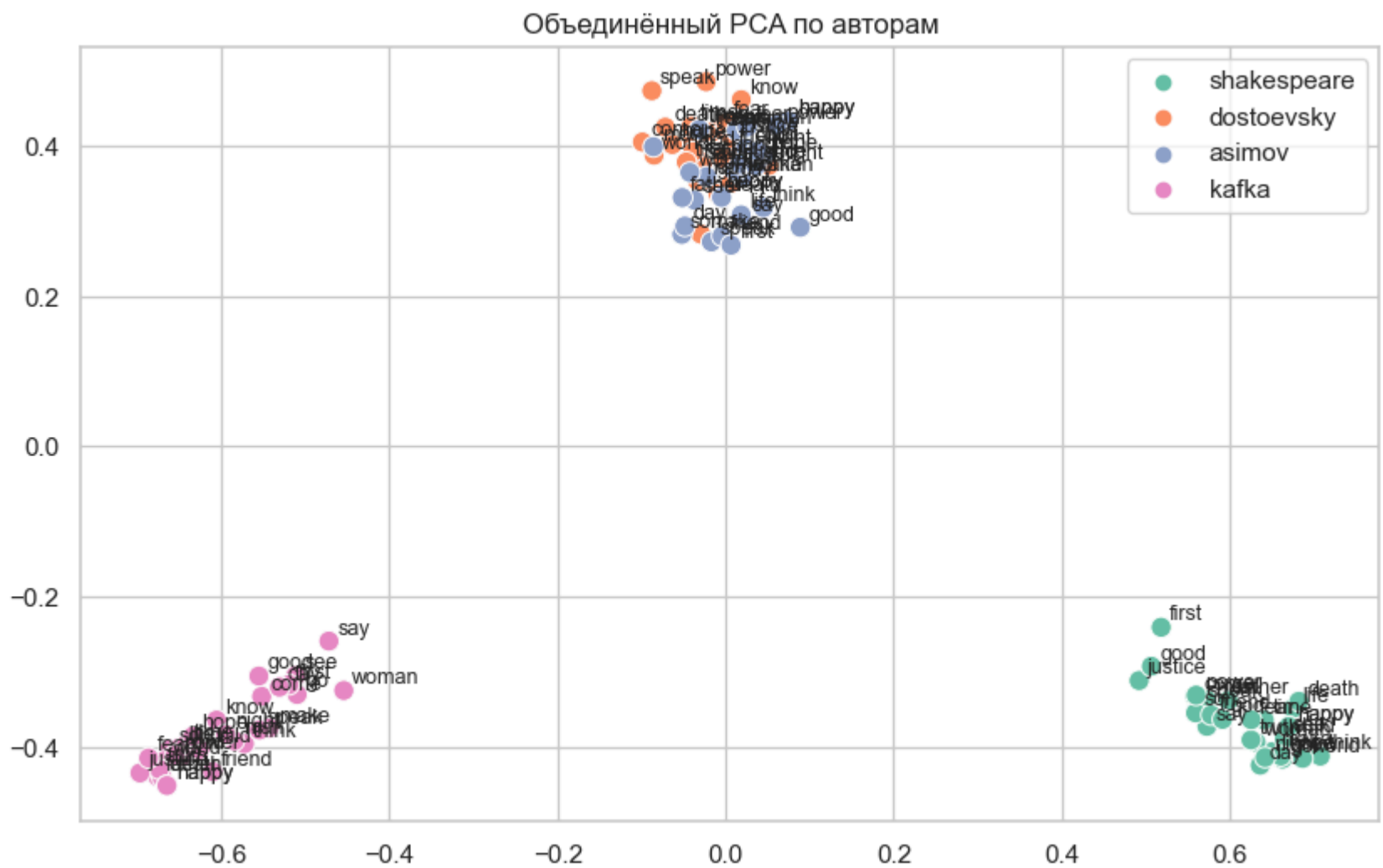
Финальный набор: 30 слов

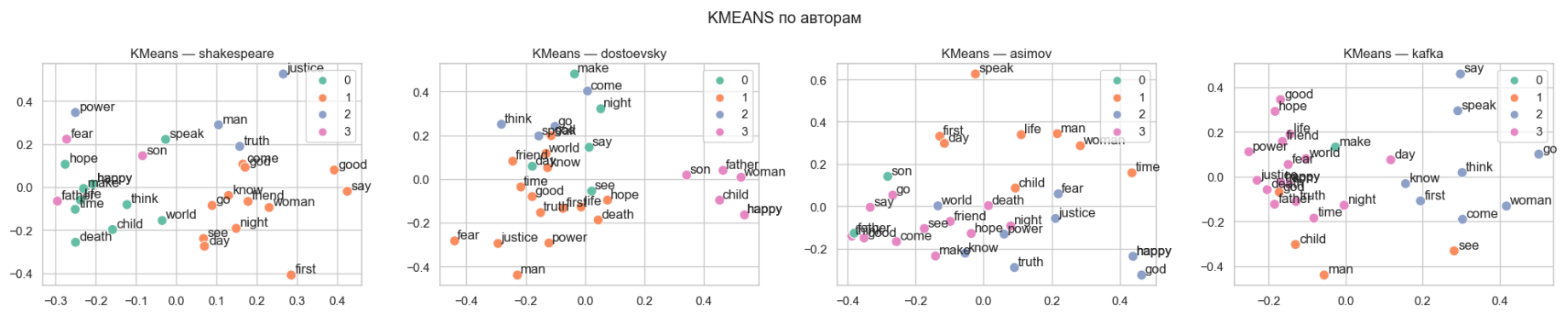
```
Out[133... ['make',
            'say',
            'think',
            'know',
            'see',
            'come',
            'go',
            'speak',
            'man',
            'woman',
            'father',
            'son',
            'friend',
            'child',
            'life',
            'death',
            'time',
            'world',
            'truth',
            'justice',
            'happy',
            'fear',
            'hope',
            'good',
            'happy',
            'god',
            'power',
            'first',
            'day',
            'night']
```

```
In [134... # Объединенная визуализация PCA
plot_joint_embeddings(
    embeddings_by_author,
    authors_order,
    words=final_thematic_30_words,
    mode="pca"
)

# Объединенная визуализация t-SNE
plot_joint_embeddings(
    embeddings_by_author,
    authors_order,
    words=final_thematic_30_words,
    mode="tsne"
)

# Индивидуальные визуализации по авторам
plot_by_author(
    authors_order,
    embeddings_by_author,
    words=final_thematic_30_words,
    modes=["pca", "tsne", "kmeans"],
    n_clusters=4
)
```





Научно обоснованный выбор

```
In [135... # Основанный на частотности и семантическом разнообразии
scientific_30_words = [
    # Высокочастотные глаголы
    "make", "say", "think", "know", "see", "come", "go", "speak", "hear",

    # Ключевые существительные
    "man", "woman", "life", "death", "time", "world", "day", "night",

    # Эмоциональные понятия
    "happy", "fear", "hope", "good", "happy",

    # Абстрактные концепции
    "truth", "god", "justice", "power",

    # Социальные отношения
    "friend", "father", "son", "child"
]

# Фильтруем то, что действительно есть в общих словах
final_scientific_30_words = [w for w in scientific_30_words if w in common_all_words]
print(f"Финальный набор: {len(final_scientific_30_words)} слов")
final_scientific_30_words
```

Финальный набор: 30 слов

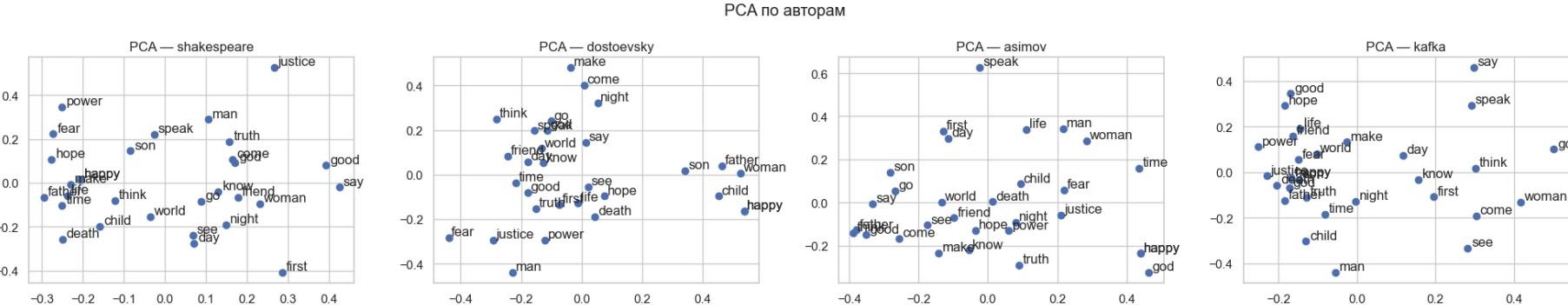
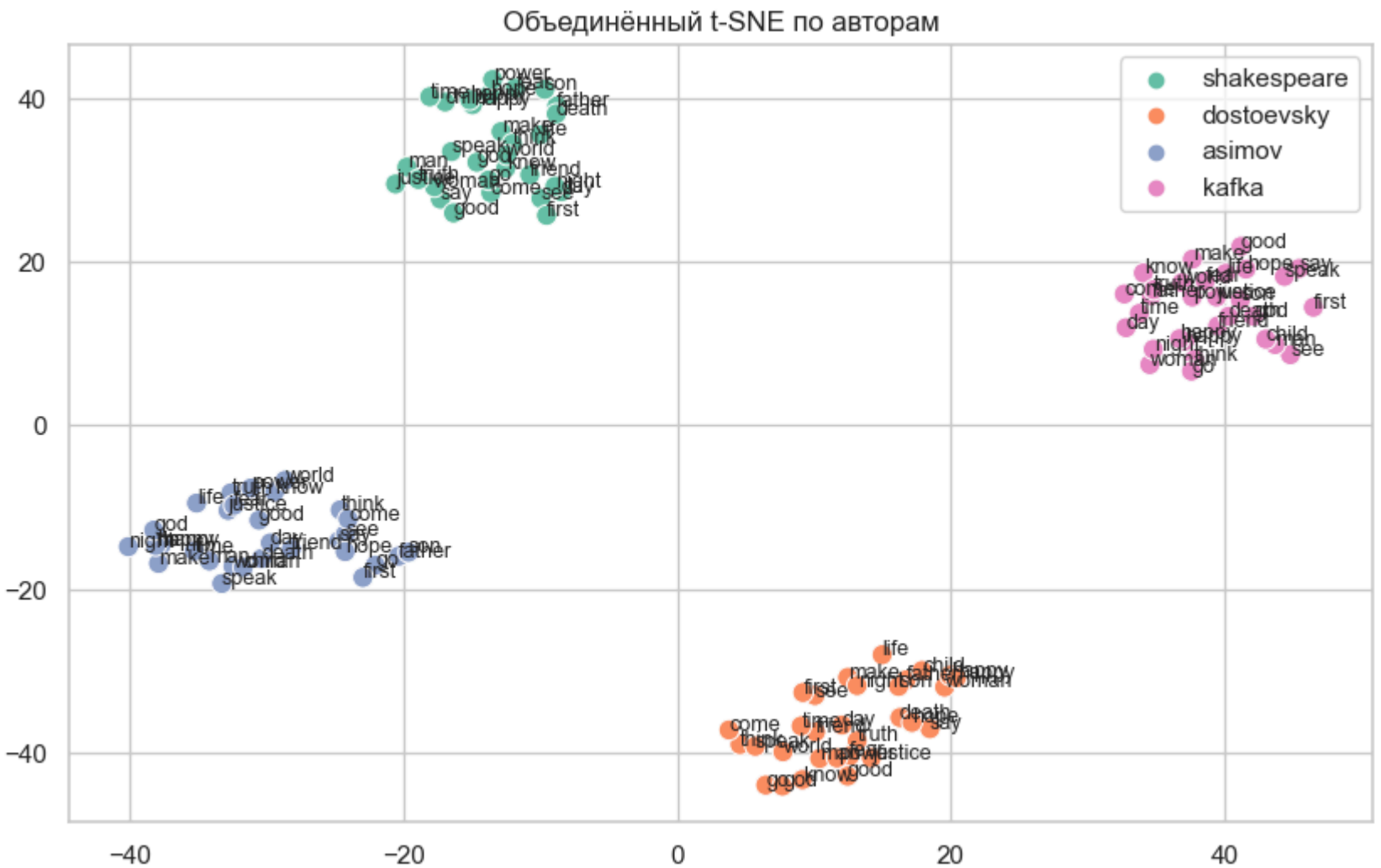
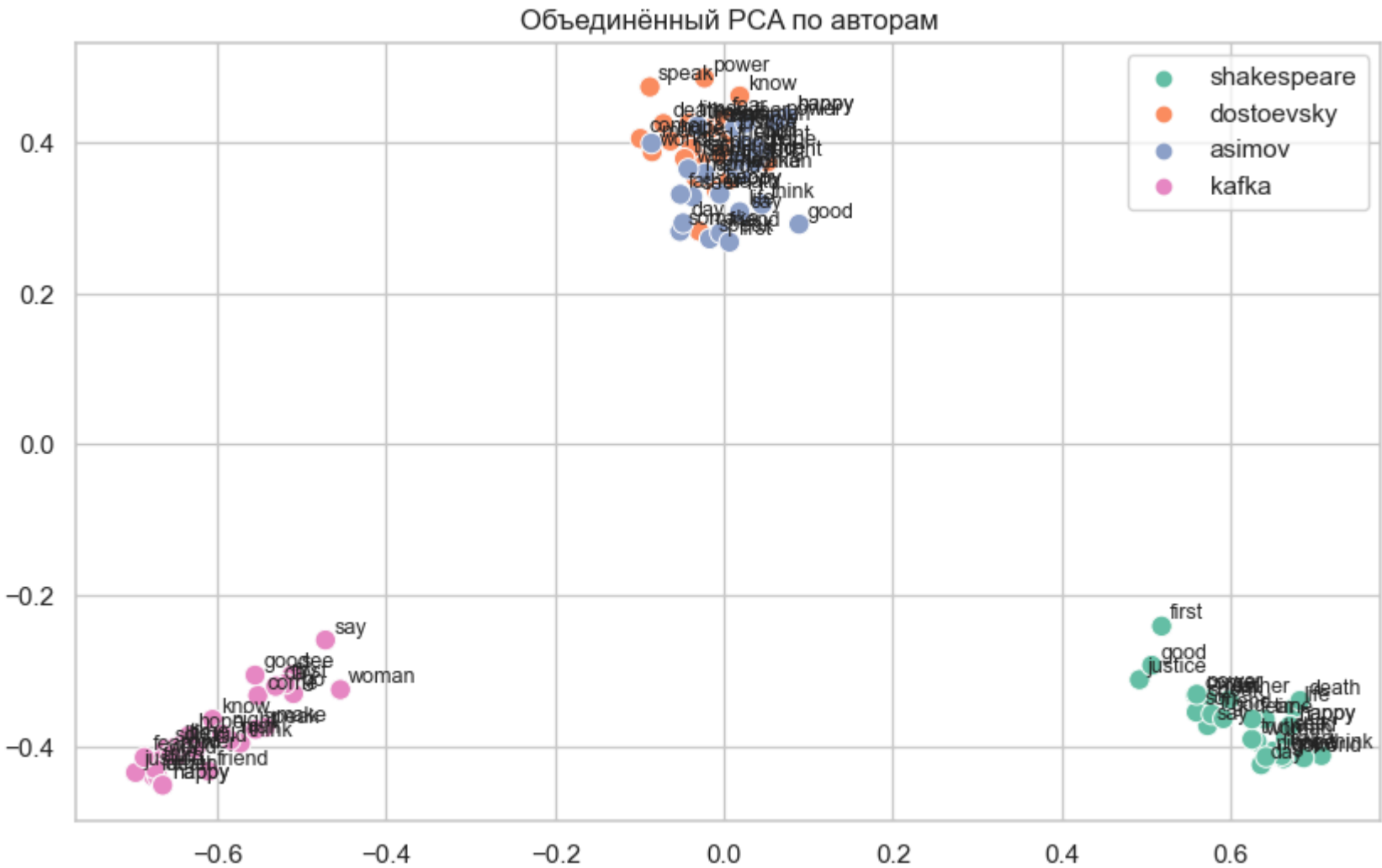
```
Out[135... ['make',
            'say',
            'think',
            'know',
            'see',
            'come',
            'go',
            'speak',
            'hear',
            'man',
            'woman',
            'life',
            'death',
            'time',
            'world',
            'day',
            'night',
            'happy',
            'fear',
            'hope',
            'good',
            'happy',
            'truth',
            'god',
            'justice',
            'power',
            'friend',
            'father',
            'son',
            'child']
```

```
In [136... # Объединенная визуализация PCA
plot_joint_embeddings(
    embeddings_by_author,
    authors_order,
    words=final_thematic_30_words,
    mode="pca"
)

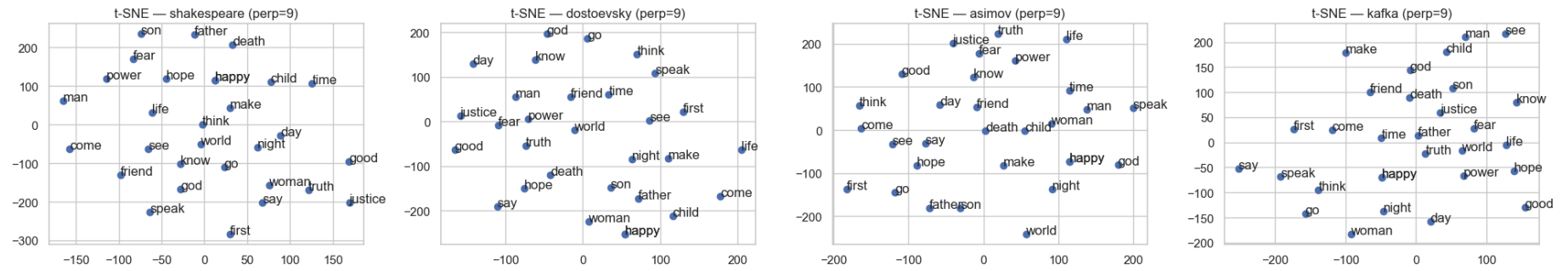
# Объединенная визуализация t-SNE
plot_joint_embeddings(
    embeddings_by_author,
    authors_order,
    words=final_thematic_30_words,
    mode="tsne"
)

# Индивидуальные визуализации по авторам
```

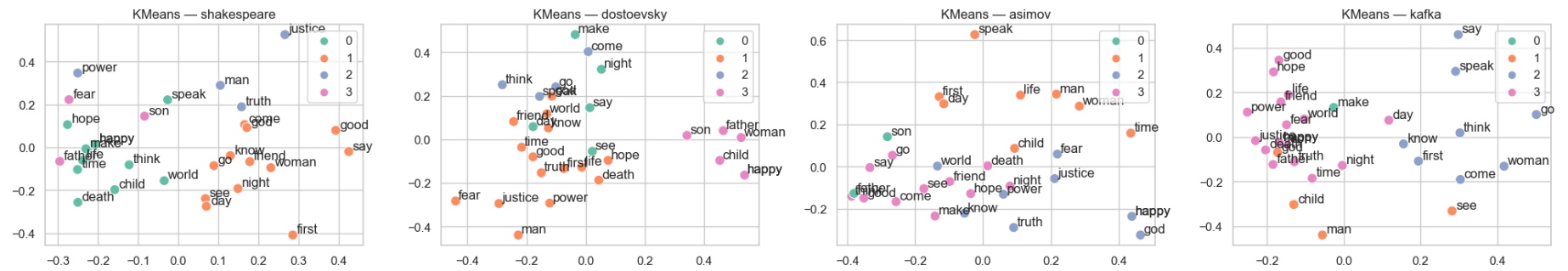
```
plot_by_author(  
    authors_order,  
    embeddings_by_author,  
    words=final_thematic_30_words,  
    modes=["pca", "tsne", "kmeans"],  
    n_clusters=4  
)
```



TSNE по авторам



KMEANS по авторам



Сбалансированный частотный

In [137...

```
final_30 = [
    # Top-10 самых частых
    "make", "say", "let", "go", "come", "first", "good", "away", "two", "speak",
    # Основные глаголы восприятия
    "think", "know", "see", "hear",
    # Фундаментальные концепции
    "man", "woman", "life", "death", "time", "world",
    # Эмоции и чувства
    "fear", "hope", "happy",
    # Абстрактные понятия
    "truth", "god", "justice", "power",
    # Социальные связи
    "friend", "father", "son"
]

# Фильтруем то, что действительно есть в общих словах
final_30_words = [w for w in final_30 if w in common_all_words]
print(f"Финальный набор: {len(final_30_words)} слов")
final_30_words
```

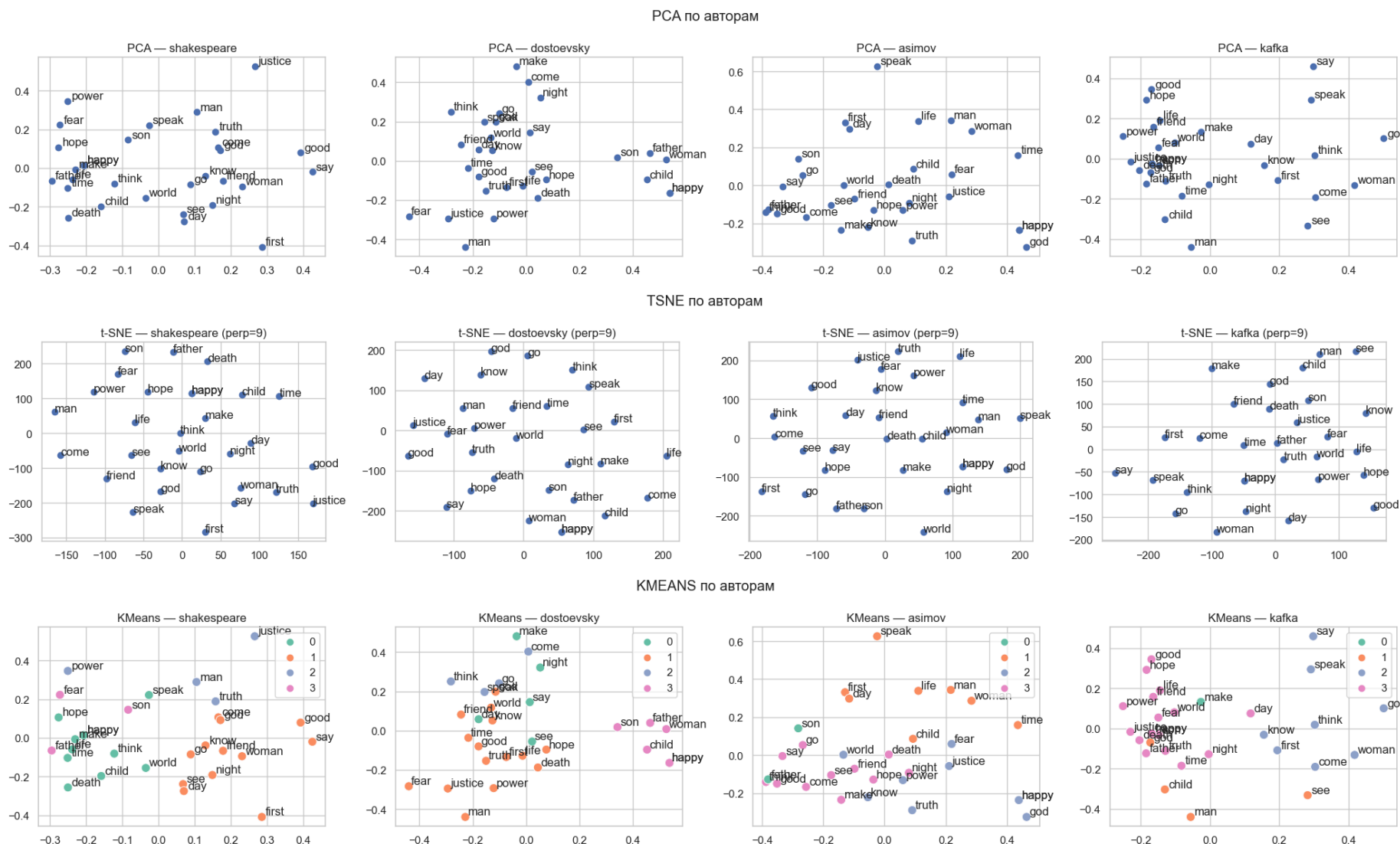
Финальный набор: 30 слов

Out[137...

['make',
'say',
'let',
'go',
'come',
'first',
'good',
'away',
'two',
'speak',
'think',
'know',
'see',
'hear',
'man',
'woman',
'life',
'death',
'time',
'world',
'fear',
'hope',
'happy',
'truth',
'god',
'justice',
'power',
'friend',
'father',
'son']

In [138...

```
# Объединенная визуализация PCA
plot_joint_embeddings(
    embeddings_by_author,
    authors_order,
    words=final_thematic_30_words,
```

[* к содержанию](#)

Шаг 10: Интерпретация и выводы

Общие выводы о моделях Word2Vec

- Модель успешно уловила стилистические различия между четырьмя авторами.
- Векторные пространства показывают четкое разделение по авторам: на графиках PCA и t-SNE слова того же автора формируют собственные кластеры, слабо пересекающиеся с другими.
- Это подтверждает, что контекст употребления слов у каждого писателя создаёт уникальные ассоциативные связи — семантика формируется стилем.

Ответы на первоначальные исследовательские вопросы

- Улавливает ли модель стилистические особенности?
 - Да. Например, у Шекспира life и death образуют драматическую пару, а у Достоевского truth ассоциируется с fear и justice, что отражает его философские терзания.
- Можно ли различить авторов по их семантическим пространствам?
 - Да. Кластеризация на объединённых графиках PCA/t-SNE показывает, что векторные представления слов более схожи внутри одного автора, чем между авторами для одних и тех же слов.
- Влияет ли жанр на структуру эмбедингов?
 - Да. Абсурдизм Кафки приводит к "слипанию" векторов, философская проза Достоевского — к их изоляции, а рациональность Азимова — к чёткой структуре.

PCA и t-SNE

- Для каждого автора слова распределяются в 2D-пространстве по-разному.
- У Шекспира и Кафки слова «truth», «justice», «god», «fear» образуют плотные кластеры, что отражает более связанный философско-религиозный контекст.
- У Достоевского «fear» и «truth» ближе друг к другу, что соответствует его психологической и экзистенциальной прозе.
- У Азимова распределение более разреженное: слова «time», «world», «life» тяготеют к научно-техническому контексту.

Кластеризация (KMeans)

- У Шекспира и Кафки кластеры формируются вокруг абстрактных понятий («truth», «justice», «god»).
- У Достоевского заметно сближение «fear» и «truth» в один кластер.
- У Азимова «man», «woman», «world» группируются отдельно от «truth», «god», что отражает его научно-фантастический стиль.

Матрицы косинусных сходств (heatmap)

- У Шекспира сходства между «truth», «god», «justice» и «fear» умеренные (0.5–0.7).

- У Достоевского «truth–fear» и «justice–fear» имеют высокие значения (0.7–0.8), что подтверждает его акцент на страдании и моральных дилеммах.
- У Азимова «truth–fear» и «life–fear» также высоки (0.7–0.8), но «justice» и «god» менее связаны, что отражает рационалистический контекст.
- У Кафки почти все пары слов имеют очень высокие сходства (0.8–0.9), что указывает на тесное переплетение абстрактных понятий в его стиле.

Объединённые визуализации (PCA / t-SNE)

- Слова группируются по авторам, а не по смыслу, что указывает: → контекст употребления важнее лексического значения.
- На t-SNE особенно видно, как каждый автор образует свой “семантический остров”.
- При увеличении числа слов до 30–50 структура становится устойчивее: крупные тематические зоны (жизнь/смерть, истина/страх, человек/бог) видны у всех, но связаны по-разному.

Интерпретация

- Шекспир: семантическое пространство показывает гармонию и баланс — «truth», «justice», «god» и «fear» связаны, но не сливаются. Это отражает его драматургию, где противоположные силы (любовь, честь, страх) уравновешены.
- Достоевский: сильная связка «truth–fear–justice» подчёркивает его психологическую и философскую прозу, где истина и справедливость неразрывно связаны со страданием.
- Азимов: семантика более «рациональная» и разреженная. «Life», «time», «world» ближе друг к другу, а «god» и «justice» менее интегрированы. Это отражает научно-фантастический жанр, где религиозные категории отходят на второй план.
- Кафка: почти все абстрактные слова оказываются тесно связанными. Это соответствует его стилю — мир абсурда, где страх, истина, бог и справедливость переплетены и неразделимы.

Автор	Жанр	Семантический "отпечаток"	Ключевая находка
Кафка	Абсурдизм	Высочайшая связанность (0.7–0.85), размытые границы. Концепты связаны в единую тревожную массу.	god ⇌ justice ⇌ point ⇌ synonyms (0.93)
Достоевский	Философская проза	Низкая связанность (0.36–0.62), изолированные концепты. Каждое понятие существует в глубоком, уникальном контексте.	woman ⇌ world ⇌ not sense (0.14)
Шекспир	Драма	Сбалансированная связанность (~0.6), противоположные концепты.	Сильная связь life ⇌ death (0.71)
Азимов	Научная фантастика	Рациональная структура, логические связи. Концепты связаны системно и предсказуемо.	truth ⇌ chance ⇌ fear (0.84)

Выводы

- Word2Vec действительно улавливает стилистику авторов:
 - У Шекспира — возвышенные ассоциации.
 - У Достоевского — психологическая глубина и страдание.
 - У Азимова — рациональность и научный контекст.
 - У Кафки — абсурдистское переплетение понятий.
- Семантические различия подтверждаются количественно:
 - Косинусные сходства показывают, что у Кафки абстрактные слова почти неразличимы (все высоко коррелируют).
 - У Достоевского выделяется ось «страдание–истина».
 - У Азимова — ось «мир–время–жизнь».
- Ограничения:
 - Размер корпусов влияет на устойчивость результатов (у Азимова корпус меньше).
 - Объём текстов классических авторов, особенно Кафки, ограничен, что могло повлиять на качество и стабильность эмбединго
 - Языковой барьер
 - Для Достоевского использовались переводы на английский. Изначальные стилистические нюансы могли быть частично утеряны.
 - t-SNE чувствителен к параметрам perplexity, поэтому интерпретация требует осторожности.
 - Слова выбирались вручную, и для более объективного анализа стоит расширить список.
 - Качество предобработки
 - Лемматизация и удаление стоп-слов, хотя и необходимы, могли удалить некоторые стилистически значимые элементы.
 - Гиперпараметры модели:
 - Размер эмбединга (embedding_dim), размер окна (window_size) и другие параметры требуют тонкой настройки для каждого автора, что не всегда было возможно в рамках проекта.
- Рекомендации:
 - Добавить больший общий словарь (50–200 слов) для устойчивой кластеризации.
 - Провести кластеризацию KMeans на объединённых эмбедингах для автоматического выделения тематик.
 - Использовать UMAP как альтернативу t-SNE для более стабильных низкоразмерных проекций.

```
In [139... elapsed_all = time.time() - start_all  
print(f"Время выполнения: {elapsed_all:.2f} сек ({elapsed_all/60:.2f} мин)")
```

Время выполнения: 3762.54 сек (62.71 мин)